SQL Injection (SQLi)	Analyzing Android App Permissions and Mobile Traffic
Cross-Site Scripting (XSS)	Testing IoT Device Security (default passwords, open ports)
Broken Authentication and Session Management	Creating and Analyzing Disk Images Using dc3dd and Autopsy (Alternative to FTK Imager)
Cross-Site Request Forgery (CSRF)	Log File Analysis for Incident Detection
Security Misconfiguration	Network Forensics with Wireshark (Analyzing captured attack traffic)
Broken Access Control	Privacy Audit of Popular Apps/Websites and Data Breach Case Study Analysis
Man-in-the-Middle (MITM) Attack Using Burp Suite	Conducting a Security Audit and Risk Assessment
API Security Testing	Cloud Security Configuration (AWS/Azure Free Tier)
Setting up a Virtual Lab Environment and Basic Firewall Configuration	
Password Strength Testing	
Analyzing Phishing Emails	
Packet Sniffing and Network Traffic Analysis with Wireshark	
Setting up a Simple IDS with Snort	
Web Application Vulnerability Scanning with OWASP ZAP	

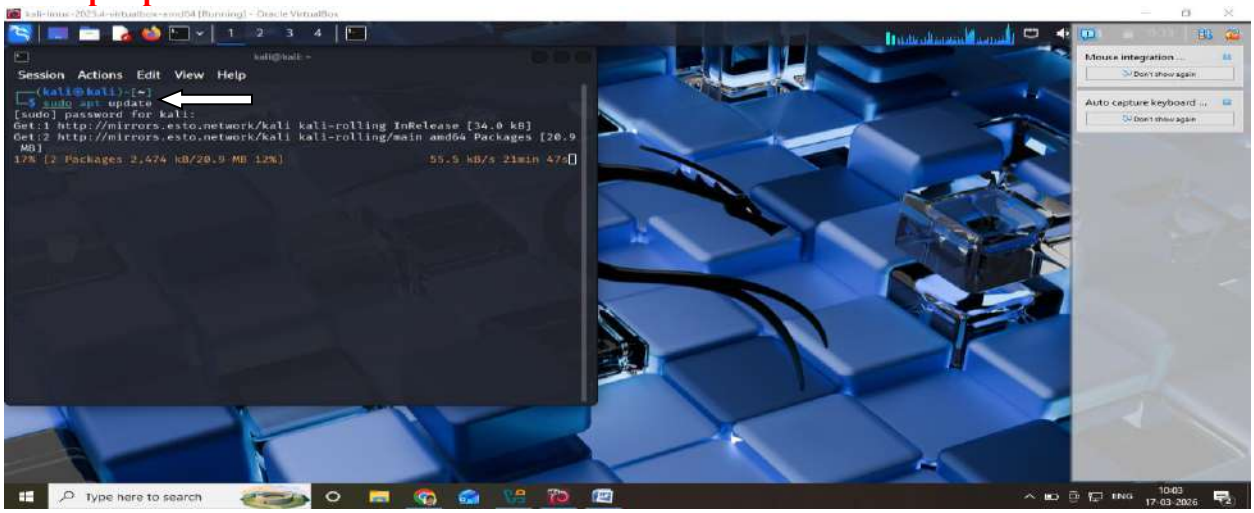
Experiment 1: SQL Injection (SQLi)

Tool: WebGoat / OWASP Juice Shop / Burp Suite
Objective: Exploit SQL injection vulnerabilities to retrieve unauthorized data from a database.
Tasks: Identify SQLi vulnerabilities, bypass authentication, and extract user details.

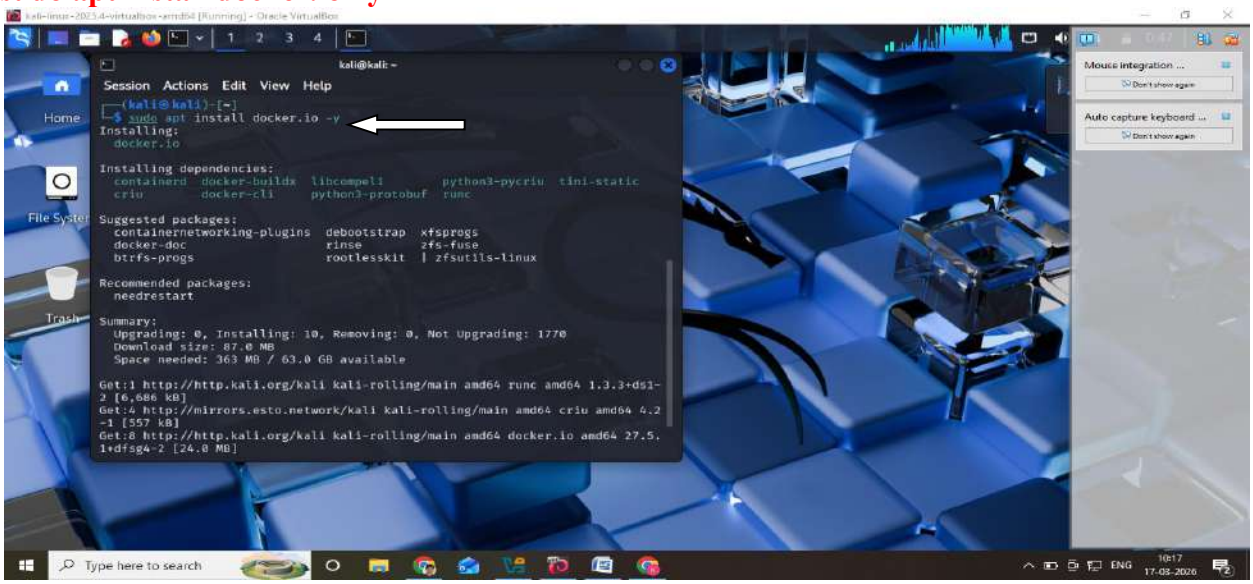
SQL Injection using Burp Suite in OWASP Juice Shop
AIM: To perform SQL Injection attack using **Burp Suite** on Juice Shop.
REQUIREMENTS

- Kali Linux
- Burp Suite
- Web browser
- Juice Shop (local or online)

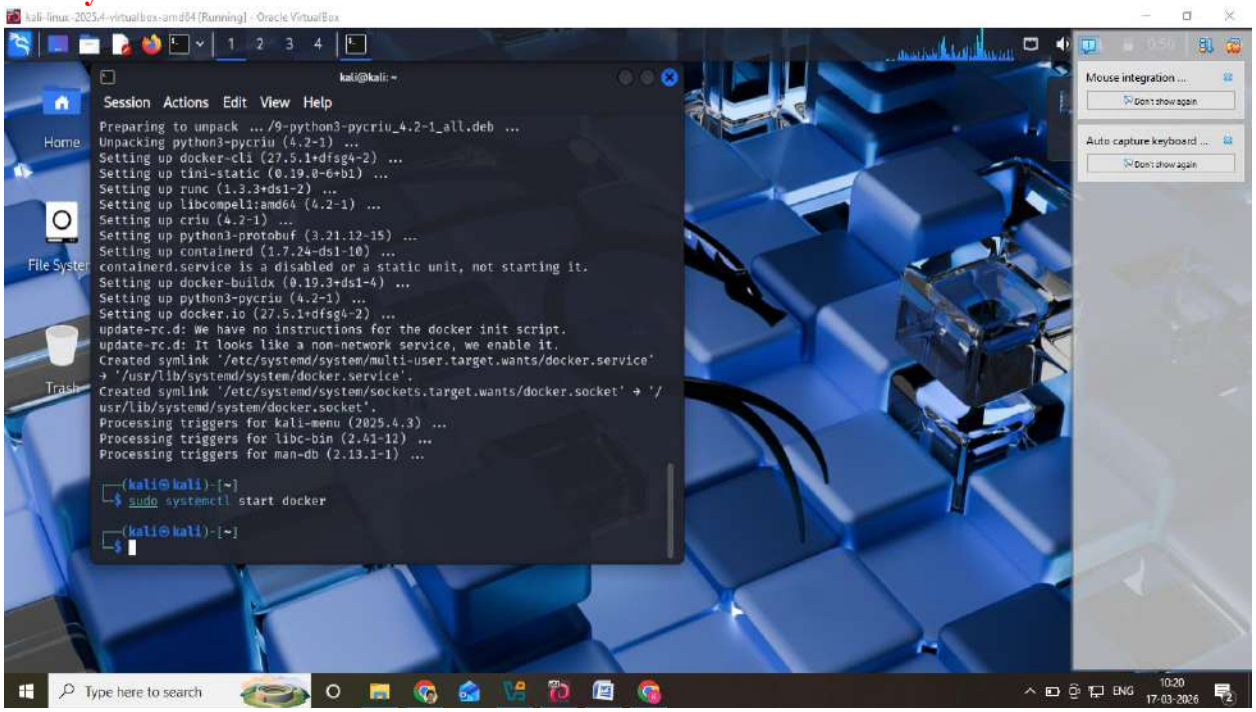
STEP-BY-STEP PROCESS
1. Start Juice Shop in Kali Linux
Juice Shop can run with Docker or Node.js. Docker is easiest.
Step 1: Install Docker (if not installed)
Open terminal in Kali:
sudo apt update



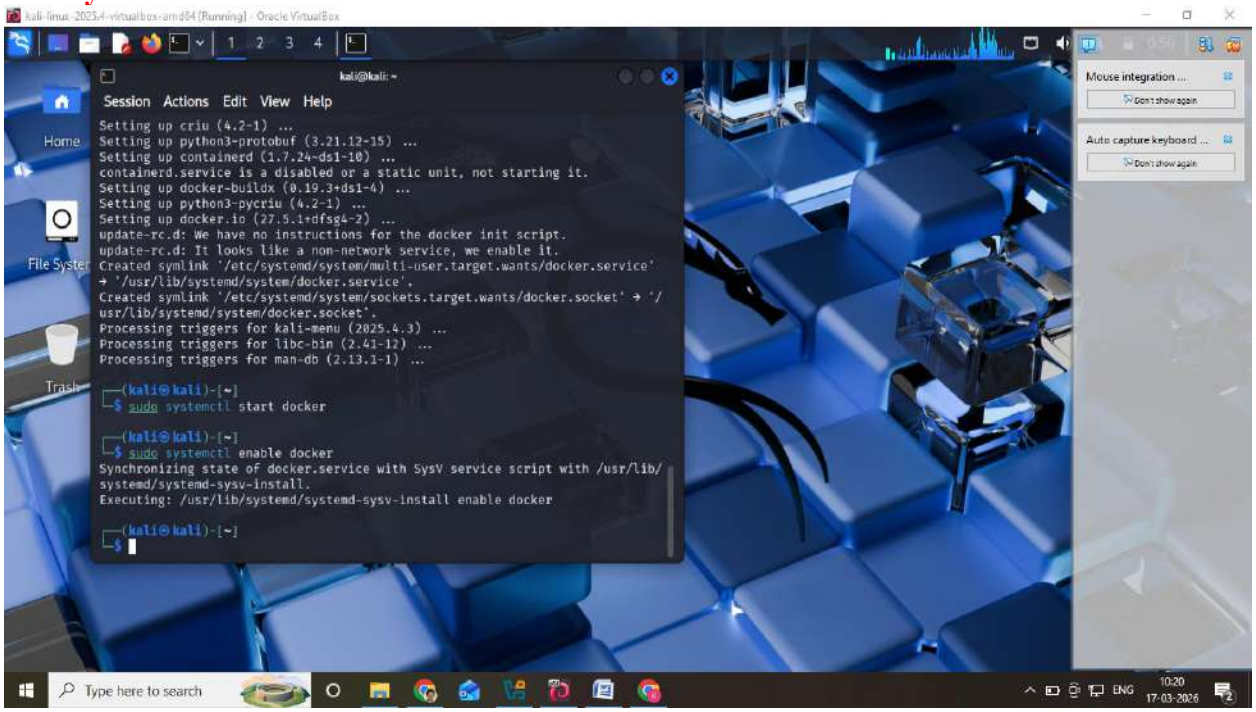
sudo apt install docker.io –y



sudo systemctl start docker

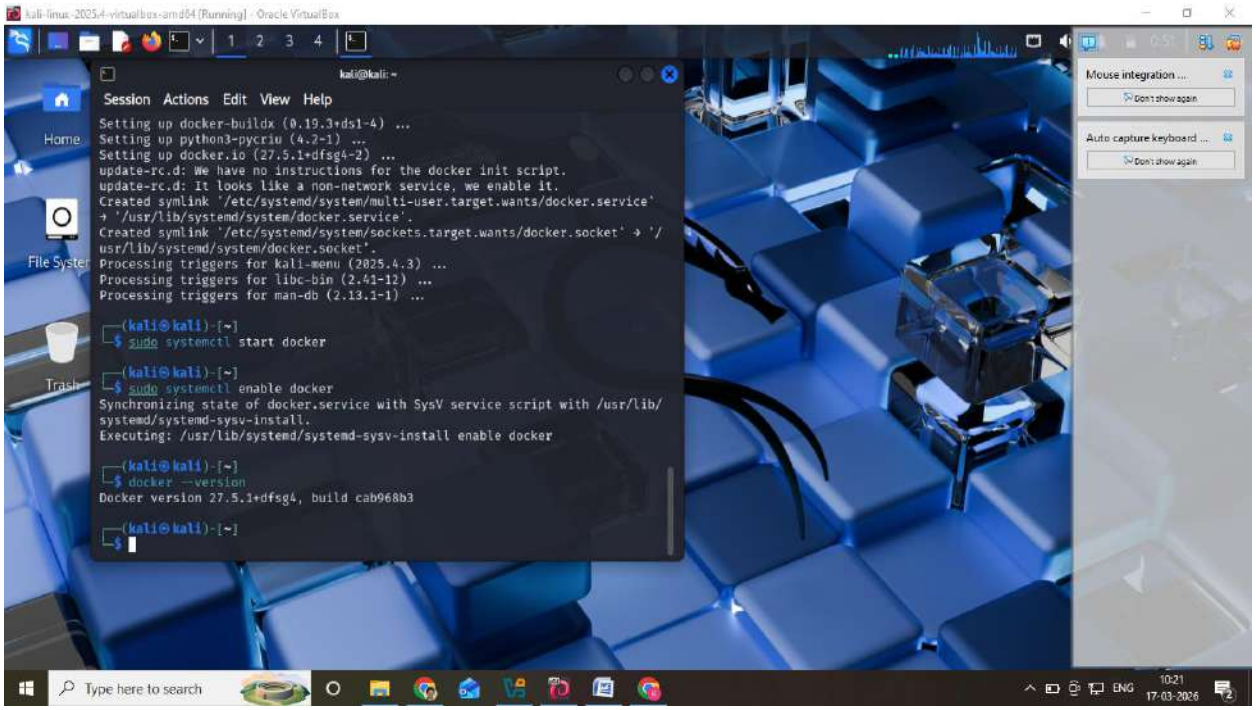


sudo systemctl enable docker



Verify:

docker --version



Run:

sudo usermod -aG docker $USER

Restart the session (important):

reboot

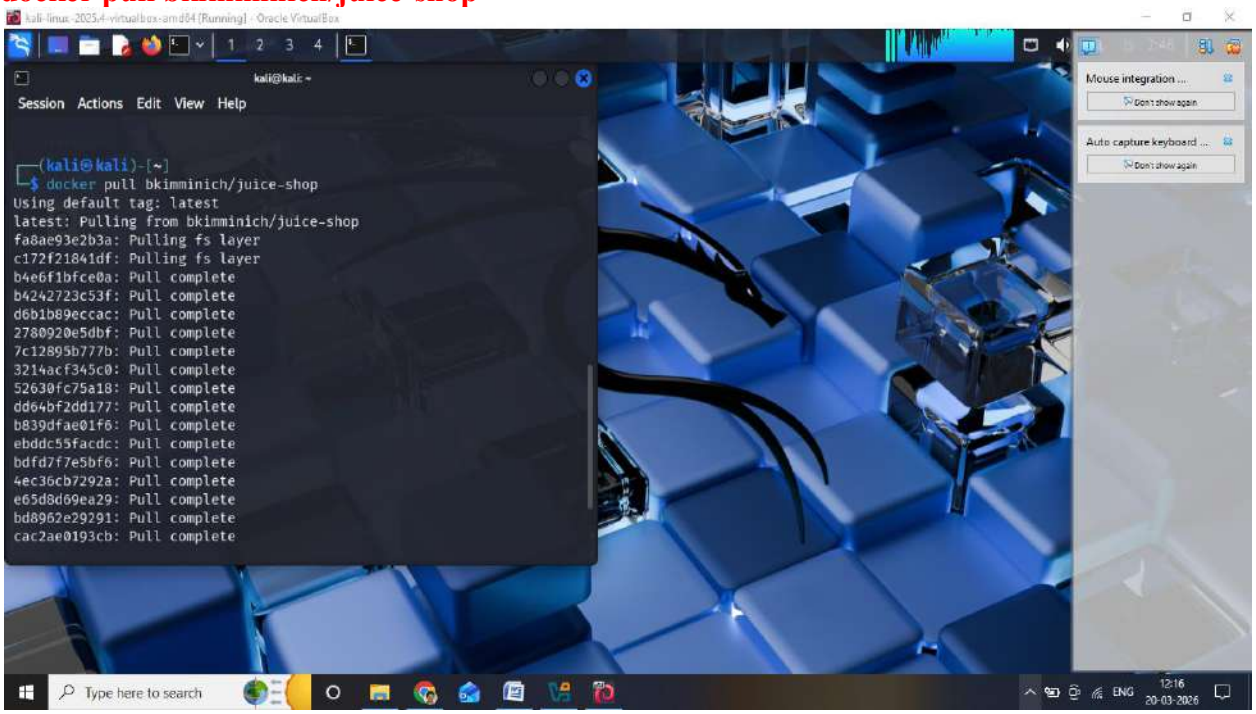
OR logout and login again.

Test Docker:

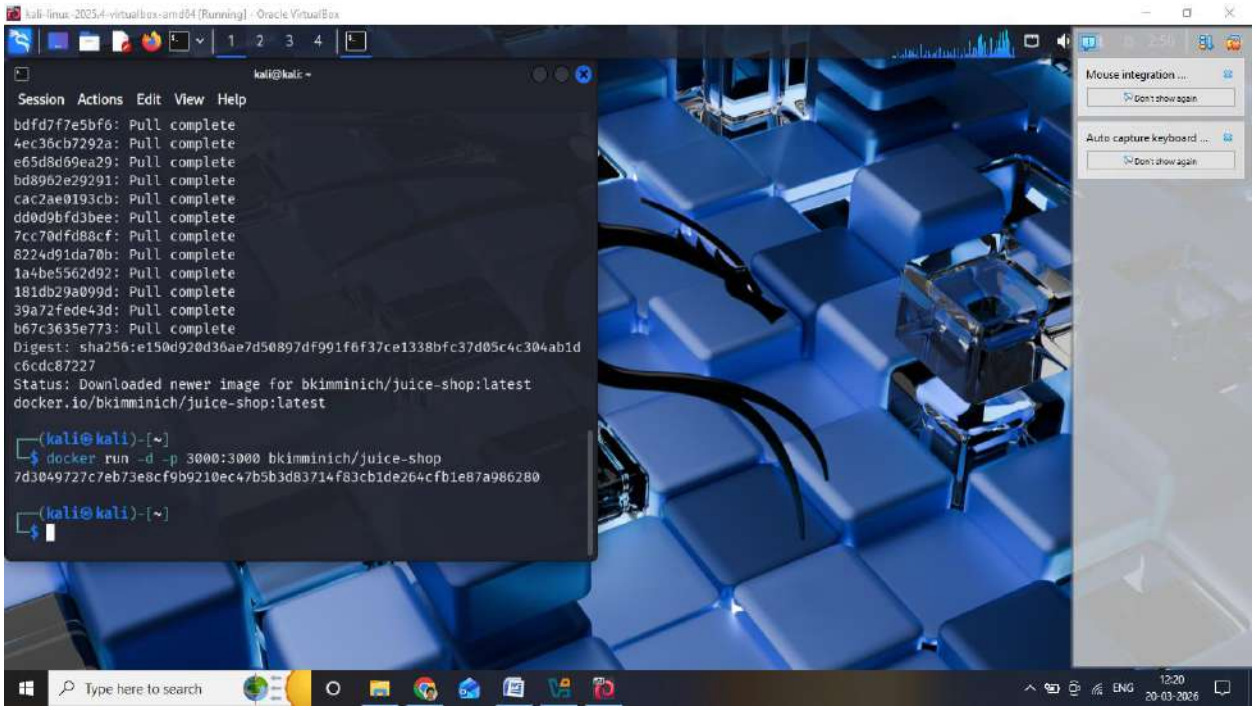
docker run hello-world

Download and Run Juice Shop

docker pull bkimminich/juice-shop

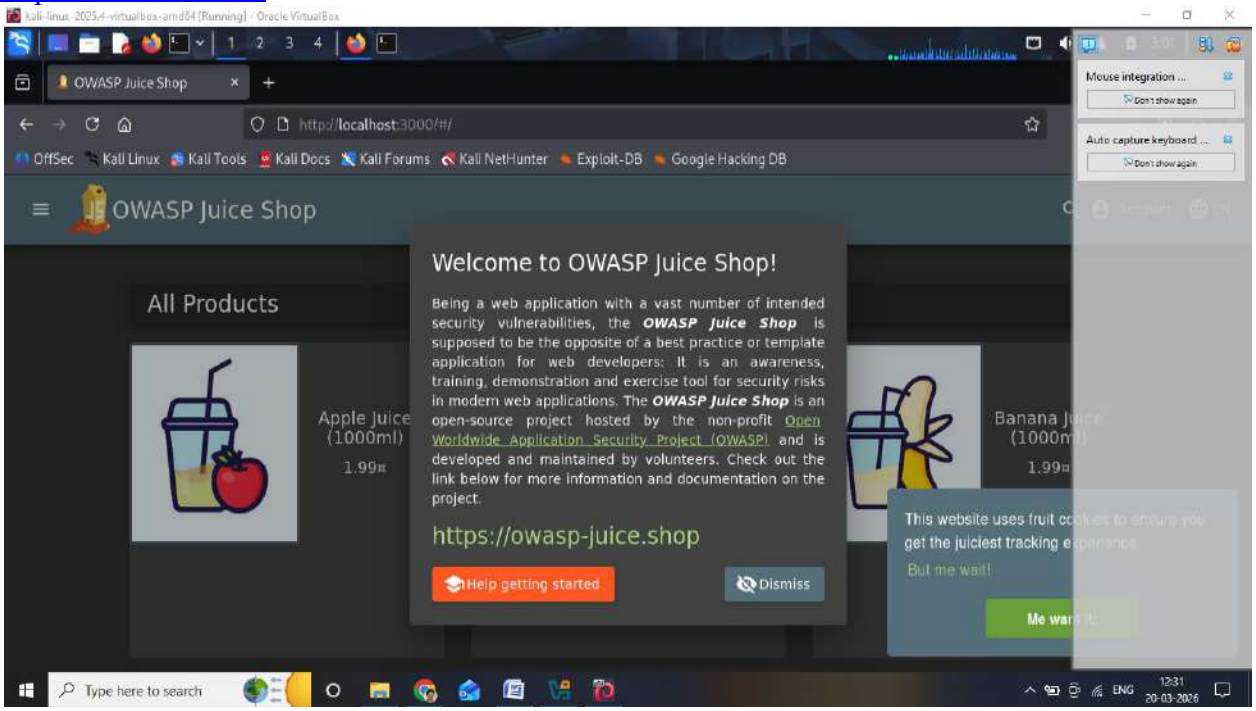


docker run -d -p 3000:3000 bkimminich/juice-shop

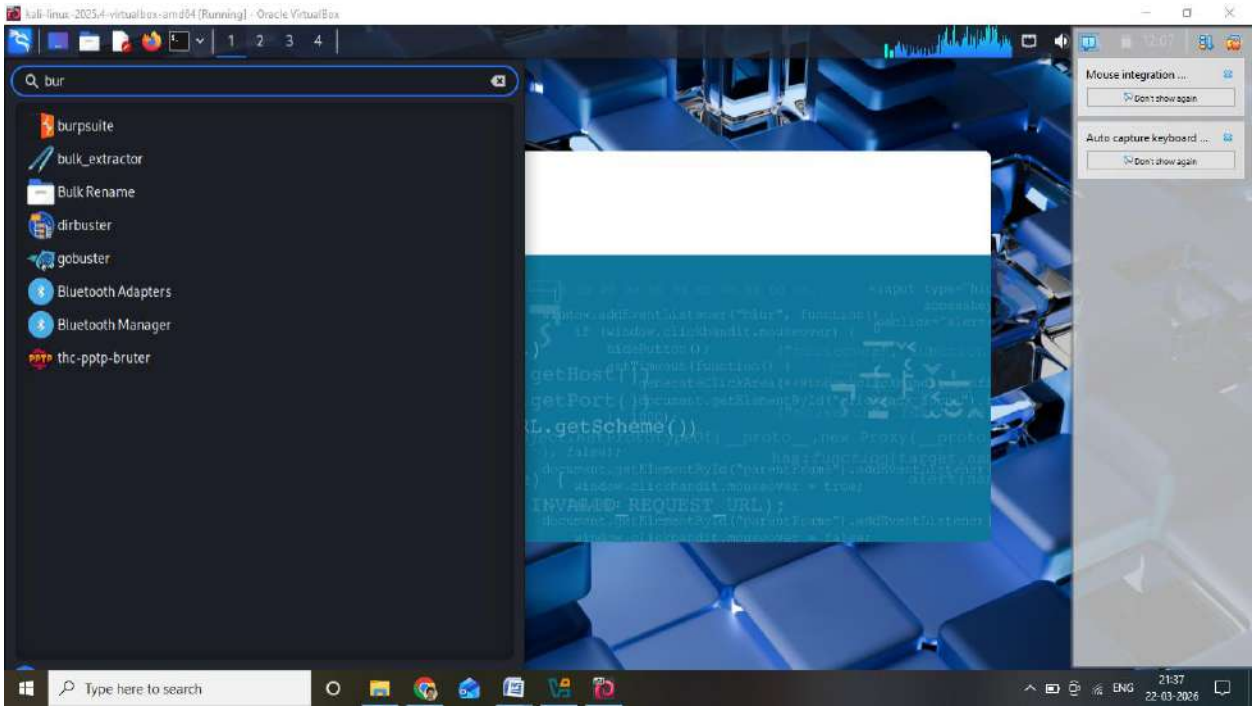


Step 3: Open the Application

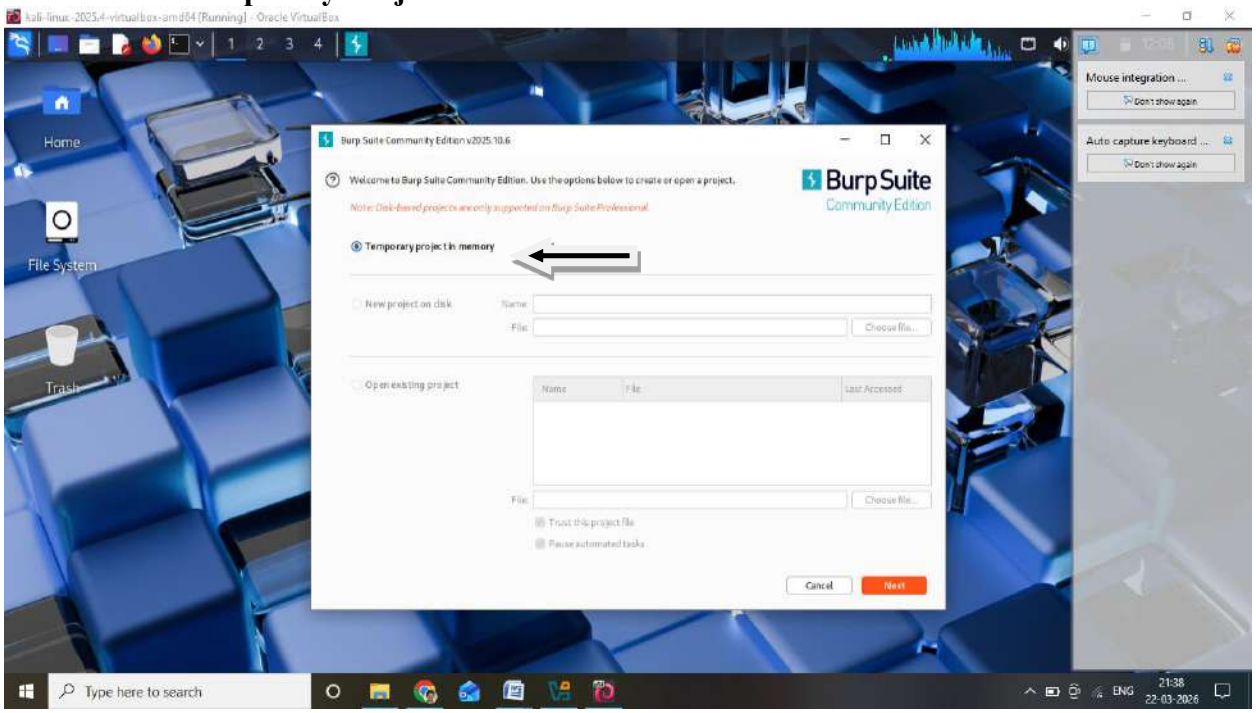
Open browser in Kali and go to:
http://localhost:3000



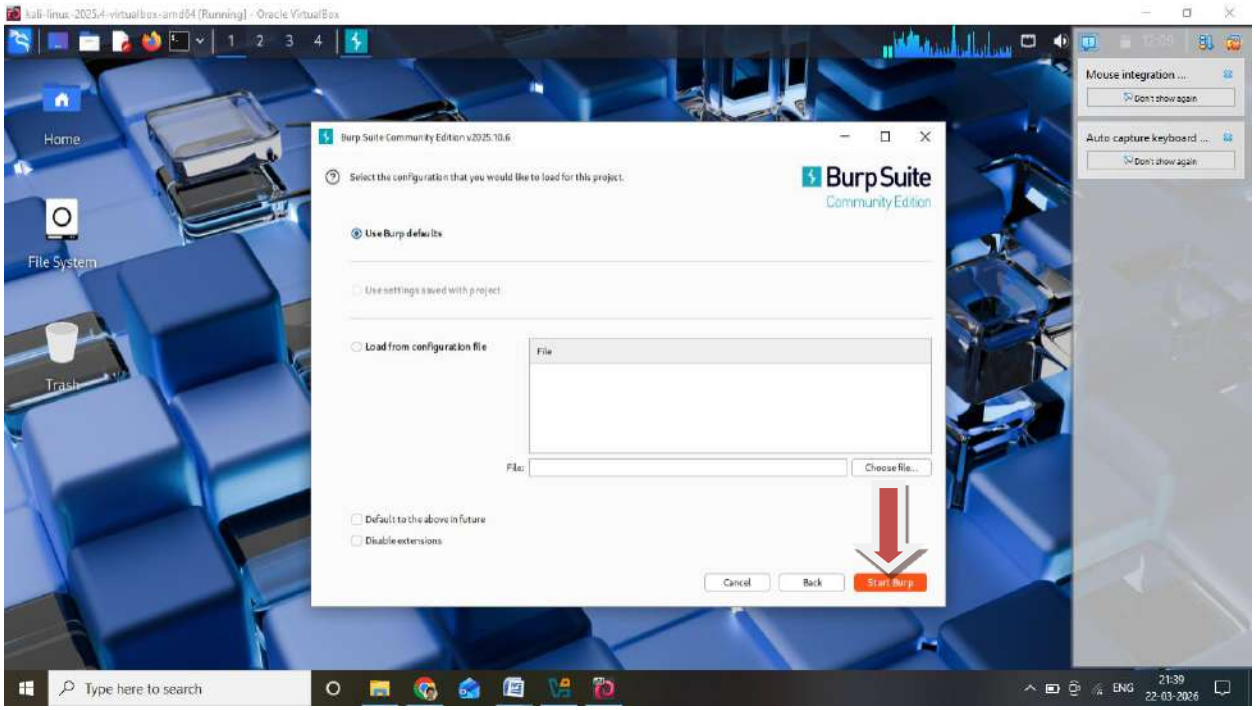
STEP 1: Open Burp Suite
Open terminal:
Burpsuite



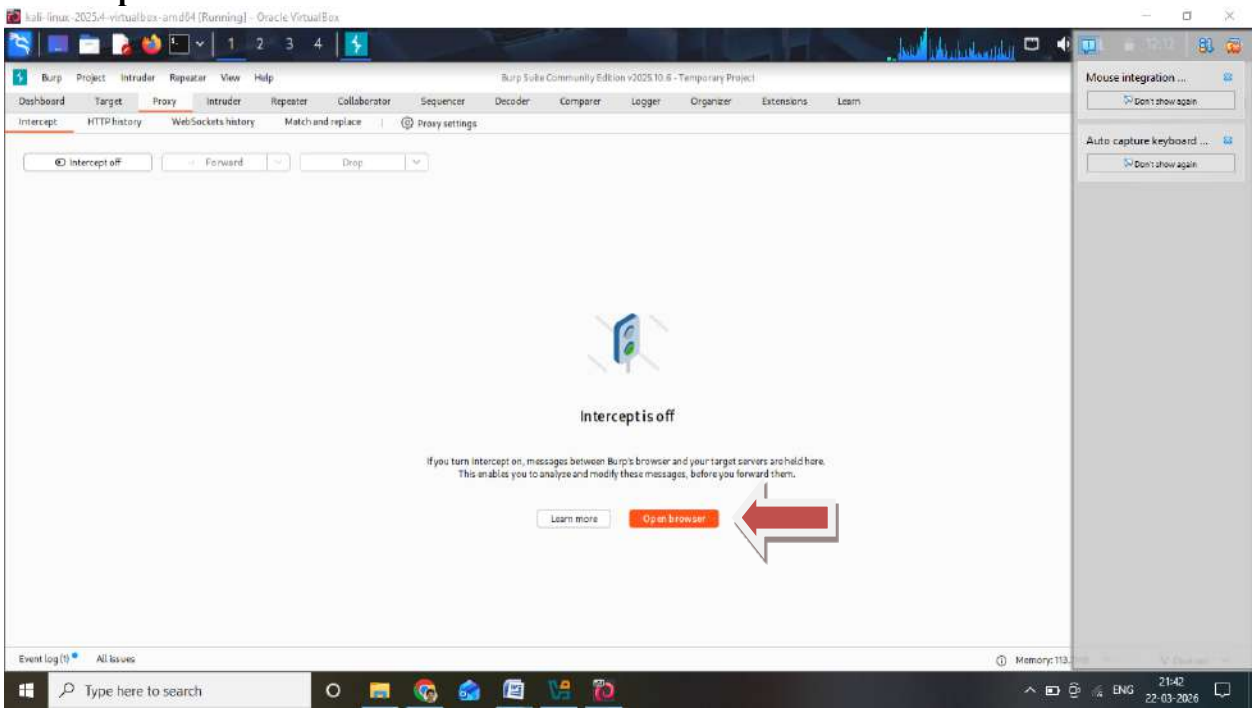
- Select **Temporary Project**



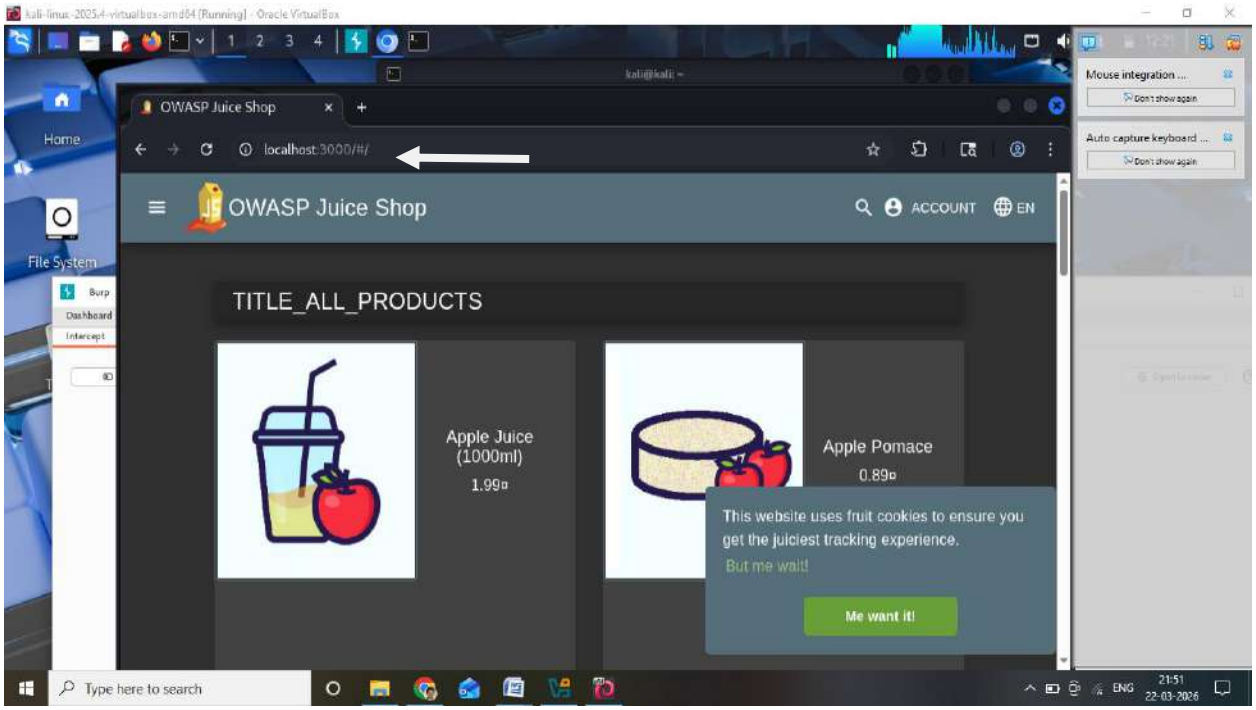
- Click **Start Burp**



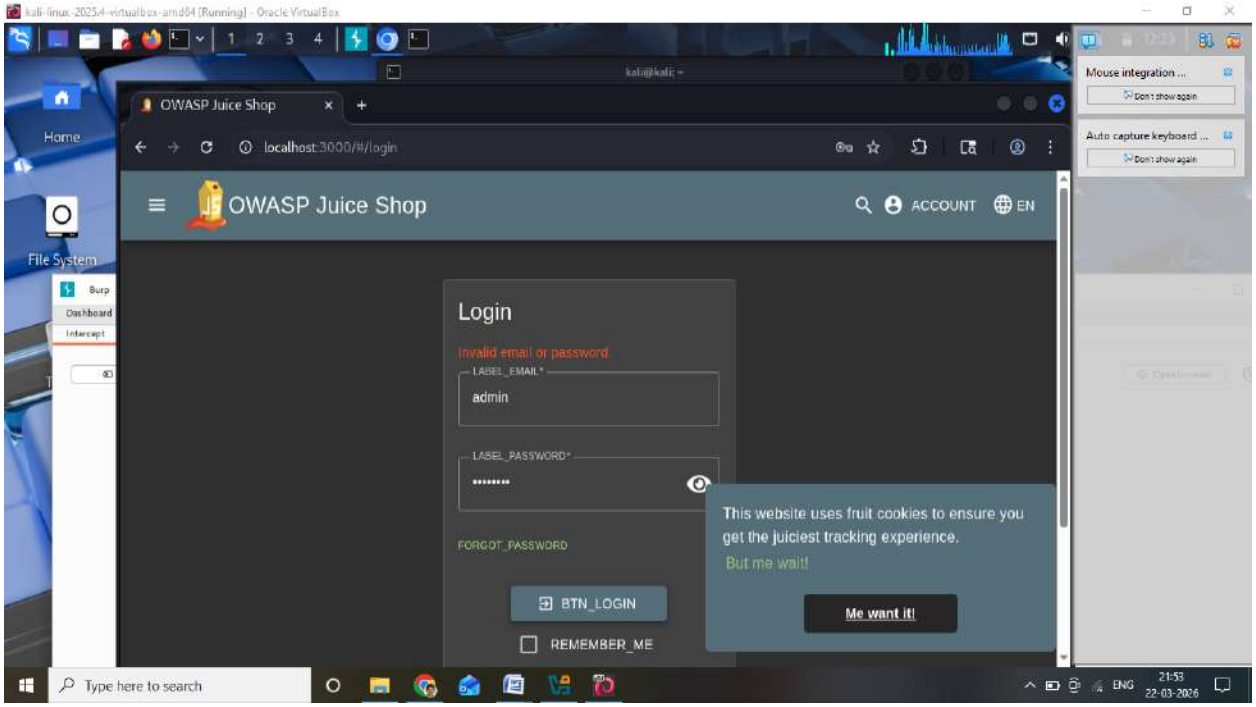
STEP 2: Open Burp Browser
In Burp:
Go to **Proxy tab**
Click **"Open Browser"**



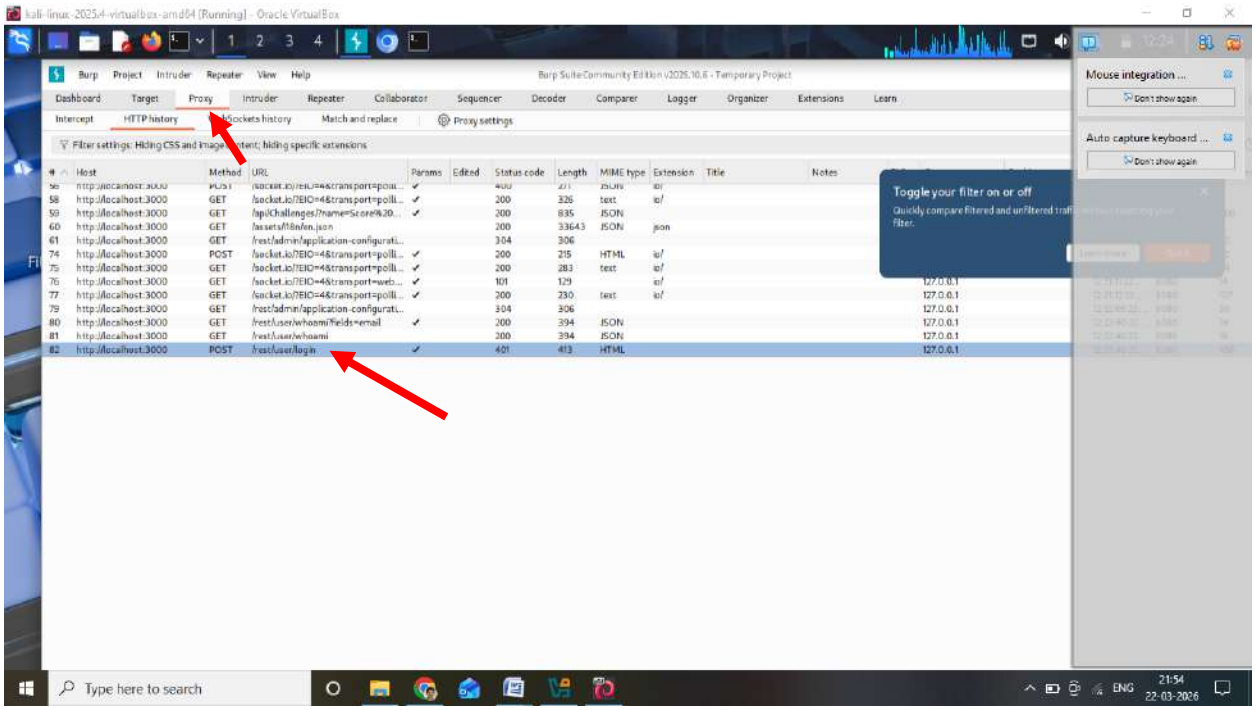
This browser is already configured with proxy (no manual setup needed)
STEP 3: Open Juice Shop Website
In Burp browser, open:
https://juice-shop.herokuapp.com/#/login
OR (local):
http://localhost:3000/#/login



STEP 4: Enter Dummy Login Details
Enter any random values:
Email: admin
Password: password
Click **Login**



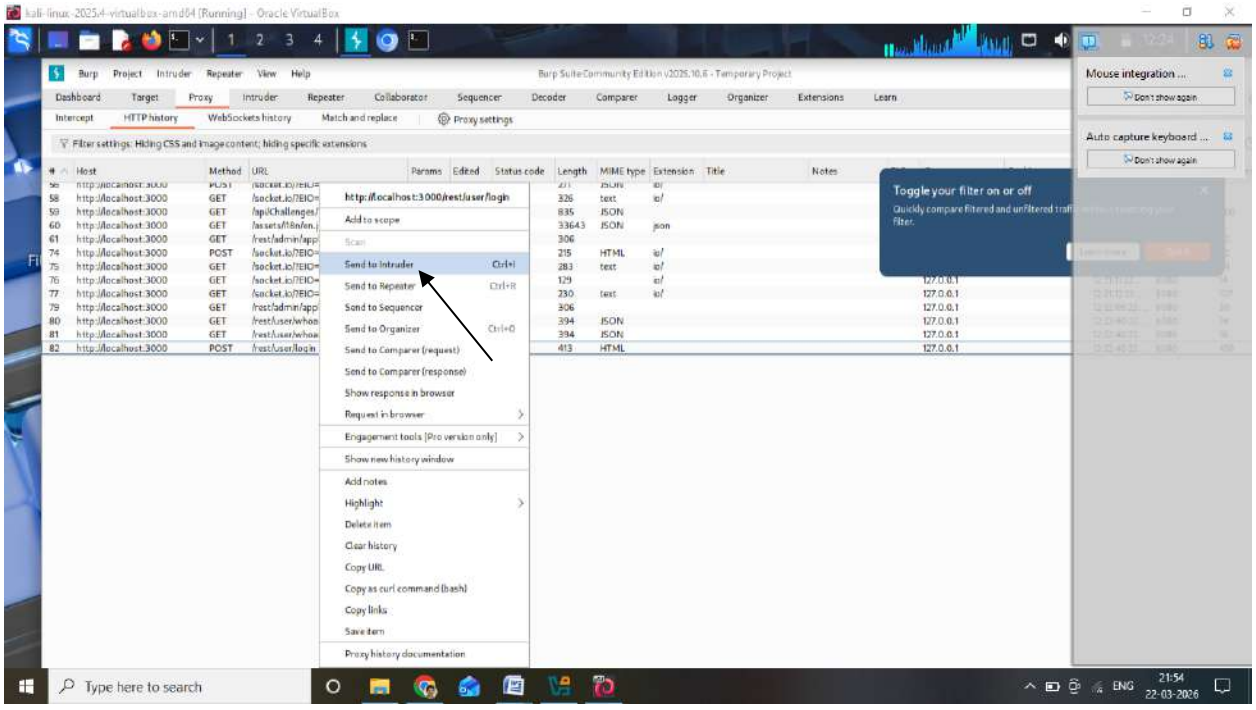
STEP 5: Capture Request in Burp
Now go to Burp:
Click **HTTP History**
Look for:
/rest/user/login
This is the login request



STEP 6: Send Request to Intruder

- Right-click on /rest/user/login
- Click:

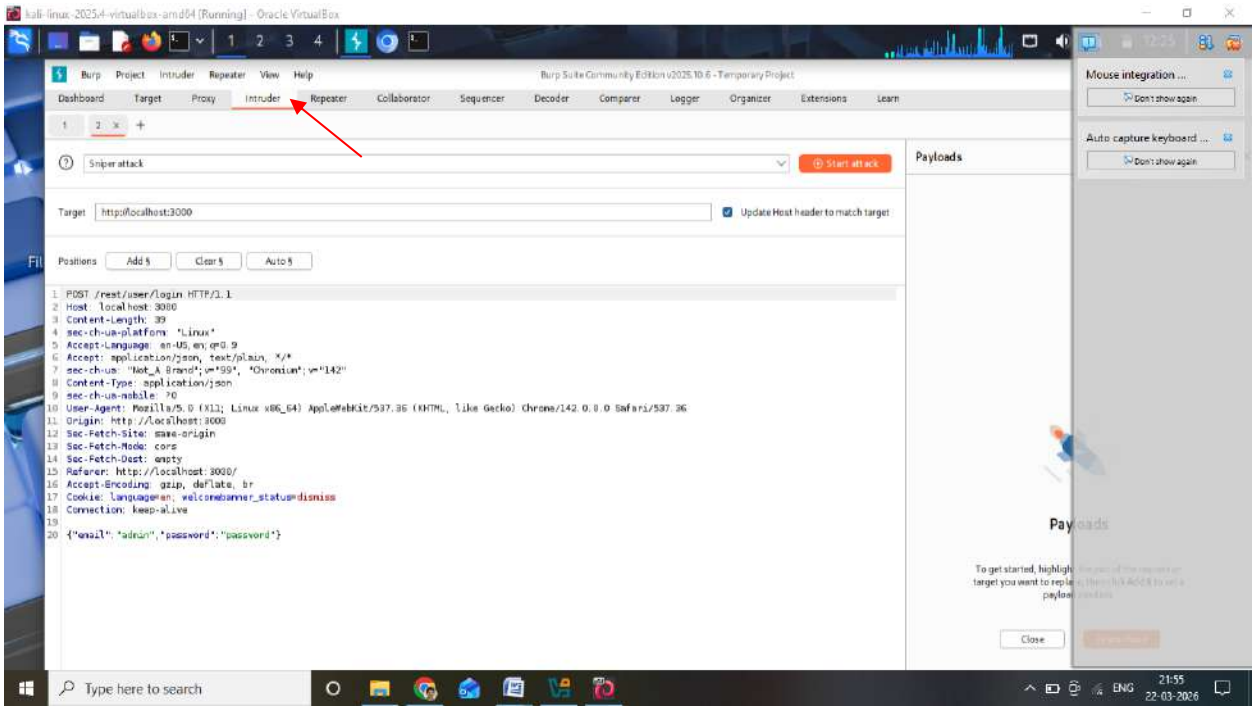
Send to Intruder



STEP 7: Configure Intruder Attack

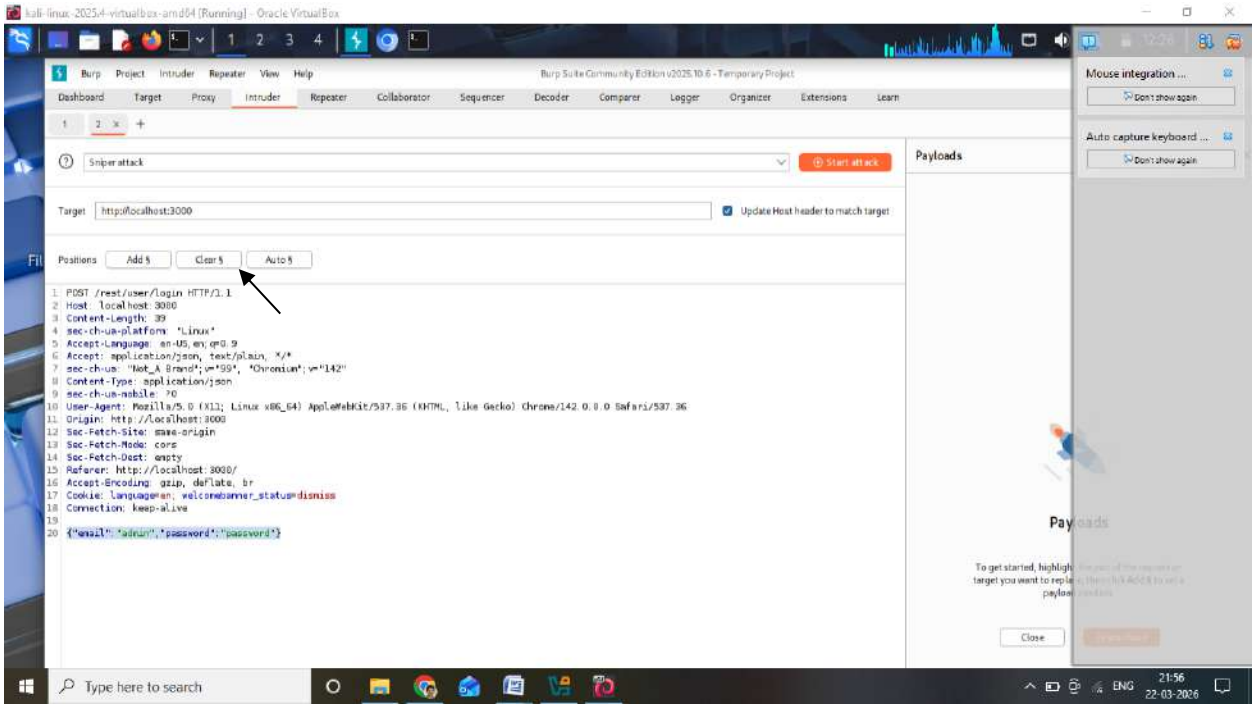
Go to:

Intruder tab



1. Positions Tab

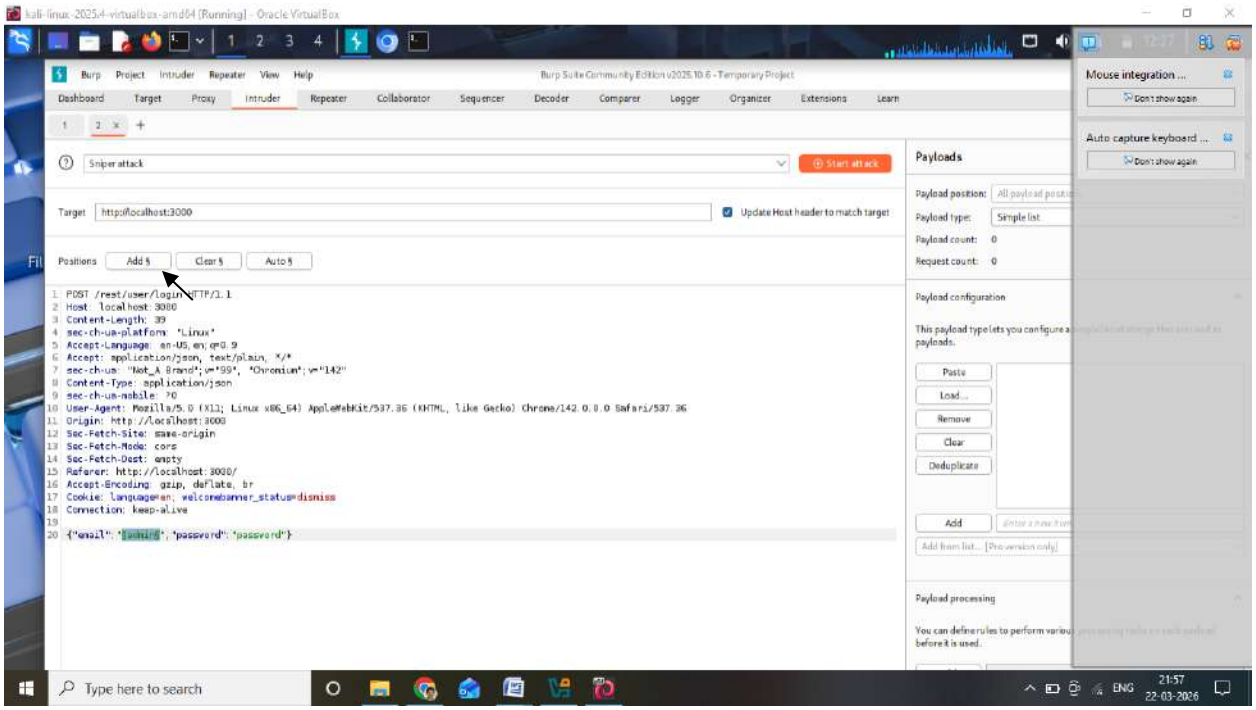
- Click **Clear §**
- Select email value:



Example:
"email":"admin"

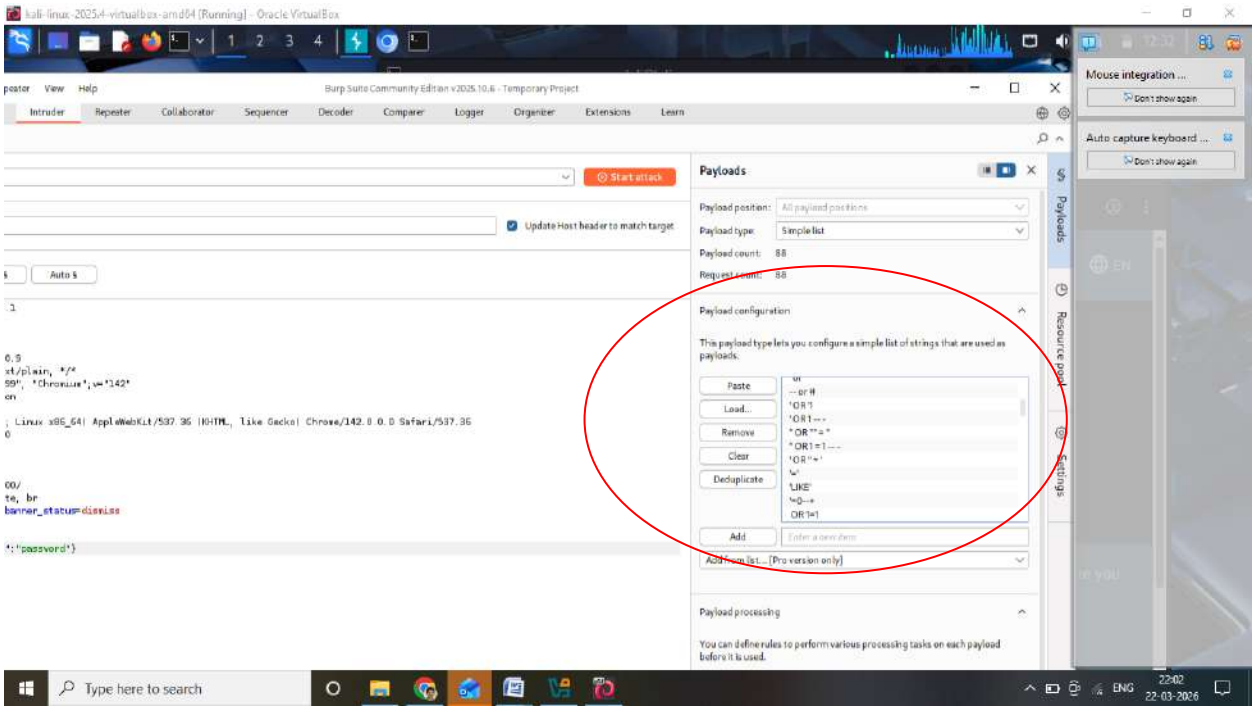
- Click **Add §**

Now it becomes:
"email":"§admin§"



STEP 8: Add SQL Payloads
Go to:
Payloads tab
Open the website: https://github.com/Ninja-Yubaraj/SQL-Injection-Payloads-List
copy and paste from: **Generic SQL Injection Payloads** Section
After unchecking URL encoder

Now paste SQL injection payloads.
Example Payload List:
' OR 1=1 --
' OR '1'='1
admin' --
' OR 1=1#
' OR ''='
These payloads will be tested automatically

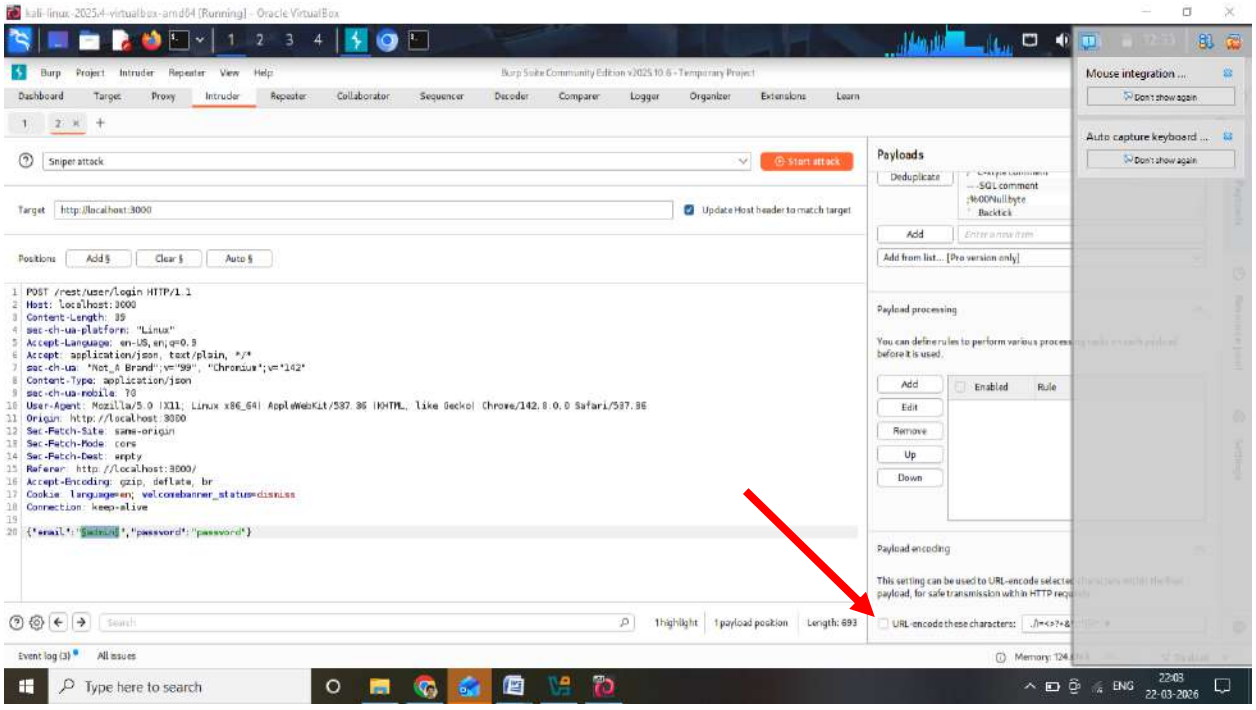


STEP 9: Disable URL Encoding

Very important:

- Uncheck:

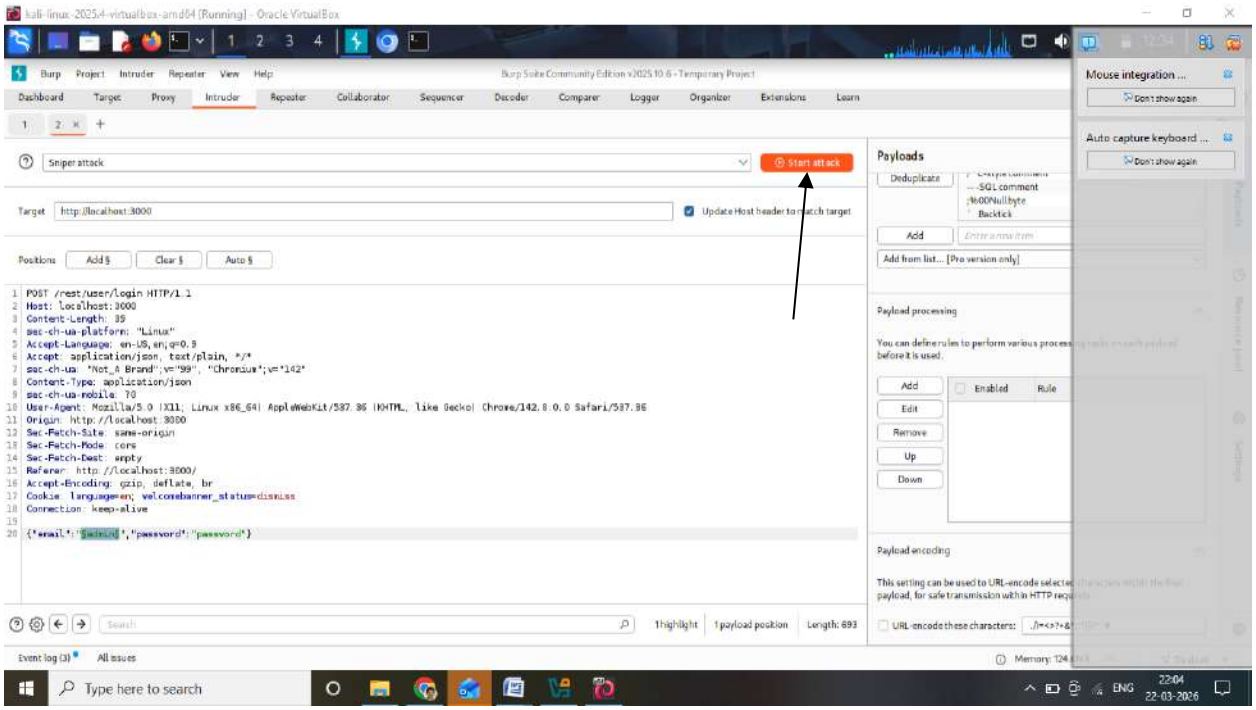
  URL Encode these characters



STEP 10: Start Attack

Click:

Start Attack



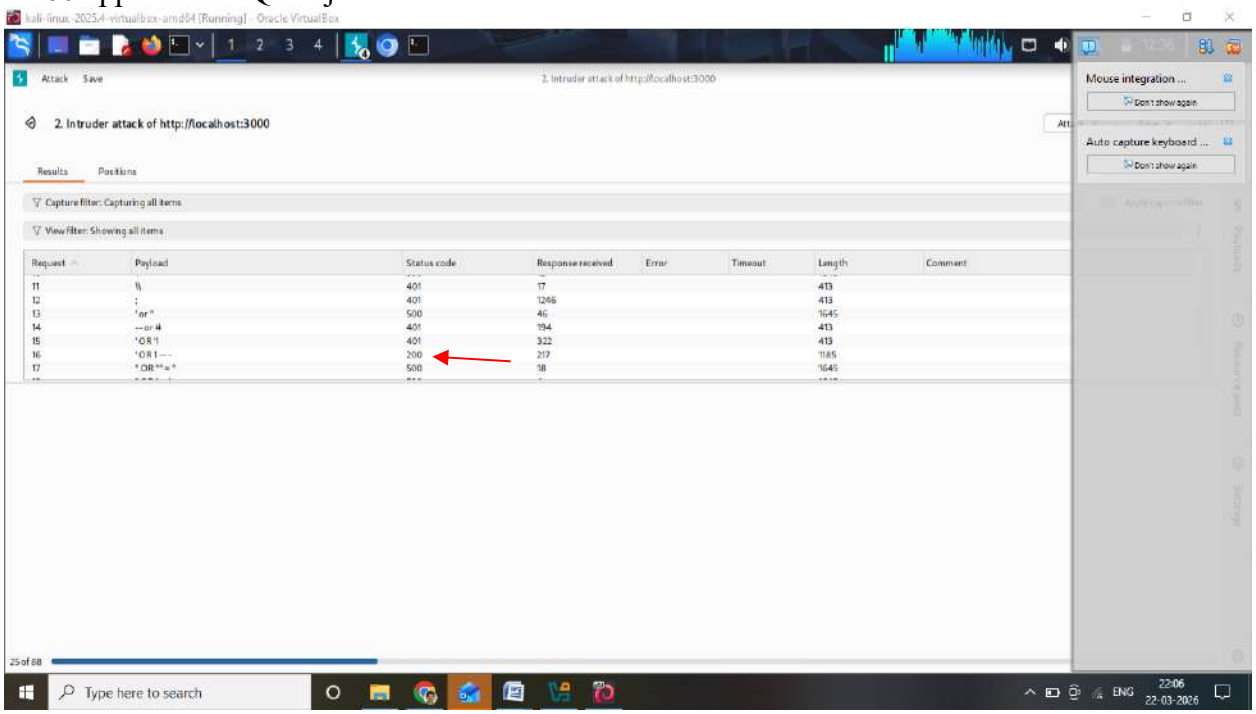
STEP 11: Analyze Results

You will see many responses.

Focus on:

- **Status Code = 200** ✓(Success)
- Other codes = Failed

If 200 appears → SQL Injection worked



STEP 12: Extract Authentication Token

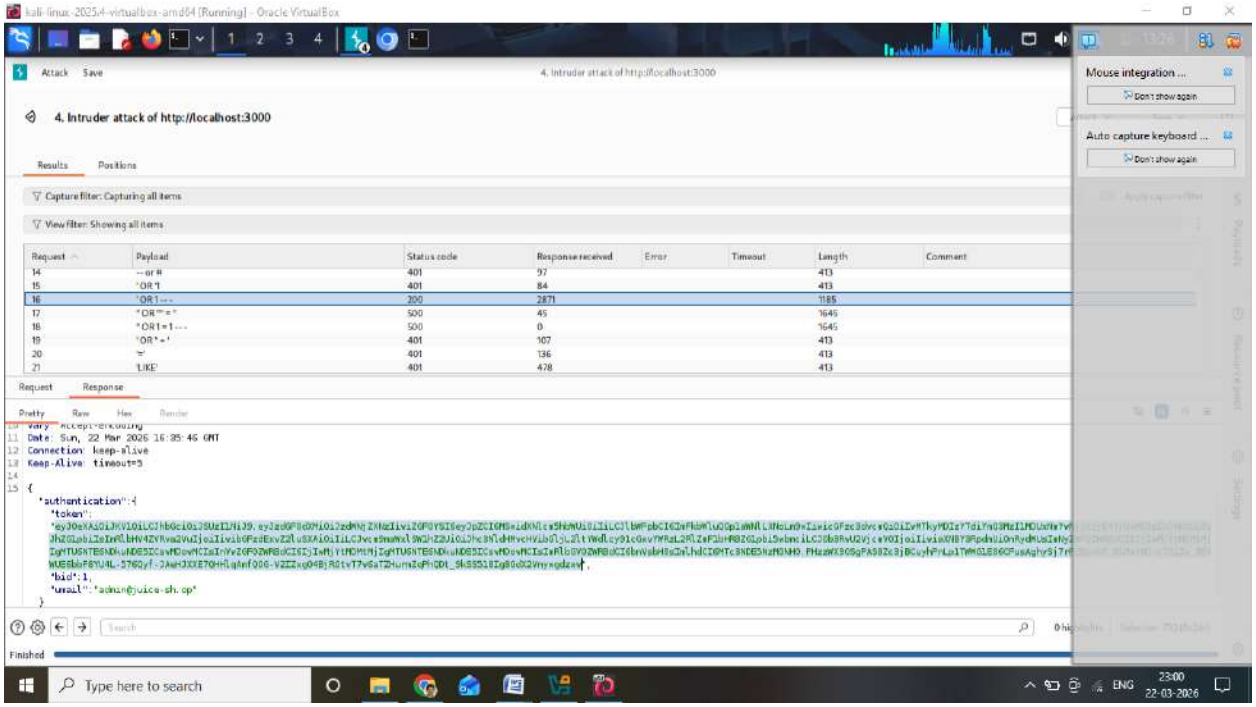
Click the **200 response**

Go to:

Response tab

Look for:

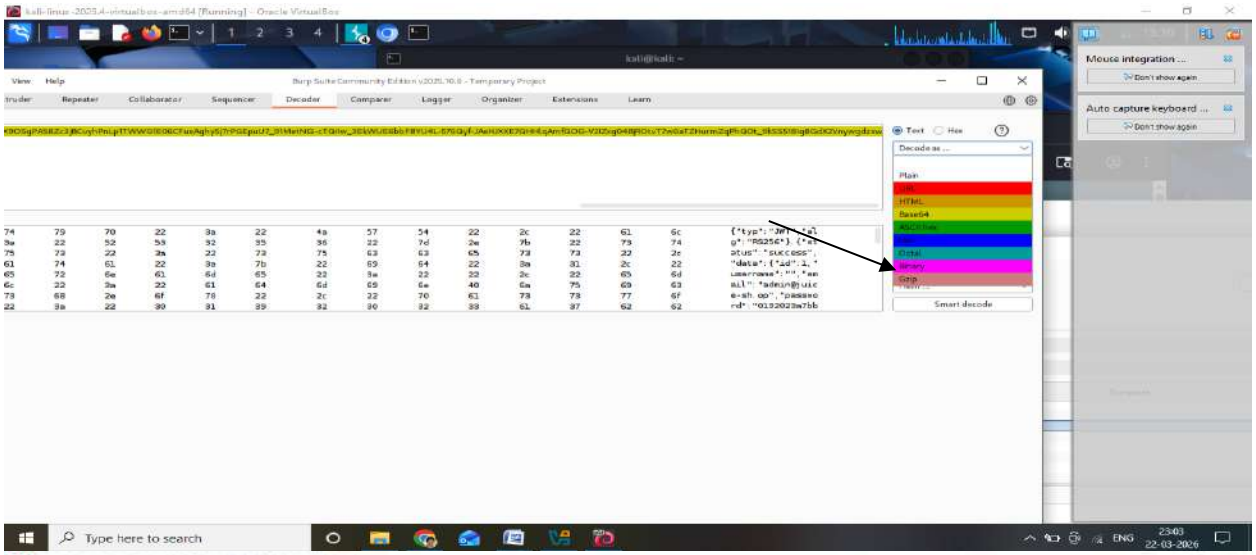
authentication token
Copy it
“eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTkyMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIiLCJwcm9maWxlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjYtMDMtMjIgMTU6NTE6NDkuNDE5ICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjYtMDMtMjIgMTU6NTE6NDkuNDE5ICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTc3NDE5NzM0NH0.PHzzWX9OSgPAS8Zc3jBCuyhPnLp1TWWG1E06CFusAghySj7rPGEpuU7_91MetNQ-cTQiIw_3EkWUE6bbF8YU4L-576Qyf-JAeHJXXE7QHHlqAmfQOG-V2IZxg04BjROtvT7w6aTZHurmZqPhQDt_9kSS518Ig8GdX2Vnywgdzxw”

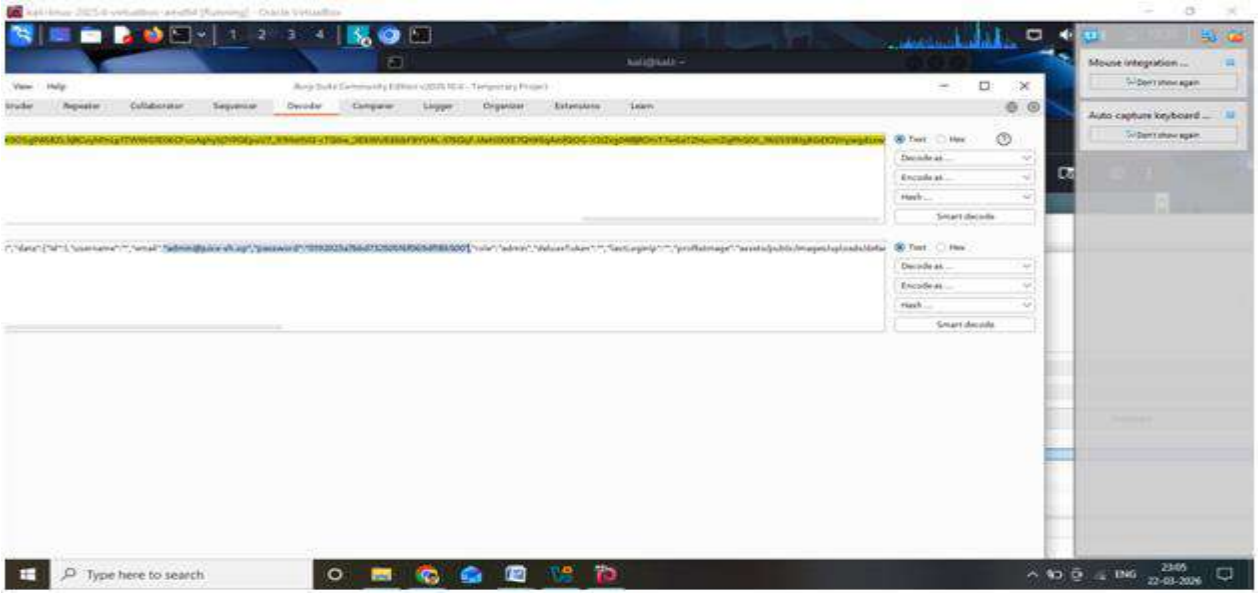


STEP 13: Decode Token
Go to:
Decoder tab

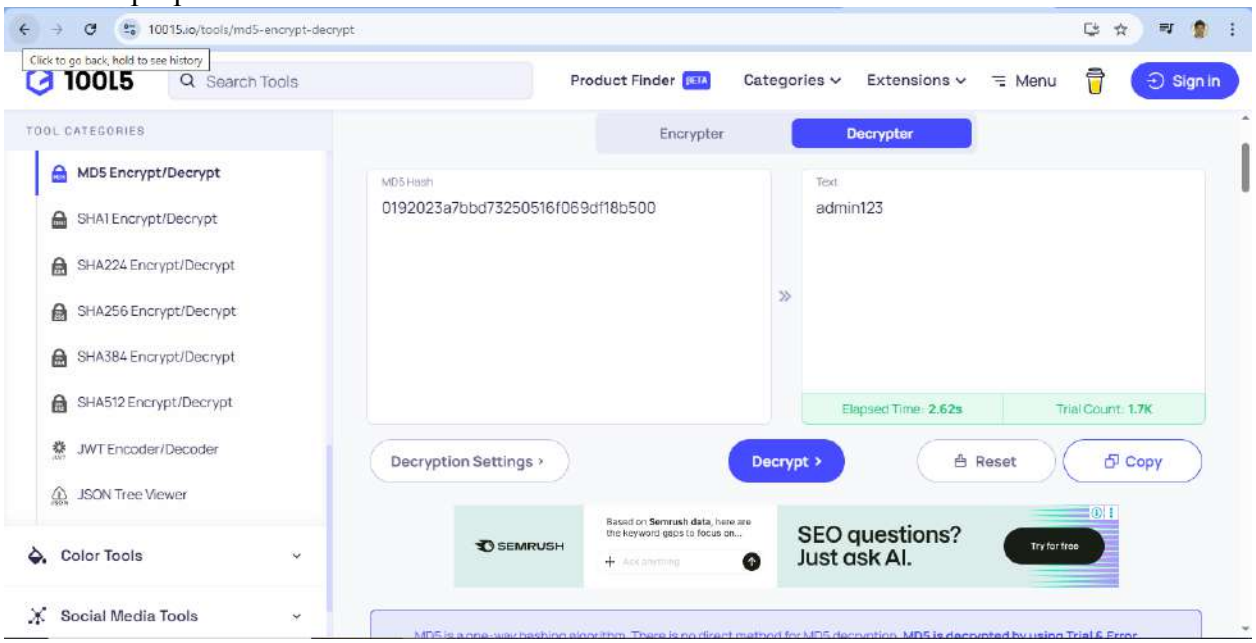
1. Paste token
2. Select:

Decode as → Base64





Password purpose: Use online MD5 decoder



STEP 14: Get Credentials

After decoding, you will see:

email: admin@juice-sh.op
password: admin123

These are extracted credentials

STEP 15: Login Using Extracted Data

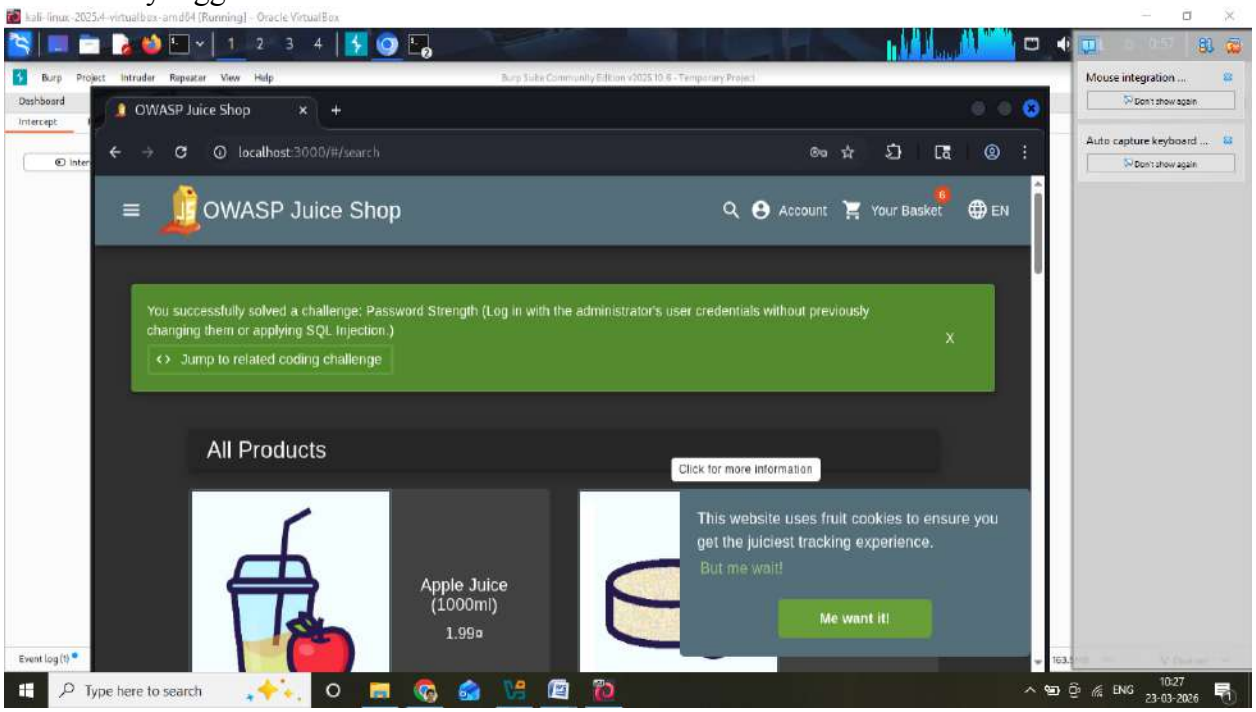
Go back to Juice Shop:

Enter:

- Email
- Password (decoded one)

Click Login

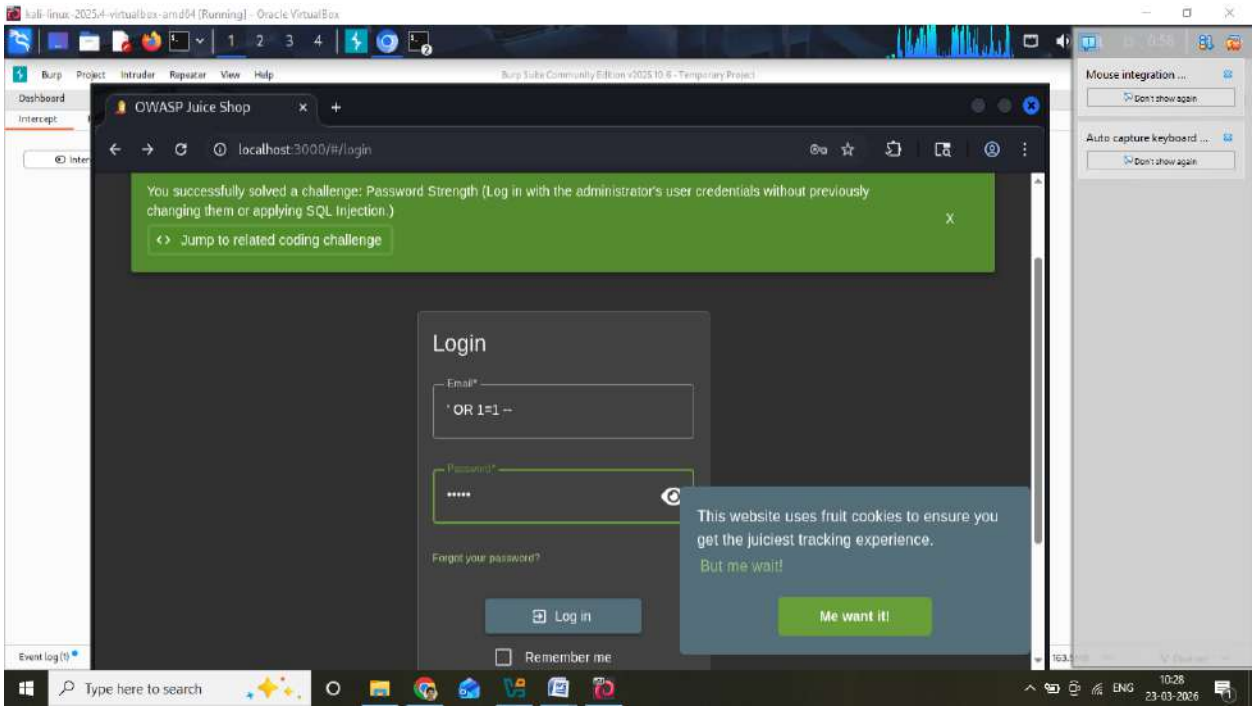
Successfully logged in



TYPES OF SQL INJECTION USED HERE

1. Authentication Bypass

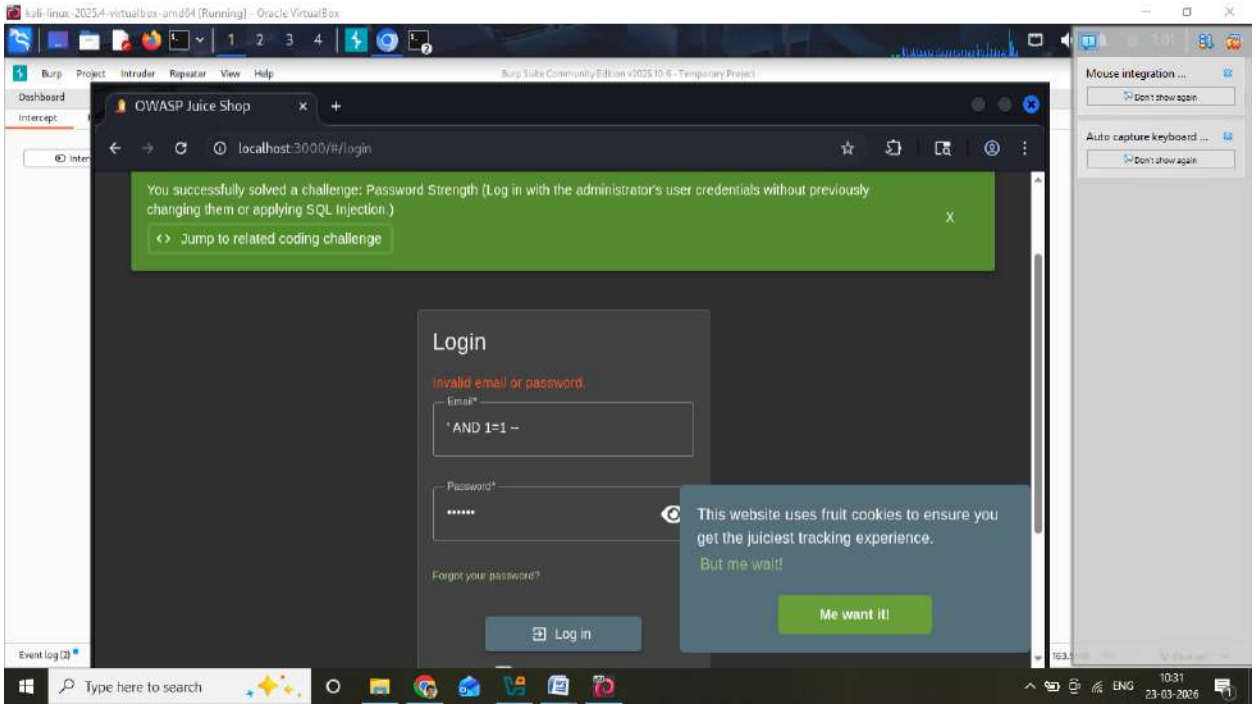
' OR 1=1 --



2. Boolean-Based (Blind)

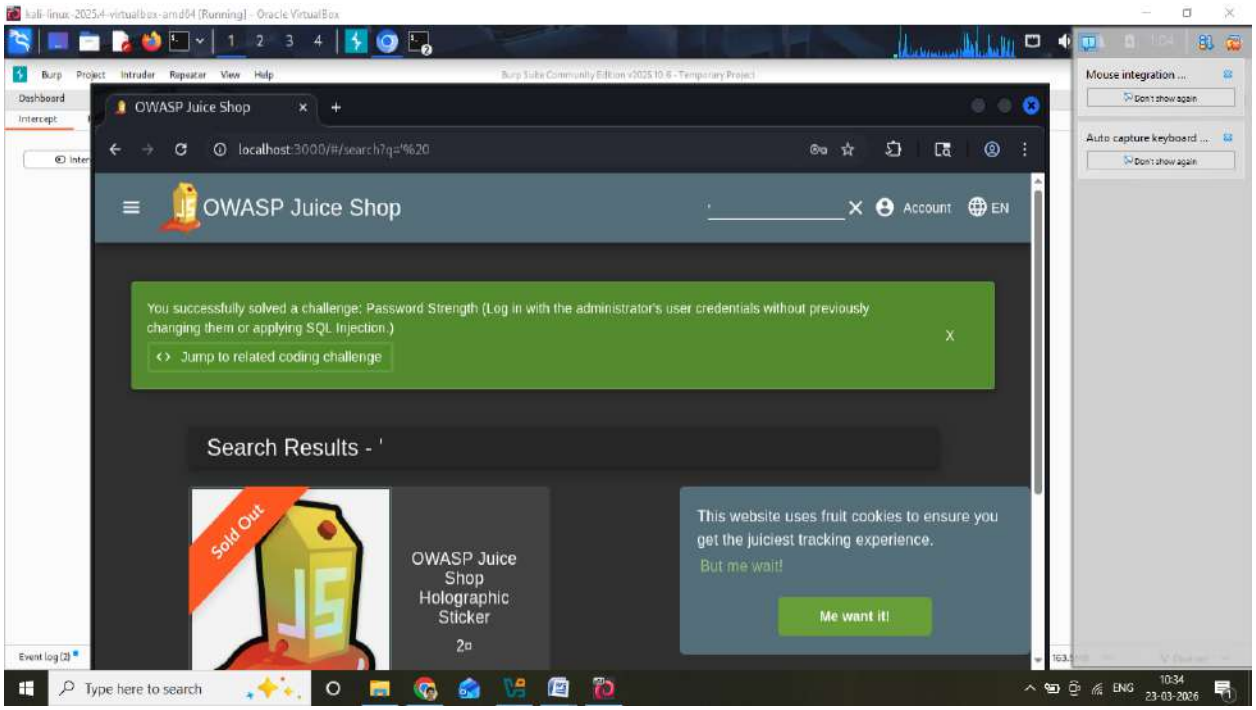
' AND 1=1 --

' AND 1=2 –



3. Error-Based

'



4. Automated Injection (Intruder)
✔ Testing multiple payloads automatically

RESULT
SQL Injection vulnerability successfully identified and exploited using Burp Suite.

Experiment 2: Cross-Site Scripting (XSS)

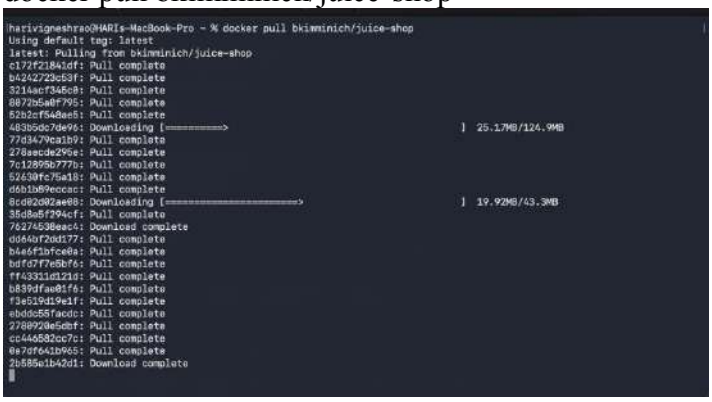
Tool: WebGoat / OWASP Juice Shop
Objective: Understand and execute reflected, stored, and DOM-based XSS attacks.
Tasks: Inject malicious scripts, steal cookies, and execute JavaScript on victim browsers.

OWASP Juice Shop is installed and running on http://localhost:3000

Pull the OWASP Juice Shop Docker Image

Open **Terminal / Command Prompt** and run:

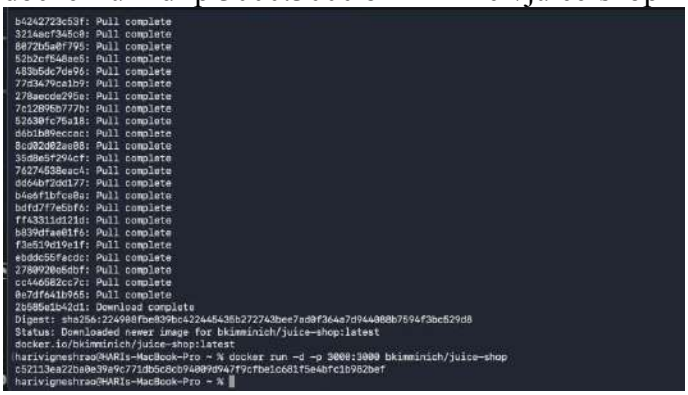
docker pull bkimminich/juice-shop



Run the Juice Shop Container

After the image is downloaded, run:

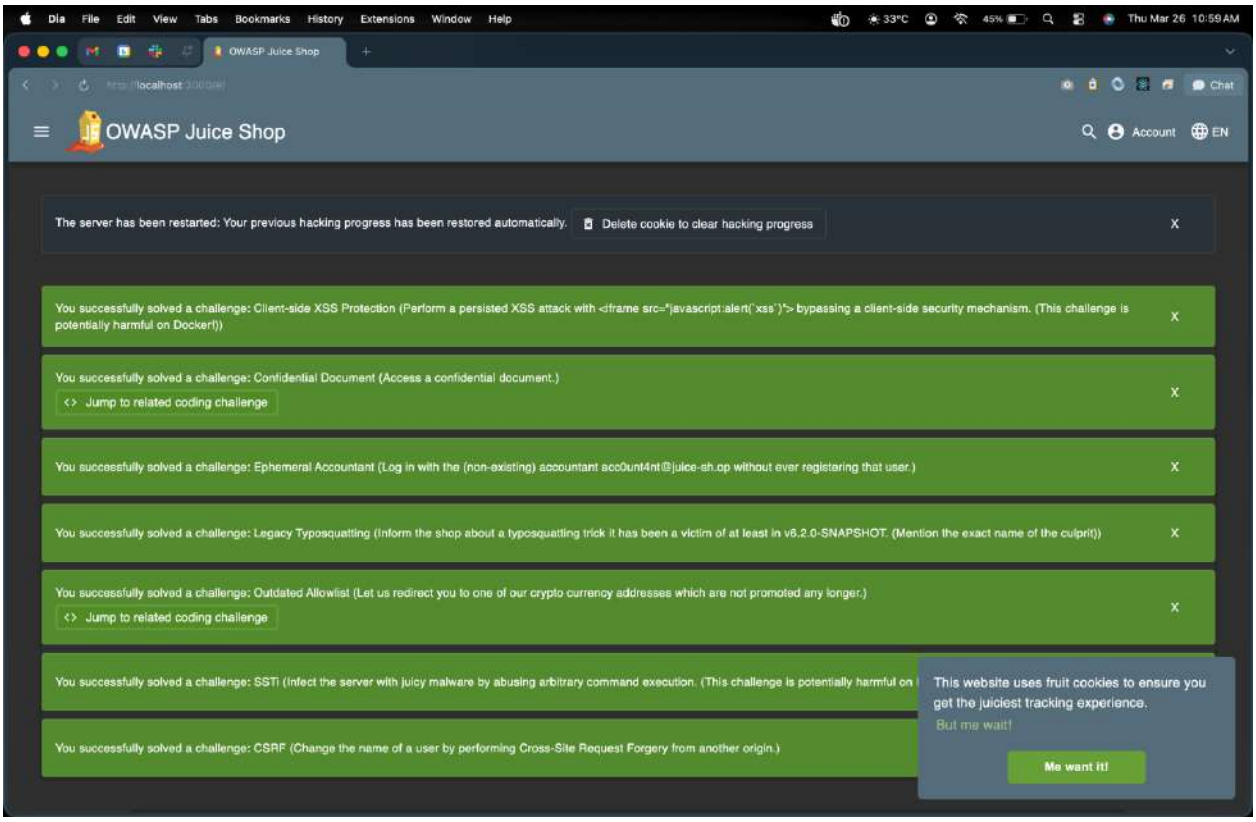
docker run -d -p 3000:3000 bkimminich/juice-shop



Access OWASP Juice Shop

Open your browser and navigate to:

http://localhost:3000

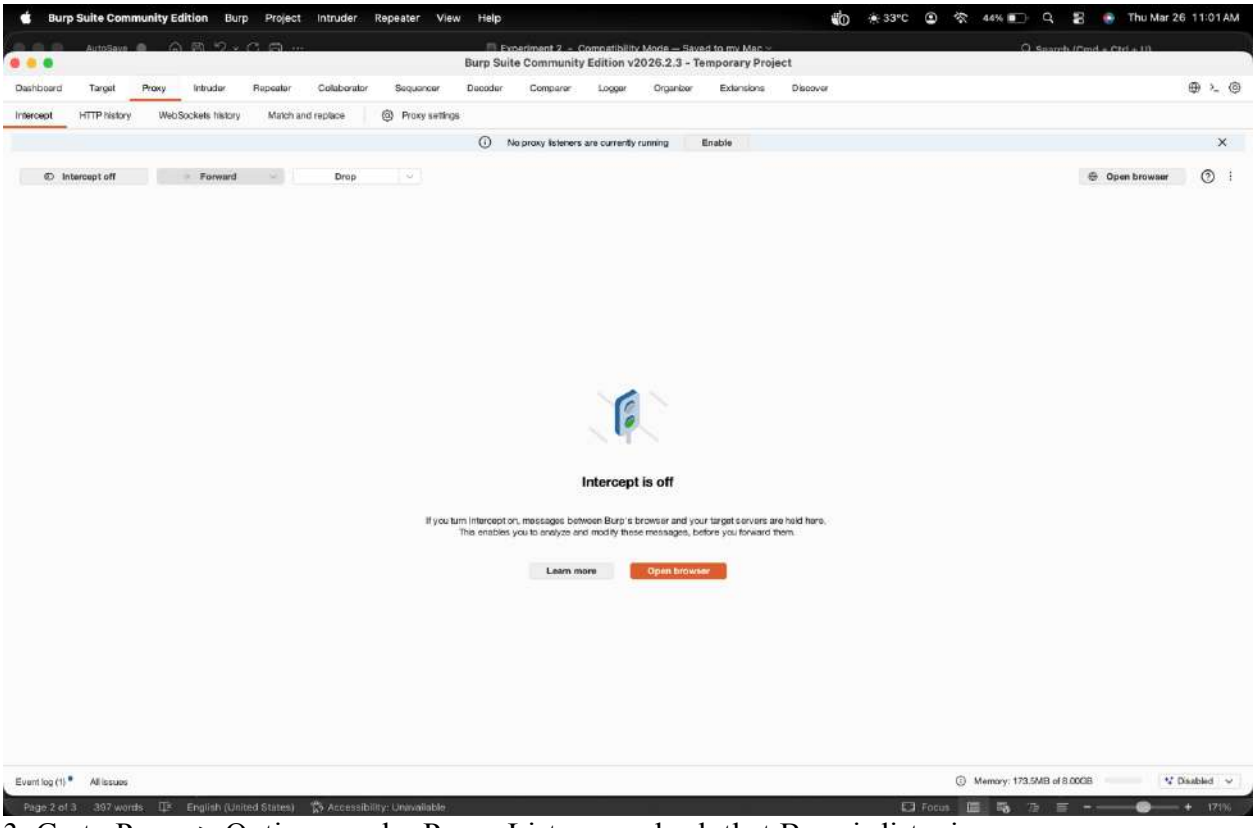


Burp Suite is installed and configured to intercept traffic

A modern web browser (Chrome/Firefox)

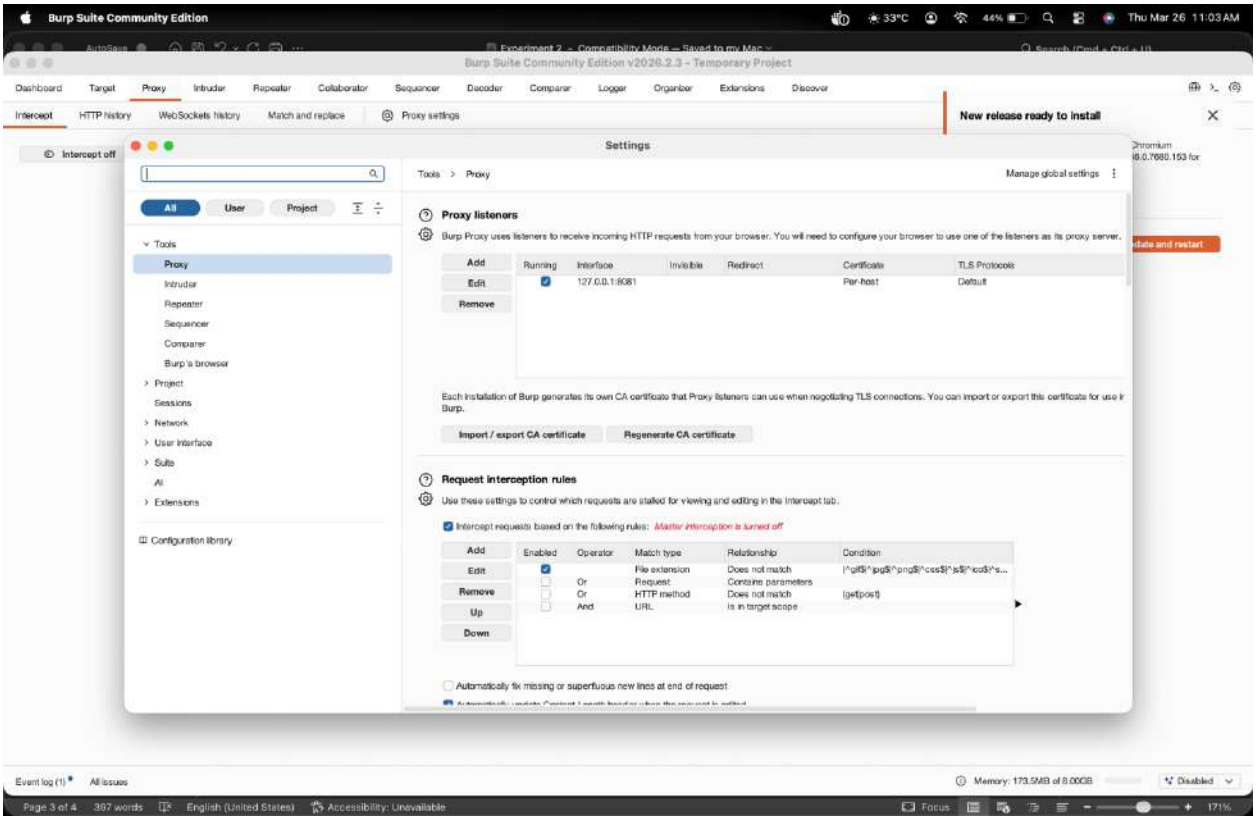
Step 1: Configure Burp Suite Proxy

1. Open Burp Suite and navigate to Proxy > Intercept.
2. Ensure Intercept is off.

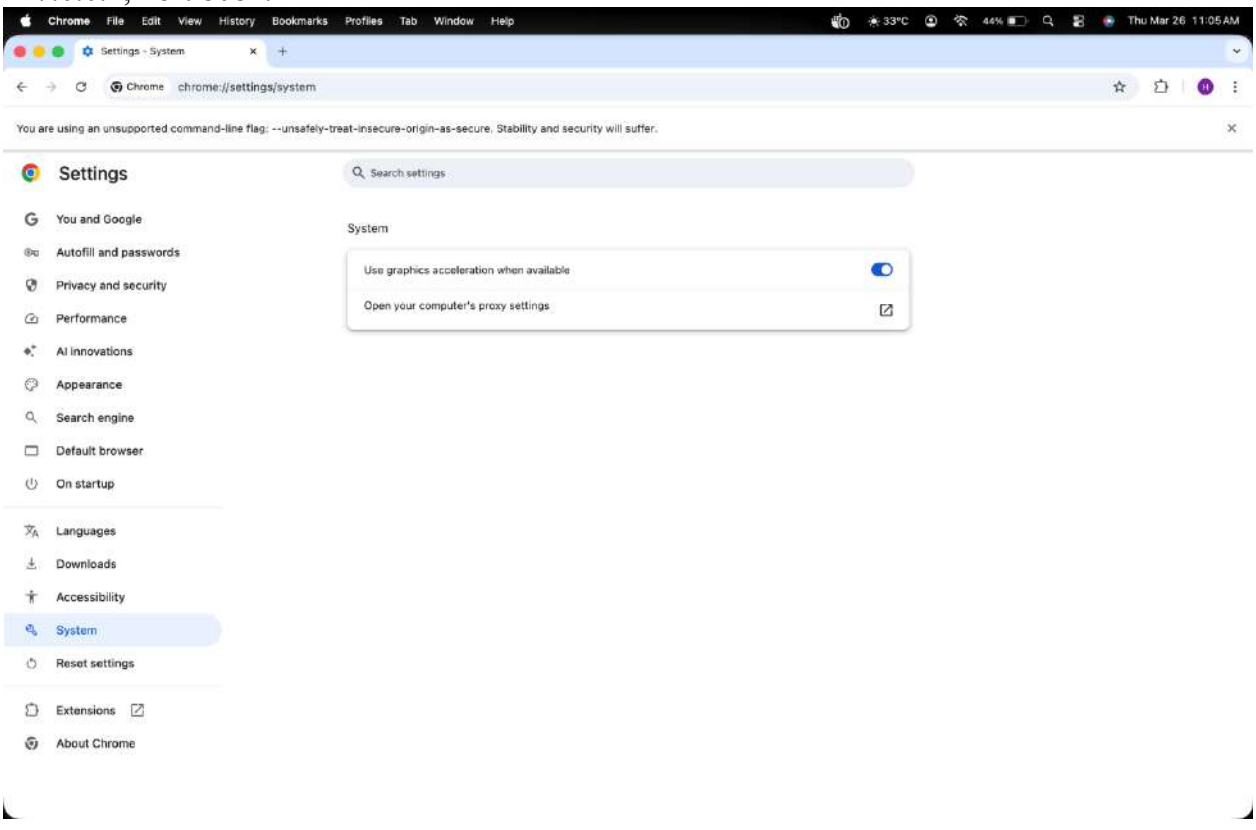


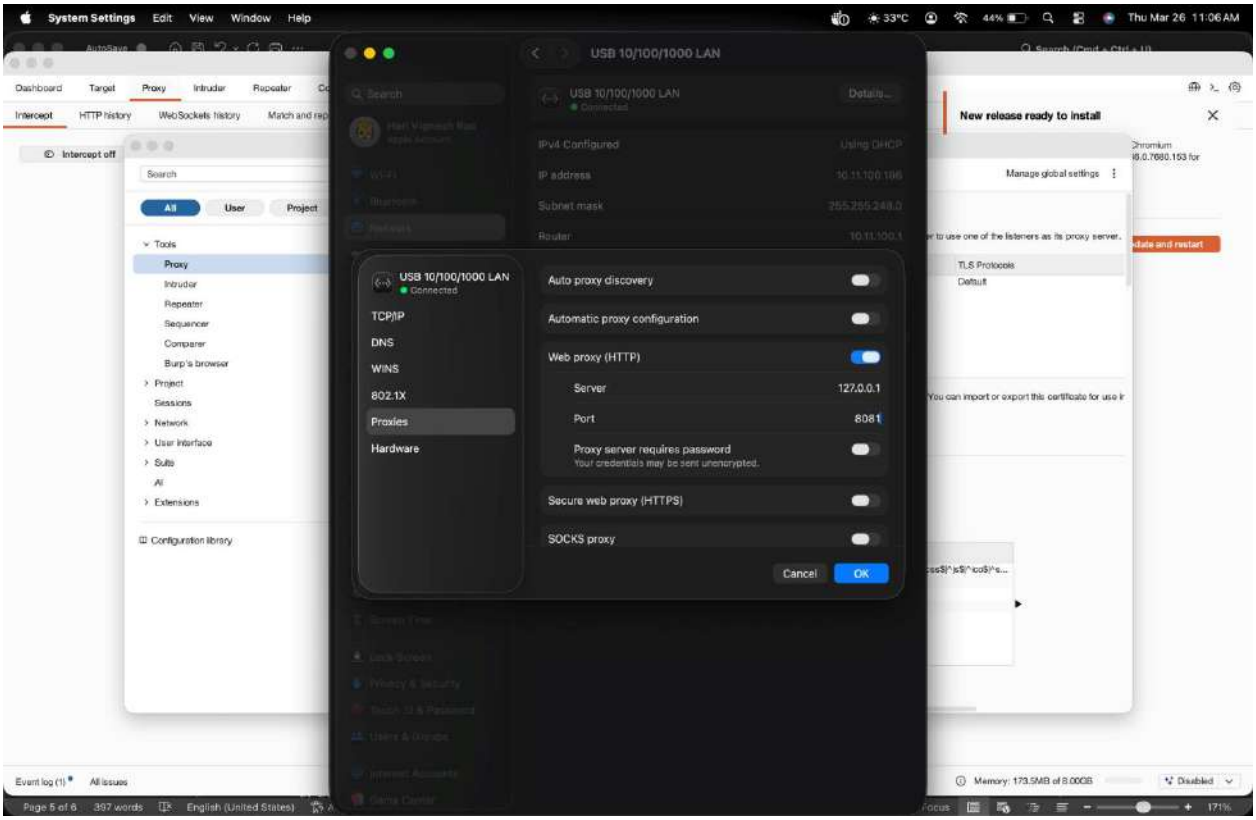
3. Go to Proxy > Options, under Proxy Listeners, check that Burp is listening on

127.0.0.1:8081.

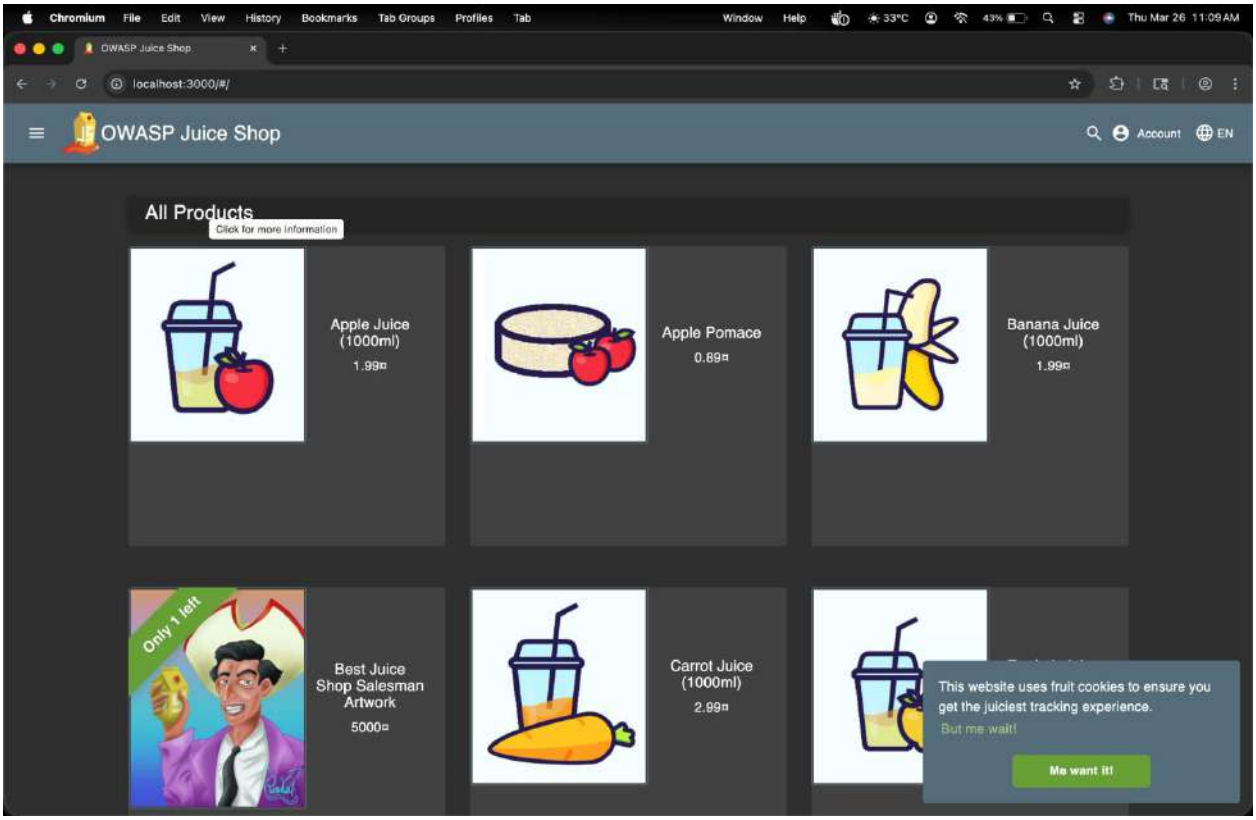


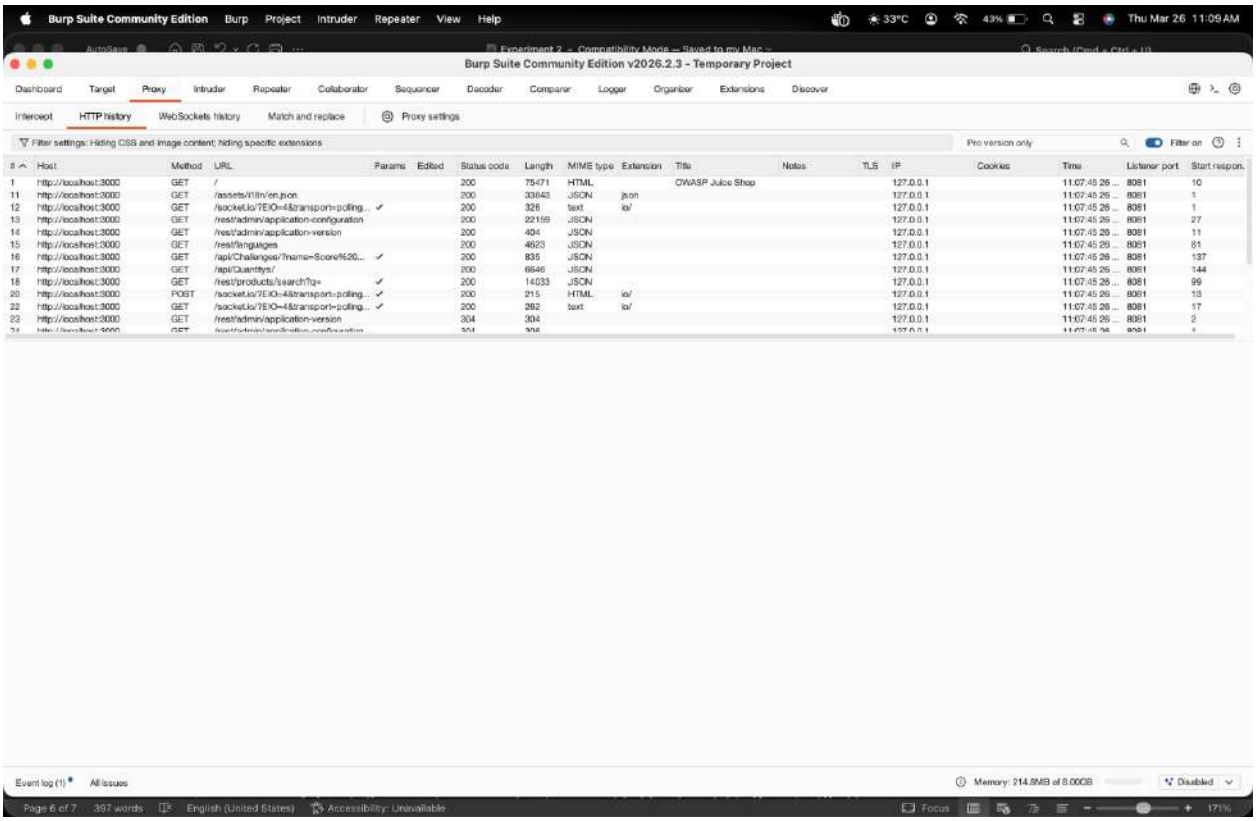
4. Open your browser and configure it to use Burp Suite as a proxy:
O Chrome: Settings > Proxy settings > Manual proxy > Set HTTP Proxy to 127.0.0.1, Port 8081.
O Firefox: Preferences > Network Settings > Manual proxy > Set HTTP Proxy to 127.0.0.1, Port 8081.





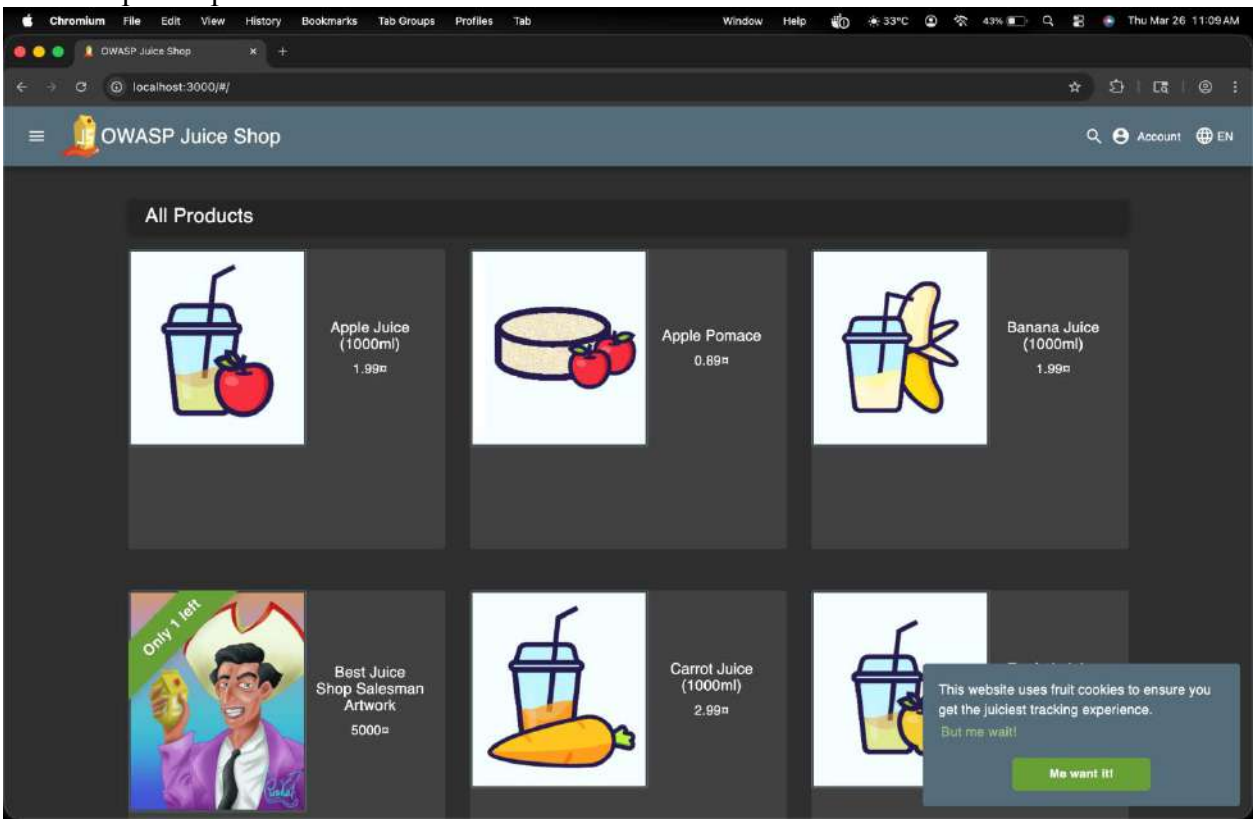
5. Verify by opening http://localhost:3000 in your browser while Burp Suite is running. Requests should appear in HTTP history.



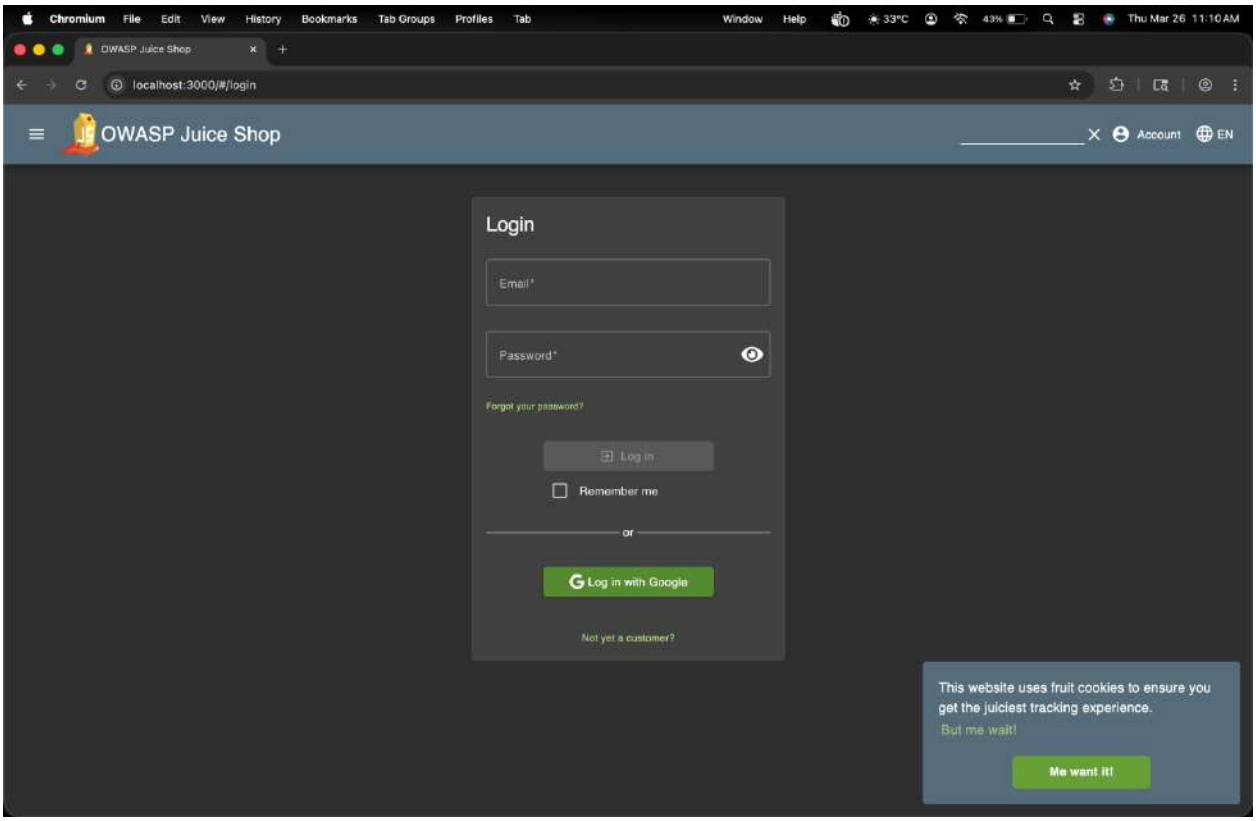


Step 2: Identify XSS Vulnerabilities

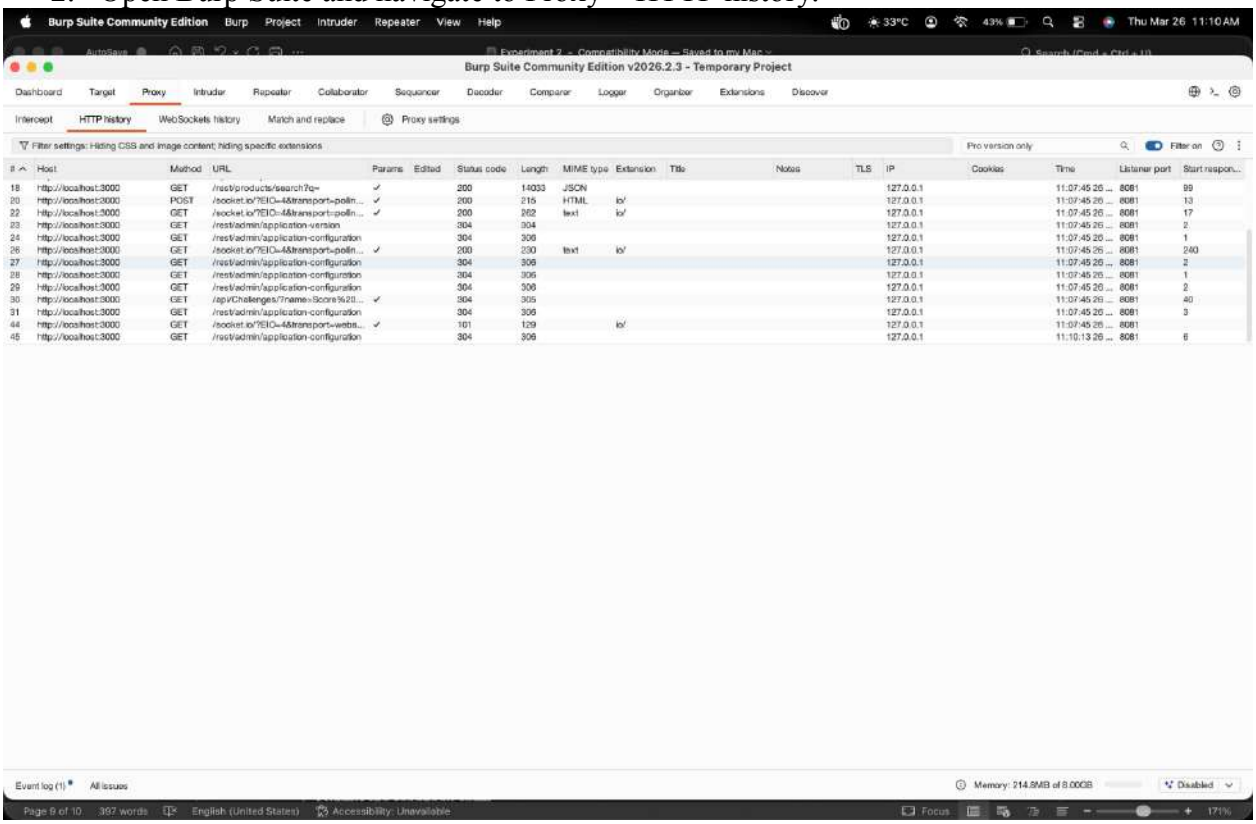
1. Open http://localhost:3000 in the browser.



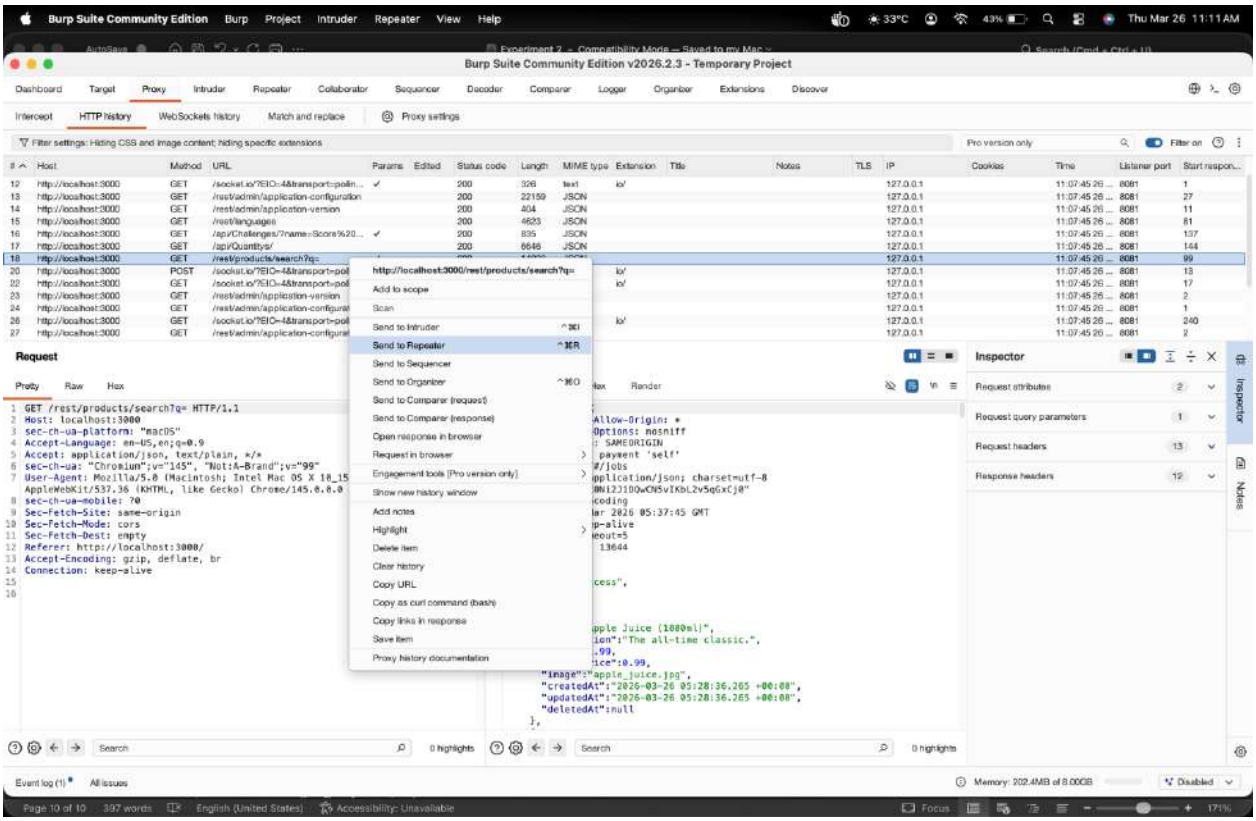
2. Browse through different sections of Juice Shop and look for input fields (such as search bars, login forms, and feedback forms).



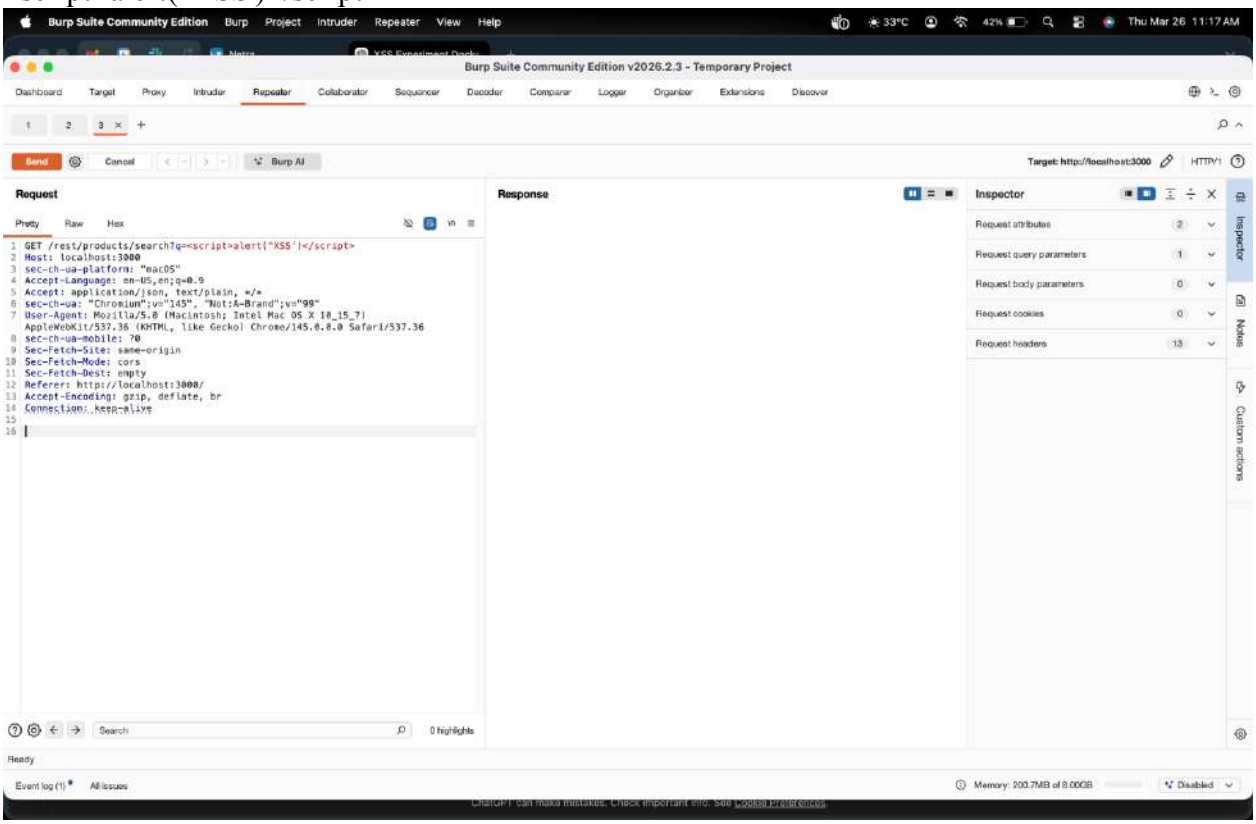
2. Open Burp Suite and navigate to Proxy > HTTP history.



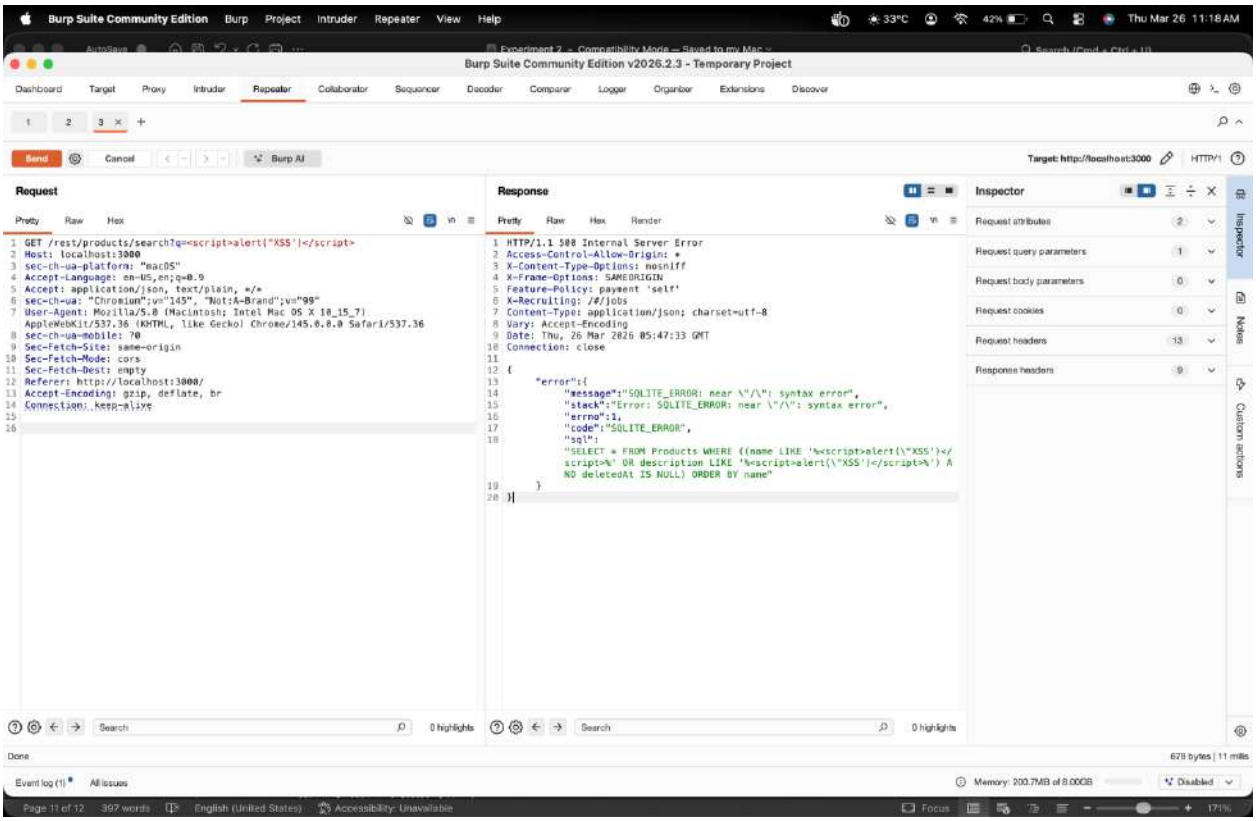
4. Locate the request corresponding to an input field and send it to Repeater (Right-click > Send to Repeater).



5. In the Repeater tab, experiment by modifying the input fields with JavaScript payloads such as:
<script>alert("XSS')</script>

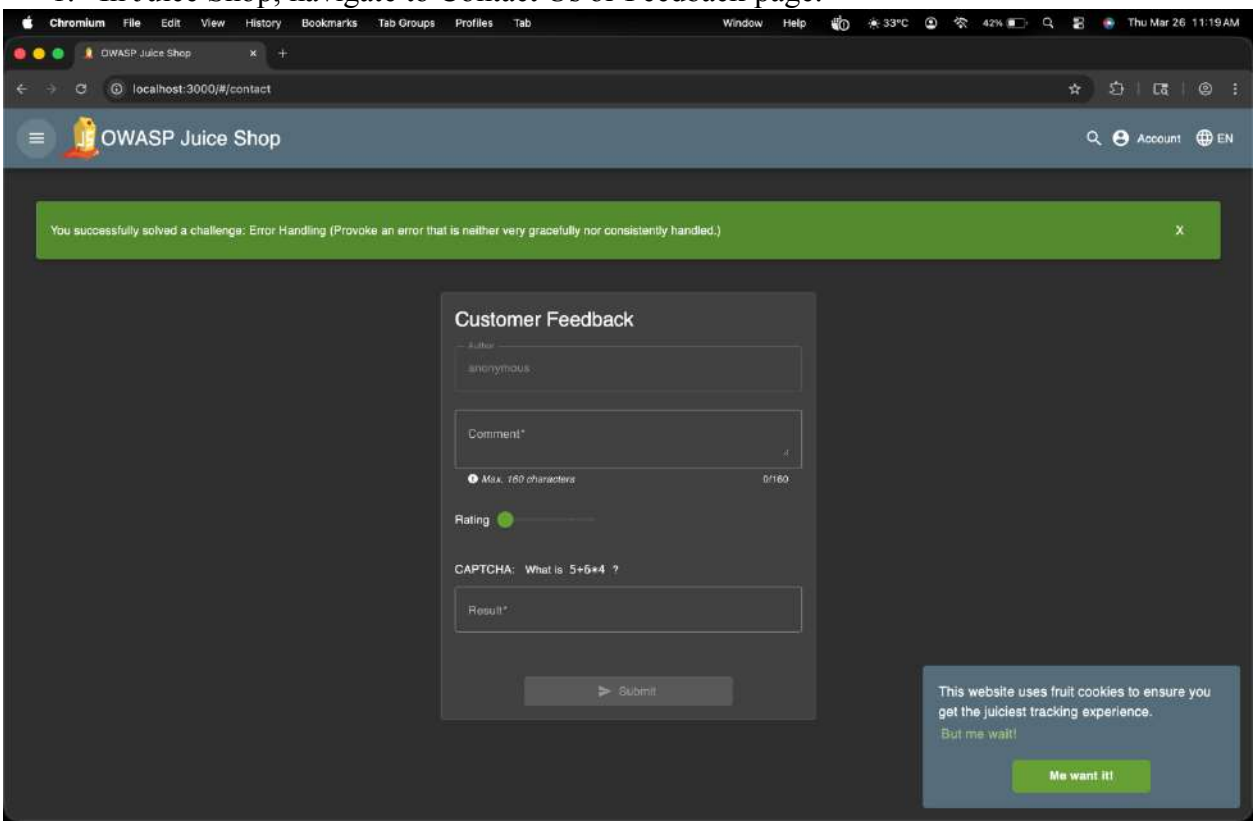


6. Click Send and observe the response for execution of the script.

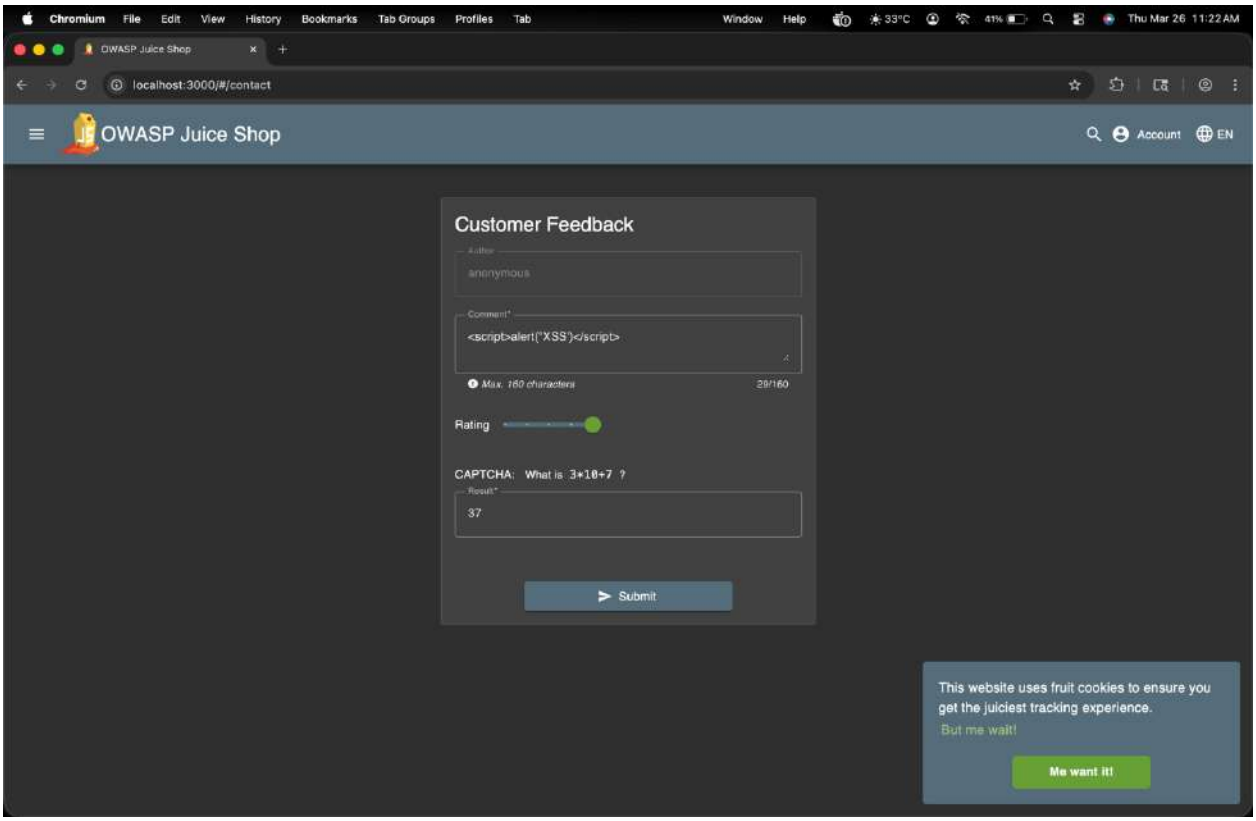


Step 3: Exploiting XSS in Feedback Form

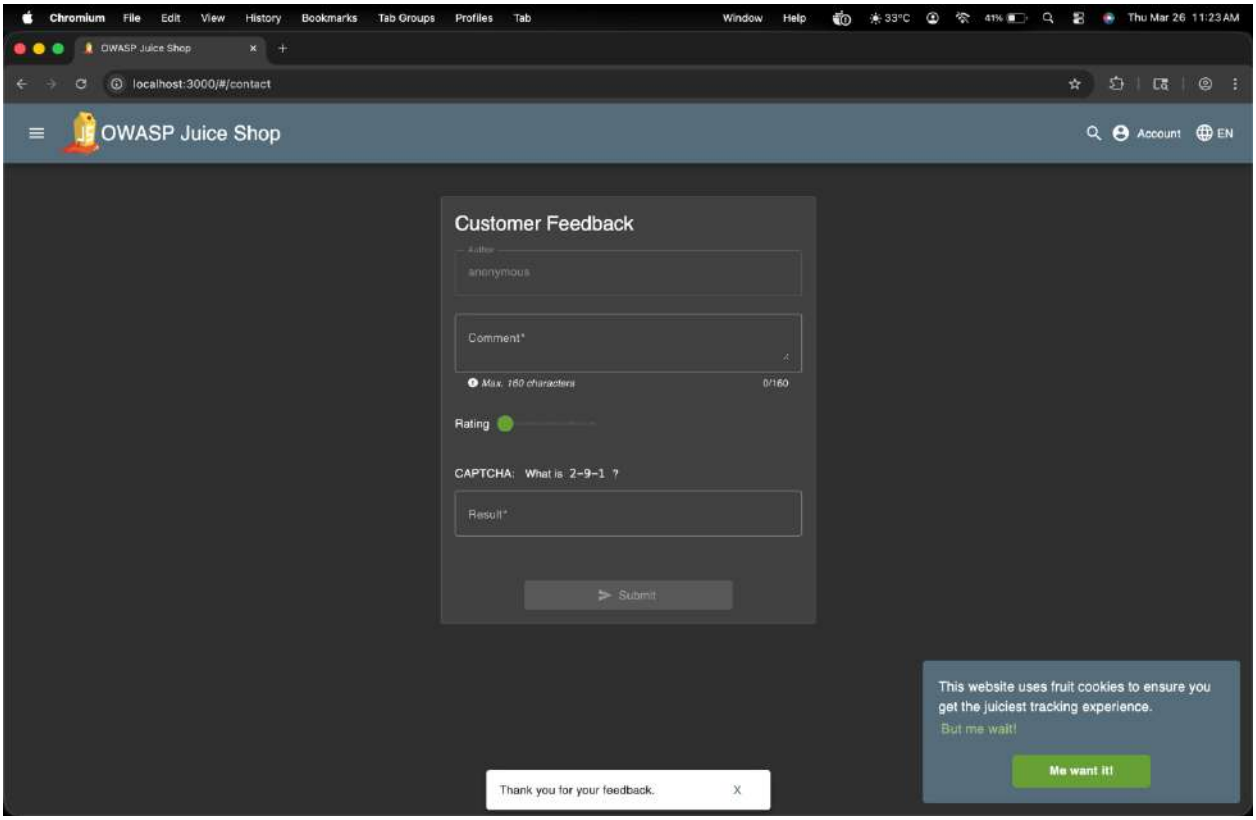
1. In Juice Shop, navigate to Contact Us or Feedback page.



2. Enter the following payload in the comment field: <script>alert("XSS')</script>



3. Submit the feedback form.

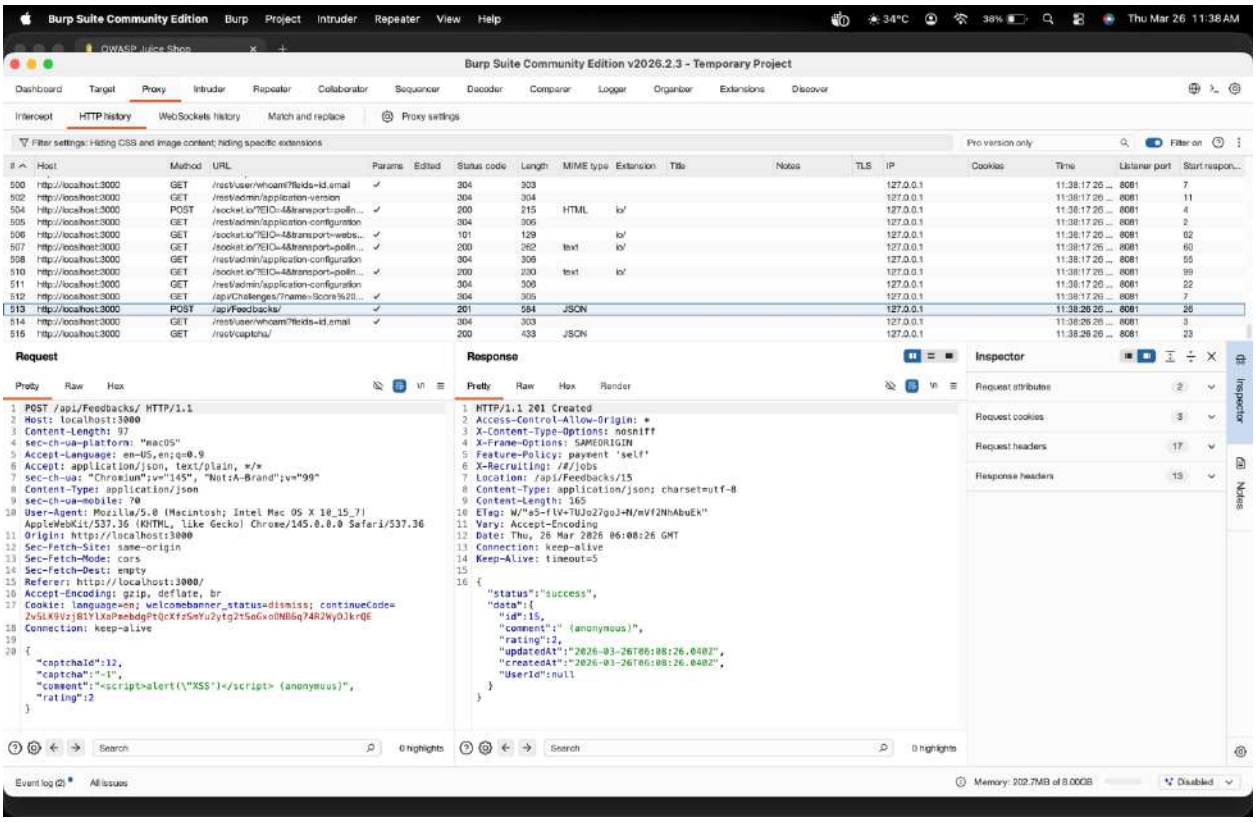


4. If the site is vulnerable, an alert box will pop up.
5. Confirm the XSS attack by checking:

O The browser console (F12> Console).

O The HTTP history tab in Burp Suite.

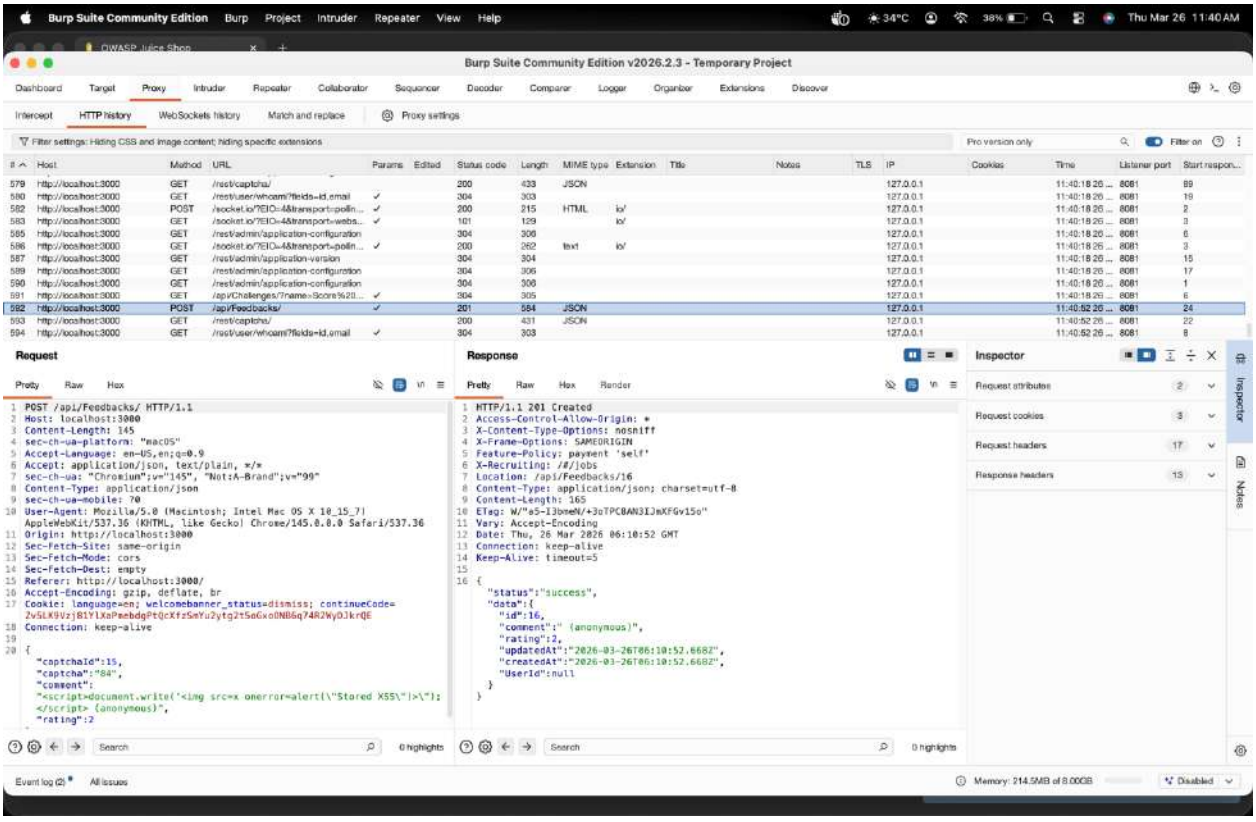
Inspecting the request and response in Burp Suite.

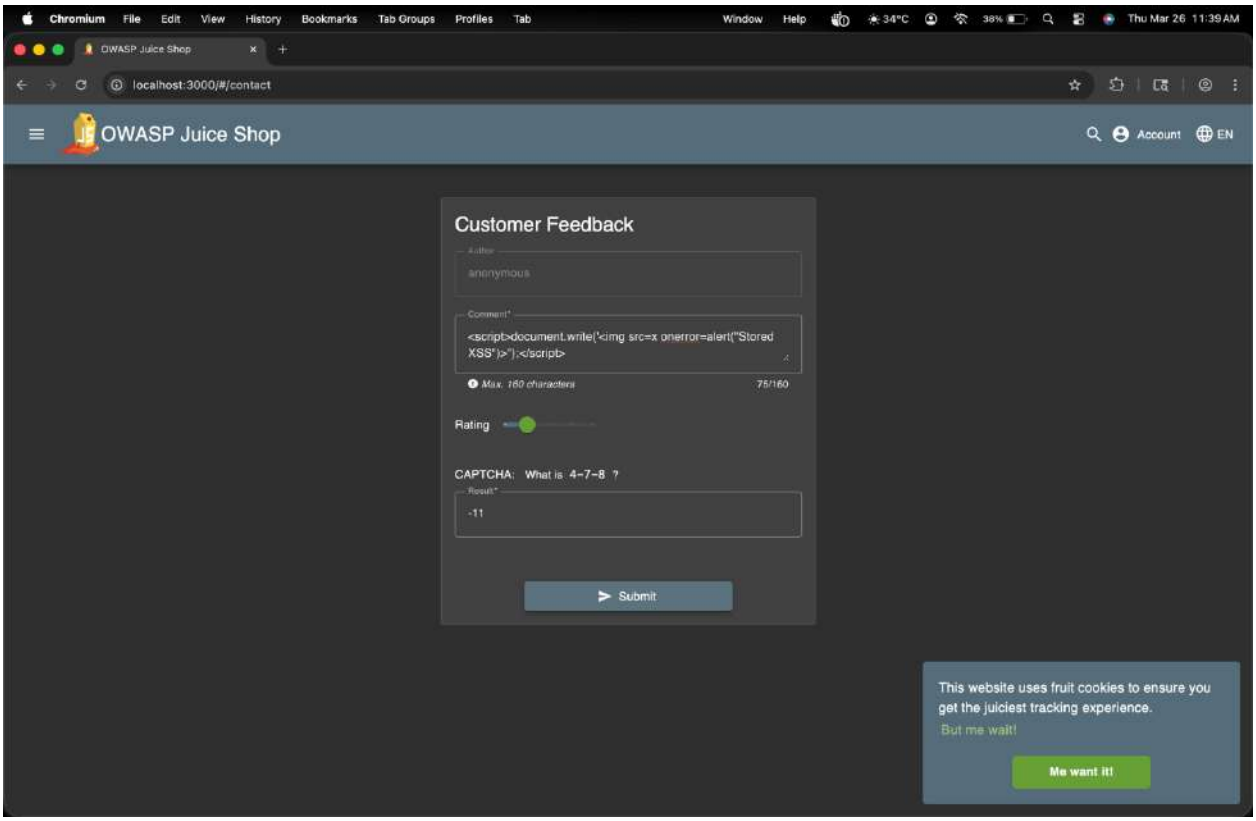


Step 4: Stored XSS Attack

1. Locate a form where the input is stored and displayed later (e.g., user profile, reviews, or feedback section).
2. Enter the payload:

<script>document.write('<img src=x onerror=alert("Stored XSS")>');</script>





3. Submit the form.
4. Reload the page where the stored input is displayed.

Experiment 3: Broken Authentication and Session Management

Tool: Juice Shop / Burp Suite
Objective: Exploit weak authentication mechanisms and hijack sessions.
Tasks: Capture and modify session cookies, brute force login credentials, and exploit session fixation.

AIM

To identify and analyze **authentication vulnerabilities** and **session management flaws** in a web application using Docker and Burp Suite.

REQUIREMENTS

- Kali Linux
- Docker
- Burp Suite
- Web Browser (Chrome/Firefox)
- Internet connection

THEORY

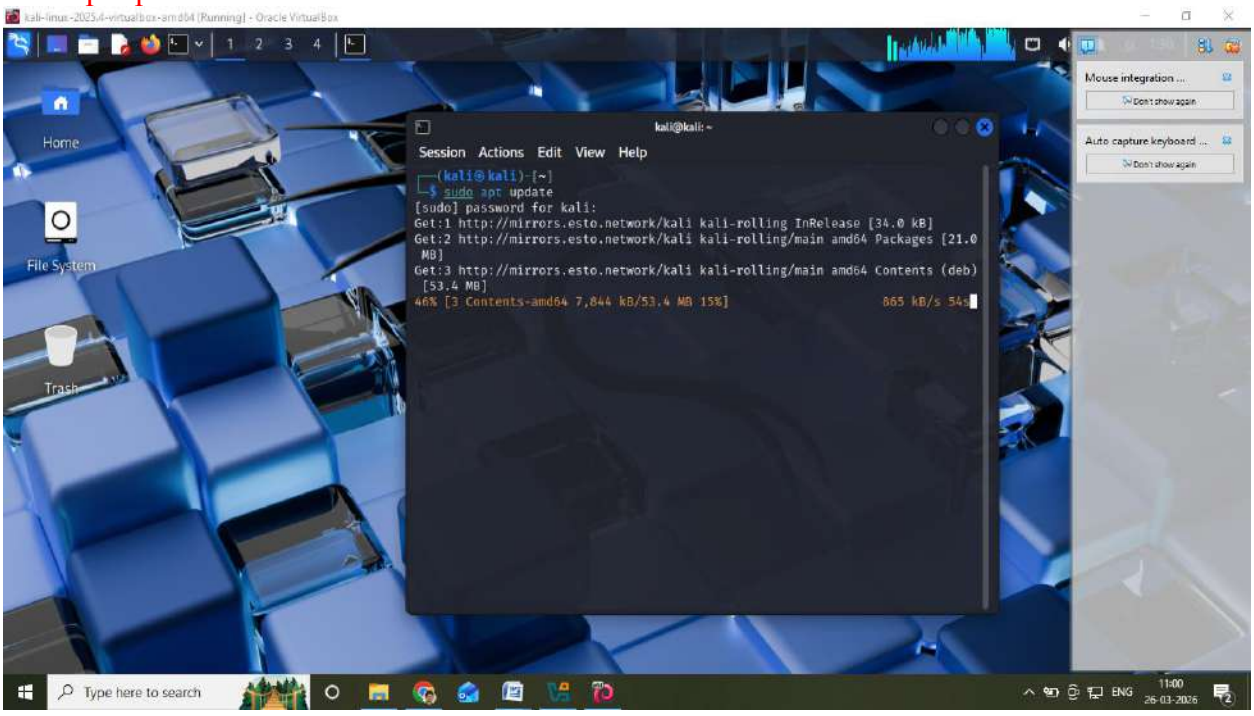
Authentication verifies user identity (login system).
Session Management maintains user state using session IDs or tokens.
Common vulnerabilities:

- SQL Injection
- Weak passwords
- Brute force attacks
- Session hijacking
- Session fixation
- Improper logout

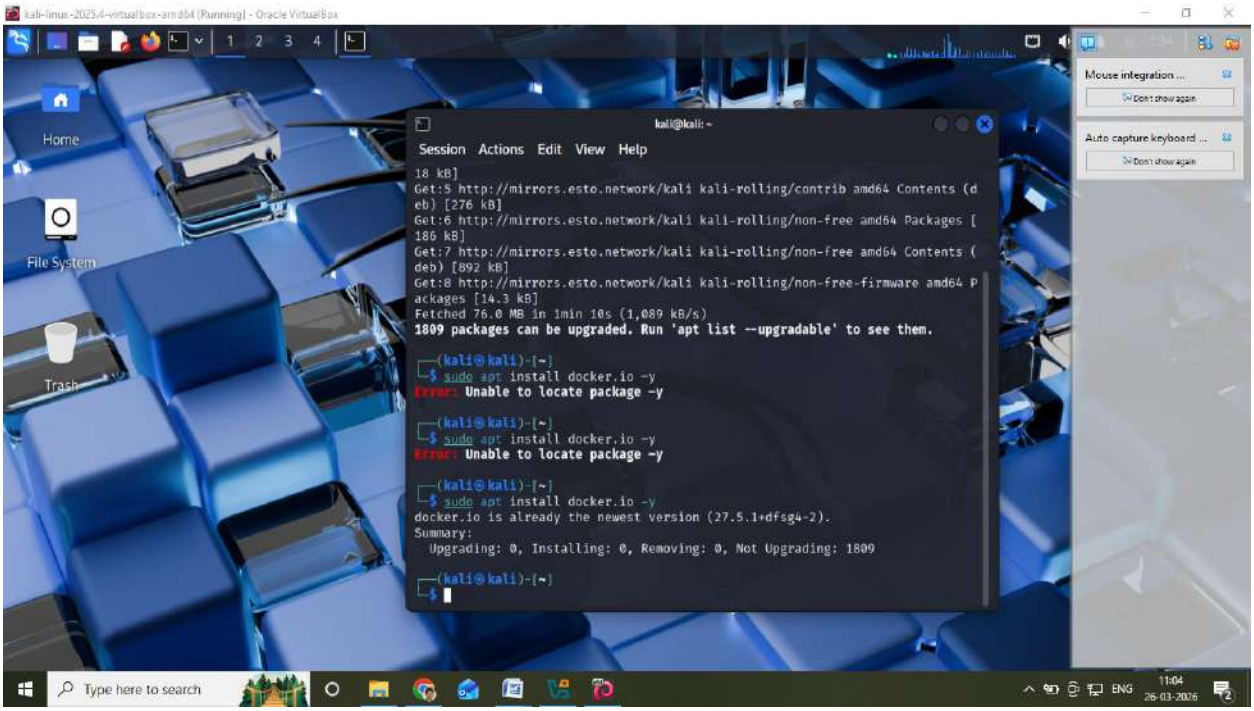
PROCEDURE

STEP 1: Install Docker

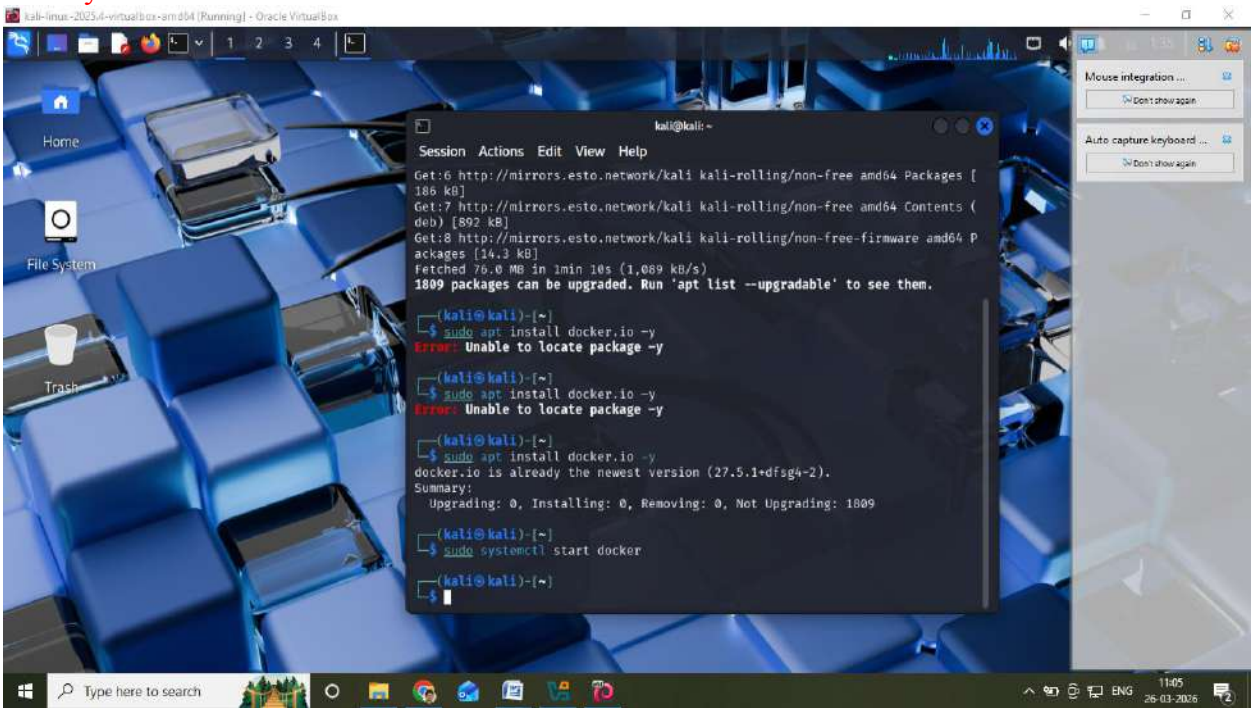
sudo apt update



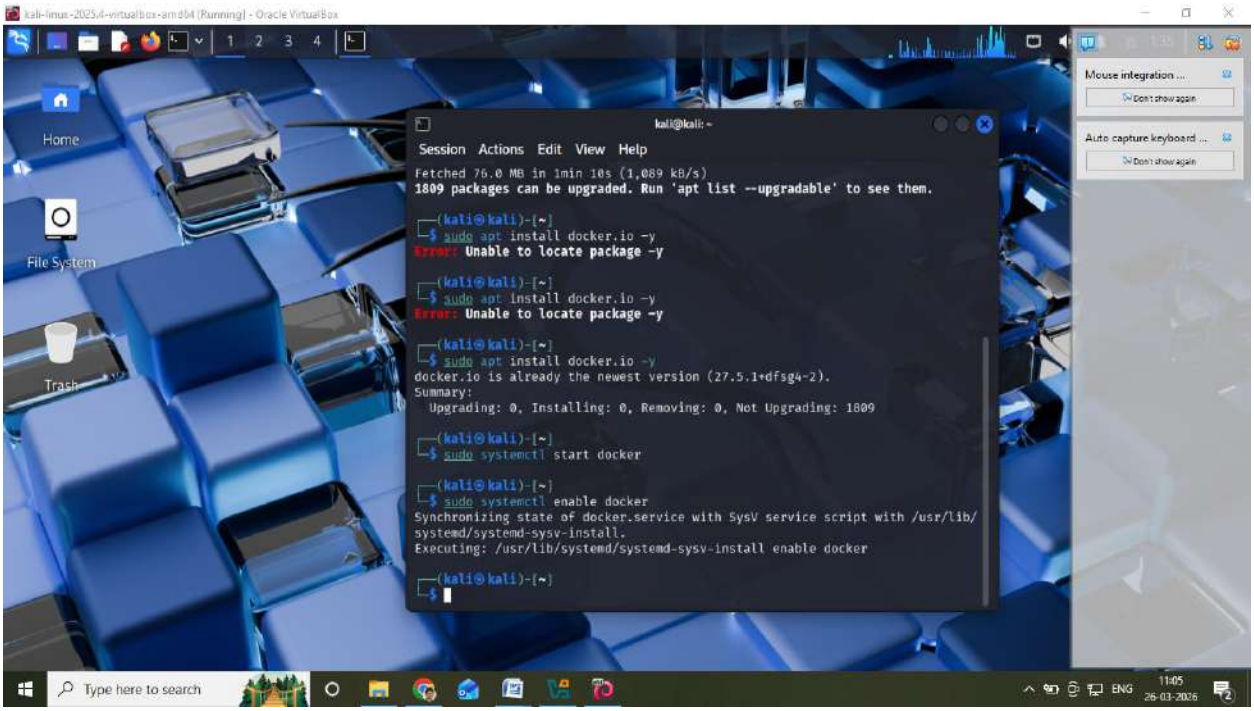
sudo apt install docker.io -y



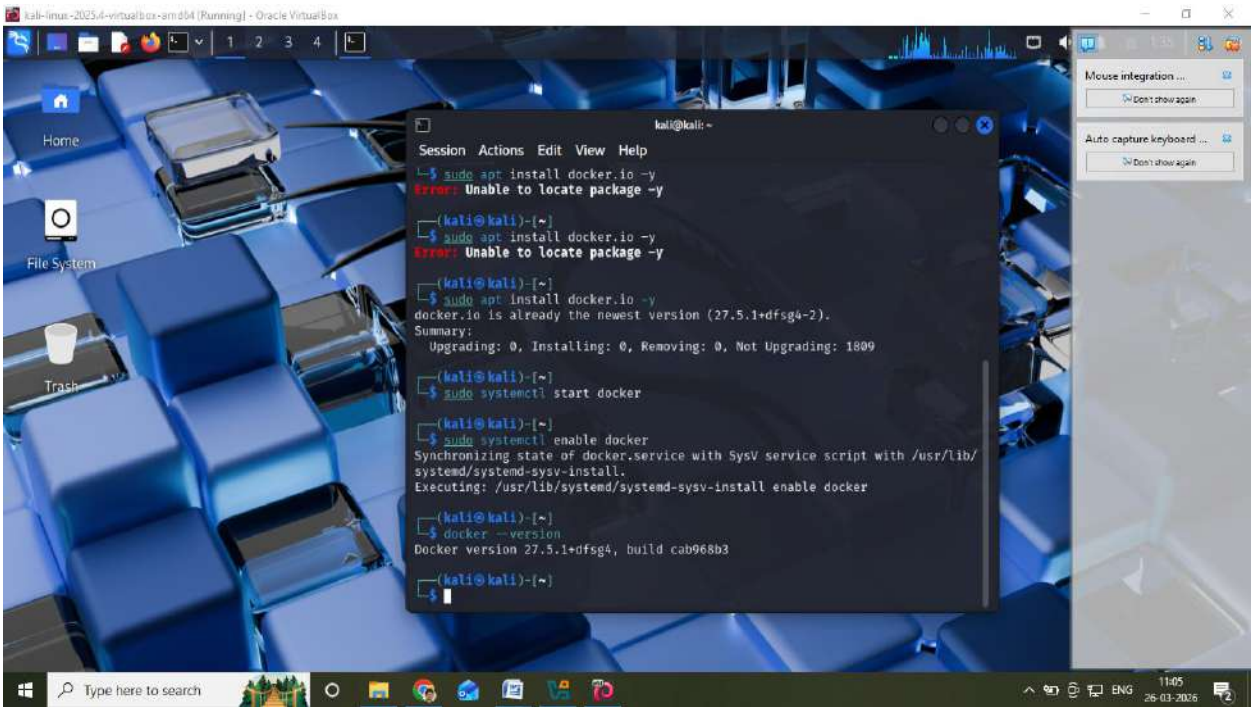
sudo systemctl start docker



sudo systemctl enable docker

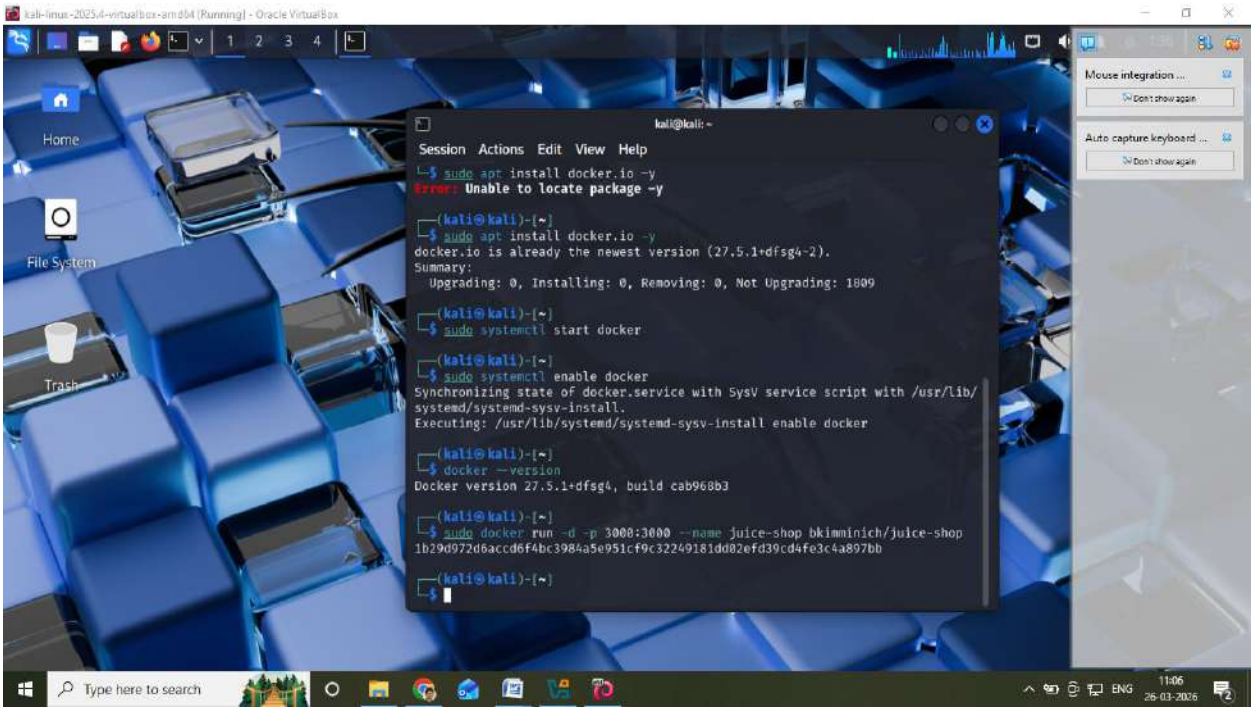


docker –version



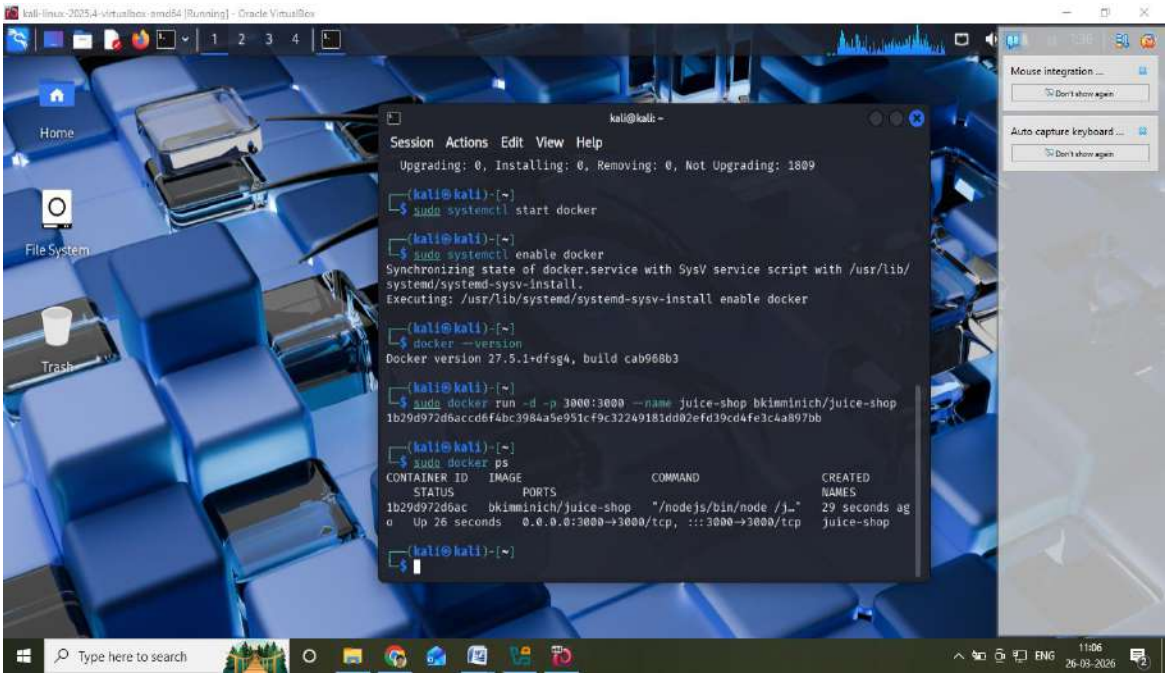
STEP 2: Run Juice Shop using Docker

sudo docker run -d -p 3000:3000 --name juice-shop bkimminich/juice-shop



Check container:

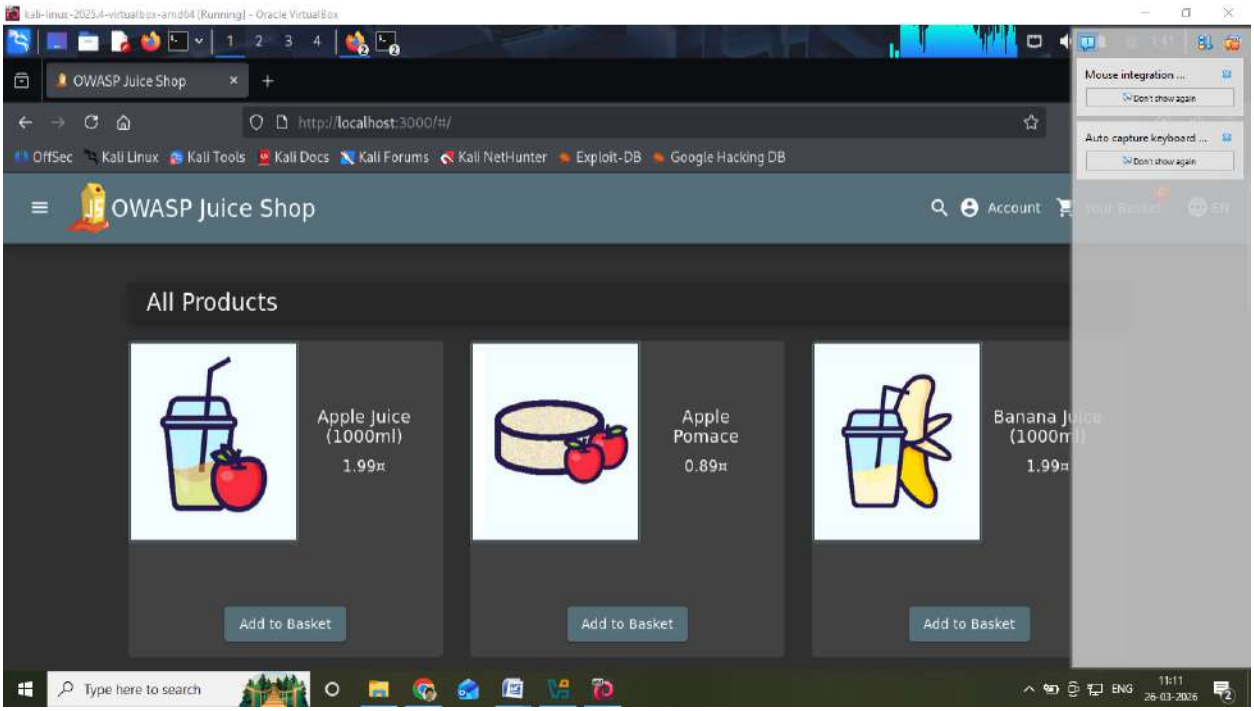
sudo docker ps



STEP 3: Open Application

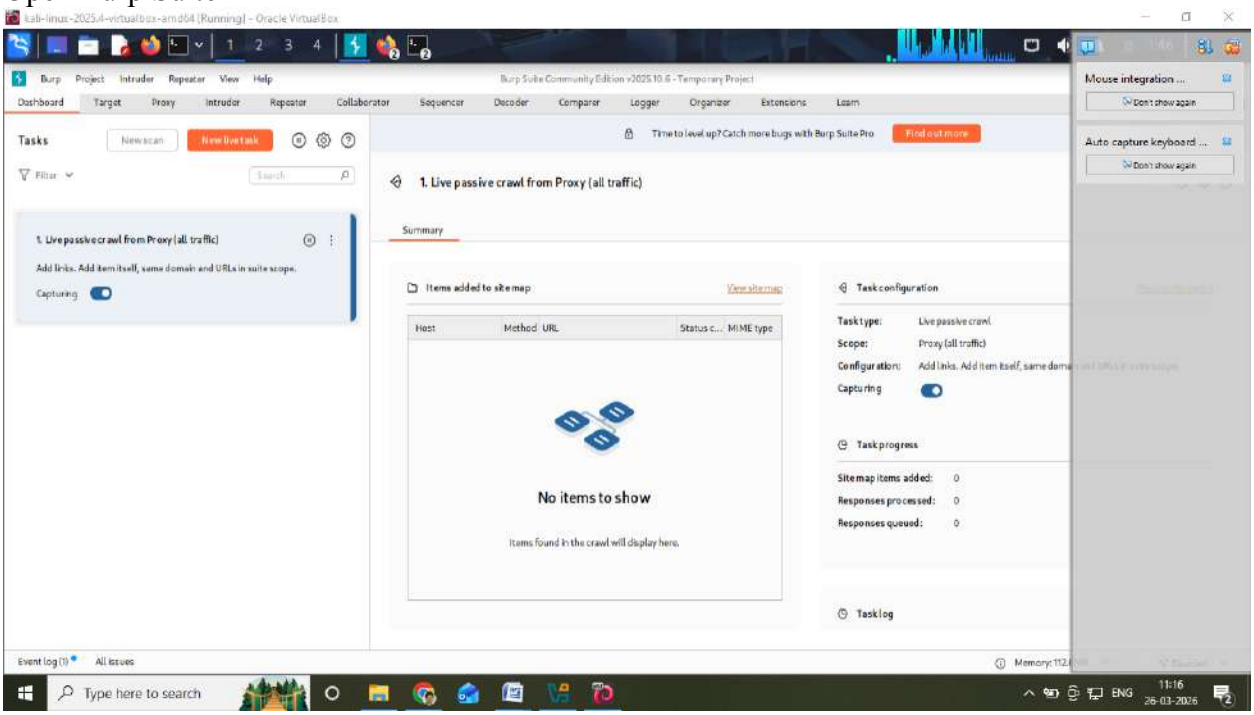
Open browser:

http://localhost:3000

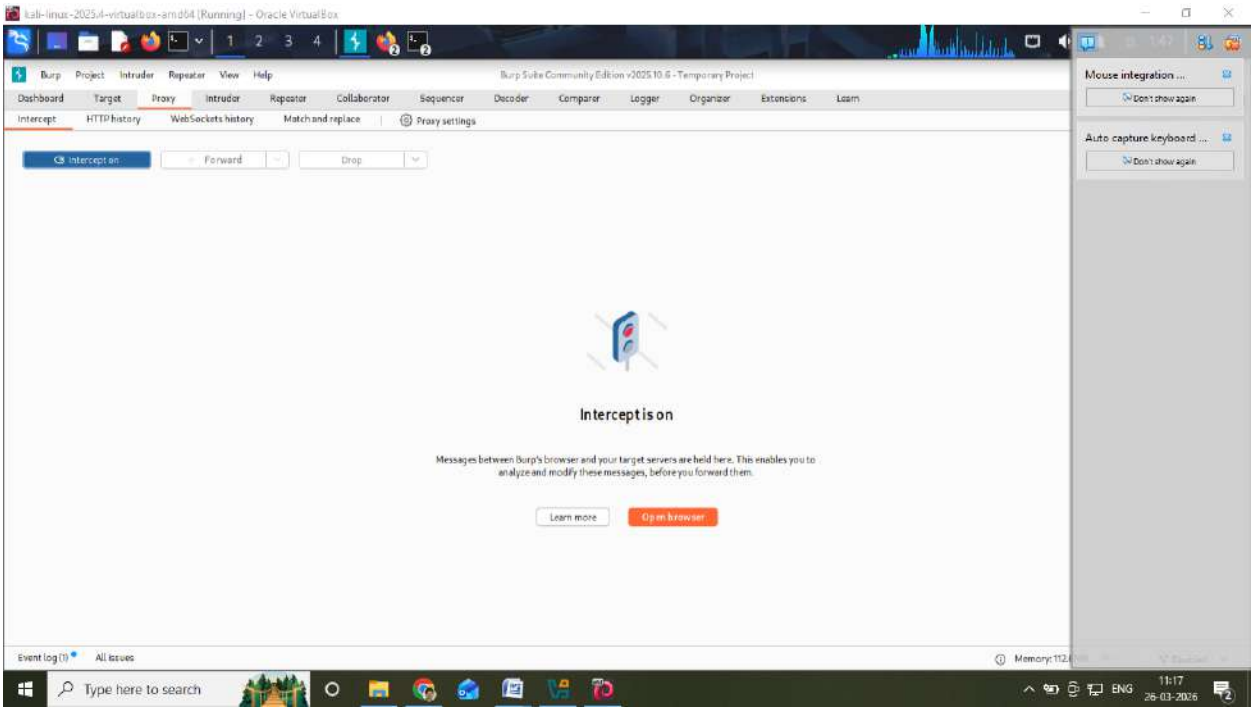


STEP 4: Configure Burp Suite

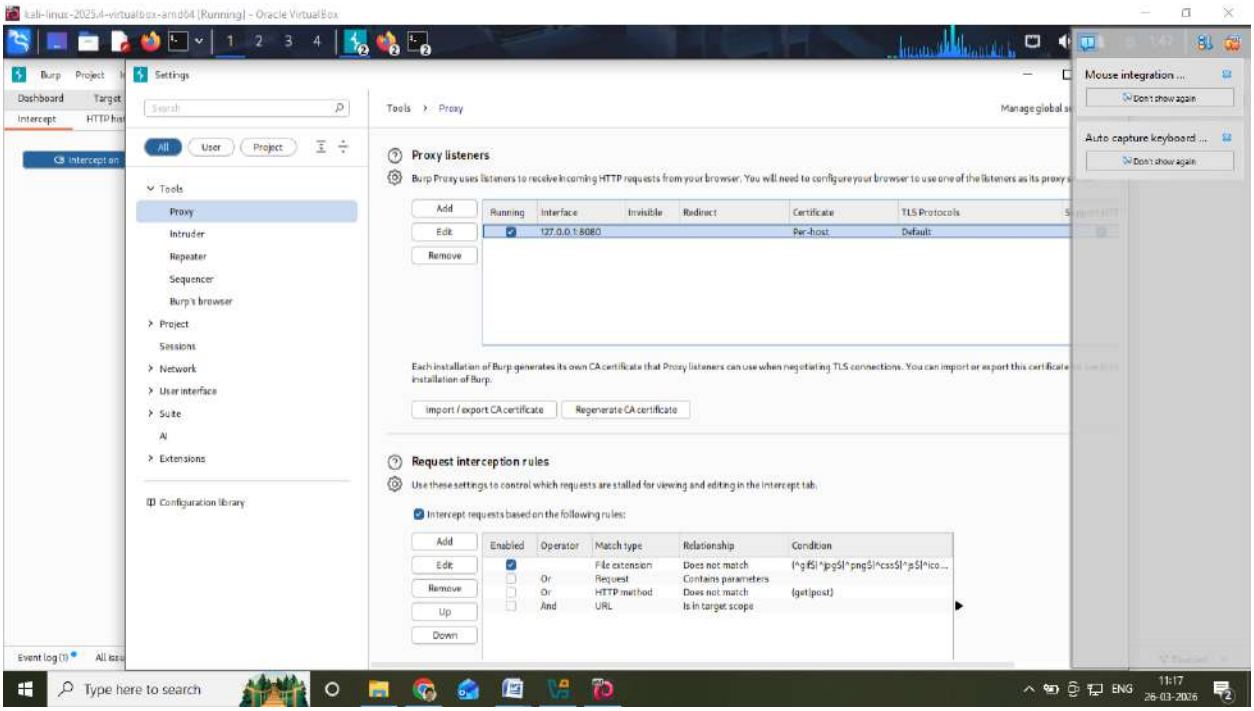
1. Open Burp Suite



2. Proxy → Intercept ON



3. Set browser proxy:
    - IP: 127.0.0.1
    - Port: 8080



PART A: AUTHENTICATION TESTING

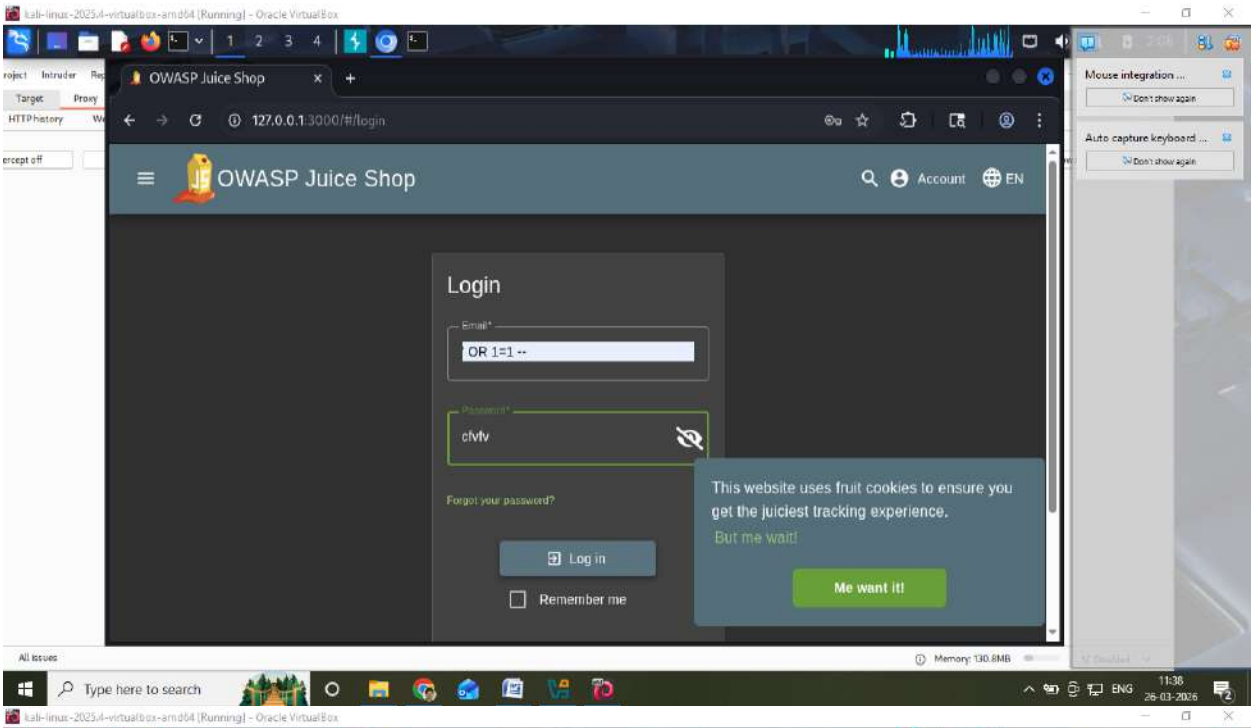
Test 1: SQL Injection Login Bypass

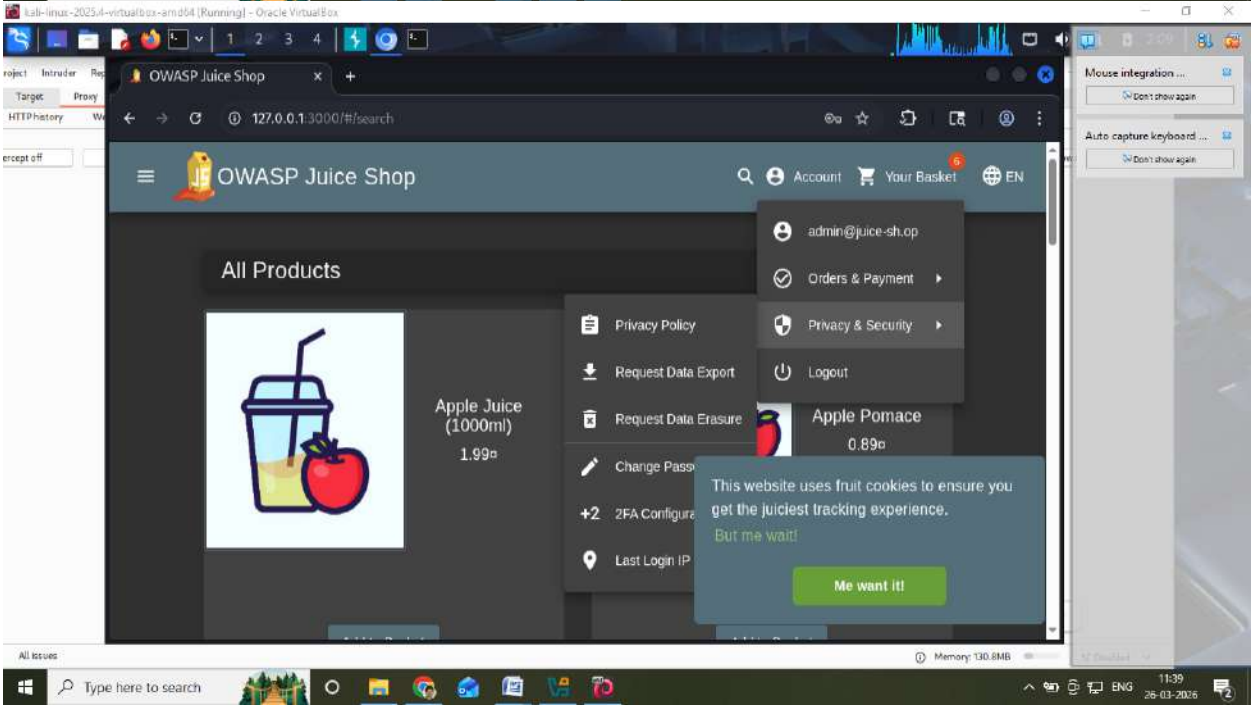
Steps:

1. Open Login page
2. Enter:

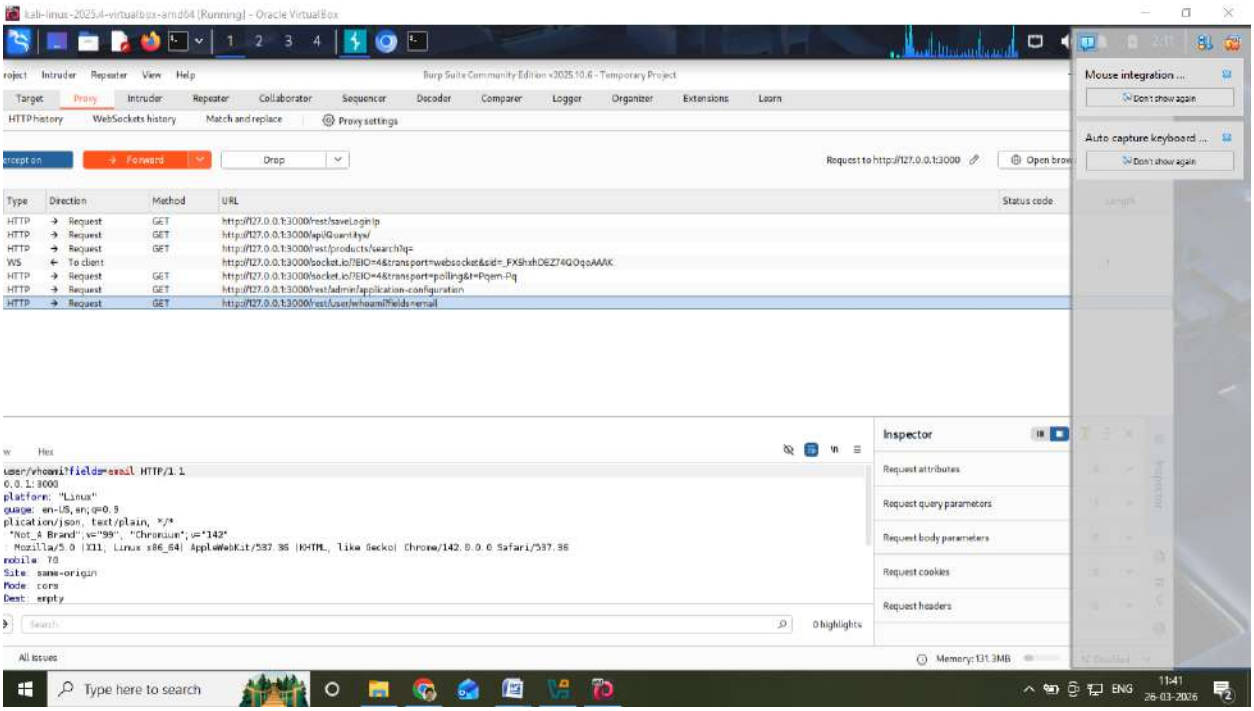
Email: ' OR 1=1 --

Password: anything





3. Forward request in Burp



Result:

Login successful without valid credentials

Inference:

Application is vulnerable to SQL Injection

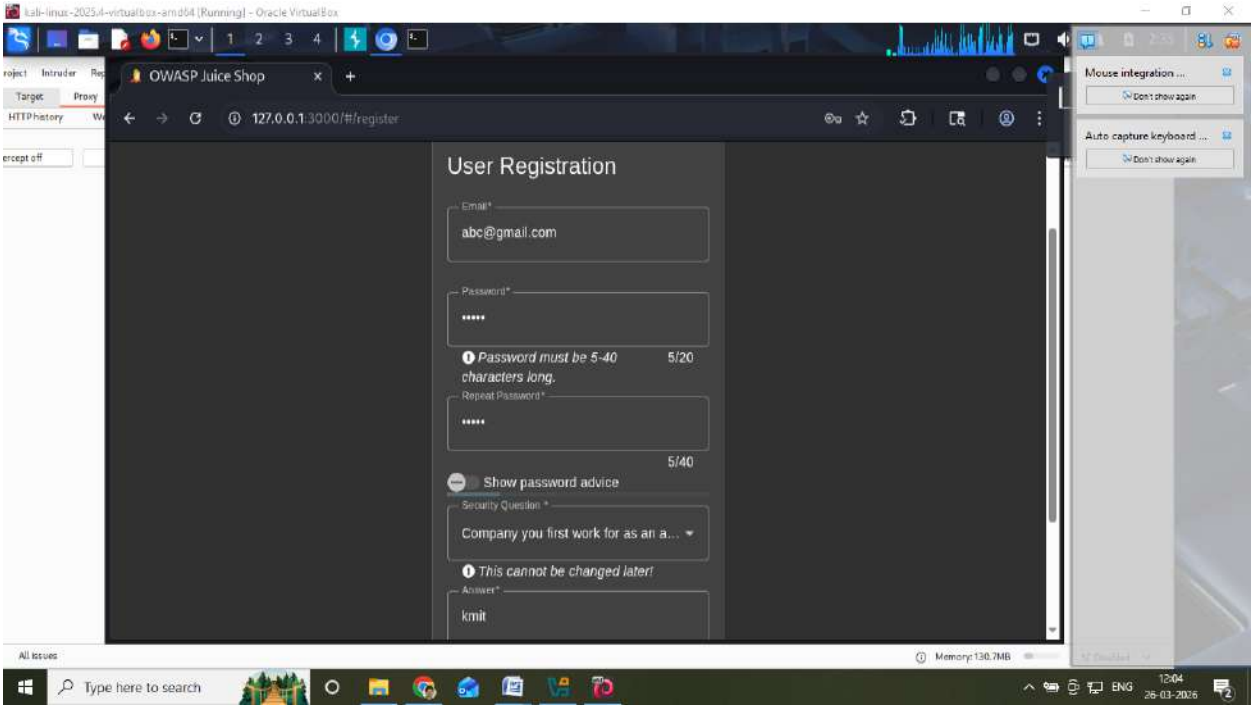
Test 2: Weak Password Policy

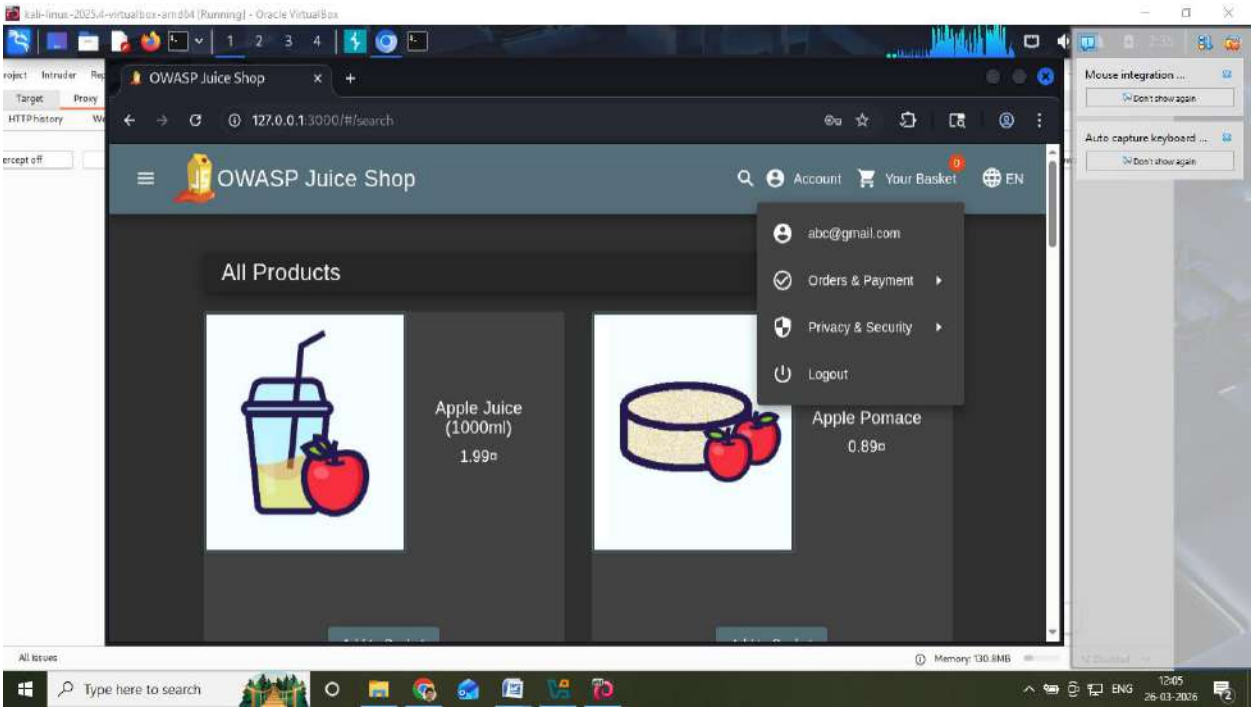
Steps:

1. Register new account
2. Use password:

12345

Result:





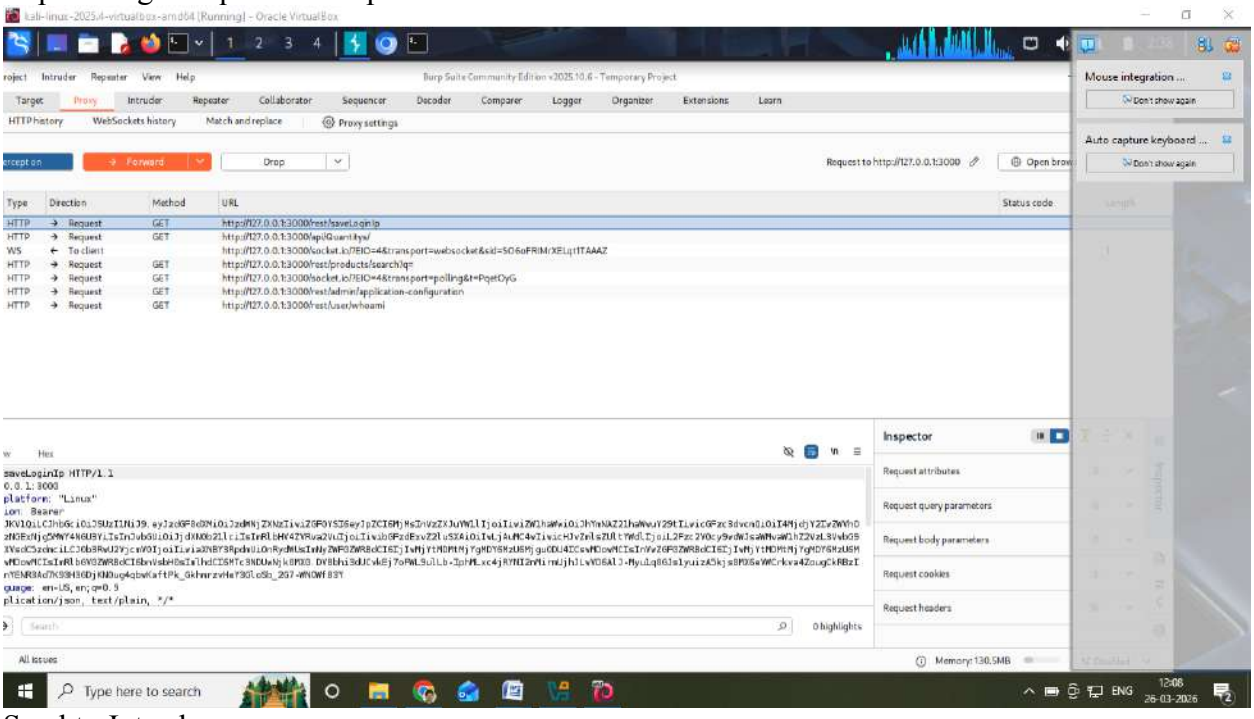
Inference:

No password complexity enforcement

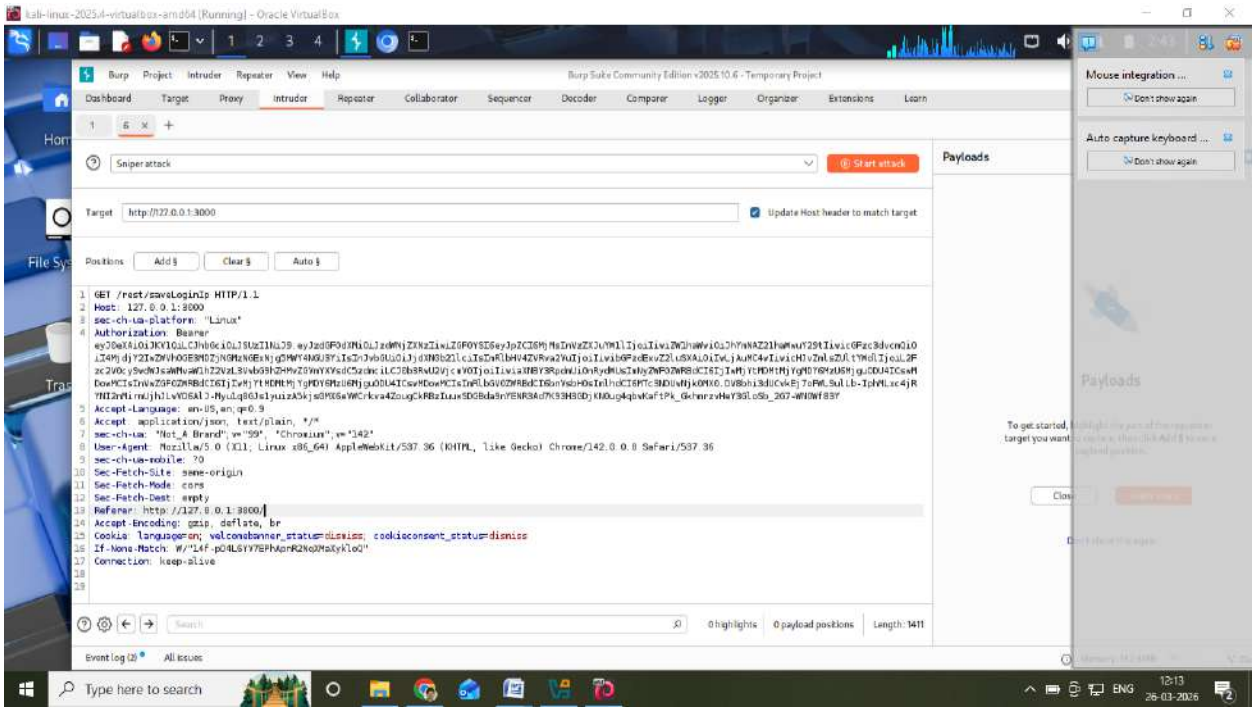
Test 3: Brute Force Attack

Steps:

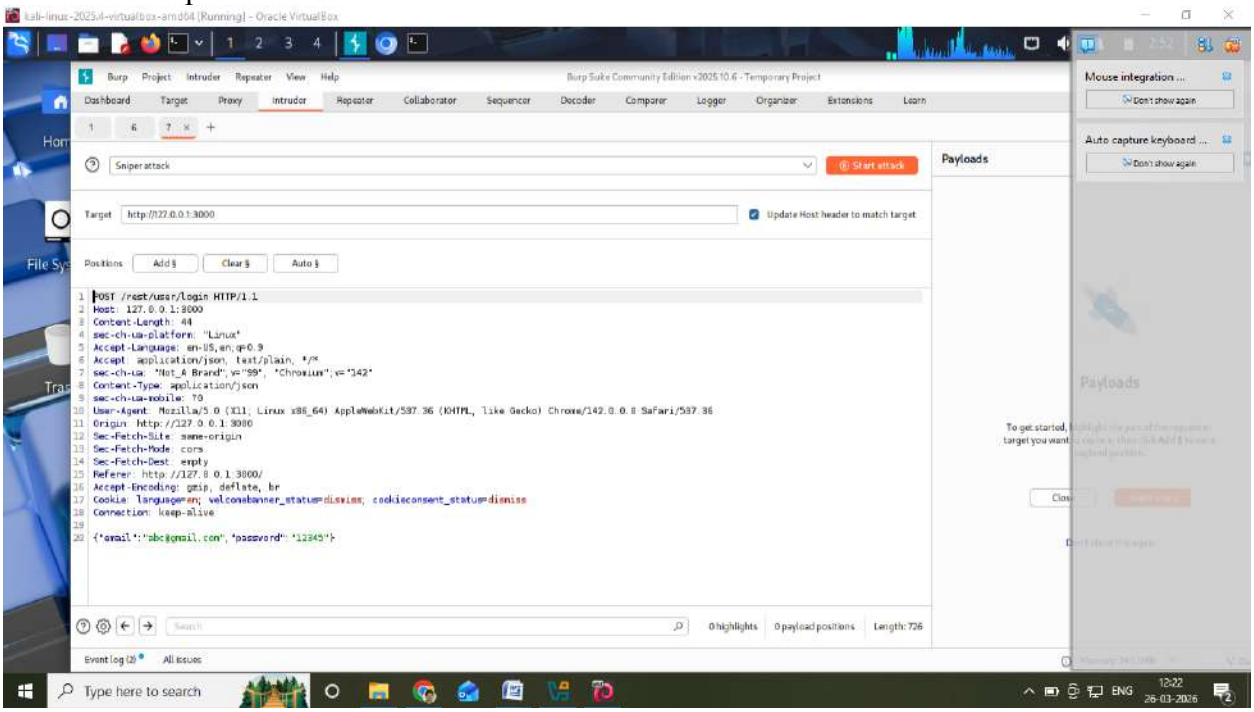
1. Capture login request in Burp

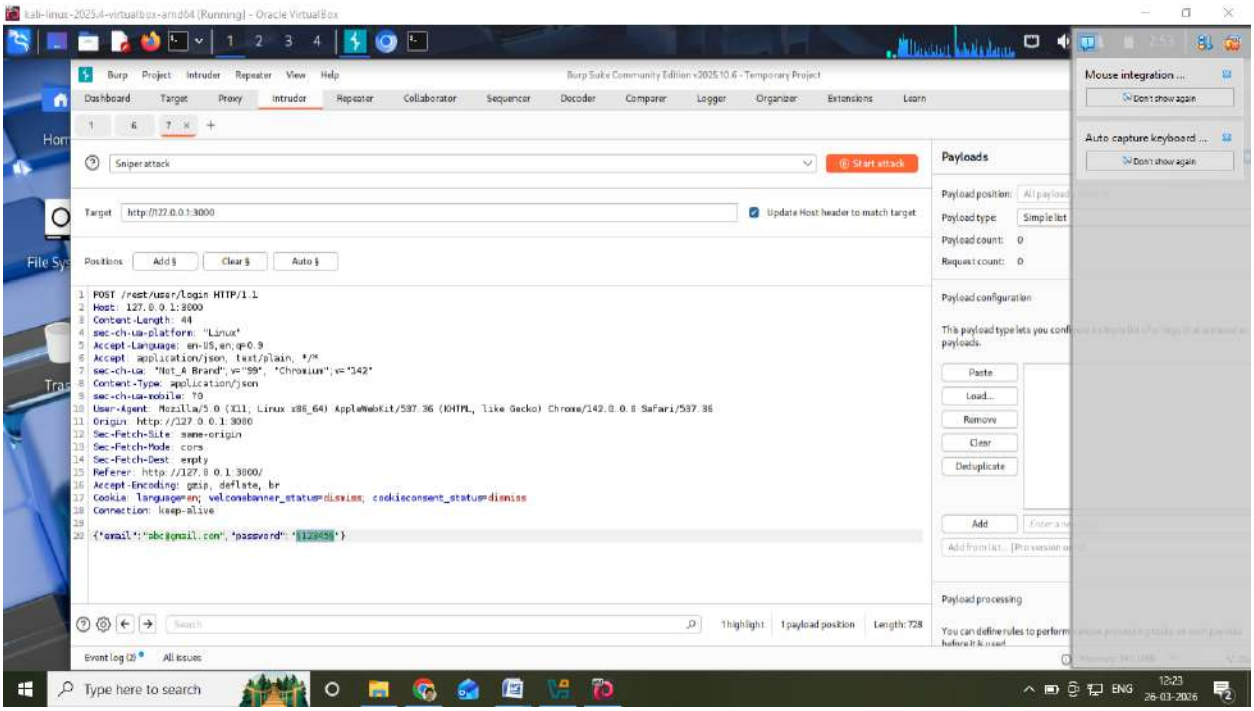


2. Send to Intruder

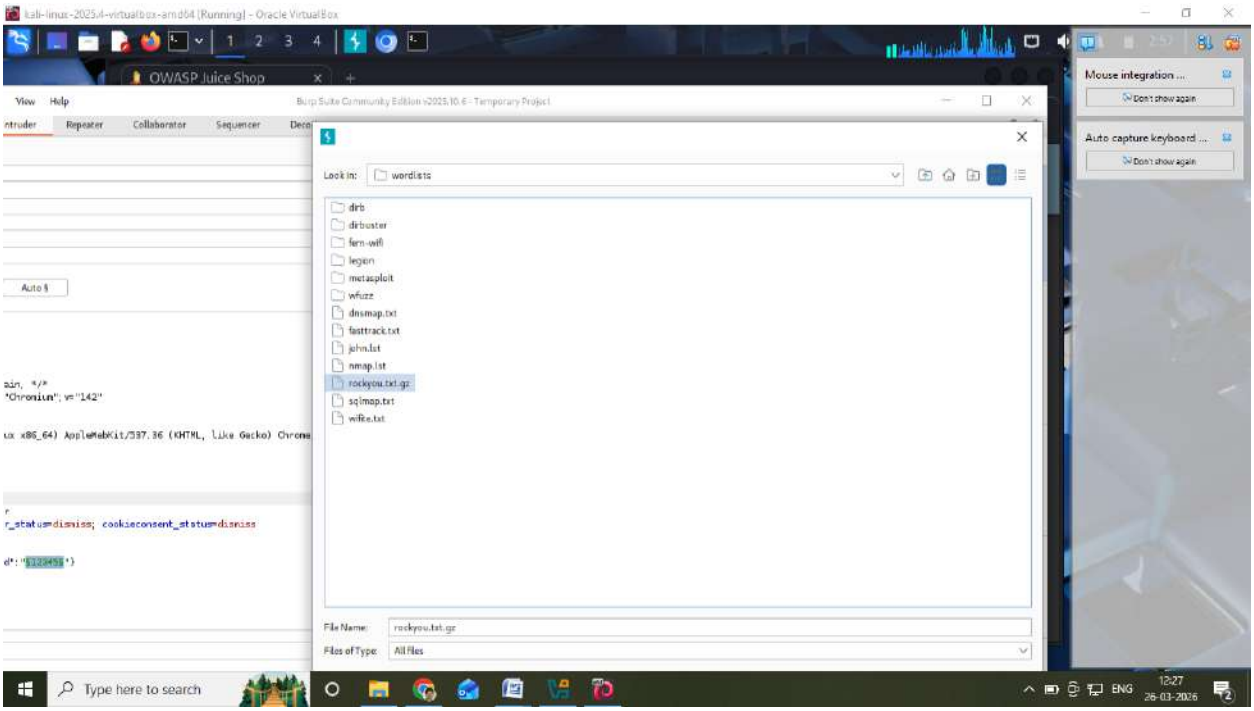


3. Select password field





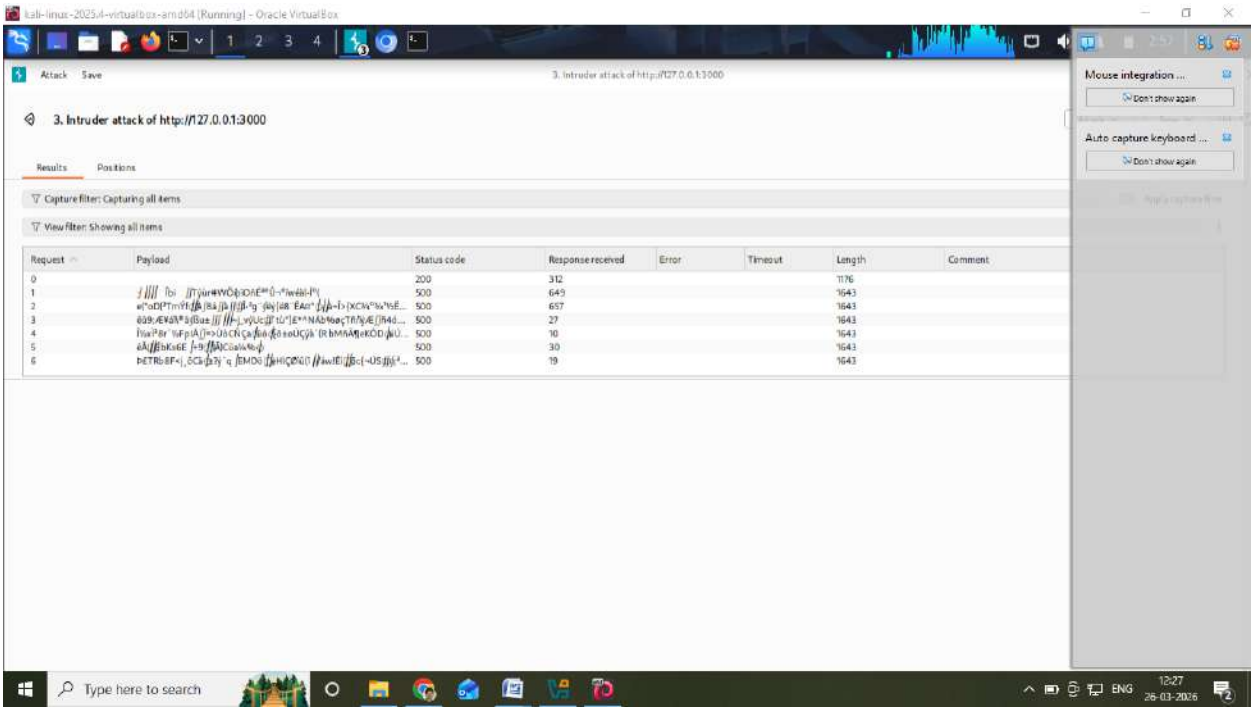
4. Load wordlist: /usr/share/wordlists/rockyou.txt



5. Start attack

Result:

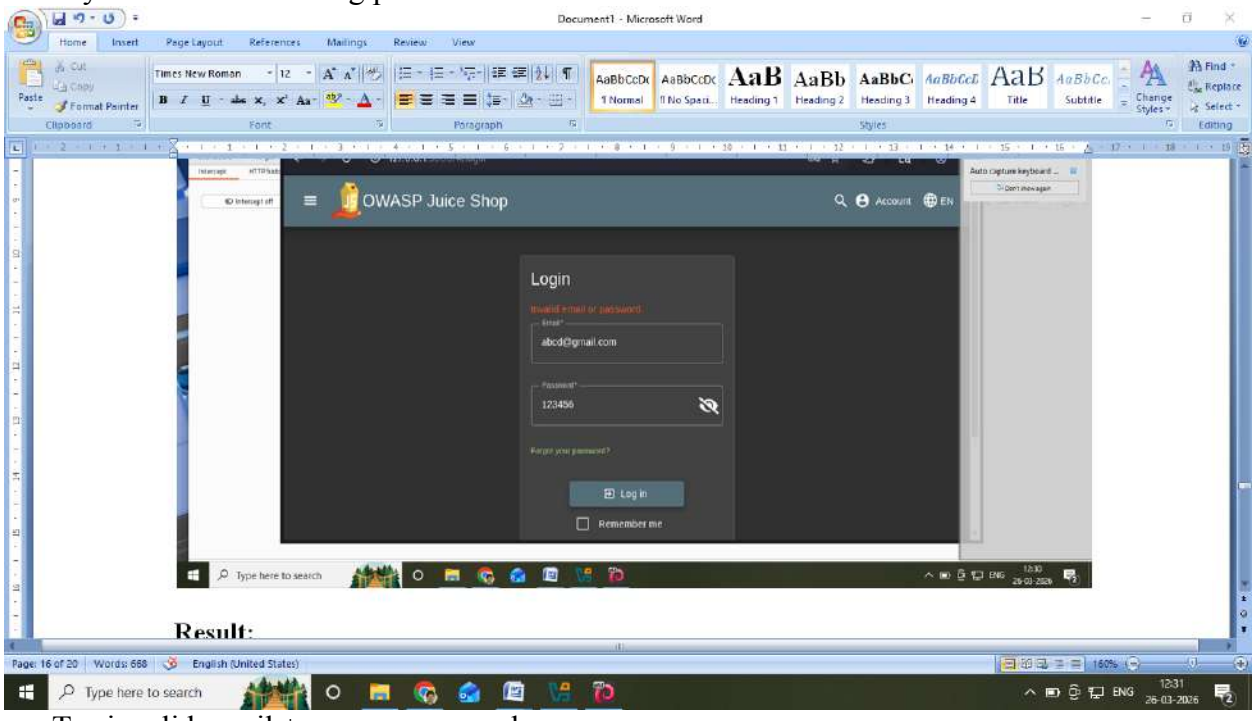
Password discovered



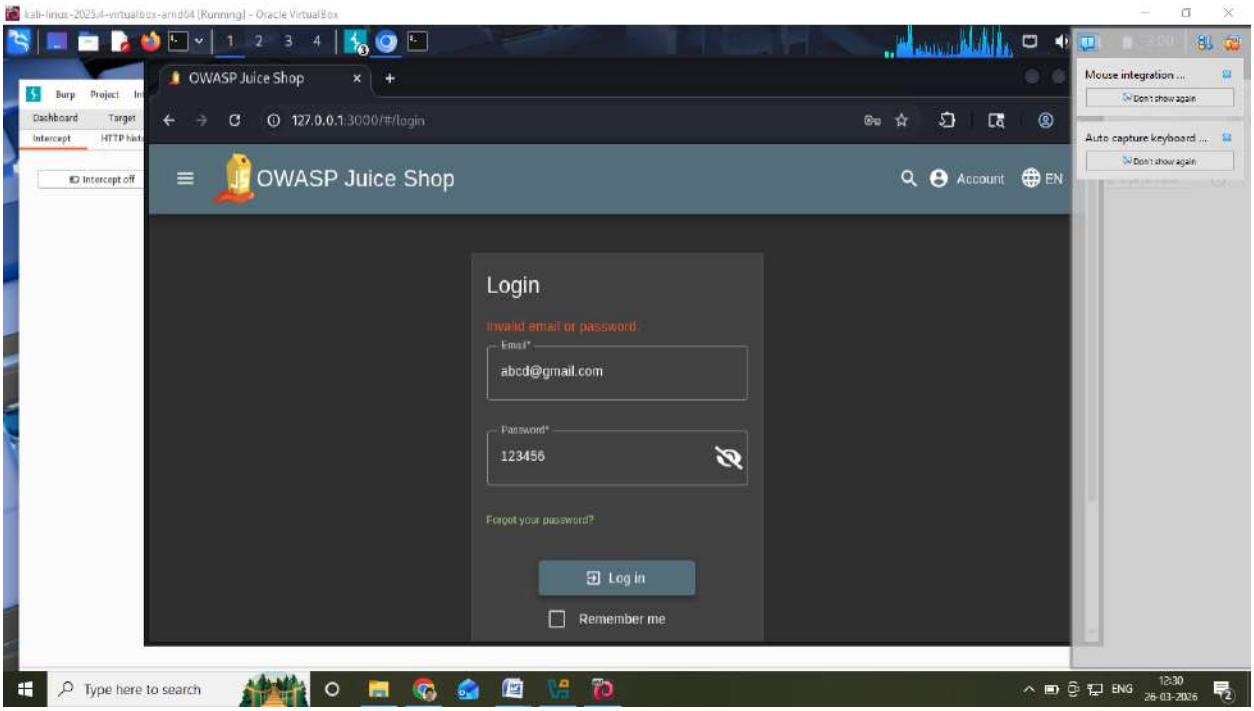
Inference:
No rate limiting or account lock

Test 4: Username Enumeration
Steps:

- Try valid email + wrong password



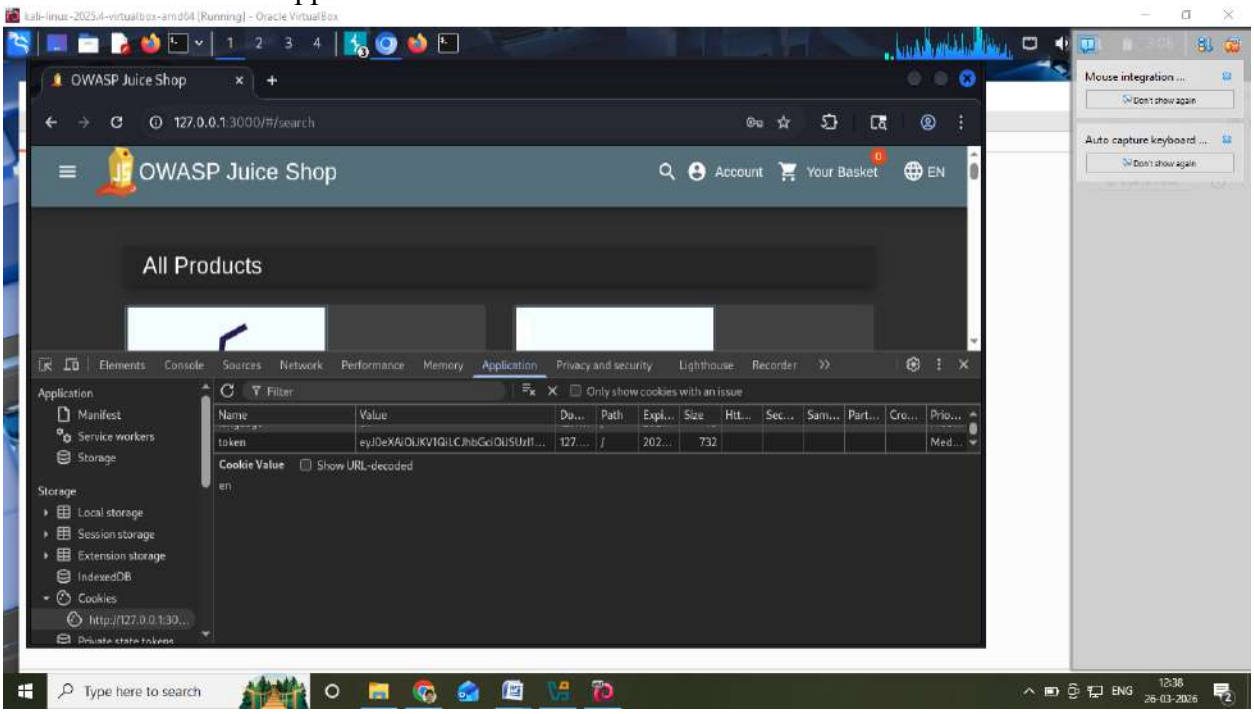
- Try invalid email + wrong password



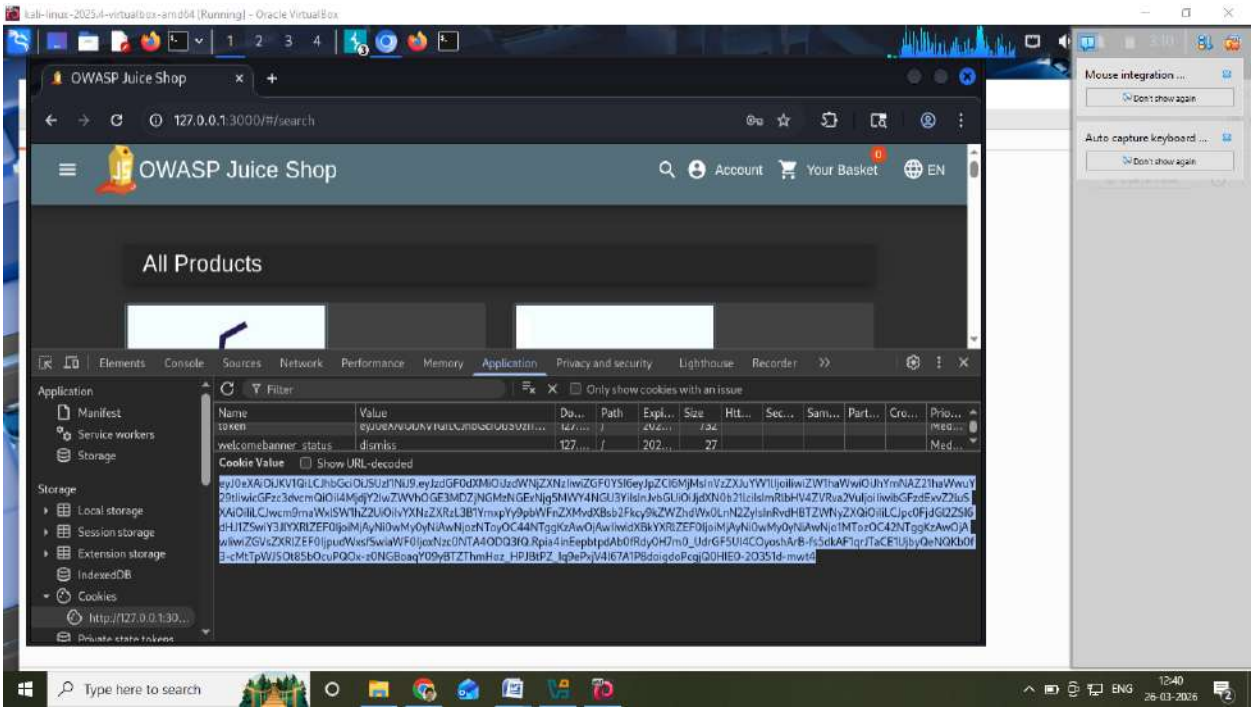
Result:
Different error messages
Inference:
System reveals valid usernames

PART B: SESSION MANAGEMENT TESTING
Test 5: Session Cookie Analysis
Steps:

1. Login
2. Press F12 → Application → Cookies



Result:
Session token visible



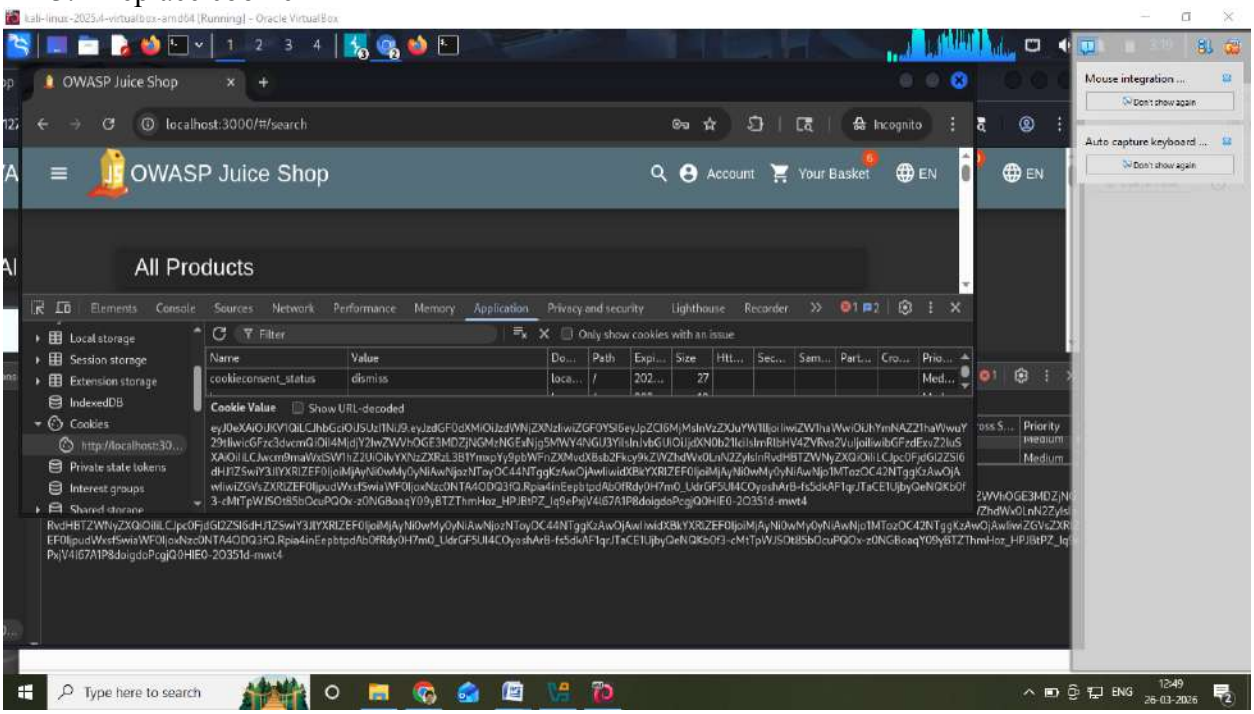
Inference:

Check for HTTPOnly & Secure flags

Test 6: Session Hijacking

Steps:

1. Copy session cookie
2. Open another browser
3. Replace cookie



Result:

Access without login

Inference:

Session not properly protected

Test 7: Session Fixation

Steps:

1. Note session ID before login
2. Login
3. Compare session ID

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTkyMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIiLCJwcm9maWxlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTc3NDUwOTg4MH0.ZSQs6sHIZMsZ0JsHgpRrfXESQ2ERIJ2zpx93QQMvCd2Jm3qKGWtVkY8H8acafz1B9HmS7lvoDZ-K2ST0RT2b5xV6wDd8J1hdlhoGIleM1ZCGVt8yuuBaAs36Mfakm2wOq7nrNowYRKcQf5rwnPUme9N5_Yy3oBhNi-Zk3CxJQSM

Using Burp Suite

Step 1: Intercept Login Request

- Turn **Intercept ON**
- Login in browser

Step 2: Check Response

Go to:

Proxy → HTTP History

Click login request

Step 3: Find Token

Check:

- **Response tab**
- OR Headers:

Set-Cookie: token=

eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJpZCI6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTkyMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjoiIiwibGFzdExvZ2luSXAiOiIiLCJwcm9maWxlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2RlZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjIwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjYtMDMtMjYgMDU6Mzc6MTUuNDU2ICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTc3NDUwOTg4MH0.ZSQs6sHIZMsZ0JsHgpRrfXESQ2ERIJ2zpx93QQMvCd2Jm3qKGWtVkY8H8acafz1B9HmS7lvoDZ-K2ST0RT2b5xV6wDd8J1hdlhoGIleM1ZCGVt8yuuBaAs36Mfakm2wOq7nrNowYRKcQf5rwnPUme9N5_Yy3oBhNi-Zk3CxJQSM

That value = **Session ID**

Result:

Same session ID

Inference:

Session fixation vulnerability exists

Test 8: Logout Mechanism

Steps:

1. Login → copy cookie
2. Logout

3. Reuse cookie

Result:
Session still active
Inference:
Session not invalidated properly
Test 9: JWT Token Analysis
Steps:
1. Copy token
2. Decode in jwt.io

Result:
Payload visible
Screenshot 13: JWT decoding
Inference:
Improper token validation possible

DOCKER COMMANDS USED
docker ps
docker stop juice-shop
docker start juice-shop
docker rm juice-shop
docker logs juice-shop
RESULT
Authentication and session vulnerabilities were successfully identified in the application, including:

- SQL Injection
- Weak password policy
- Brute force vulnerability
- Session hijacking
- Session fixation

Experiment 4: Cross-Site Request Forgery (CSRF)

Tool: WebGoat / Burp Suite
Objective: Perform CSRF attacks to manipulate user actions without consent.
Tasks: Create a forged request to perform unintended actions on behalf of the victim.
Aim

To understand how a Cross-Site Request Forgery (CSRF) attack works, perform the attack in a controlled lab environment, and study methods to prevent it.

Objective

After completing this experiment, the student will be able to:

- Understand the concept of CSRF.
- Identify websites vulnerable to CSRF.
- Perform a CSRF attack in a safe lab setup.
- Analyze the impact of the attack.
- Implement protection techniques against CSRF.

Requirements

- Web Browser
- OWASP Juice Shop
- Node.js or Docker (to run Juice Shop)
- Burp Suite or Browser Developer Tools
- Kali Linux (optional)

Lab Setup

1. Install and run OWASP Juice Shop locally or using Docker.
2. Open the application in the browser.
3. Create an account and login.
4. Navigate to the profile or password change section.
5. Keep the session active in the browser.

Procedure

Step 1: Login to OWASP Juice Shop

- Open OWASP Juice Shop in the browser.
- Login using a registered account.
- Open the browser developer tools and go to the Network tab.
- Navigate to a feature that performs a state-changing action such as updating the user profile.

Step 2: Observe the Legitimate Request

Suppose the application updates the user profile through the following HTTP request:

```
POST /rest/user/profile HTTP/1.1
Host: localhost:3000
Content-Type: application/json
Cookie: connect.sid=<session_cookie>

{
  "username": "student",
  "email": "attacker@example.com"
}
```

If the application does not verify a CSRF token, a forged request can be sent automatically from another webpage while the victim is logged in.

Step 3: Create a Malicious HTML File

Create a file named csrf.html with the following content:

```
<html>
<head>
   <title>CSRF Attack Demo</title>
```

```
</head>
<body onload="document.forms[0].submit()">
  <form action="http://localhost:3000/rest/user/profile" method="POST">
    <input type="hidden" name="username" value="student">
    <input type="hidden" name="email" value="attacker@example.com">
  </form>
</body>
</html>
```

Step 4: Execute the Attack

- Ensure the victim is already logged in to OWASP Juice Shop.
- Open the csrf.html file in the same browser.
- The form automatically submits a forged request.
- The victim's profile data is modified without their knowledge.

Output

The email or profile information of the logged-in user changes automatically.
Example output message:
Profile updated successfully.
```text

Password Changed.
```

Result

The CSRF attack was successfully demonstrated in the lab environment. The vulnerable web application allowed an attacker to perform actions on behalf of the user.

Prevention Techniques

The following methods can be used to prevent CSRF attacks:

1. CSRF Tokens
    - Generate a unique token for each session.
    - Validate the token before processing the request.
2. SameSite Cookies
    - Set cookies with SameSite=Strict or SameSite=Lax.
3. Re-authentication
    - Ask the user to enter the password again before critical actions.
4. Use POST Requests
    - Avoid performing sensitive actions using GET requests.
5. Check Referrer or Origin Header
    - Verify that the request is coming from the correct domain.

Sample Secure Code Using CSRF Token

```
<?php
session_start();

if (!isset($_SESSION['token'])) {
  $_SESSION['token'] = bin2hex(random_bytes(32));
}
?>
```

```
<form method="POST" action="change_password.php">
   <input type="hidden" name="token" value="<?php echo $_SESSION['token']; ?>">
   <input type="password" name="password">
   <input type="submit" value="Change Password">
</form>
```

Validation:

```
if ($_POST['token'] !== $_SESSION['token']) {
   die("Invalid CSRF Token");
}
```

Viva Questions

1. What is CSRF?
2. How is CSRF different from XSS?
3. Why does a CSRF attack require the victim to be logged in?
4. What is the purpose of a CSRF token?
5. Why are GET requests unsafe for sensitive operations?
6. How do SameSite cookies help prevent CSRF?

Experiment 5: Security Misconfiguration

Tool: Juice Shop

Objective: Identify and exploit misconfigurations in security settings.

Tasks: Access unintended admin functionalities, find sensitive information leaks, and exploit debug mode vulnerabilities

Aim

To identify and exploit security misconfigurations in a web application and understand how improper settings can expose sensitive data and functionalities.

Objective

- To understand security misconfiguration vulnerabilities
- To identify misconfigured components in a web application
- To exploit exposed admin panels, debug features, and sensitive files
- To learn preventive measures against misconfigurations

Requirements

- Web Browser (Chrome/Firefox)
- OWASP Juice Shop
- Node.js or Docker
- Burp Suite (optional)

Lab Setup

1. Install Node.js or Docker
2. Run Juice Shop using:
   npm install -g juice-shop
   juice-shop
   OR using Docker:
   docker run -d -p 3000:3000 bkimminich/juice-shop
3. Open browser and go to:
   http://localhost:3000
4. Register and login

Procedure

Step 1: Access Application

- Open Juice Shop in browser
- Login with your account

Step 2: Identify Misconfigurations

- Explore the application manually
- Look for:
  - Hidden directories (e.g., /admin, /ftp)
  - Error messages
  - Debug information

Step 3: Access Admin Panel

- Try accessing:
  http://localhost:3000/#/administration
- Check if access control is properly enforced

Step 4: Exploit Debug/Hidden Features

- Inspect page source (Right-click → Inspect)
- Check JavaScript files for hidden endpoints
- Look for:
  - Hardcoded credentials
  - API endpoints

Step 5: Access Sensitive Files

- Try accessing:
  http://localhost:3000/ftp
- Download exposed files and check for sensitive data

Step 6: Use Burp Suite

- Intercept requests
- Modify requests to bypass restrictions
- Observe server responses

Output

- Unauthorized access to admin panel OR
- Access to sensitive files OR
- Exposure of debug information

Example:
Admin panel accessed successfully
OR
Sensitive file downloaded

Result

Security misconfiguration vulnerabilities were successfully identified and exploited in the Juice Shop application.

Viva Questions

1. What is security misconfiguration?
2. Give examples of misconfiguration vulnerabilities
3. Why is debug mode dangerous?
4. How can admin panels be protected?
5. What are best practices to avoid misconfiguration?

Experiment 6: Broken Access Control

Tool: WebGoat / Juice Shop
Objective: Bypass access control mechanisms to access restricted pages or APIs.
Tasks: Escalate privileges, access unauthorized resources, and test improper role-based access control (RBAC).

Aim

To understand and exploit Broken Access Control vulnerabilities in a web application.

Objective

- Identify access control weaknesses

- Bypass authorization mechanisms
- Access restricted resources
- Perform privilege escalation
- Understand Role-Based Access Control (RBAC) flaws

Requirements

- Web Browser
- OWASP Juice Shop
- WebGoat
- Burp Suite (optional)

Lab Setup

1. Install and run Juice Shop (Node.js/Docker)
2. Open browser → http://localhost:3000
3. Register and login as a normal user

Procedure

Step 1: Login as Normal User

- Open Juice Shop
- Login with a regular account

Step 2: Try Accessing Admin Page

- Enter URL:

http://localhost:3000/#/administration

- Observe whether access is restricted

Step 3: Perform IDOR Attack

- Go to "Order History" or similar feature
- Modify request/URL like:

/rest/orders/1
/rest/orders/2

- Check if other users' data is accessible

Step 4: Modify API Requests (Using DevTools or Burp)

- Open **Inspect → Network Tab**
- Intercept request using Burp Suite
- Modify parameters like:
    - userId
    - role

Step 5: Privilege Escalation

- Try changing role in request:

```
{
  "role": "admin"
}
```

- Observe if application allows escalation

Step 6: Access Unauthorized Resources

- Try accessing:

/admin

/rest/user/all

- Check if data is exposed

Output

- Unauthorized access to admin panel OR
- Access to other users' data OR
- Successful privilege escalation

Example:
Access Granted to Admin Panel
OR
User data accessed without authorization

Result

Broken Access Control vulnerability was successfully identified and exploited.

1.

Viva Questions

1. What is Broken Access Control?
2. What is IDOR?
3. What is RBAC?
4. How can privilege escalation occur?
5. How do you prevent access control issues?

.

Experiment 7: Man-in-the-Middle (MITM) Attack Using Burp Suite

Tool: Burp Suite
Objective: Intercept and modify HTTP requests and responses.
Tasks: Capture credentials, manipulate API requests, and tamper with application logic.

Aim

To perform a Man-in-the-Middle (MITM) attack by intercepting and modifying HTTP requests and responses using Burp Suite.

Objective

- Understand MITM attack concepts
- Intercept HTTP/HTTPS traffic
- Capture sensitive data like credentials
- Modify requests and responses
- Analyze application behavior

Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

- Web Browser (Chrome/Firefox)
- Burp Suite
- Target application (e.g., OWASP Juice Shop)

Lab Setup

1. Install and open Burp Suite
2. Go to **Proxy → Intercept → Turn ON**
3. Configure browser proxy:
    - IP: 127.0.0.1
    - Port: 8080
4. Install Burp CA Certificate (for HTTPS interception)
5. Open target application in browser

Procedure

Step 1: Start Interception

- Open Burp Suite
- Turn **Intercept ON**

Step 2: Capture Login Request

- Open application login page
- Enter credentials and click login
- Request will be captured in Burp

Example:

```
POST /rest/user/login HTTP/1.1
Host: localhost:3000

{
  "email": "user@test.com",
  "password": "123456"
}
```

Step 3: Analyze Request

- Observe:
    - URL
    - Parameters
    - Headers
- Identify sensitive data (credentials, tokens)

Step 4: Modify Request

- Change values before forwarding:

```
{
  "email": "admin@test.com",
  "password": "admin123"
}
```

- Click **Forward**

Step 5: Capture Response

- Observe server response:
    - Status codes
    - Messages

- Tokens

Step 6: Manipulate API Requests

- Intercept other requests (e.g., profile update, orders)
- Modify parameters like:
  - userId
  - role
  - price

Step 7: Tamper Application Logic

- Change values like:

```
{
  "amount": 1
}
```

(instead of higher value)

- Forward request and observe behavior

Output

- Captured login credentials
- Modified request successfully processed
- Changed application behavior

Example:
Login successful with modified credentials
OR
Price changed due to tampered request

Result

MITM attack was successfully performed using Burp Suite. Requests and responses were intercepted and modified.

Viva Questions

1. What is a MITM attack?
2. How does Burp Suite work?
3. Why is HTTPS important?
4. What is certificate pinning?
5. How can MITM attacks be prevented?

Experiment 8 : API Security Testing

Tool: Burp Suite / Juice Shop
Objective: Identify and exploit API vulnerabilities like improper authentication, rate-limiting bypass, and injection flaws.
Tasks: Perform fuzzing on API endpoints, check for IDOR vulnerabilities, and exploit weak

Aim

To identify and exploit vulnerabilities in web APIs such as improper authentication, IDOR, and rate-limiting issues.

Objective

- Understand API security risks
- Intercept and analyze API requests
- Perform fuzzing on API endpoints
- Identify IDOR vulnerabilities
- Test weak authentication mechanisms

Requirements

- Web Browser
- OWASP Juice Shop
- Burp Suite

Lab Setup

1. Start Juice Shop:
2. `docker run -d -p 3000:3000 bkimminich/juice-shop`
3. Open browser → `http://localhost:3000`
4. Configure Burp Suite proxy (127.0.0.1:8080)
5. Turn **Intercept ON**

Procedure

Step 1: Intercept API Requests

- Open Juice Shop
- Perform actions (login, view products, orders)
- Capture API requests in Burp

Example:

```
GET /rest/products/search?q=apple HTTP/1.1
Host: localhost:3000
```

Step 2: Analyze API Endpoints

- Identify:
  - URLs
  - Parameters
  - Authentication tokens

Step 3: Perform IDOR Attack

- Modify API request:

```
rest/user/1
/rest/user/2
```

- Check if other users' data is accessible

Step 4: Fuzzing API Endpoints

- Send request to **Repeater/Intruder (Burp)**
- Try different inputs:
  - IDs
  - Strings
  - Special characters

Example:

```
' OR 1=1 --
../../etc/passwd
```

Step 5: Test Rate Limiting

- Send multiple login/API requests quickly
- Observe if system blocks repeated attempts

Step 6: Test Authentication Weakness

- Remove or modify token:

```
Authorization: Bearer invalid_token
```

- Check if API still responds

Step 7: Injection Testing

- Inject malicious input in API parameters
- Observe response changes

Output

- Access to unauthorized data (IDOR)
- API accepts invalid tokens
- No rate limiting observed
- Injection payload affects response

Example:

```
User data accessed without authorization
OR
API responded without valid authentication
```

Result

API vulnerabilities such as IDOR, weak authentication, and improper validation were

Viva Questions

1. What is API security?
2. What is IDOR?
3. What is rate limiting?
4. How does fuzzing work?
5. How can APIs be secured?

Experiment 9: Basic Firewall Configuration

Windows Defender Firewall filters incoming and outgoing network traffic using predefined rules. By default, it allows most standard traffic and only asks permission when an application tries to connect in a non-standard way. However, it does not automatically block known malicious IP addresses. To improve security, we can manually add firewall rules and also automate rule creation using a script that blocks a list of known malicious IPs.

Software Requirements:

- Python 3.x
- Windows OS
- PowerShell
- Internet Connection
- Visual Studio Code / Any Python IDE

Manual Firewall Configuration (Inbound Rule):

This part explains how to manually block a specific IP using the Windows Firewall GUI.
Steps:

1. Open **Windows Defender Firewall with Advanced Security.**
2. Click on **Inbound Rules**.
3. Click **New Rule,**on the right panel.
4. Select **Custom** and click **Next**.
5. Choose **All Programs** → Click **Next**.
6. Protocol and Ports: Keep default → Click **Next**.
7. Scope:
    - Under **Remote IP address**, select **These IP addresses**.
    - Click **Add** and enter the IP you want to block (example: 1.2.3.4).
    - Click **OK** → Next.
8. Action: Select **Block the connection** → Next.
9. Profile: Select **Domain, Private, Public** → Next.
10. Name the rule (example: Manual_Block_IP) → Finish.

This rule will now block all incoming traffic from that specific
IP. Automated Firewall Configuration (Using Python Script):
Instead of blocking one IP, the script downloads a list of malicious IPs and blocks all of them automatically.

Problem Statement:

Write a Python program that downloads a list of malicious IP addresses from a trusted online source and automatically creates firewall rules to block those IPs from accessing the system.

Program: (File name: firewall.py)

```
import requests, csv,
subprocess # source: Abuse CH
response = requests.get(
   "https://feodotracker.abuse.ch/downloads/ipblocklist.csv"
).text
rule = 'netsh advfirewall firewall delete rule name="BadIP"'
subprocess.run(["PowerShell", "-Command", rule])
mycsv = csv.reader(
   filter(lambda x: not x.startswith("#"), response.splitlines())
)
```

```
for row in mycsv:
  ip = row[1]
    if ip != "dst_ip":
    print("Added Rule to block:", ip)
   rule = "netsh advfirewall firewall add rule name='BadIP' Dir=Out Action=Block RemoteIP=" +
   ip

      subprocess.run(["PowerShell", "-Command", rule])
```

Sample Output:

Added Rule to block: 45.9.148.221
Added Rule to block: 103.17.48.5
Added Rule to block: 185.234.219.12

Execution Steps:

1. Open **Command Prompt** and select **Run as Administrator**. (Firewall rules require admin rights.)
2. Navigate to the folder where firewall.py is saved.
   Example: cd C:\Users\YourName\Desktop
3. Make sure Python is installed: python --version
4. install required library (if not already installed):- python -m pip install requests
5. Execute the program: python firewall.py
6. **Output will appear in Command Prompt like:**
   Added Rule to block: 45.9.148.221
   Added Rule to block: 103.17.48.5

For every IP, you will see:

- "Added Rule to block: <IP>"
- PowerShell/Command Prompt will also show **OK** message for successful rule creation.

7. Cross-verify the rules:
   - Open **Windows Defender Firewall with Advanced Security**
   - Go to **Outbound Rules**
   - Search for rule name: **BadIP**
   - You will see many blocked IP addresses listed

Experiment 10: Password Strength Testing using Python

Write a Python program that accepts a password from the user and checks whether it is weak, medium, or strong based on the following rules:

- Minimum 8 characters
- Must contain at least one digit
- Must contain at least one uppercase letter
- Must contain at least one lowercase letter
- Must contain at least one special character

Program: (File name: password_checker.py)

```
import re
def check_password_strength(password): if
  len(password) < 8:
    return "Weak: Password must be at least 8 characters long."
  if not any(char.isdigit() for char in password):
    return "Weak: Password must include at least one number."
  if not any(char.isupper() for char in password):
    return "Weak: Password must include at least one uppercase letter."
  if not any(char.islower() for char in password):
    return "Weak: Password must include at least one lowercase letter."


if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password

return "Medium: Add special characters to make your password stronger."
  return "Strong: Your password is secure!"
def password_checker():
  print("Welcome to the Password Strength Checker!")
  while True:
    password = input("\nEnter your password (or type 'exit' to quit): ")
    if password.lower() == "exit":
      print("Thank you for using the Password Strength Checker! Goodbye!")
      break
    result = check_password_strength(password) print(result)
if __name___== "__main__":
  password_checker()
```

Execution Steps:-

1. Open Command Prompt or VS Code Terminal.
2. Navigate to the folder where password_checker.py is saved
3. Run the program: python password_checker.py

The program will display:

Welcome to the Password Strength Checker!
Enter your password (or type 'exit' to quit):

4. Type a password and press Enter.
5. The program will display whether the password is:
    - Weak
    - Medium
    - Strong

6. To stop the program, type: exit

Sample Input & Output:

Input:
Enter your password: abc123

Output:

Weak: Password must be at least 8 characters long.

Input:

Enter your password: Abcdef12

Output:

Medium: Add special characters to make your password stronger.

Input:

Enter your password: Abc@1234

Output:

Strong: Your password is secure!

Experiment 11: Analyzing Phishing Emails

To analyze a suspicious email using EML Analyzer and VirusTotal and identify whether it is phishing based on technical indicators.

Tools Used:

- Online EML Analyzer
- VirusTotal
- Sample file: 2020-05-05-phishing-email-example-01.eml

Questions to Answer:

1. What is the full email address of the sender?
2. What domain is used to send this email?
3. What is the sender's IP address from the header?
4. Is the sender IP blacklisted?
5. What is the result of SPF authentication?
6. What is one suspicious URL found in the email body?

Part A: EML Analyzer Results

The file 2020-05-05-phishing-email-example-01.eml was uploaded to the EML Analyzer. EML Analyzer Results:

Key Observations:

- Subject: Warning: Final Notice
- From: malware-traffic-analysis.net Support sues@nnwifi.com
- To: brad@malware-traffic-analysis.net
- Content Type: text/html
- Message-ID: Missing

Header Analysis:

- Sender IP: 94.100.31.27
- Reverse DNS: 94-100-31-27.static.hvvc.us
- Mail Server: mail.nnwifi.com

Authentication:

- SPF: Failed
- DKIM: Not signed
- DMARC: Not aligned

Part B: VirusTotal Analysis

The sender IP 94.100.31.27 was checked on VirusTotal.

VirusTotal Result:

- Detection Ratio: 1 / 93 vendors flagged as malicious
- Location: Netherlands
- ASN: AS29802 (HVC-AS)

This means the IP is not widely blacklisted but has suspicious reputation.

Suspicious Link Identified

From EML Analyzer, the following URL was extracted:
https://servervirto.com.co/ed/trn/update?email=brad@malware-traffic-analysis.net Reasons it is suspicious:

- Does not match sender domain (nnwifi.com)
- Uses foreign domain (.com.co)
- Requests confirmation of ownership (credential harvesting pattern)

Answers to Given Questions

1. Full sender email address: sues@nnwifi.com
2. Domain used to send the email: nnwifi.com
3. Sender's IP address: 94.100.31.27
4. Is sender IP blacklisted? :Yes (1/93 vendors flagged it as malicious in VirusTotal)
5. SPF authentication result: Fail

6. One suspicious URL in email body:

https://servervirto.com.co/ed/trn/update?email=brad@malware-traffic-analysis.net

Conclusion:-

Phishing indicators found:

- Urgent subject: "Warning: Final Notice"
- Fake display name pretending to be malware-traffic-analysis.net
- Actual sender domain is unrelated (nnwifi.com)
- SPF authentication failed
- Message-ID missing
- HTML-only email
- Suspicious external link
- IP partially flagged by VirusTotal

Final Verdict:-

This email is classified as PHISHING.

Experiment 12: Packet Sniffing and Network Traffic Analysis with Wireshark

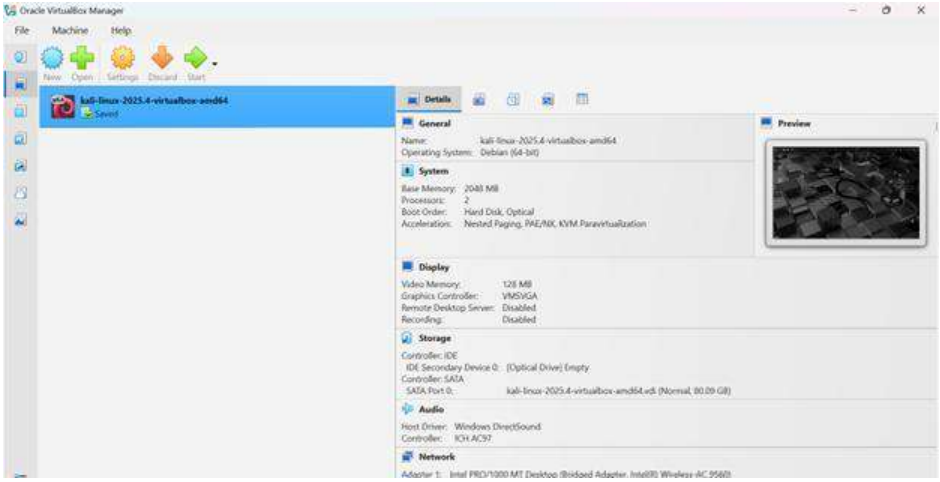
In this experiment, capture live network packets, analyze them, and understand what information an attacker can see, a local HTTP server running on port 8080 is used. Since HTTP is not encrypted, all transmitted data can be viewed in plain text by anyone who captures the traffic. Tools like tcpdump are used to capture the packets, and Wireshark is used to analyze them.

Requirements

- Kali Linux (in Oracle VirtualBox) [comes with build in Wireshark]
- Local HTTP Server (Python)

Procedure:

Step 1: Open Kali Linux.



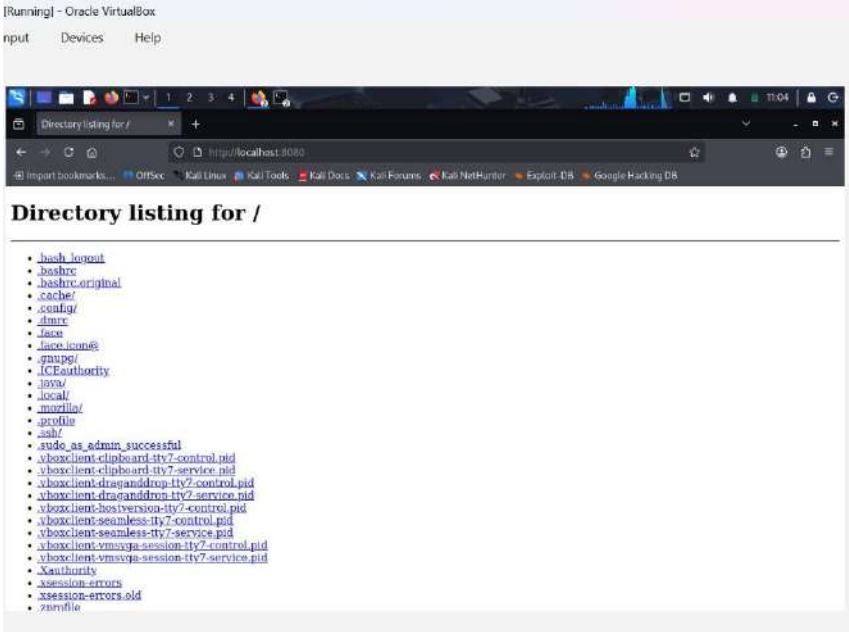
Step 2: Open Terminal in Kali Linux.

Step 3:

Start a local HTTP server on port 8080 using :
python3 -m http.server 8080

Step 4: In another new terminal start packet capture: sudo
tcpdump -i any -w capture.pcap port 8080

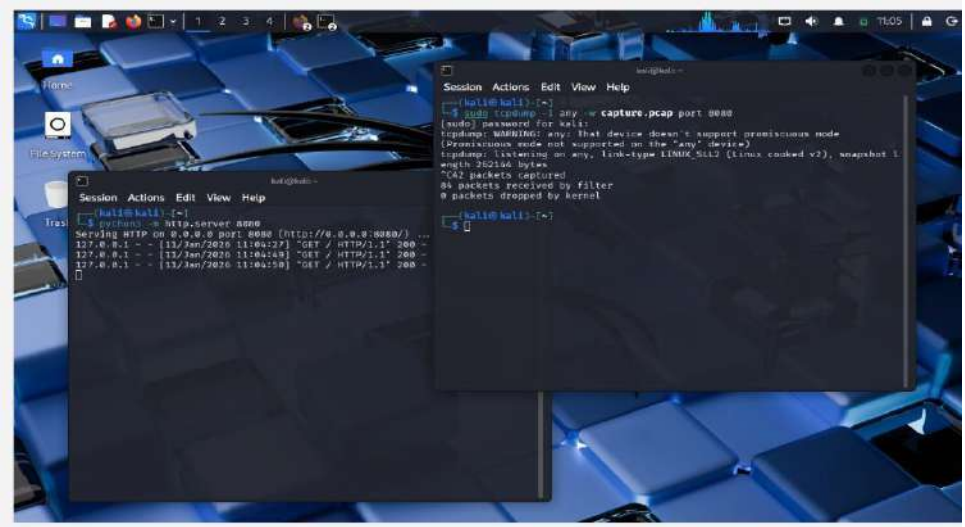
Step 5: Open Firefox in kali Linux and go to:
http://localhost:8080
Refresh the page to generate traffic.



Step 6:

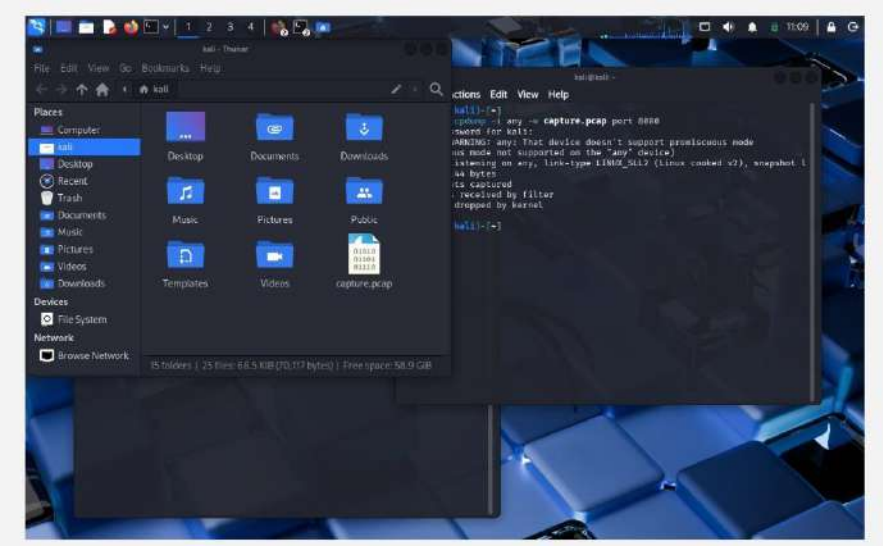
Go back to tcpdump terminal.
Stop packet capturing by using Ctrl + C.
It will stop and show how many packets were captured

The packets are saved as capture.pcap.



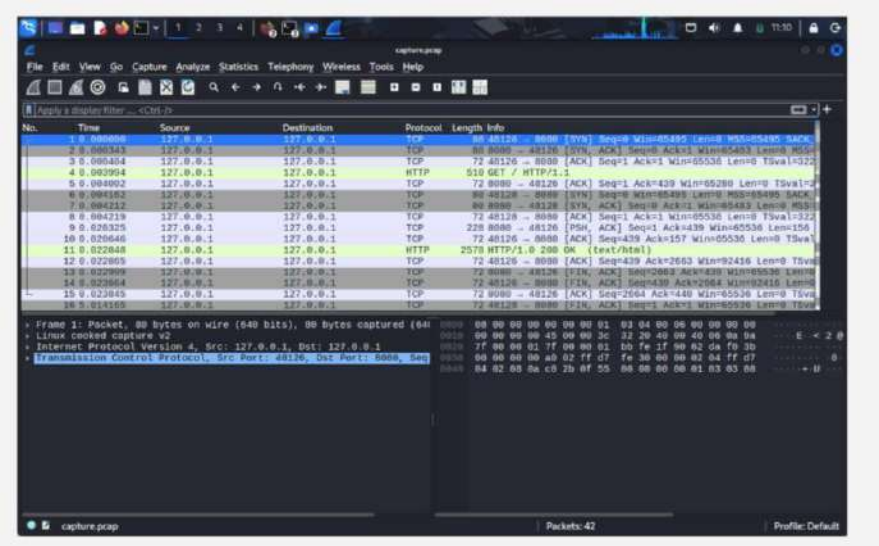
Step 7:

- Click the Kali Linux dragon icon (top left).
- Type: File Manager and open it.
- Your Home folder will open.
- You will see the file: capture.pcap.



Step 8:-

- Since Wireshark is pre-installed in Kali, just double-click capture.pcap.
- The file will open directly in Wireshark for analysis.



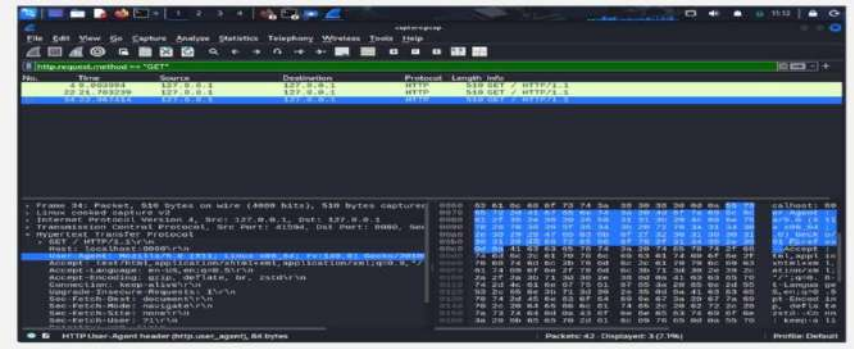
Step 9:-

Filter Login Packets

In Wireshark filter bar, type: http.request.method == "POST"

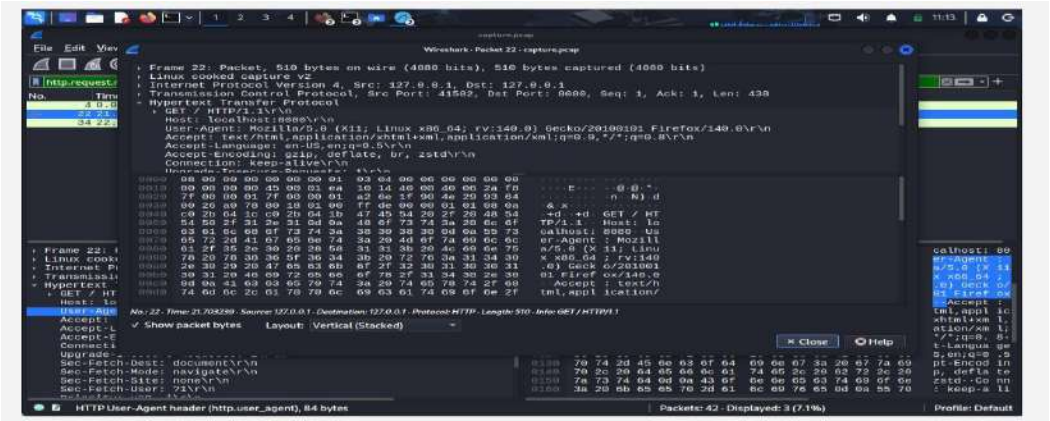
Press Enter.

Now only important packets will show.



Step 10:-

Click on any one of the packet and the following data is displayed.
Browser details such as OS, browser version, language, and visited URLs are visible.If a form is



submitted, username and password can be seen in plain text.This proves HTTP is insecure

Conclusion :-

Packets were successfully captured and analyzed. Sensitive data is visible when HTTP is used.Packet sniffing and network traffic analysis show that unencrypted communication is unsafe. HTTPS is necessary to protect data.

Experiment 13: Setting up a Simple IDS using Snort

Aim: To install and configure Snort as a simple Intrusion Detection System (IDS) on Windows and test detection using Nmap from Kali Linux

Tools Used

- Windows 10/11 Machine
- Npcap (Packet Capture Driver)
- Snort 2.9 (Windows Version)
- Kali Linux (Attacker Machine)
- Nmap Tool

Network Setup

Windows Machine (Snort IDS) IP: 192.168.0.104
Kali Linux Machine (Attacker) IP: 192.168.0.X
Both systems connected in the same local network.

Procedure

1. Step 1: Install Npcap and enable WinPcap API-compatible mode.
2. Step 2: Download and install Snort to C:\Snort.
3. Step 3: Configure snort.conf with correct paths for dynamic modules and log directory.
4. Step 4: Create local.rules inside C:\Snort\rules and add detection rules.
5. Step 5: Run Snort using: snort -i <interface_number> -c C:\Snort\etc\snort.conf -A console.
6. Step 6: Perform SYN scan from Kali using: nmap -sS -T4 192.168.0.104(windows machine IP address).

Custom Rules Used:-

ICMP Detection Rule: alert icmp any any -> any any (msg:"Ping Detected"; sid:1000001; rev:1;)

SYN Scan Detection Rule: alert tcp any any -> $HOME_NET any (msg:"Possible Port Scan"; flags:S; threshold:type threshold, track by_src, count 5, seconds 10; sid:1000002; rev:1;)

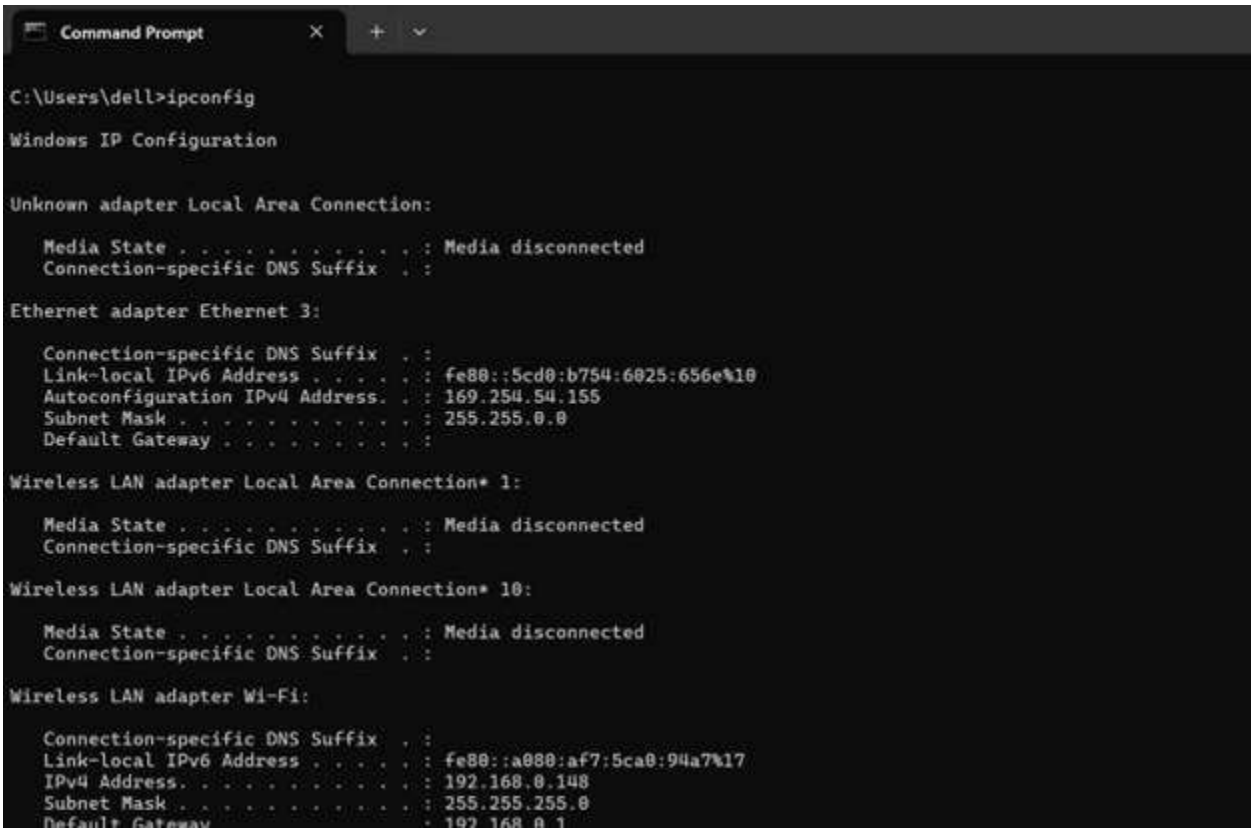
Testing and Observations:-

- Snort initialized successfully and began packet processing.
- Ping command triggered ICMP alert in Snort console.
- Nmap SYN scan triggered port scan detection alert.
- Snort logged alerts in the configured log directory.

Output :-

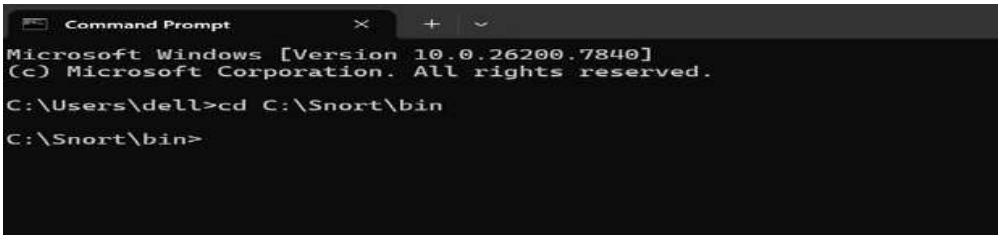
Title: Verifying IP Configuration of Windows Machine

- This screenshot shows the ipconfig command output on the Windows machine.
  It confirms that the system has a valid IPv4 address (192.168.0.148) and is connected to the local network.
  This IP address is later used as the target system for IDS monitoring and Nmap scanning.



Title: Navigating to Snort Installation Directory

This screenshot shows the Command Prompt navigating to the Snort installation directory (C:\Snort\bin).
This directory contains the Snort executable required to run the IDS in console mode.

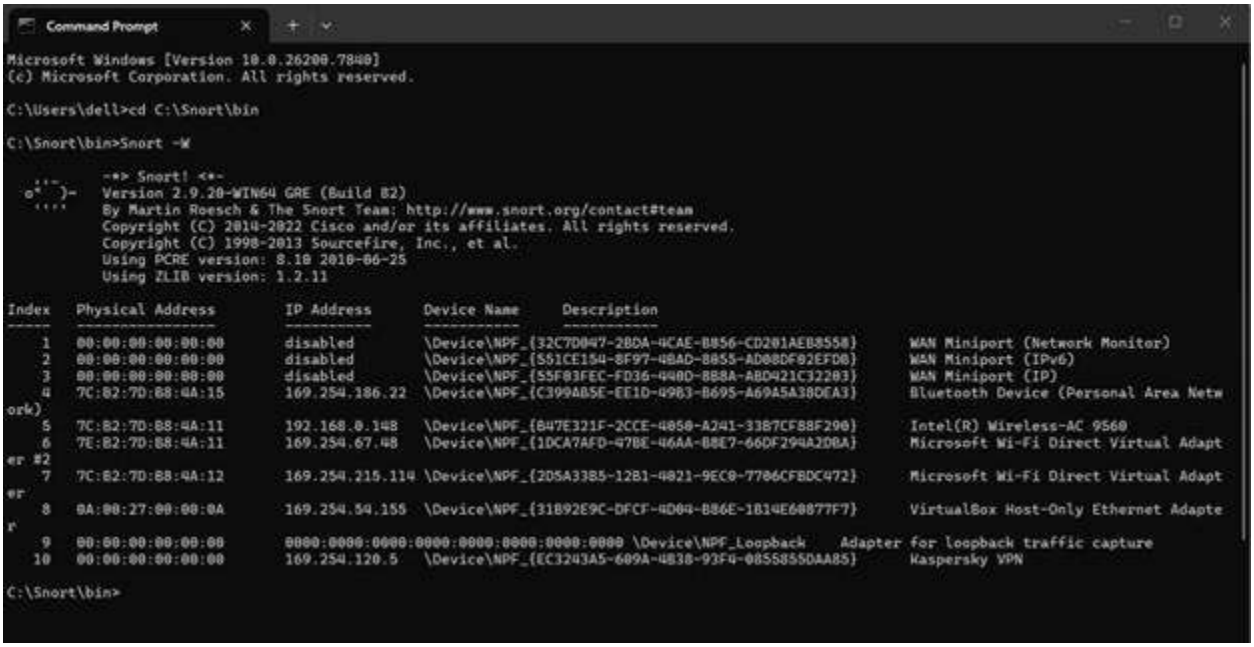


Title: Listing Available Network

Interfaces Using Snort

Description:

The command snort -W is executed to list all available network interfaces.
This helps identify the active network adapter number required to start Snort in IDS mode.
The correct interface number is selected for monitoring live network traffic.



Title: Running Snort in IDS Mode

Snort is started in IDS mode using the command:

snort -i 5 -c C:\Snort\etc\snort.conf -A console

The screenshot shows successful initialization of Snort including:

- Loading preprocessors
- Loading rules
- Initializing detection engine
- Starting packet processing

This confirms that the IDS is running successfully.

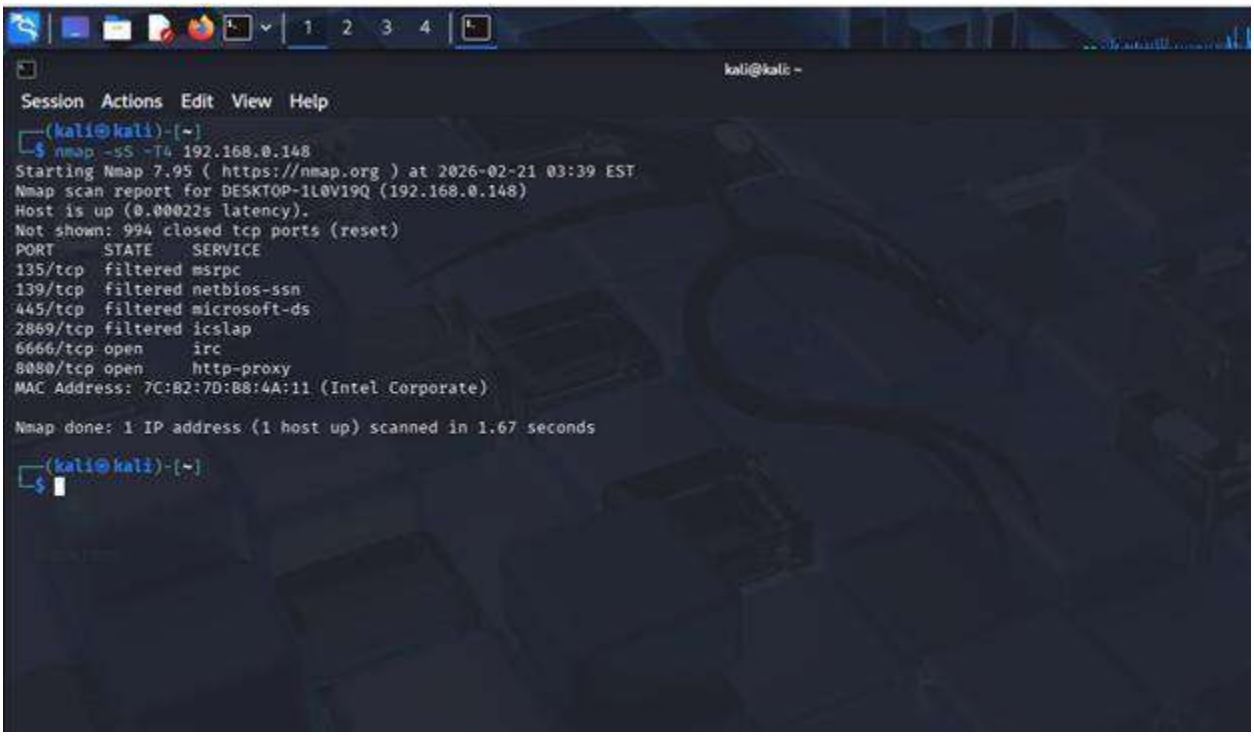
```
C:\Snort\bin>snort -i 5 -c C:\Snort\etc\snort.conf -A console
Running in IDS mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "C:\Snort\etc\snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001
7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 9080 9090:9091
9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
PortVar 'SSH_PORTS' defined :  [ 22 ]
PortVar 'FTP_PORTS' defined :  [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined :  [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined :  [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2381 2381 2809 3037 3128 3702 4343 4848 5250 69
88 7000:7001 7144:7145 7510 7777 7779 8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8888 8899 9000 9060 90
80 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined :  [ 2123 2152 3386 ]
Detection:
   Search-Method = AC-Full-Q
    Split Any/Any group = enabled
    Search-Method-Optimizations = enabled
    Maximum pattern length = 20
```

Title: Performing SYN Scan from Kali Linux Using Nmap

This screenshot shows the attacker machine (Kali Linux) performing a SYN scan using:

nmap -sS -T4 192.168.0.148

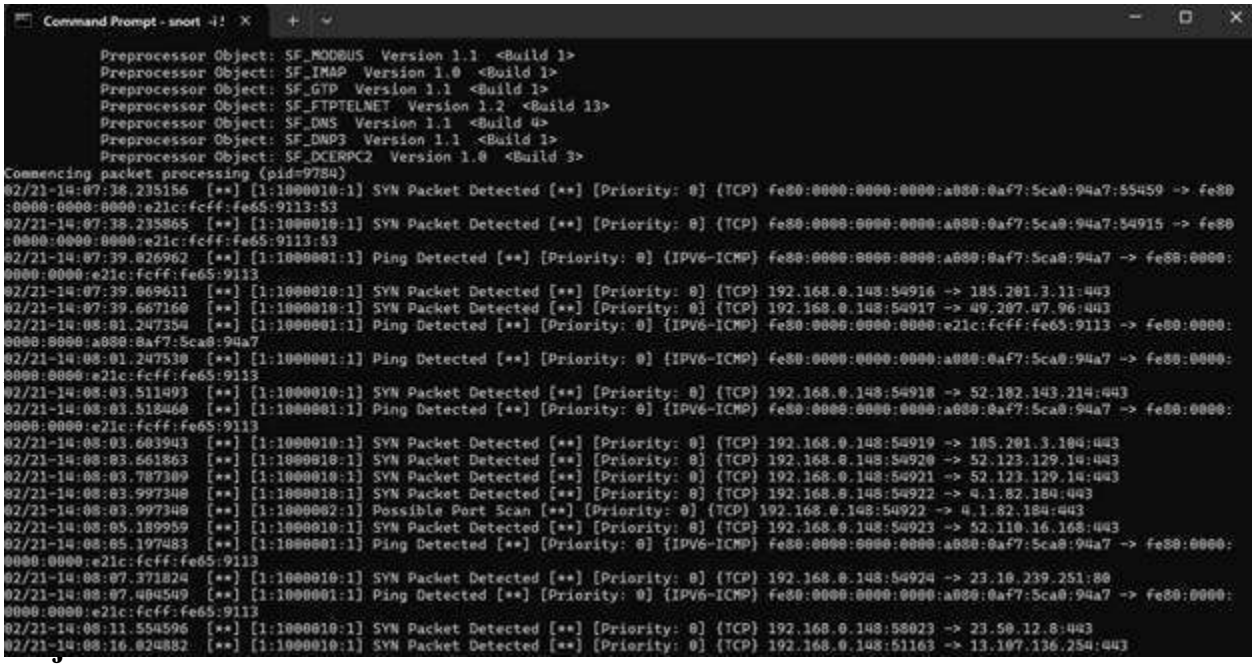
The scan checks for open and filtered TCP ports on the Windows machine.
This simulates a real-world reconnaissance attack.



Title: Snort Detecting SYN Scan and Generating Alerts

This screenshot shows Snort detecting suspicious TCP SYN packets generated by the Nmap scan.

The IDS generates alerts such as:
SYN Packet Detected
This confirms that Snort successfully monitored live network traffic and detected port scanning activity.



- Perform automated vulnerability scanning.
- Identify common vulnerabilities (XSS, SQL Injection, etc.).
- Interpret alerts and generate a report.

Experiment 14: Web Application Vulnerability Scanning with OWASP ZAP

This lab demonstrates **dynamic application security testing (DAST)** using OWASP ZAP against a deliberately vulnerable web application.

Objective

- Perform automated vulnerability scanning.
- Identify common vulnerabilities (XSS, SQL Injection, etc.).
- Interpret alerts and generate a report.

Lab Environment Setup

Step 1: Install OWASP ZAP on Windows

1. Go to the official site
   https://www.zaproxy.org/download/
2. Download the **Windows Installer**
3. Run the installer and follow the setup.

After installation open:
OWASP ZAP

Step 2: Install Node.js

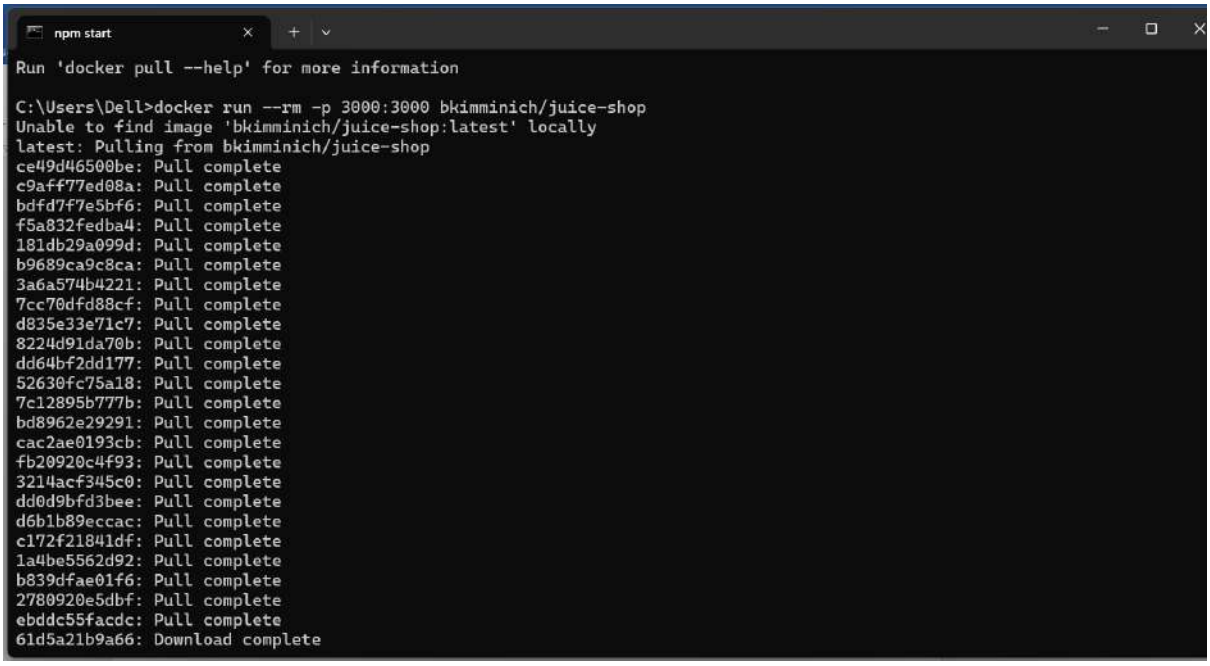
Juice Shop runs using **Node.js**.

1. Go to
   https://nodejs.org
2. Download **LTS version**
3. Install with default settings.

Step 3: Install OWASP Juice Shop

Open **Command Prompt**.
Run:
docker run --rm -p 3000:3000 bkimminich/juice-shop

OR
docker run -d -p 3000:3000 bkimminich/juice-shop



Go to the folder:

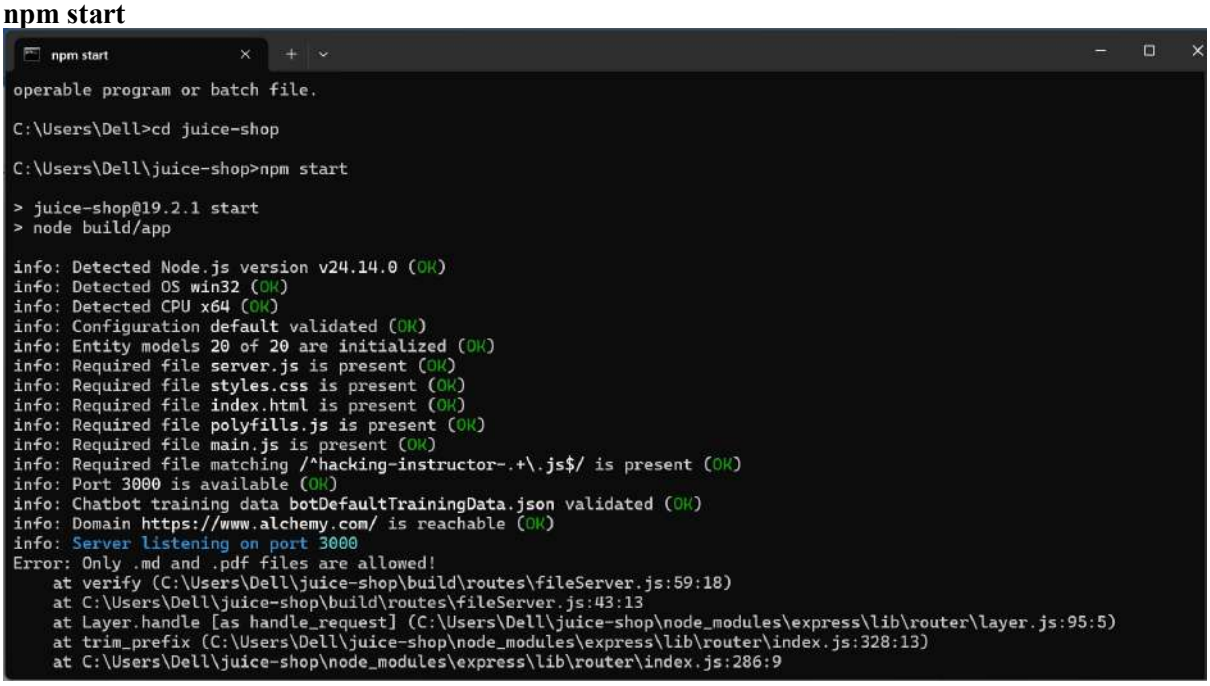
cd juice-shop

Install dependencies:

npm install

Start the application:

npm start

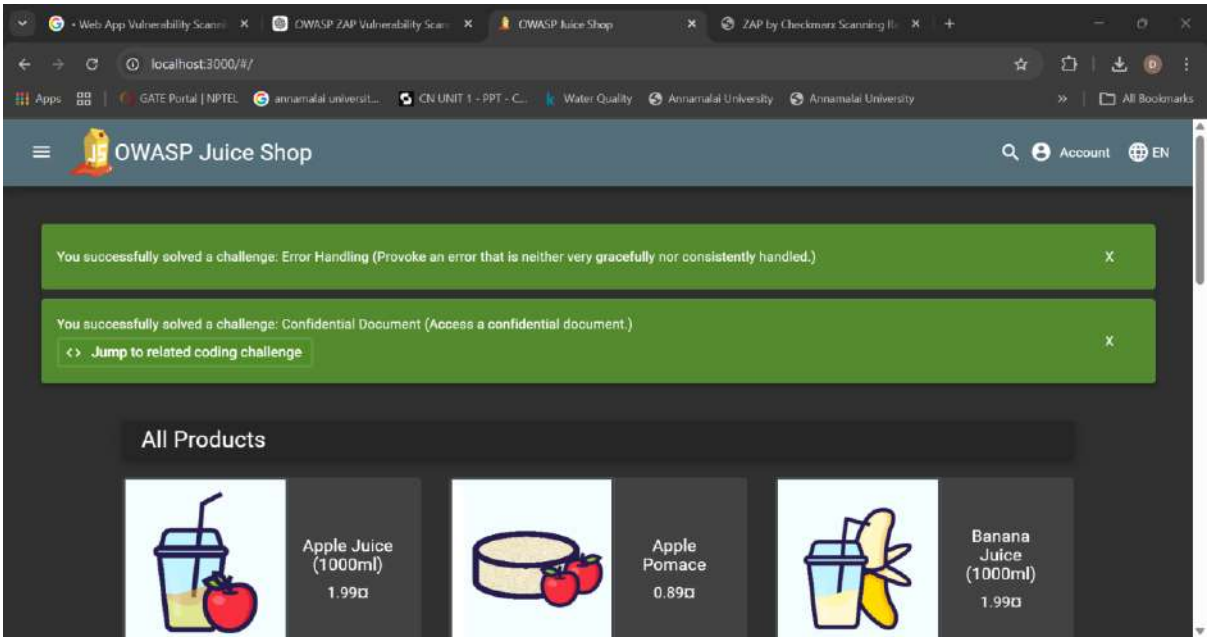


Step 4: Open Juice Shop

Open browser and go to:

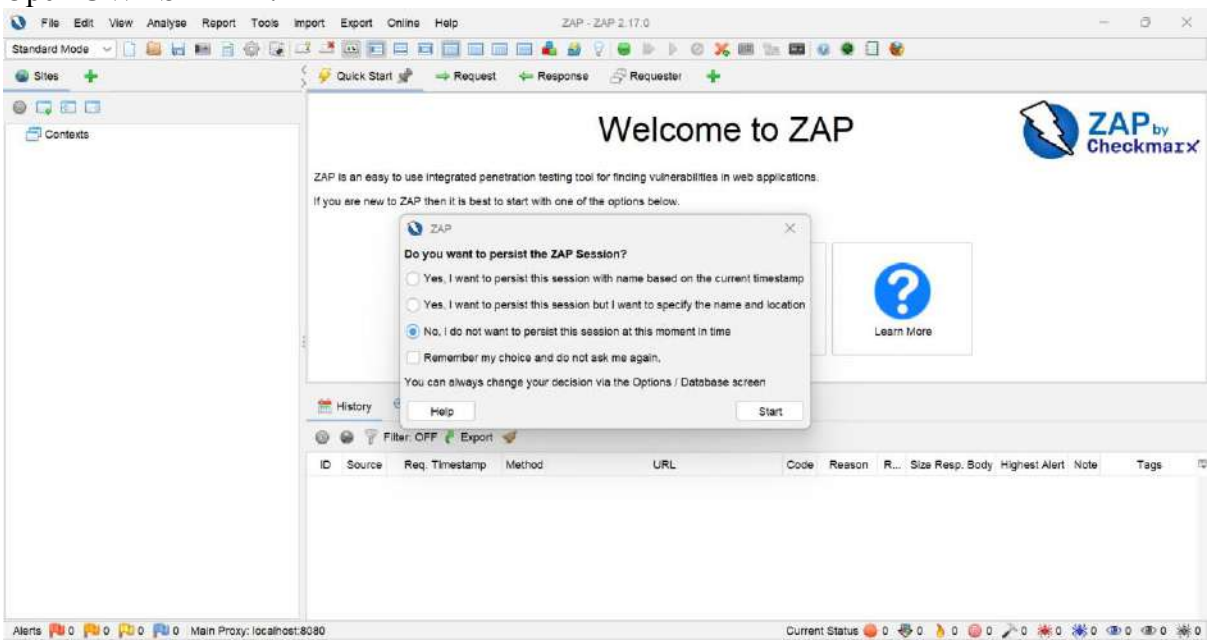
http://localhost:3000

You will see the **Juice Shop web application**.



Step 5: Start OWASP ZAP

Open **OWASP ZAP**.



When prompted choose:

No, I do not want to persist this session

Click **Start**.

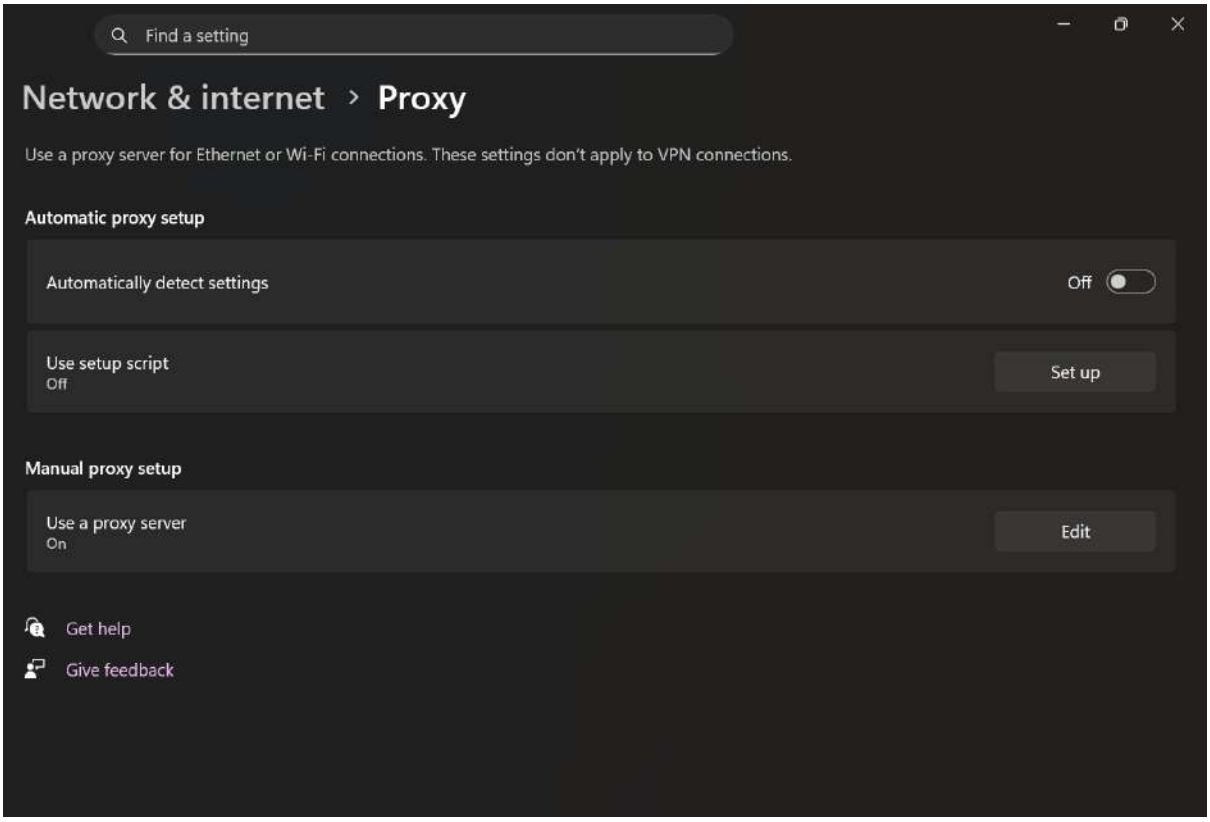
Click on automated scan

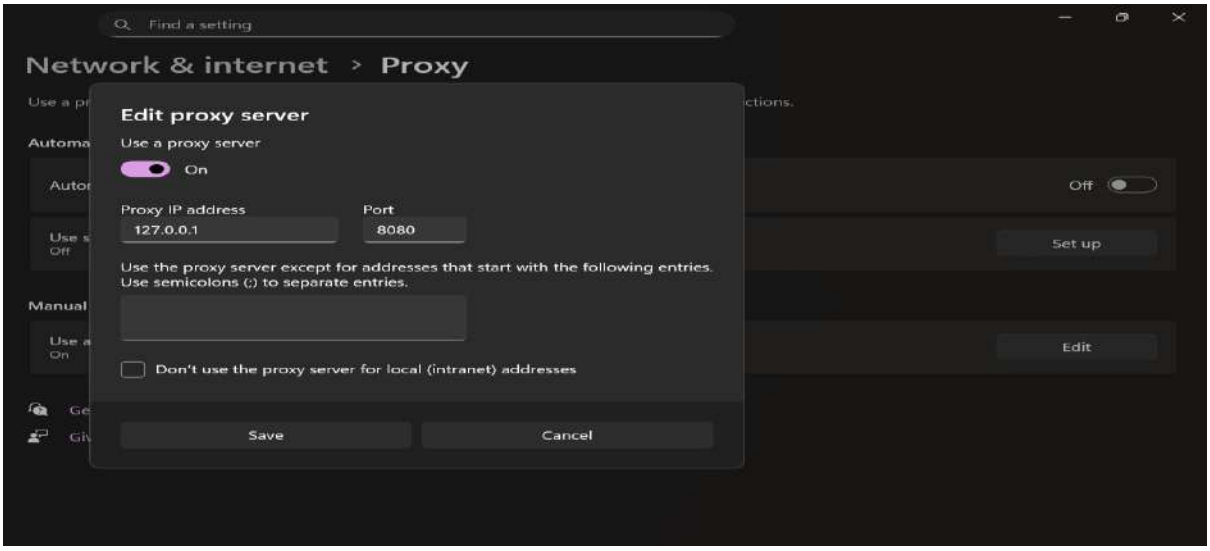
Step 6: Configure Browser Proxy

In browser proxy settings configure:

HTTP Proxy : 127.0.0.1

Port : 8080





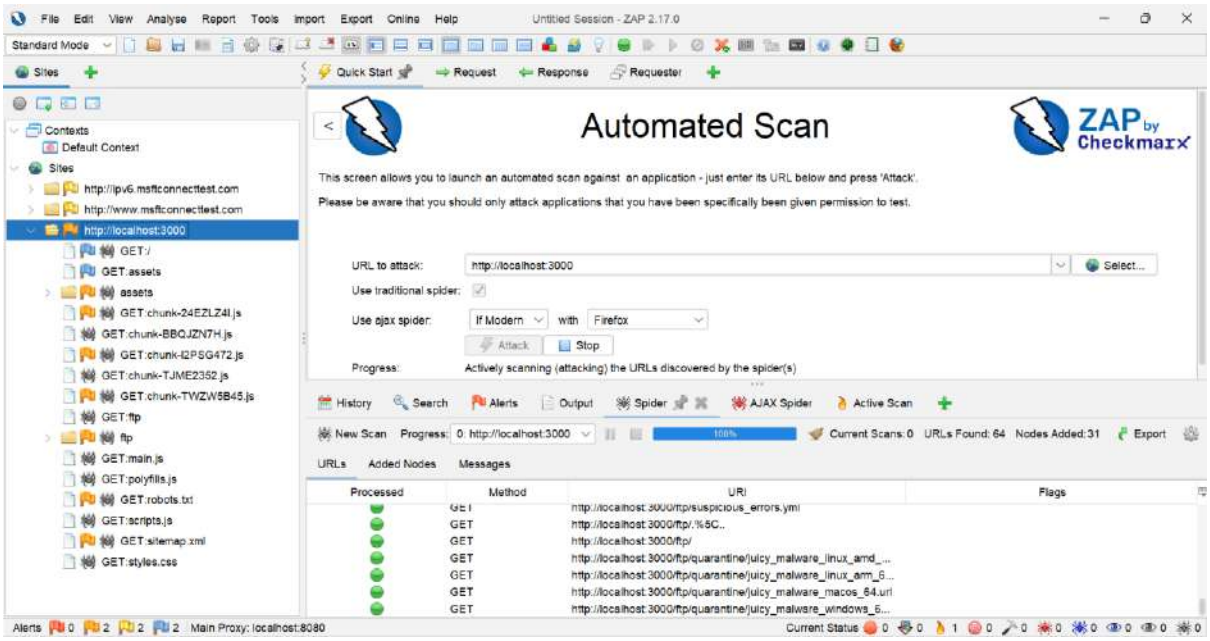
Now browser traffic passes through **ZAP**.

Step 7: Confirm Connection

Open:
http://localhost:3000
In ZAP **Sites panel** you will see:
http://localhost:3000

Step 8: Perform Spider Scan

Right-click:
http://localhost:3000
Select:
Attack → Spider
Click **Start Scan**.



ZAP will discover all pages.

Step 9: Perform Active Scan

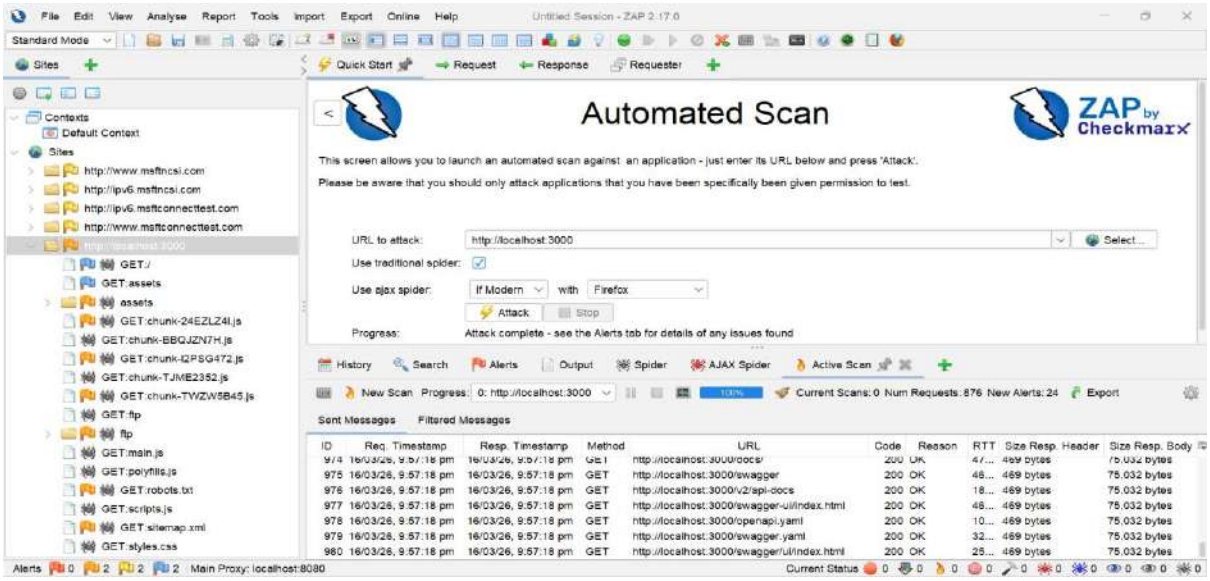
Right-click again:

http://localhost:3000

Select:

Attack → Active Scan

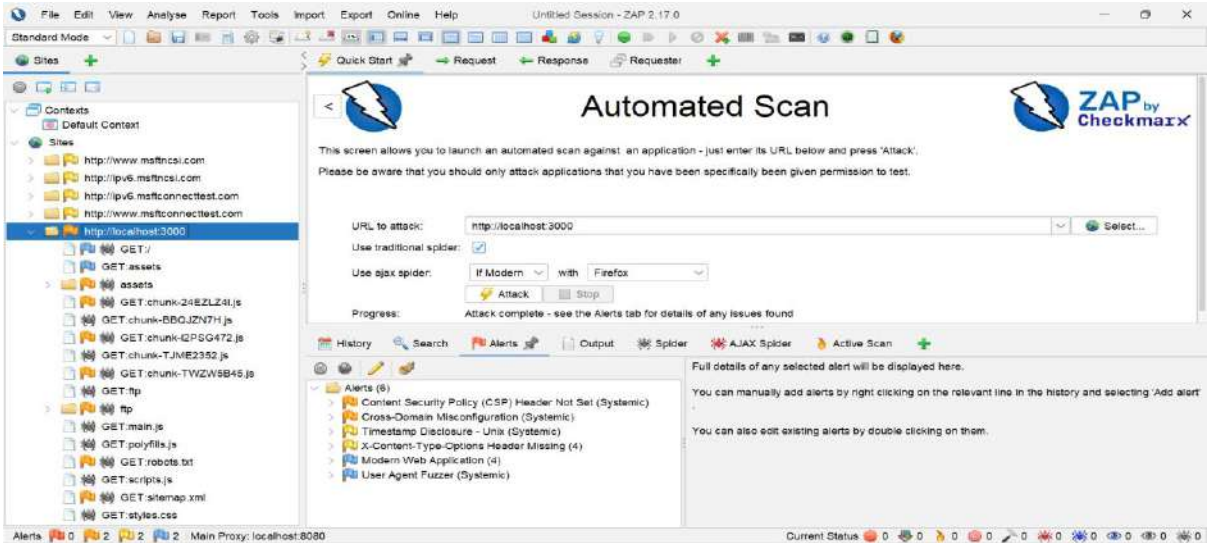
Click **Start Scan**.



ZAP will test vulnerabilities.

Step 10: Analyze Vulnerabilities

Open the **Alerts tab**.



You may see vulnerabilities like:

- Cross Site Scripting
- Missing security headers
- Cookie security issues
- Information disclosure

Analysing Results

Step 11: View Alerts

Go to:

Alerts Tab

Alerts are categorized by risk:

Risk Level	Meaning
High	Critical vulnerability
Medium	Exploitable issue
Low	Minor issue
Informational	No direct risk

Click any alert to view:

- Description
- Evidence
- Attack payload used
- Suggested remediation

Step 12: Export Scan Report

1. Go to **Report**
2. Choose format:
    - HTML
    - PDF
    - XML
3. Select:
    - Risk levels
    - Confidence level
4. Generate report

Save file.

Result

The vulnerable web application was scanned using **OWASP ZAP**, and multiple vulnerabilities were identified in **OWASP Juice Shop**.

Experiment: 15 Analysing Android App Permissions and Mobile Traffic

1. Aim

To analyse Android application network traffic and permissions by intercepting HTTP and HTTPS requests using an Android Emulator and Burp Suite.

2. Objective

- To create and run an Android Virtual Device (AVD).
- To configure a proxy between the Android Emulator and Burp Suite.
- To capture HTTP traffic from the Android device.
- To intercept and analyse HTTPS traffic by installing the Burp Suite CA certificate.

3. Required Tools / Software

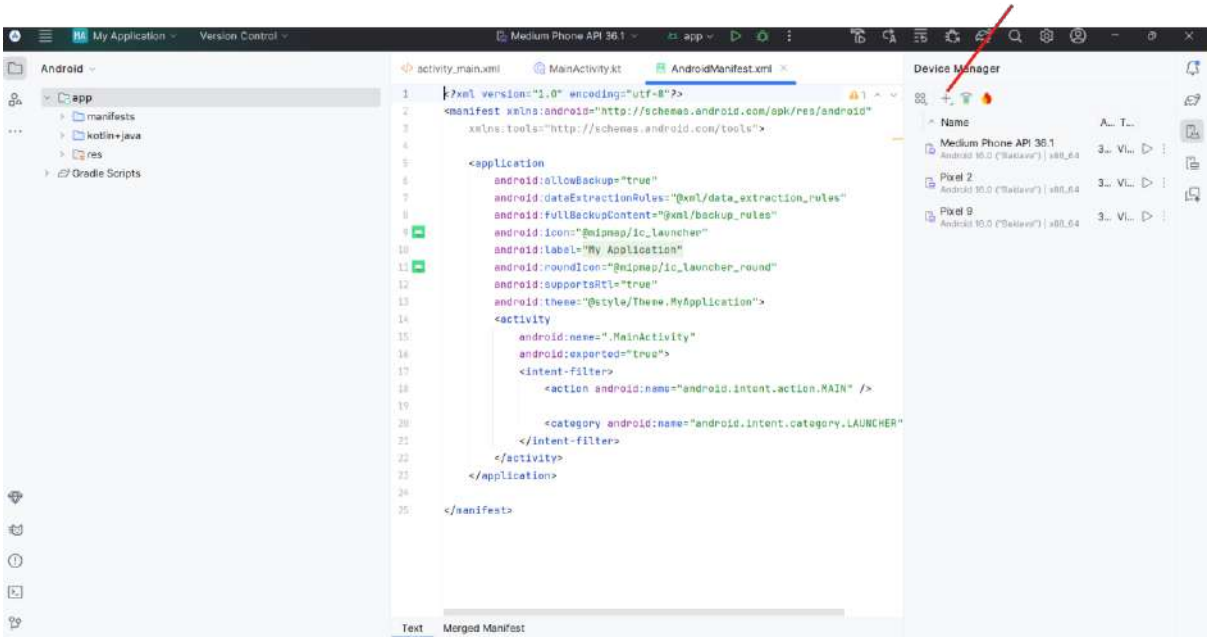
- Android Studio
- Android Emulator (AVD Manager)
- Burp Suite
- ADB (Android Debug Bridge)
- Google Chrome (inside emulator)
- Internet connection

4. Procedure

Step 1: Create Android Virtual Device

1. Install **Android Studio**.
2. Open **AVD Manager**.
3. Create a new Virtual Device.

Step-1: Download Android Studio->Create Virtual Device

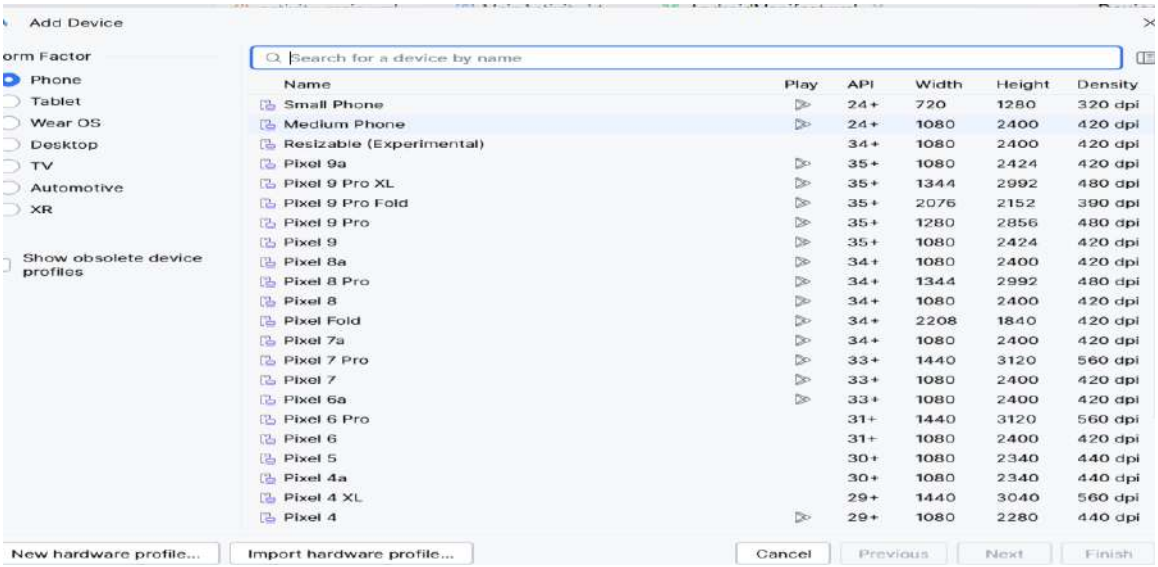


Step 2: Select Device

Choose any device such as:

- Pixel 2

- Pixel 9



Step-3: Start AVD from Command Prompt1

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\emulator
C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -list-avds
Medium_Phone_API_36.1
Pixel_2
Pixel_9
C:\Users\Admin\AppData\Local\Android\Sdk\emulator>emulator -avd Pixel_9 -http-proxy http://10.0.2.2:8080

Step-4: Setup Proxy:

In AVD

Settings-> Network and Internet-> internet->select icon -> top right corner there is edit->advanced options -> proxy->Change proxy None->Manual->Hostname as 10.0.2.2 ->save

Note:if save option is not working then follow the steps in command prompt

Command Prompt2:

C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices

List of devices attached

emulator-5554 device

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080

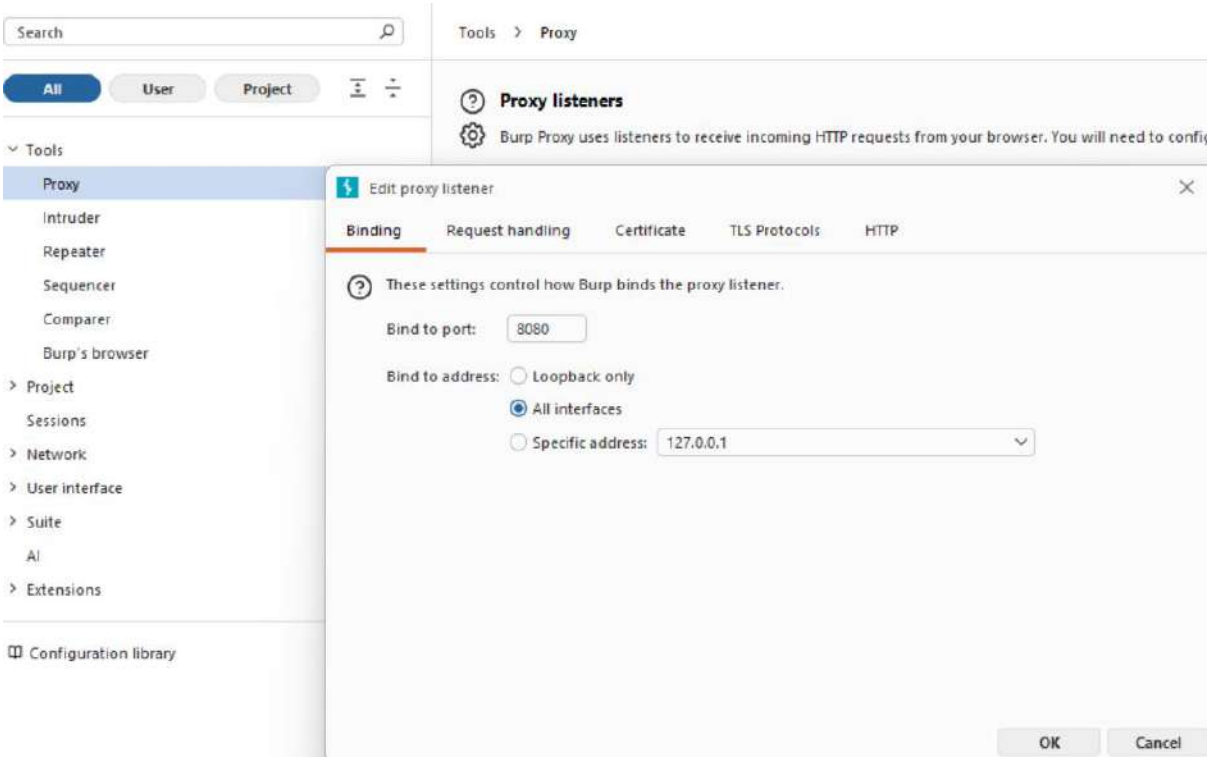
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy

10.0.2.2:8080

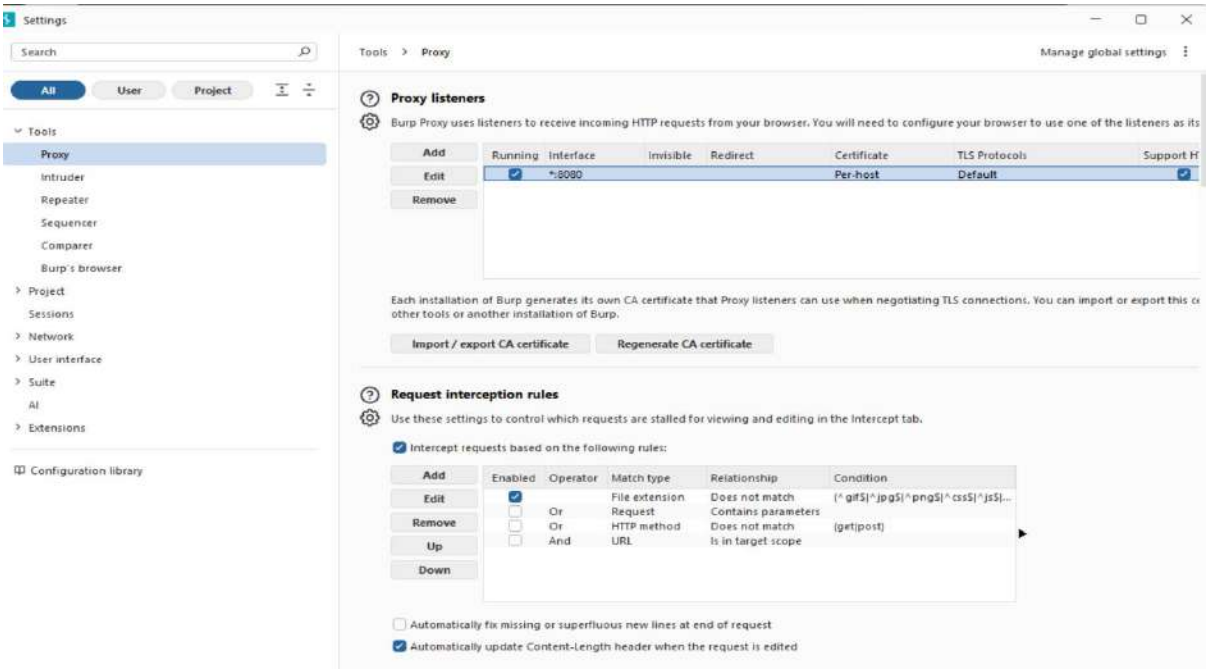
```
C:\Users\Admin>cd C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb devices
List of devices attached
emulator-5554   device


C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings put global http_proxy 10.0.2.2:8080

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb shell settings get global http_proxy
10.0.2.2:8080

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>
```

Step 5: In BurpSuit

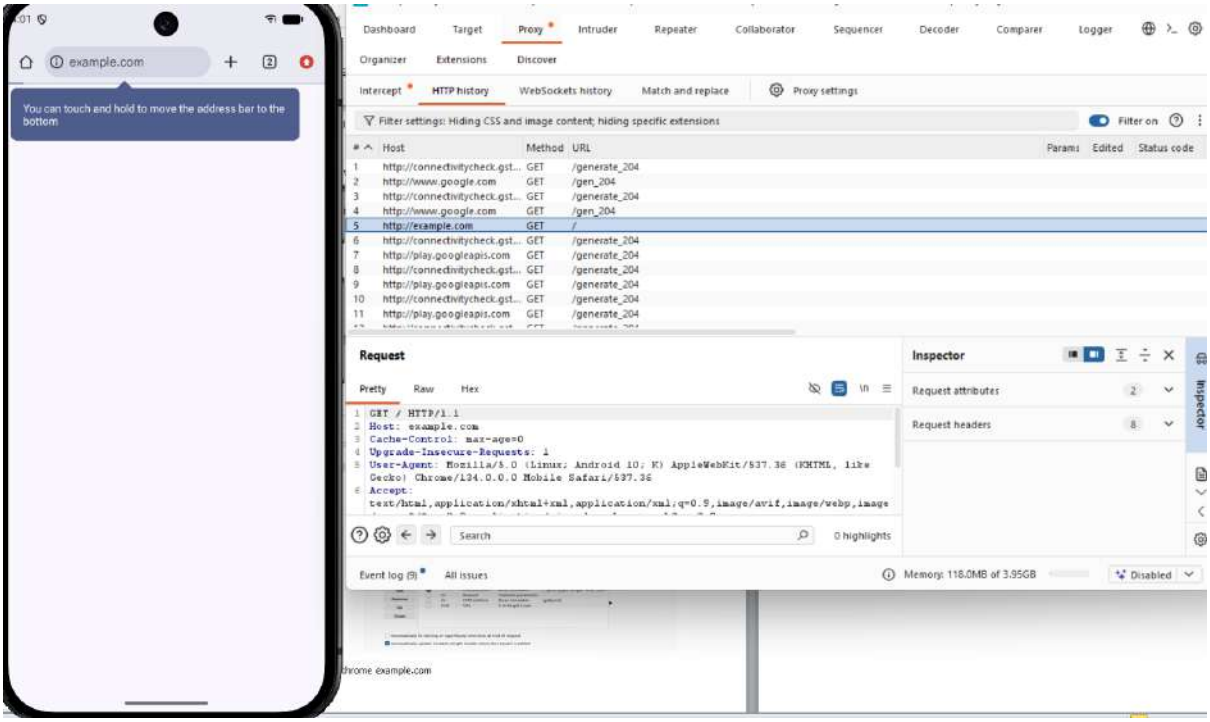
Go procy->options/proxy settings->under Proxy Listeners->Edit->select All interfaces->ok
Go proxy->intercept on



You can see BurpSuit is accepting all interfaces traffic

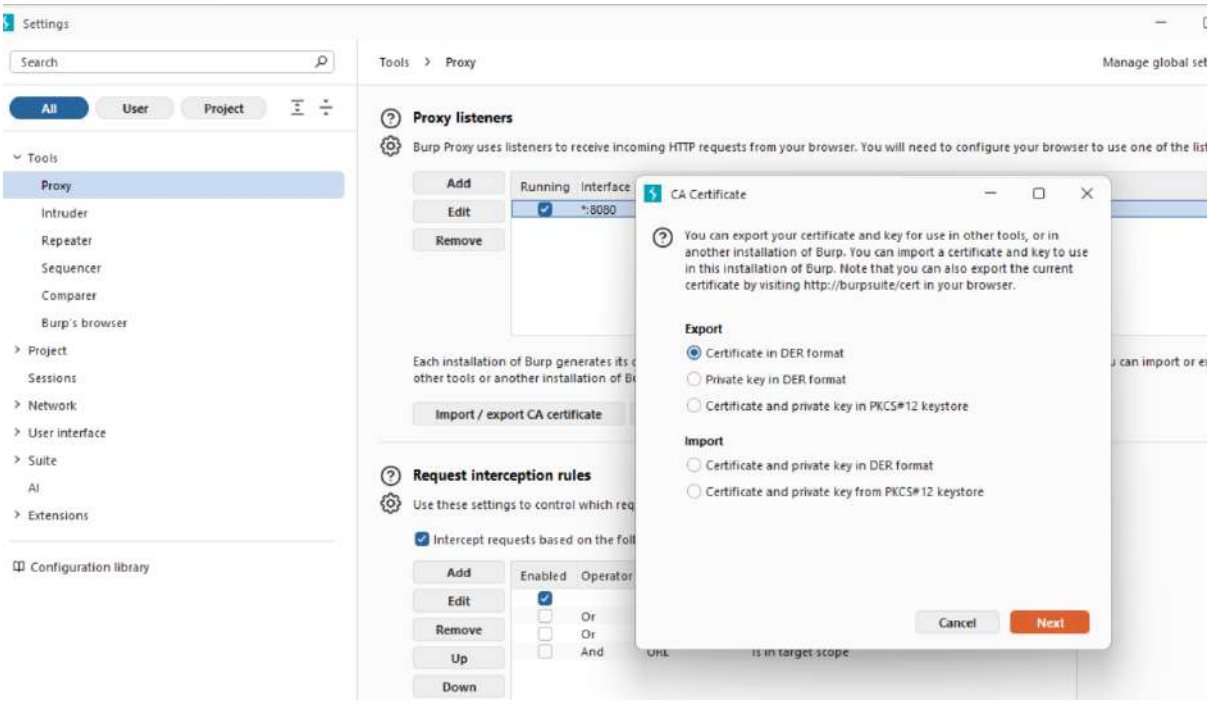


Step 6: in AVD type in chrome example.com and you can see burpsuit HTTP history



Step 7: in Step 5 we only able to http request and response only but if you want to analyse https request and response follow the Procedure to download CA certificate from BunrpSuit.

Go procy->options/proxy settings->import/export CA certificate->Certificate in DER format->select file-> Downloads->give name as burpcer.der



Step 8: Sending CA certificate to AVD

Go to command Prompt2

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/

```
C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>adb push C:\Users\Admin\Downloads\burpcer.cer /sdcard/Download/
C:\Users\Admin\Downloads\burpcer.cer: 1 file pushed, 0 skipped. 0.0 MB/s (939 bytes in 0.022s)

C:\Users\Admin\AppData\Local\Android\Sdk\platform-tools>
```

To install Certificate in AVD

Settings

→ Security & privacy

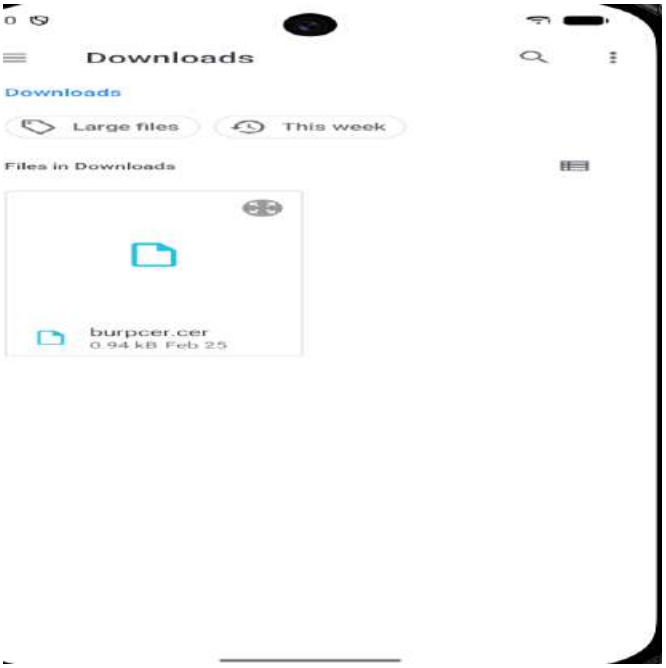
→ More security & privacy

→ Encryption & credentials

→ Install a certificate

→ CA certificate

In AVD we can see burpcer.der certificate



Step 9:verify HTTPS Interception

1. Open browser in emulator.
2. Visit any HTTPS website.

Example:
https://example.com or https://wikipedia.com

3. In Burp Suite, check **Proxy → HTTP History**.

Now HTTPS requests and responses will be visible.

5. Result

Android application network traffic was successfully intercepted and analysed using Burp Suite by configuring proxy settings in the Android emulator and installing the Burp Suite CA certificate.

6. Precautions

- Ensure Burp Suite proxy port (8080) is correctly configured.
- Verify the emulator is running before executing ADB commands.
- Install the CA certificate properly to capture HTTPS traffic.
- Disable intercept when not required to allow traffic to pass normally.

Experiment 16: Testing IoT Device Security (Default Passwords & Open Ports)

Aim

To analyze security weaknesses in a simulated IoT device by identifying:

- Open ports
- Running services
- Service version disclosure
- Default/weak authentication
- Unencrypted communication

Tools Used

- Windows 10 (Host System)
- Docker Desktop
- OWASP Juice Shop
- Kali Linux
- Nmap

Experimental Setup

- A vulnerable web application (Juice Shop) was deployed using Docker on Windows.
- Kali Linux was used as the attacker machine.
- The Windows system IP address was scanned from Kali Linux to simulate IoT network testing.

Attack Scenario Description

In this experiment:

- Kali Linux acted as the attacker machine.
- Windows system hosting the Docker container acted as the IoT device.
- The vulnerable web application (Juice Shop) simulated the IoT dashboard.
- Network scanning was performed from Kali to the Windows IP address over LAN.

This simulates a real-world scenario where an attacker scans an IoT device connected to the same network.

Procedure

Step 1: Deploy Vulnerable IoT Simulation

The following command was used to run Juice Shop in Docker:
docker run -d -p 8090:3000 --name juiceshop bkimminich/juice-shop

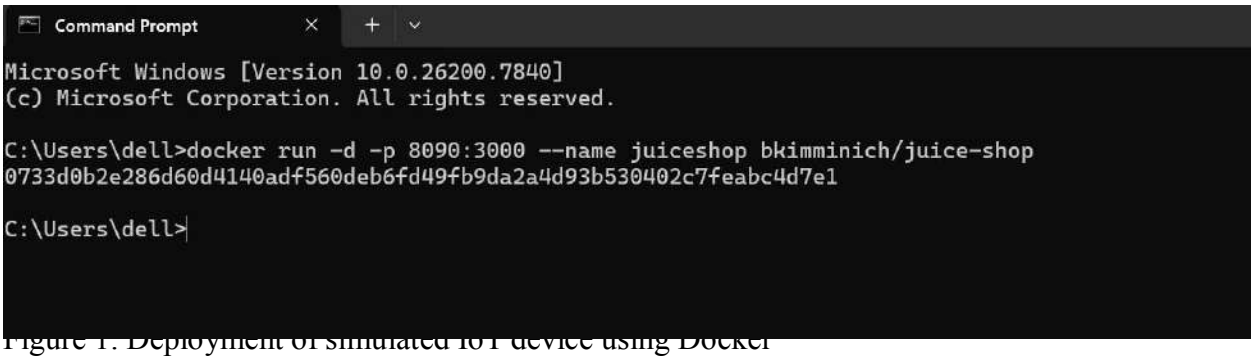


Figure 1: Deployment of simulated IoT device using Docker

The application was accessed at:

http://localhost:8090

Step 2: Identify Host IP Address

On Windows: ipconfig

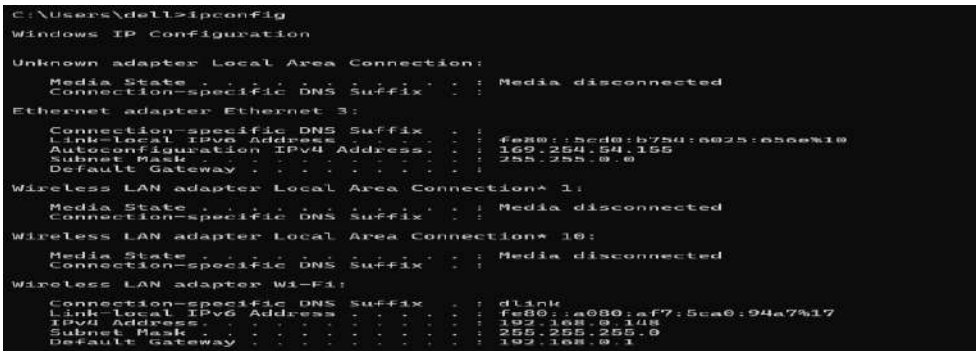


Figure 2: Host system IP address identification

Step 3: Perform Network Scan from Kali Linux

From Kali Linux terminal:

nmap -sV 192.168.0.148

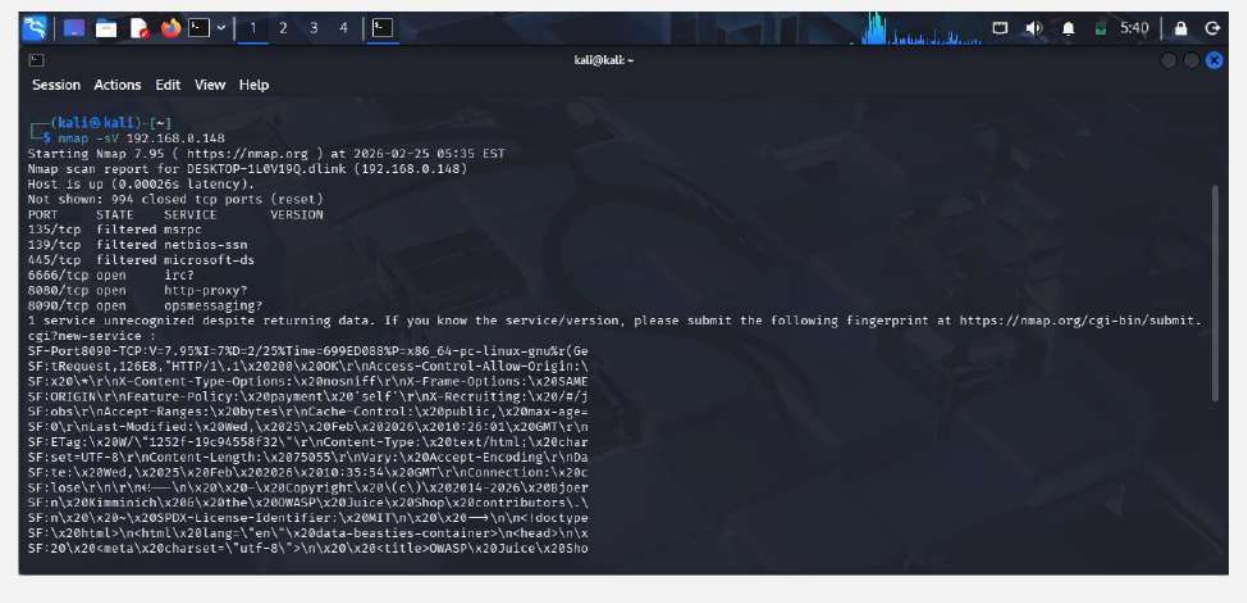
This scan performed:

- Port scanning
- Service detection
- Version detection
- Banner grabbing

Figure 3: Open ports and service enumeration using Nmap

Step 4: Nmap Version Scan (-sV)

- nmap -sV 192.168.x.x output
- Showing service fingerprint / version detection



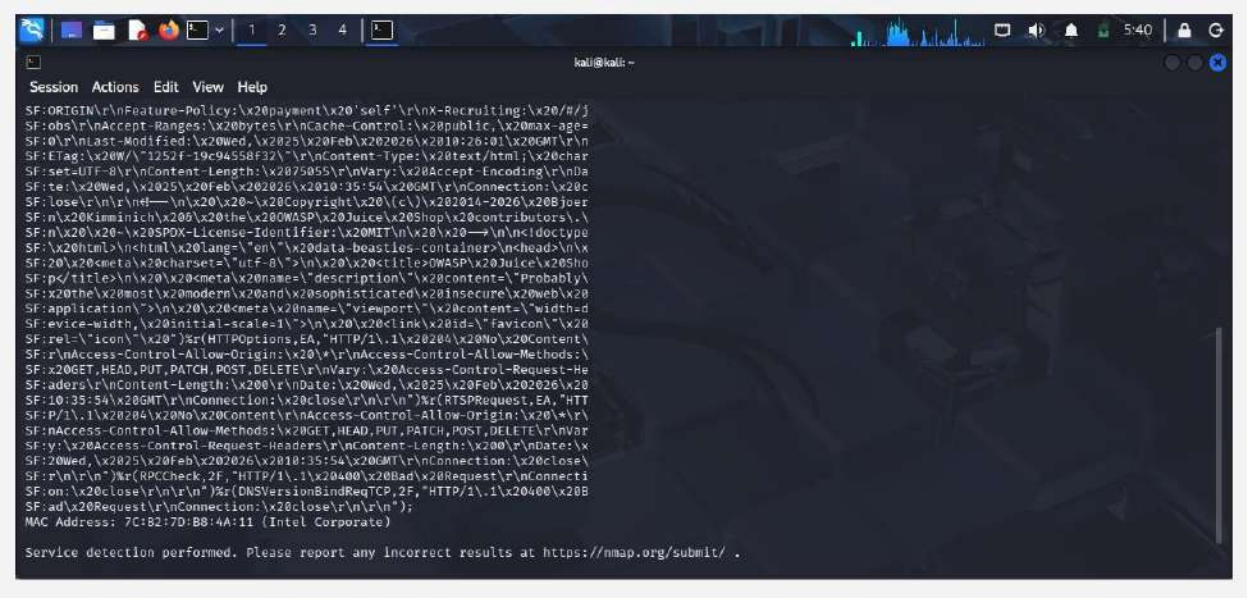


Figure 4: Service enumeration and version detection

Step 5: Accessing the IoT Device Web Dashboard

The simulated IoT device dashboard was accessed through the browser using: http://localhost:8090

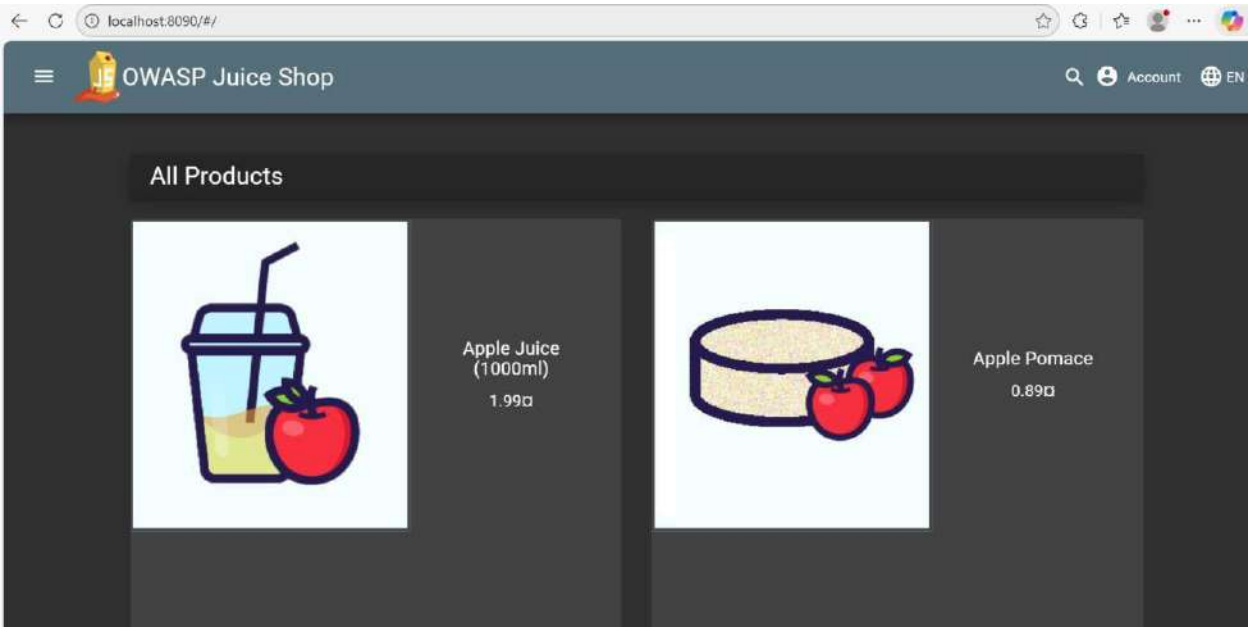


Figure 5: Simulated IoT device web dashboard running on port 8090

Step 6: Testing Default/Weak Credentials

The login functionality was tested using administrator credentials to check for default password vulnerability

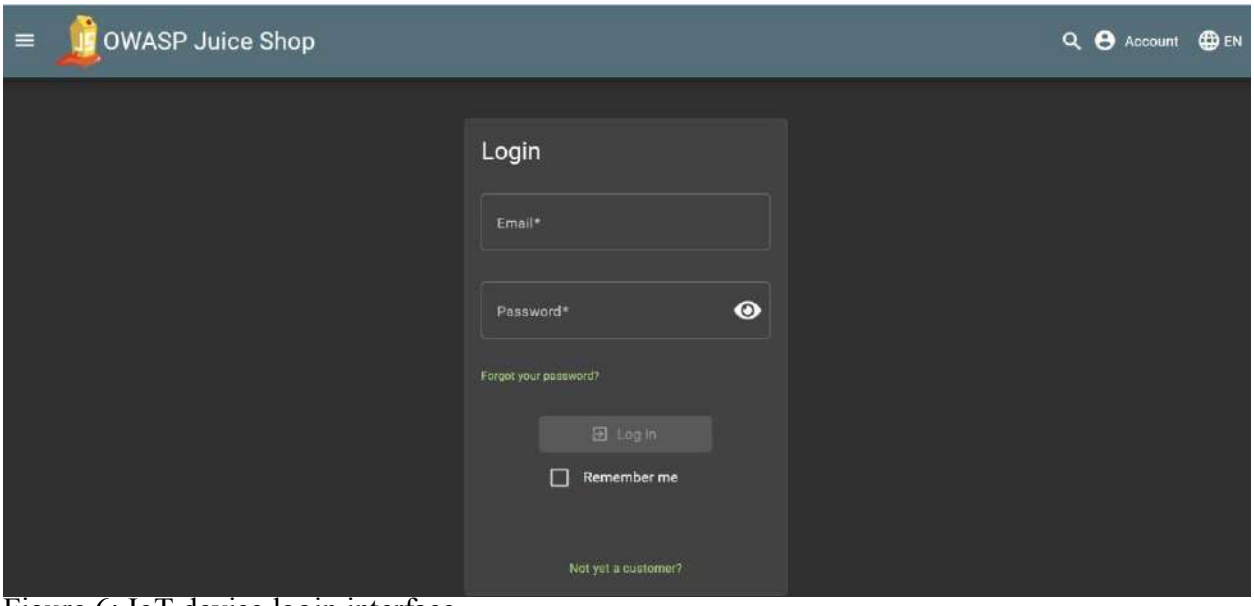


Figure 6: IoT device login interface

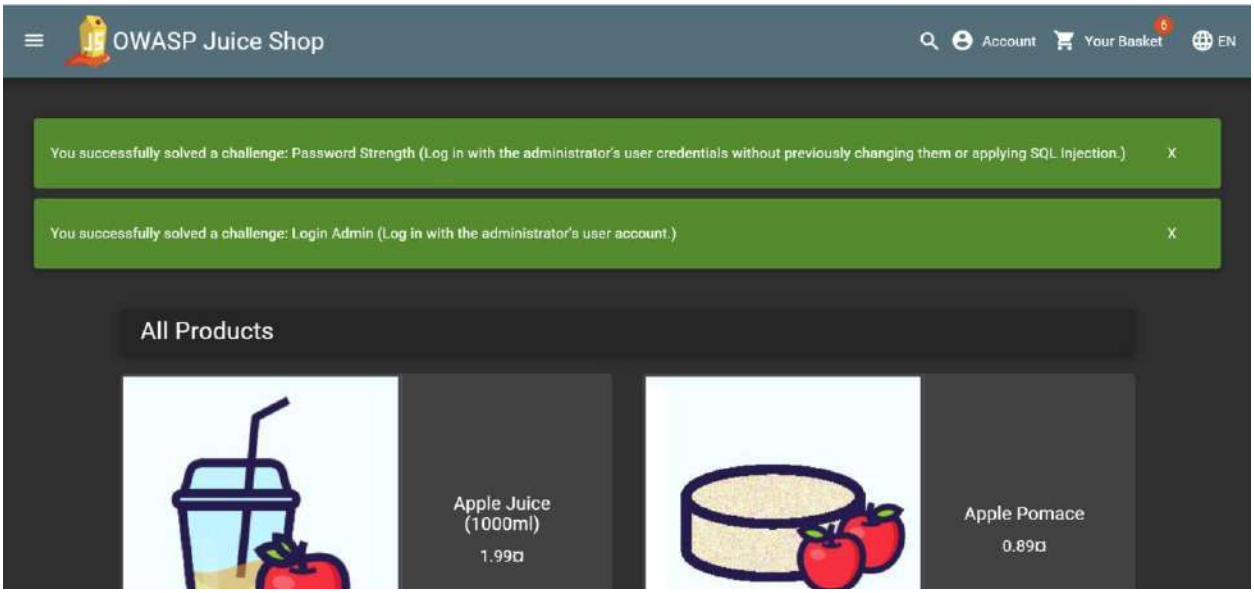


Figure 7: Successful login using default/weak credentials

Successful login using administrator credentials demonstrates the risk of default or weak passwords in IoT devices. If default credentials are not changed after deployment, attackers can gain unauthorized administrative access.

Step 7: Developer Tools → Network Tab

- Network tab showing HTTP requests
- Plain text traffic visible

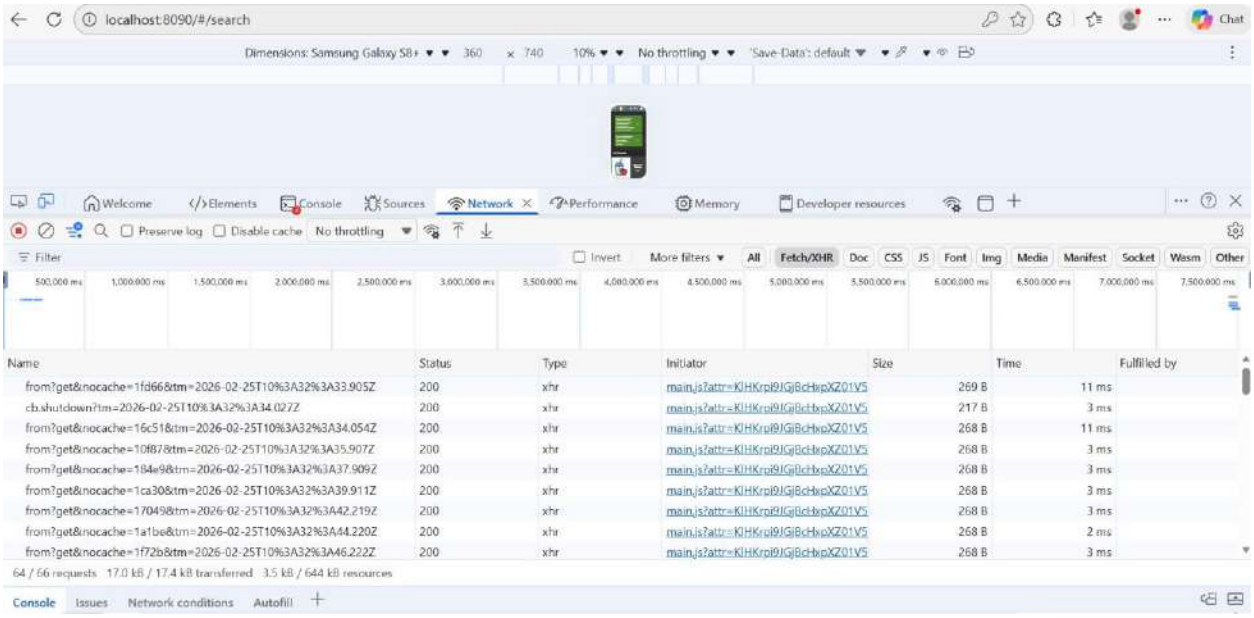


Figure 8: Unencrypted HTTP communication analysis

Observations

The scan revealed:

- Port 8090 open (HTTP service running).
- Service fingerprint identified as Juice Shop.
- Windows system ports (135, 139, 445) visible.
- Service version information exposed.
- MAC address detected.
- Web communication running over HTTP (not HTTPS).

Using Browser Developer Tools:

- Network requests visible in plain text.
- No SSL/TLS encryption detected.

Vulnerabilities Identified

1. **Open Port Exposure**
   - Port 8090 accessible over network.
   - Increases attack surface.
2. # Service Enumeration
   - Service version and fingerprint visible.
   - Enables targeted exploit attempts.
3. # Unencrypted Communication
   - Application runs over HTTP.
   - Data transmitted in plain text.
4. # Weak Authentication Risk
   - Login interface exposed.
   - No strict account lockout mechanism.
5. # Information Disclosure
   - Banner reveals application identity.
   - Server headers visible.

Conclusion

The experiment demonstrated how an attacker on the same network can identify open ports, detect running services, and test default credentials on an IoT device. The presence of exposed services and unencrypted communication highlights common IoT security risks. Proper configuration, strong authentication, and secure communication protocols are essential to protect IoT devices from exploitation.

Experiment 17: Creating and Analyzing Disk Images Using dc3dd and Autopsy (Alternative to FTK Imager)

1. Aim

To create a forensic disk image and analyze digital evidence using **dc3dd** and **Autopsy** in **Kali Linux**.

2. Objectives

After completing this experiment, students will be able to:

- Understand the concept of **digital forensics and disk imaging**
- Create a **bit-by-bit forensic copy of a storage device**
- Maintain **data integrity using hashing**
- Analyze disk images to identify **files, deleted files, and system artifacts**

3. Introduction /

Theory Digital

Forensics

Digital forensics is the process of **identifying, collecting, preserving, analyzing, and presenting digital evidence** from computers or storage devices.
Digital evidence may include:

- Documents
- Emails
- Browser history
- Logs
- Deleted files

To preserve evidence, investigators create a **forensic disk image**.

Disk Imaging

Disk imaging is the process of creating an **exact replica of a storage device**.

Characteristics of forensic disk images:

- Bit-by-bit copy of storage
- Preserves deleted files
- Maintains data integrity
- Allows investigation without altering original evidence

Hashing in Forensics

Hashing is used to verify the **integrity of digital evidence**.

Common hash algorithms:

Algorithm	Purpose
MD5	evidence integrity
SHA1	verification
SHA256	stronger security

If the hash value of the original disk and the disk image match, the **evidence is considered intact**.

4. Tools Used

Tool	Purpose
Kali Linux	forensic analysis environment
dc3dd	disk image acquisition
Autopsy	forensic investigation and analysis

5. System Requirements

Component	Specification
Processor	Intel i3 or higher
RAM	Minimum 4 GB
Disk	20 GB free space
Software	Kali Linux VM

6. Algorithm

1. Start the Kali Linux environment.
2. Generate activity on the system to create forensic artifacts.
3. Identify the disk partition using system commands.
4. Create a forensic disk image using dc3dd.
5. Generate hash values for integrity verification.
6. Start the Autopsy forensic analysis tool.
7. Import the disk image into Autopsy.
8. Analyze files, logs, and deleted artifacts.
9. Extract relevant digital evidence.

7. Procedure

Step 1: Open Terminal in Kali Linux

Press:

Ctrl + Alt + T

This opens the terminal.

Step 2: Create Sample Evidence

Generate a file that will later be discovered during forensic analysis.

```
echo "Cybersecurity Lab Evidence" > evidence.txt
```

Verify the file creation.

```
Ls
```

Step 3: Identify Disk Partition

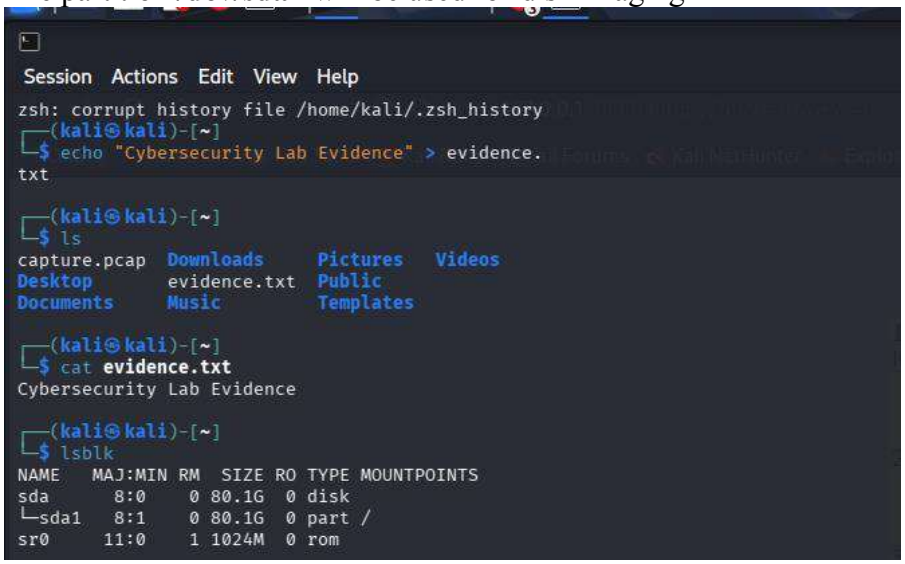
List storage devices.

```
lsblk
```

Example output:

```
sda
 ├─sda1
 ├─sda2
```

The partition **/dev/sda1** will be used for disk imaging



Install dc3dd

```
┌──(kali㉿kali)-[~]
└─$

┌──(kali㉿kali)-[~]
└─$ sudo apt install dc3dd
Installing:
  dc3dd

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not U
pgrading: 1757
  Download size: 117 kB
  Space needed: 489 kB / 60.2 GB available

Get:1 http://kali.download/kali kali-rolling/main
 amd64 dc3dd amd64 7.3.1-4 [117 kB]
Fetched 117 kB in 5s (21.9 kB/s)
Selecting previously unselected package dc3dd.
(Reading database ... 454289 files and directorie
s currently installed.)
Preparing to unpack .../dc3dd_7.3.1-4_amd64.deb .
..
Unpacking dc3dd (7.3.1-4) ...
Setting up dc3dd (7.3.1-4) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.4.3) ...

┌──(kali㉿kali)-[~]
└─$ dc3dd --help
─────
usage:
─────

        dc3dd [OPTION 1] [OPTION 2] ... [OPTION N
]

                *or*

        dc3dd [HELP OPTION]

        where each OPTION is selected from the ba
sic or advanced
        options listed below, or HELP OPTION is s
elected from the
        help options listed below.
```

Step 4: Create Disk Image

Step 1: Create a Small Disk File

dd if=/dev/zero of=practice_disk.dd bs=1M count=100

```
┌──(kali㉿kali)-[~]
└─$ dd if=/dev/zero of=practice_disk.dd bs=1M cou
nt=100
100+0 records in
100+0 records out
104857600 bytes (105 MB, 100 MiB) copied, 0.28316
5 s, 370 MB/s
```

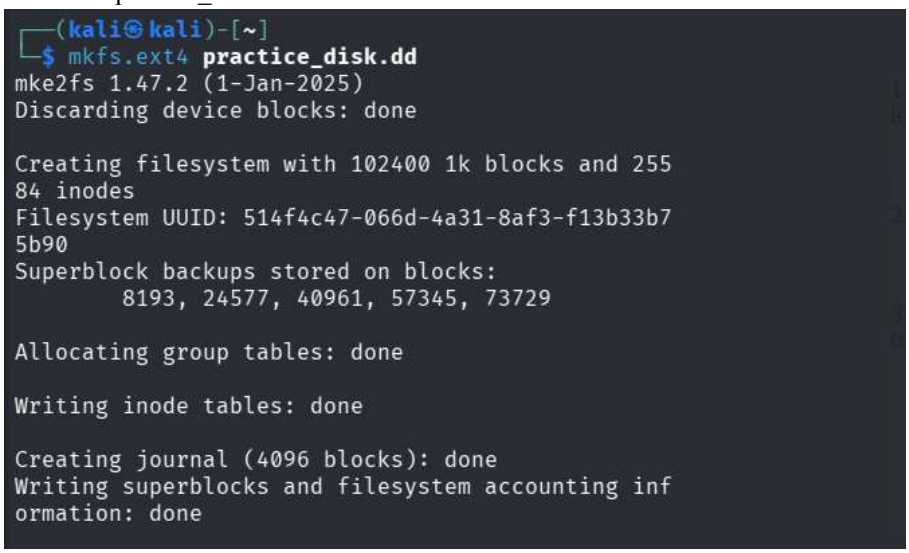
Explanation:

Parameter	Description
if	input disk
of	output image file
hash	generate MD5 hash
log	acquisition log file

Creates a **100 MB disk**.

Step 2: Mount the Disk

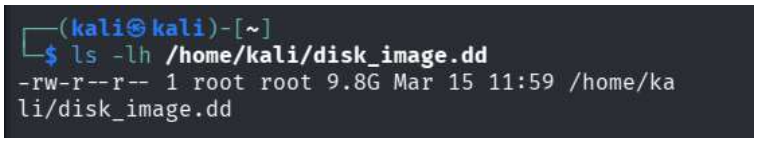
mkfs.ext4 practice_disk.dd



Step 5: Verify Image Creation

Check the created image.

ls -lh /home/kali/disk_image.dd



Check log file.

cat /home/kali/acquisition.log

The log will contain:

- image size
- hash value
- acquisition details

```
┌──(kali㉿kali)-[~]
└─$ cat /home/kali/acquisition.log

dc3dd 7.2.646 started at 2026-03-15 11:52:30 -040
0
compiled options:
command line dc3dd if=/dev/sda1 of=/home/kali/dis
k_image.dd hash=md5 log=/home/kali/acquisition.lo
g
device size: 167966702 sectors (probed),   85,998
,951,424 bytes
sector size: 512 bytes (probed)
 10503356416 bytes ( 9.8 G ) copied ( 12% ), 432.
546 s, 23 M/s

input results for device `/dev/sda1':
   20514368 sectors in
   0 bad sectors replaced by zeros
   ce827a581c4dfce5b976603df85bb5e0 (md5)

output results for file `/home/kali/disk_image.dd
':
   20514368 sectors out

dc3dd aborted at 2026-03-15 11:59:42 -0400
```

Step 6: Start Autopsy

Launch the forensic analysis tool.

Autopsy

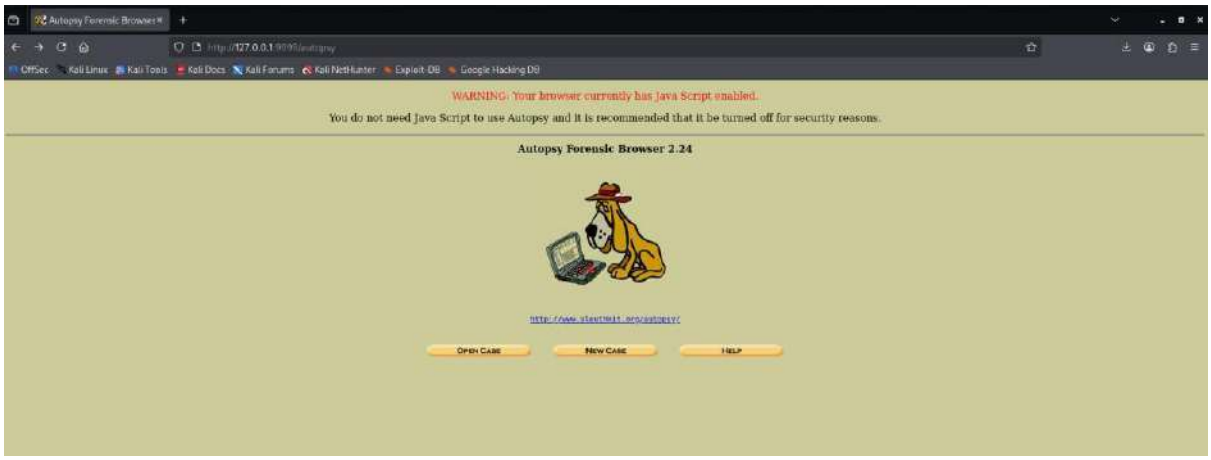
```
┌──(kali㉿kali)-[~]
└─$ autopsy

═══════════════════════════════════════════════════
═══════════════════╤═══════

                     Autopsy Forensic Browser
                  http://www.sleuthkit.org/autops
y/
                              ver 2.24

═══════════════════════════════════════════════════
═══════════════════════════
Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:01:18 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste
 this URL in it:

    http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exi
```

```
┌──(kali㉿kali)-[~]
└─$ sudo autopsy
[sudo] password for kali:

                        Autopsy Forensic Browser
                    http://www.sleuthkit.org/autops
y//
                                ver 2.24

Evidence Locker: /var/lib/autopsy
Start Time: Sun Mar 15 12:16:57 2026
Remote Host: localhost
Local Port: 9999

Open an HTML browser on the remote host and paste
 this URL in it:

    http://localhost:9999/autopsy

Keep this process running and use <ctrl-c> to exi
t
```

The terminal will display a URL:

http://localhost:9999/autopsy Open this link

in the browser.



Step 7: Create a New Case

In the Autopsy interface:

1. Click **Create New Case**
2. Enter case details:

Step 8: Add Disk Image

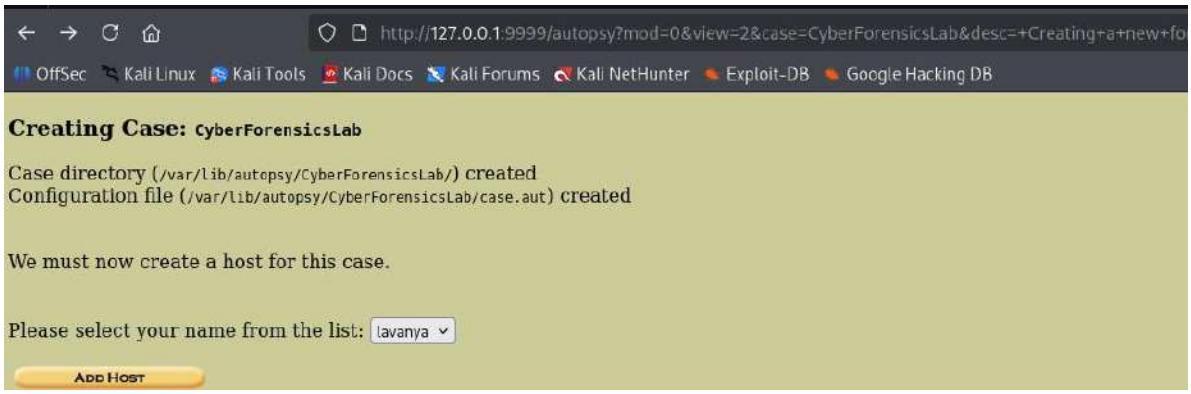
Select:

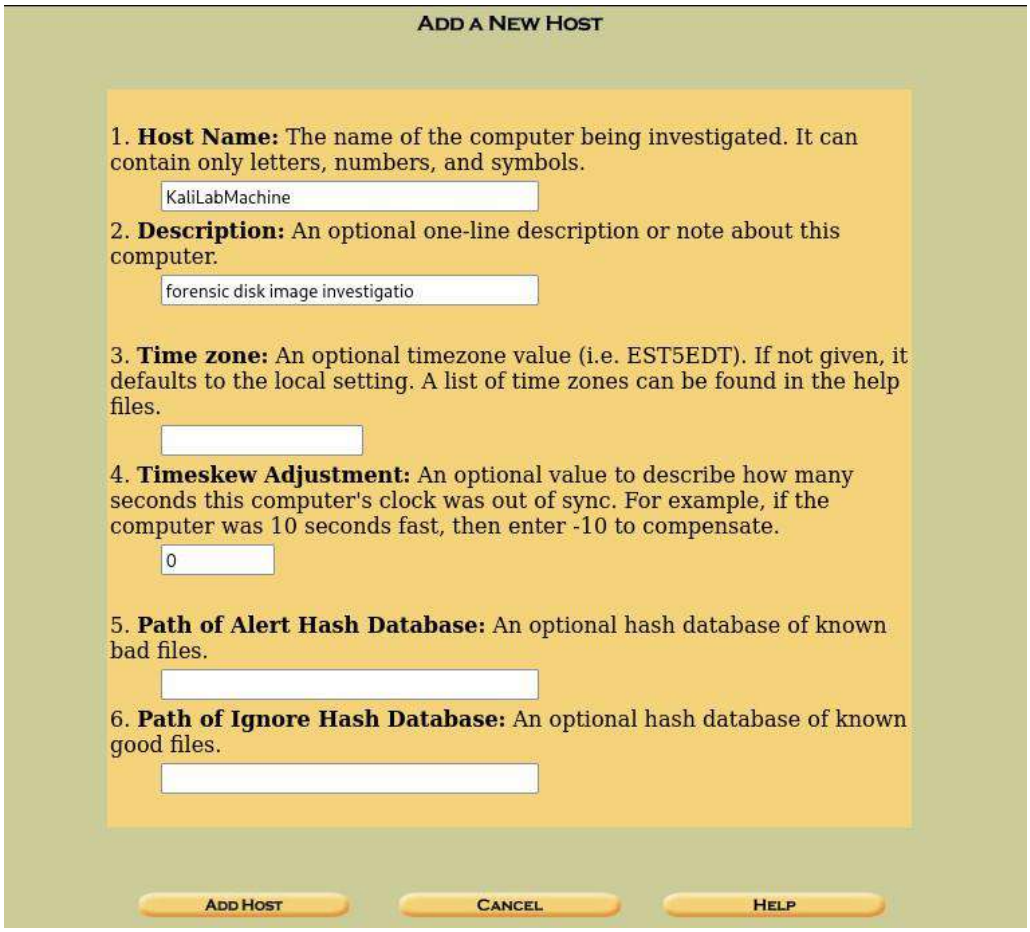
Add Host → Add Image

Browse and select:

/home/kali/disk_image.dd

Autopsy will begin analyzing the disk image.





Recommended Values

Host Name

KaliLabMachine

Description

Kali Linux virtual machine used for forensic disk image investigation

Time Zone

Leave Blank

Time Skew Adjustment

Leave Blank

Alert Hash Database

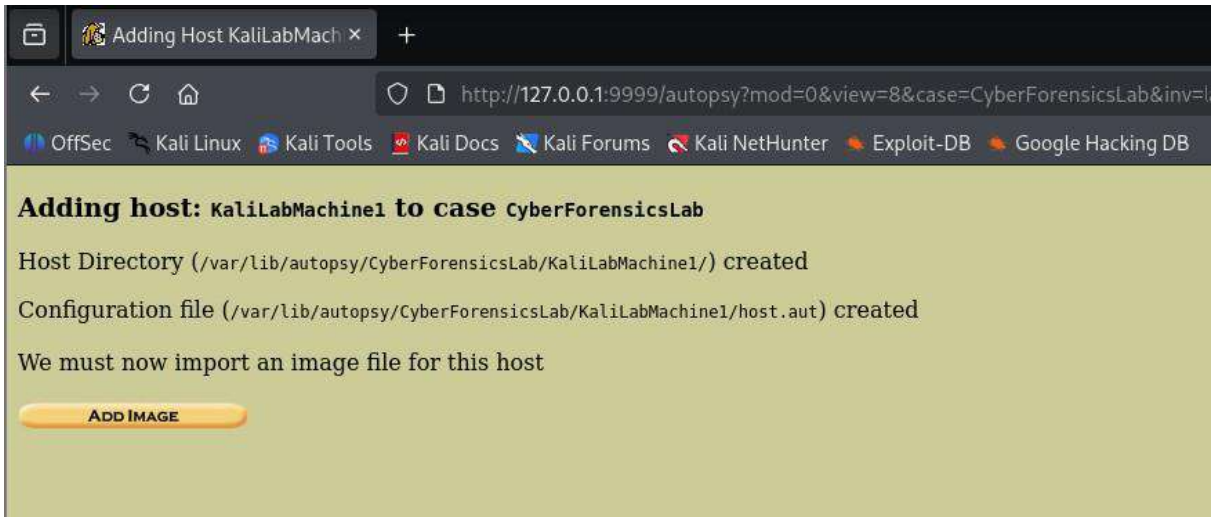
Leave Blank

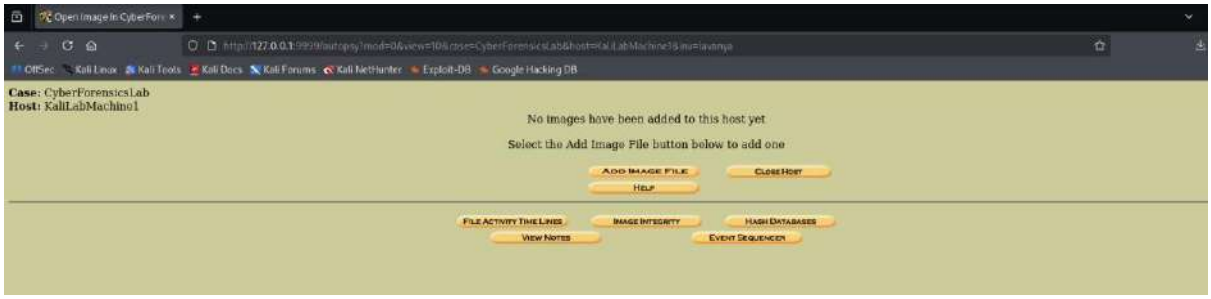
Ignore Hash Database

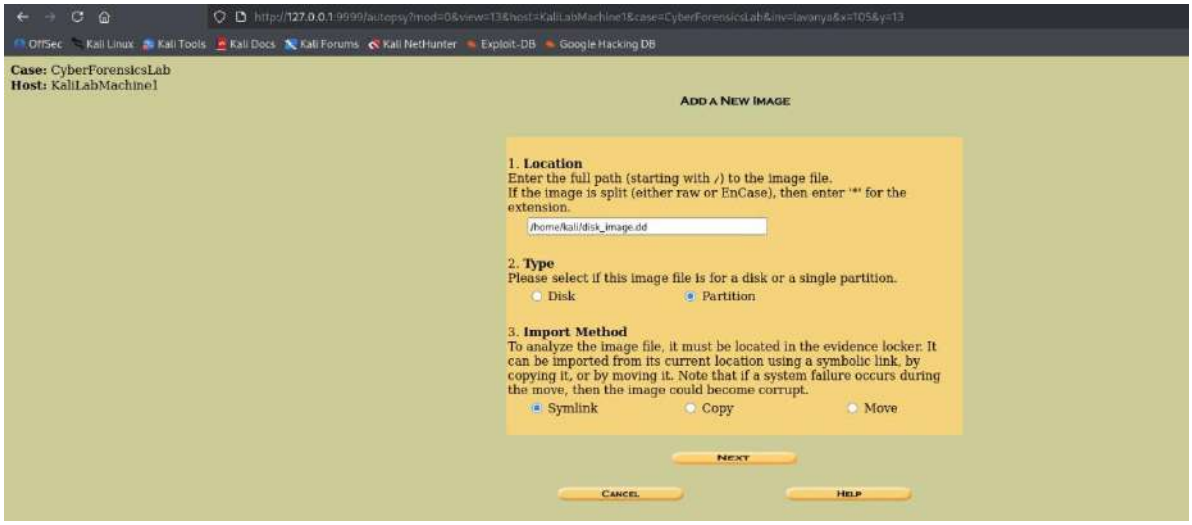
Leave Blank

Then click:

Next







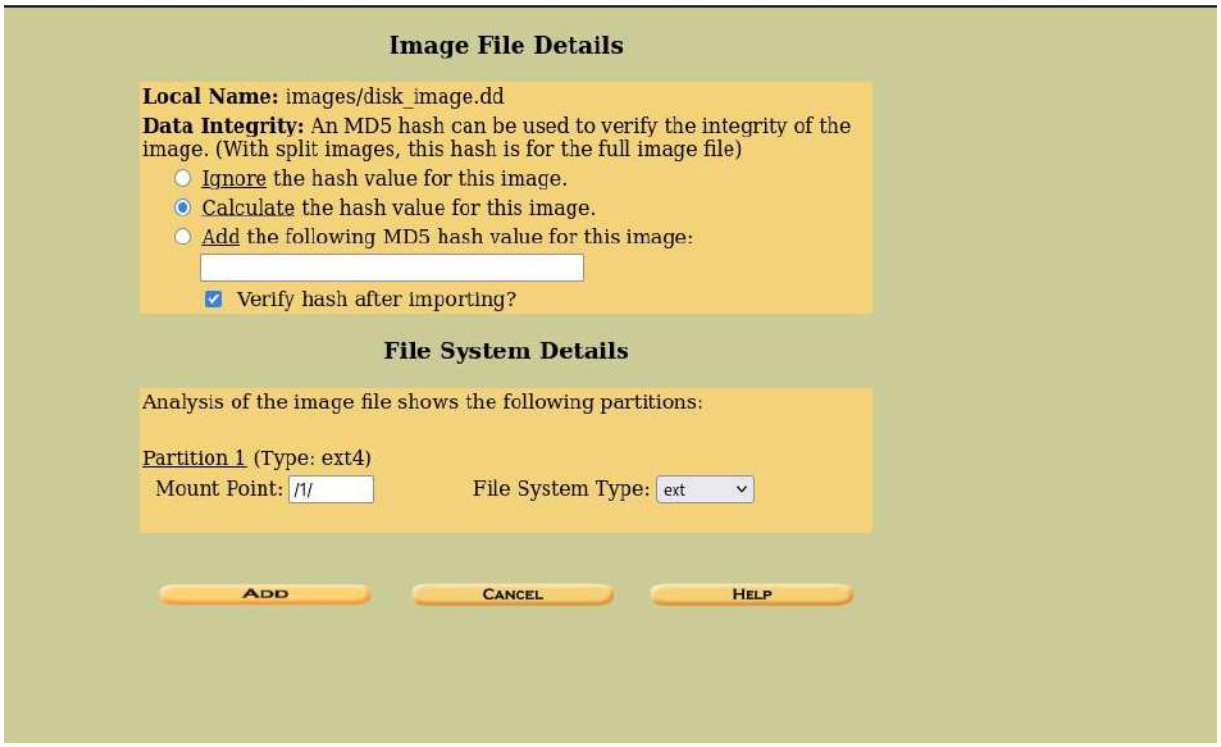
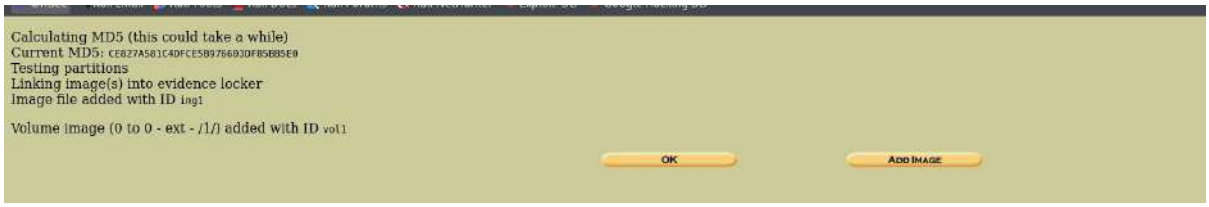
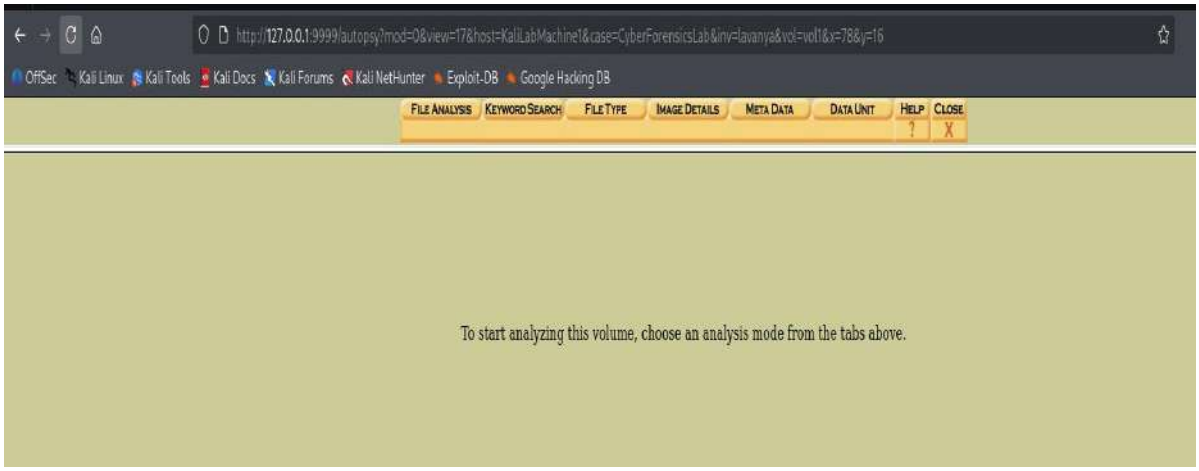
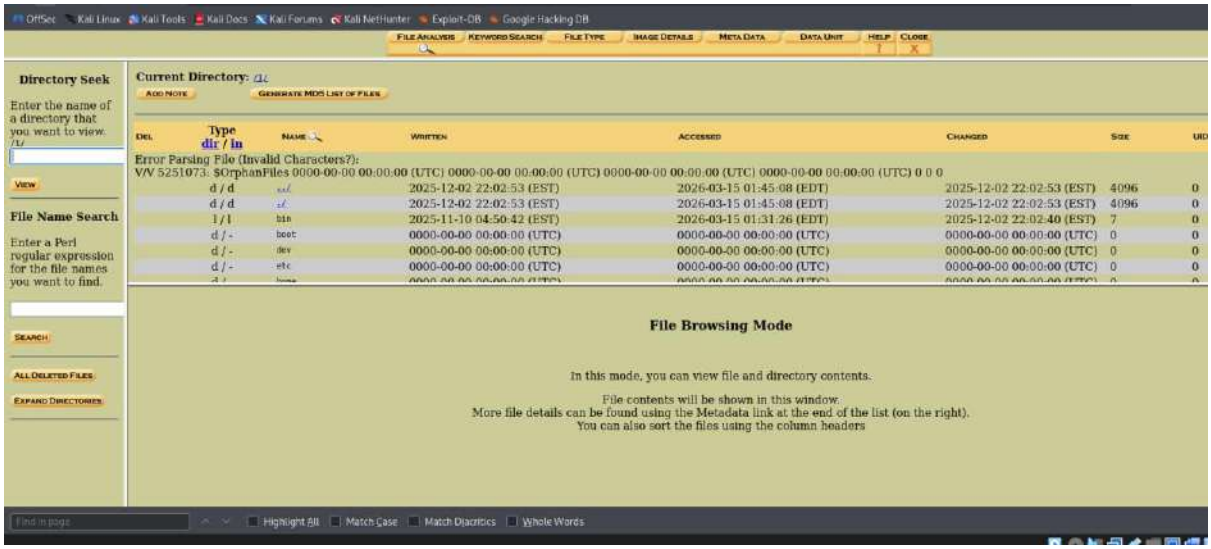


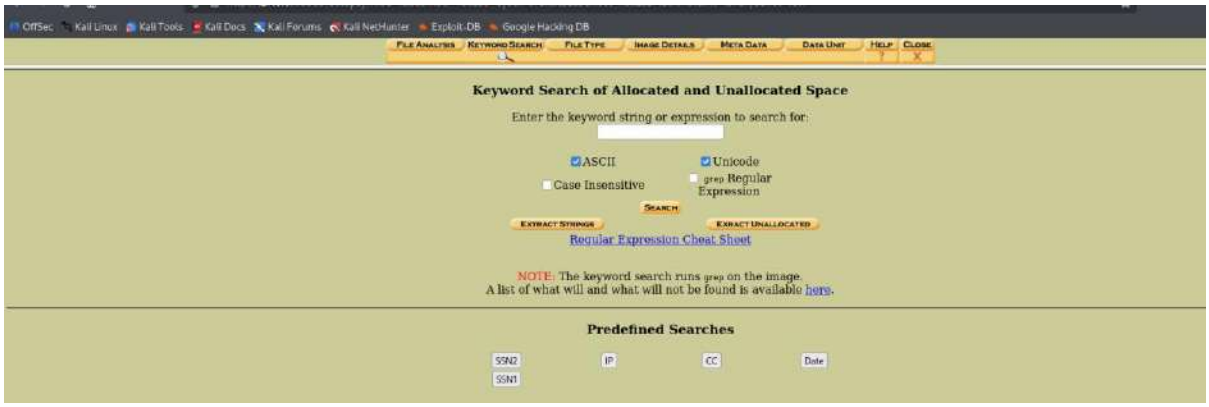
Image File Details
Local Name: images/disk_image.dd
Data Integrity: An MD5 hash can be used to verify the integrity of the image. (With split images, this hash is for the full image file)
Ignore the hash value for this image.
Calculate the hash value for this image.
Add the following MD5 hash value for this image:
Verify hash after importing?
File System Details
Analysis of the image file shows the following partitions:
Partition 1 (Type: ext4)
Mount Point: /1/
File System Type: ext
ADD
CANCEL
HELP

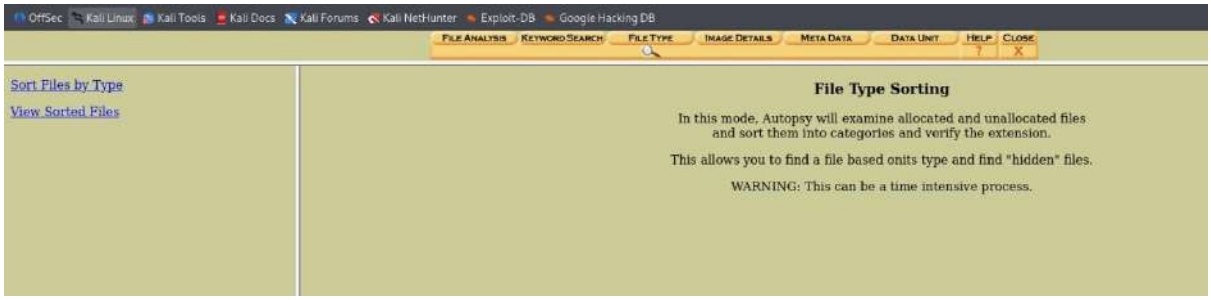

Calculating MD5 (this could take a while)
Current MD5: CE827A581C4DFCE5B976603DFB5B85E0
Testing partitions
Linking image(s) into evidence locker
Image file added with ID img1
Volume image (0 to 0 - ext - /1/) added with ID vol1
OK
ADD IMAGE


Case: CyberForensicsLab
Host: KaliLabMachine1
Select a volume to analyze or add a new image file.
CASE GALLERY
HOST GALLERY
HOST MANAGER
mount
name
fs type
/1/
disk_image.dd-0-0
ext
details
ANALYZE
ADD IMAGE FILE
CLOSE HOST
HELP
FILE ACTIVITY TIME LINES
IMAGE INTEGRITY
HASH DATABASES
VIEW NOTES
EVENT SEQUENCER


http://127.0.0.1:9999/autopsy?mod=0&view=17&host=KaliLabMachine1&case=CyberForensicsLab&inv=lavanya&vol=vol1&x=78&y=16
FILE ANALYSIS
KEYWORD SEARCH
FILE TYPE
IMAGE DETAILS
META DATA
DATA UNIT
HELP
CLOSE
To start analyzing this volume, choose an analysis mode from the tabs above.







Step 9: Analyze Evidence

Experiment 18 : LOG FILE ANALYSIS FOR INCIDENT DETECTION LAB

Aim

To analyze system log files and detect suspicious activities such as failed login attempts and identify suspicious IP addresses.

Objective

- Understand system log files
- Detect suspicious activities and incidents
- Analyze logs using Linux tools
- Identify attack patterns (failed logins, brute force, etc.)

Requirements

- Kali Linux / Ubuntu VM
- Basic Linux command knowledge
- Tools:
- Kali Linux
- journalctl
- ssh
- grep, awk, sort
- grep
- awk
- less
- cat
- journalctl

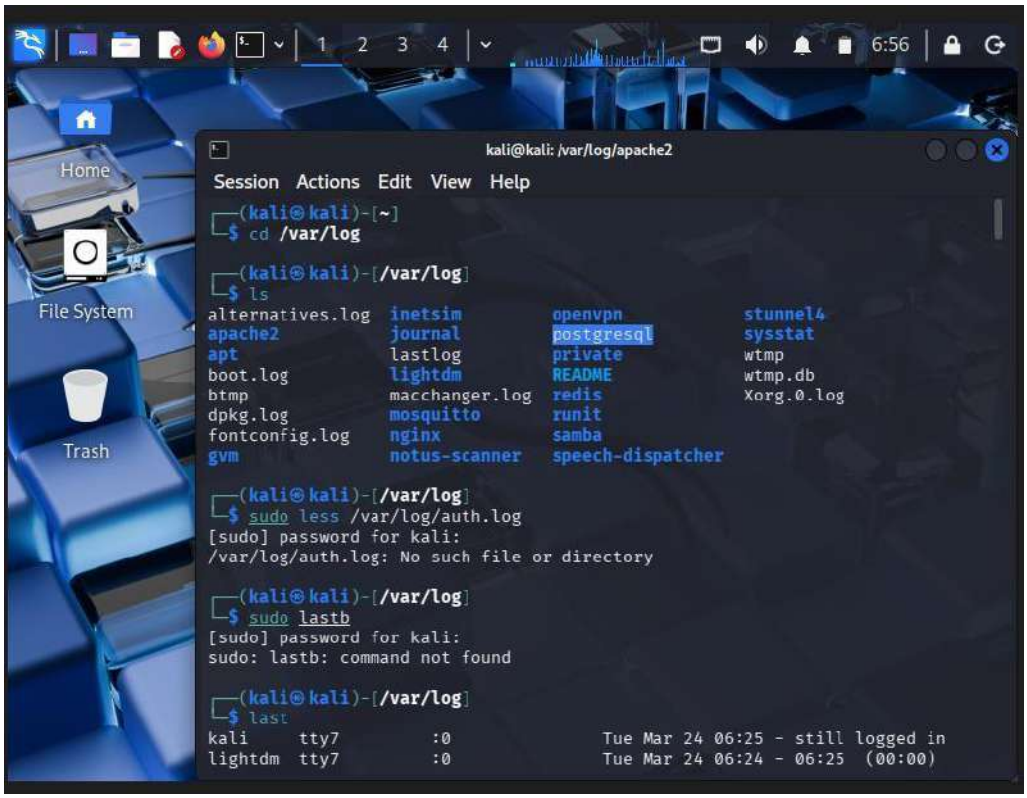
Understanding Log

Files Navigate to Log

Directory cd /var/log

ls

The /var/logdirectory contains system and application logs such as Apache, Nginx, and login records used for security analysis.



What Your Output

Shows From your ls:

- ✔ **dpkg.log → package activity**
- ✔ **apache2 → web server logs**
- ✔ **nginx → web logs**
- ✔ **journal → system logs (IMPORTANT)**
- ✔ **wtmp, btmp → login records**

Successful Logins (wtmp)

Command:

last

Capture:

- Login history

This displays successful login sessions. It helps identify legitimate and suspicious user access.

System Logs using journalctl

Command:

journalctl | less

Capture:

- Logs output

Failed SSH Attempts

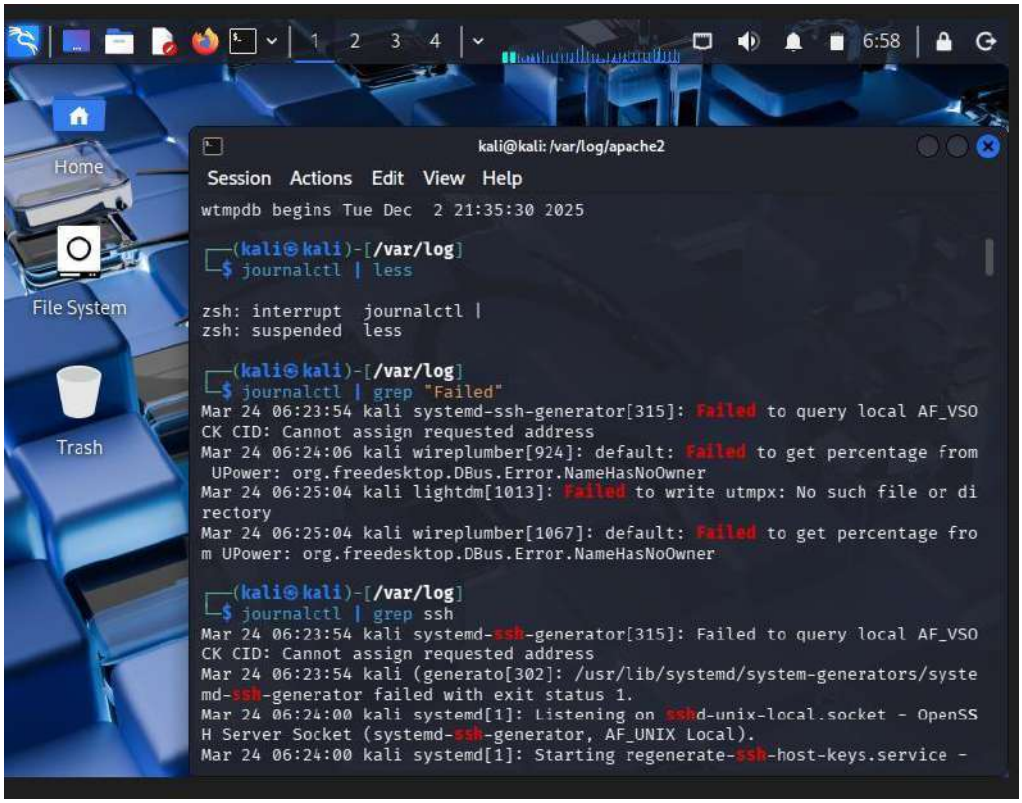
Command:

journalctl | grep "Failed"

SSH Login Activity

Command:

journalctl | grep ssh

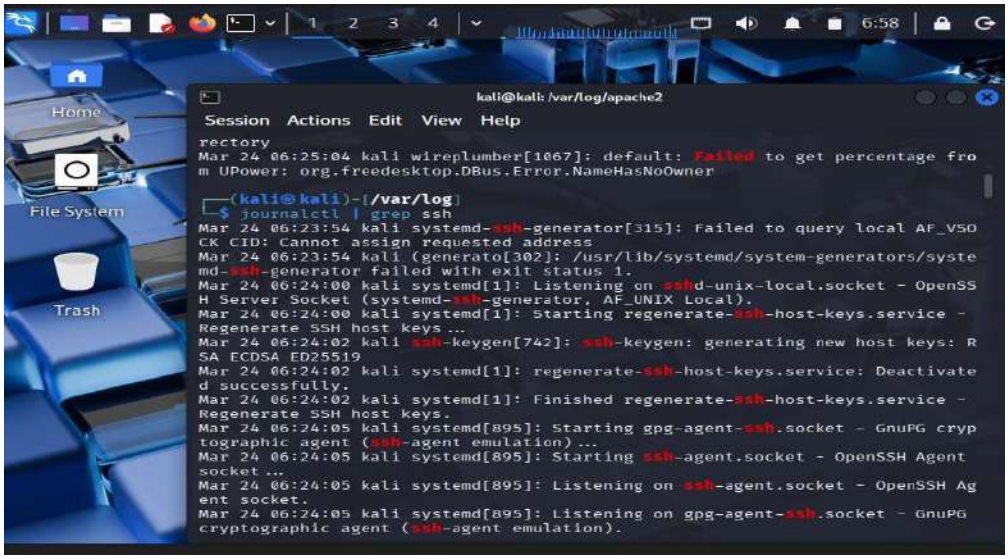


Explanation:

The journalctl command provides centralized system logs including authentication, services, and system events.

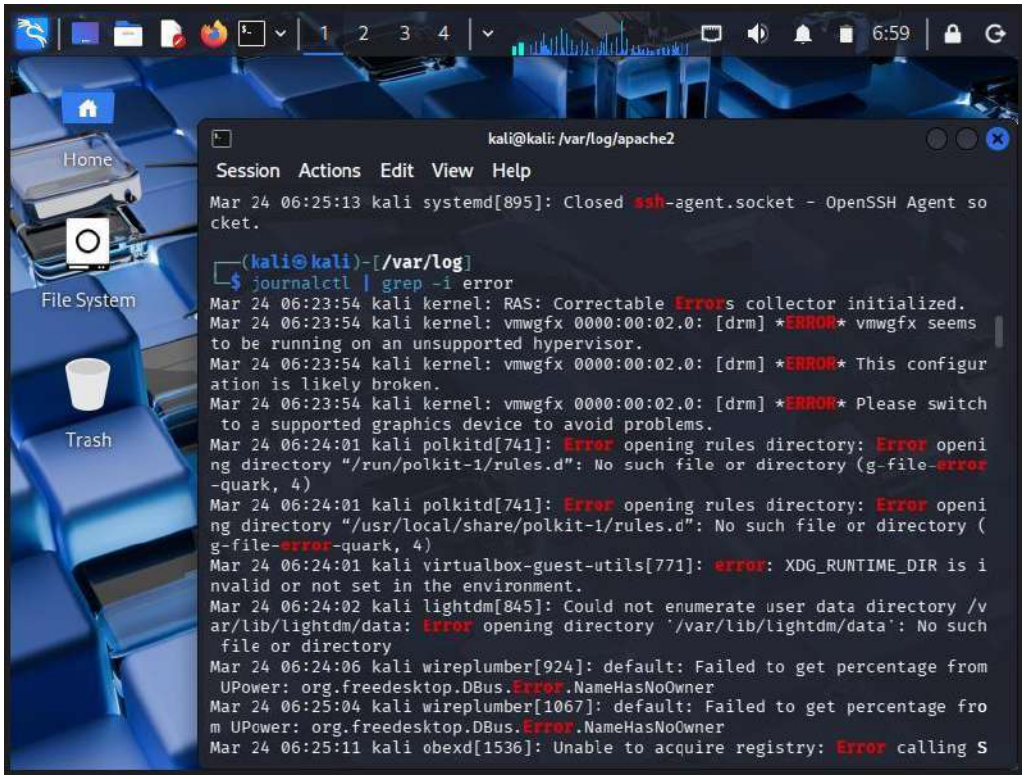
This screenshot shows failed authentication attempts, which may indicate unauthorized access attempts.

This screenshot also displays SSH-related logs, including login attempts and connections.



Error Detection

Command: journalctl | grep -i error



The above screenshot highlights system errors, which may indicate abnormal behavior or system issues.

Apache Log Analysis

Command:

cd /var/log/apache2 ls

Then:

sudo less access.log
Apache logs record web requests. These logs help identify suspicious web activity such as repeated requests or attacks.

Suspicious Requests Command:

grep "404" /var/log/apache2/access.log
This shows failed web requests (404 errors), which may indicate scanning or probing attacks.

Package Activity

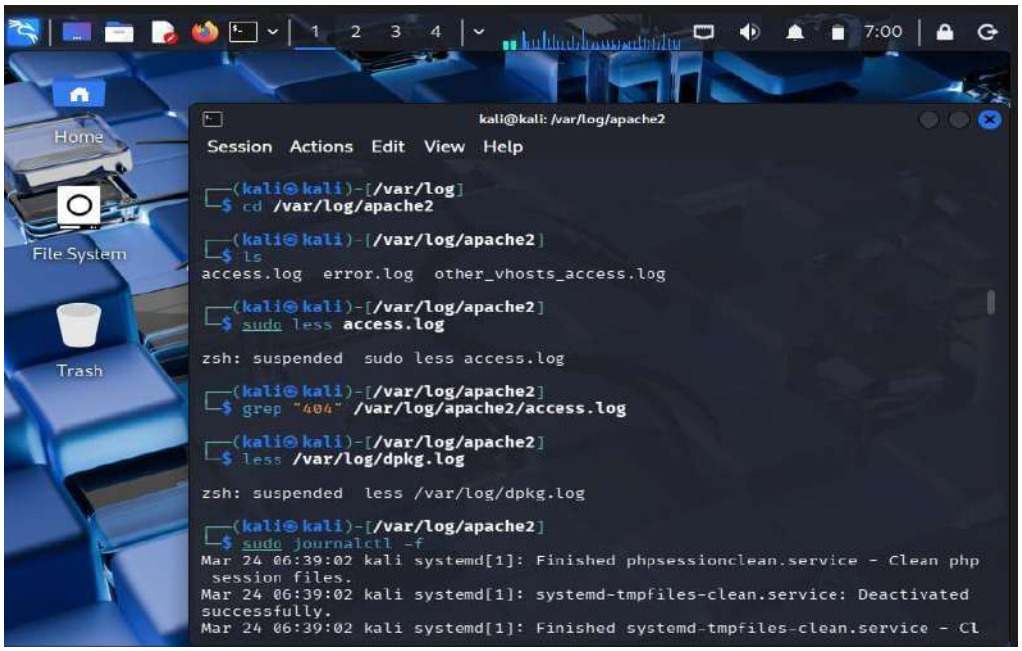
Command:

less /var/log/dpkg.log
This shows software installation and update logs, useful for detecting unauthorized changes.

Real-Time Monitoring Command:

sudo journalctl -f

This shows real-time system log monitoring, allowing detection of incidents as they occur.



QUICK CHECKLIST

Check	Command	Suspicious If
Failed logins	lastb	Many attempts
SSH activity	`journalctl	grep ssh`
Login history	last	Odd timing
Errors	`journalctl	grep error`
Web logs	grep 404	Many failed requests

STEP-BY-STEP ANALYSIS OF YOUR LOG

1. System Services (NORMAL)

Finished phpsessionclean.service Finished systemd-tmpfiles-clean.service

✔ These are routine background tasks
✔ No issue
Regular system maintenance services were observed, indicating normal system operation.

2. CRON Jobs (NORMAL)

CRON: session opened for user root (root) CMD (debian-sa1)

✔ Automated scheduled task
✔ Runs periodically

Scheduled cron jobs were observed, which are normal automated system processes.

3. Your Own SUDO Activity (NORMAL)

kali : COMMAND=/usr/bin/less access.log kali : COMMAND=/usr/bin/journalctl -f

✔ This is YOU running commands
✔ Not suspicious

Sudo activity corresponds to legitimate administrative actions performed by the user.

4. XFCE / GUI Services (NORMAL)

xfconfd.service started

✔ Desktop environment service
✔ Normal

5. IMPORTANT PART

● Repeated Kernel Messages

dndHGCM: Received message HOST_DND_FN_GH_REQ_PENDING dnd: No guest source window

These lines are:

Repeated MANY times
Coming from VirtualBox (VBoxDRM / DnD)

Q Is This Suspicious?

Technically:

- NOT an attack
- But **abnormal repetitive behavior**

! Why it matters:

- Repeated logs = unusual system activity
- Could indicate:
  - Misconfiguration
  - Resource issue
  - VirtualBox interaction problem

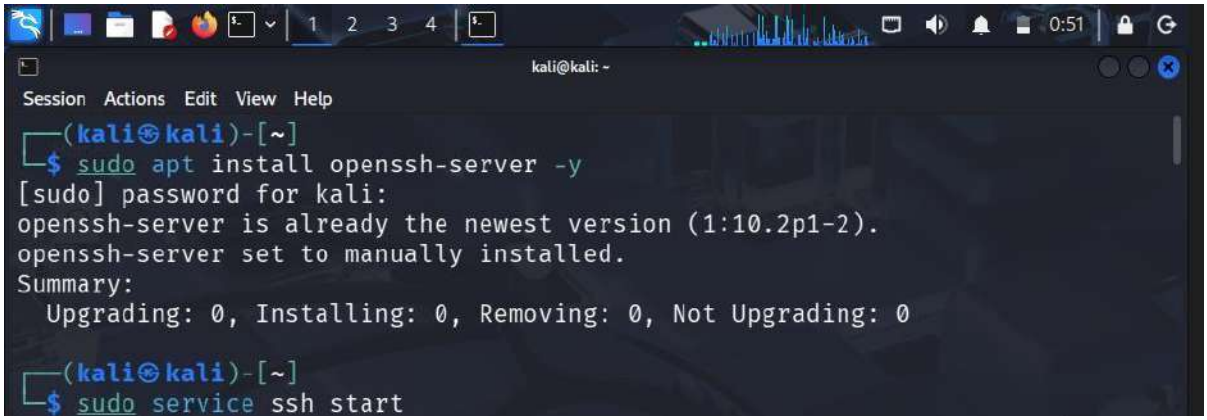
METHOD 1: SIMULATE FAILED SSH LOGIN

Step 1: Ensure SSH is Installed

sudo apt install openssh-server –y

Step 2: Start SSH Service

sudo service ssh start

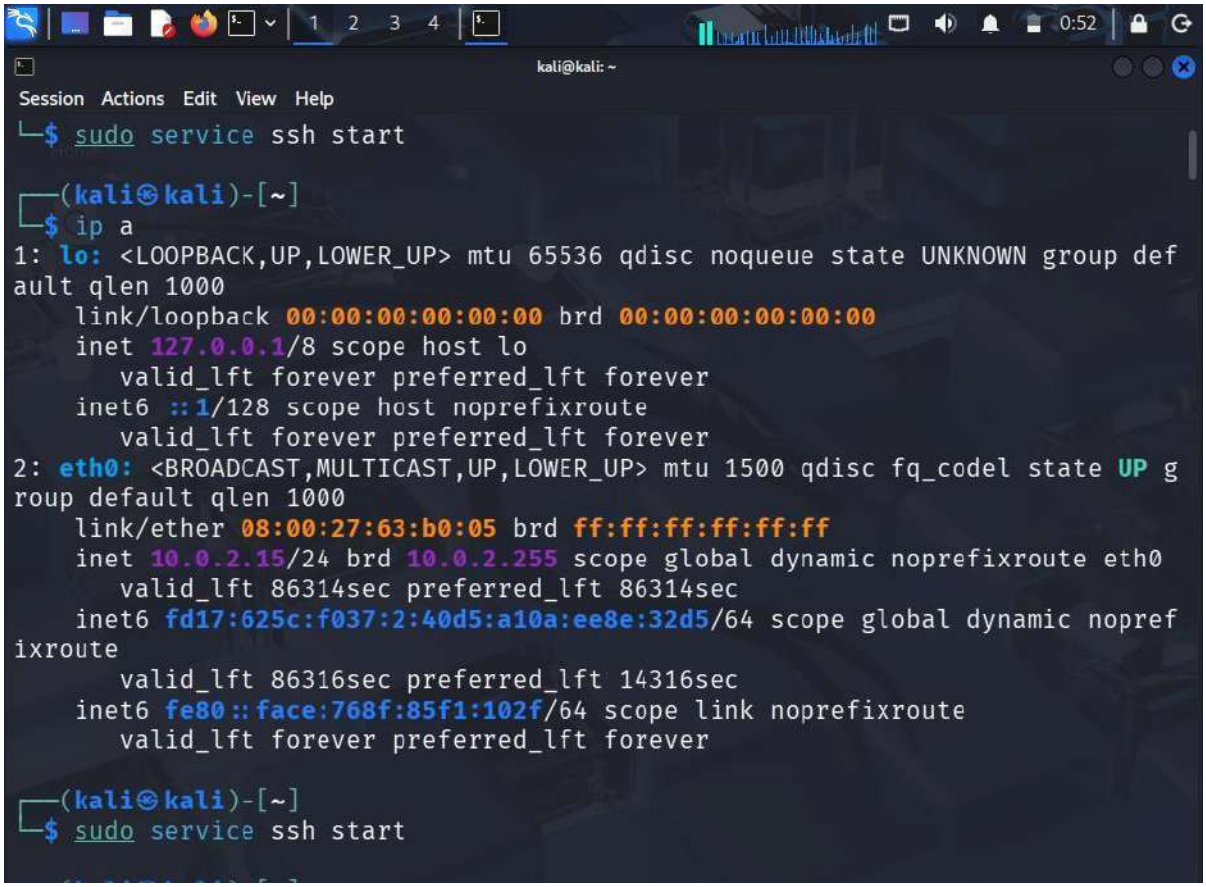


Step 3: Find Your IP Address

ip a

Look for something like:

inet 192.168.x.x



Step 4: Attempt Wrong Login (GENERATE FAILURES)

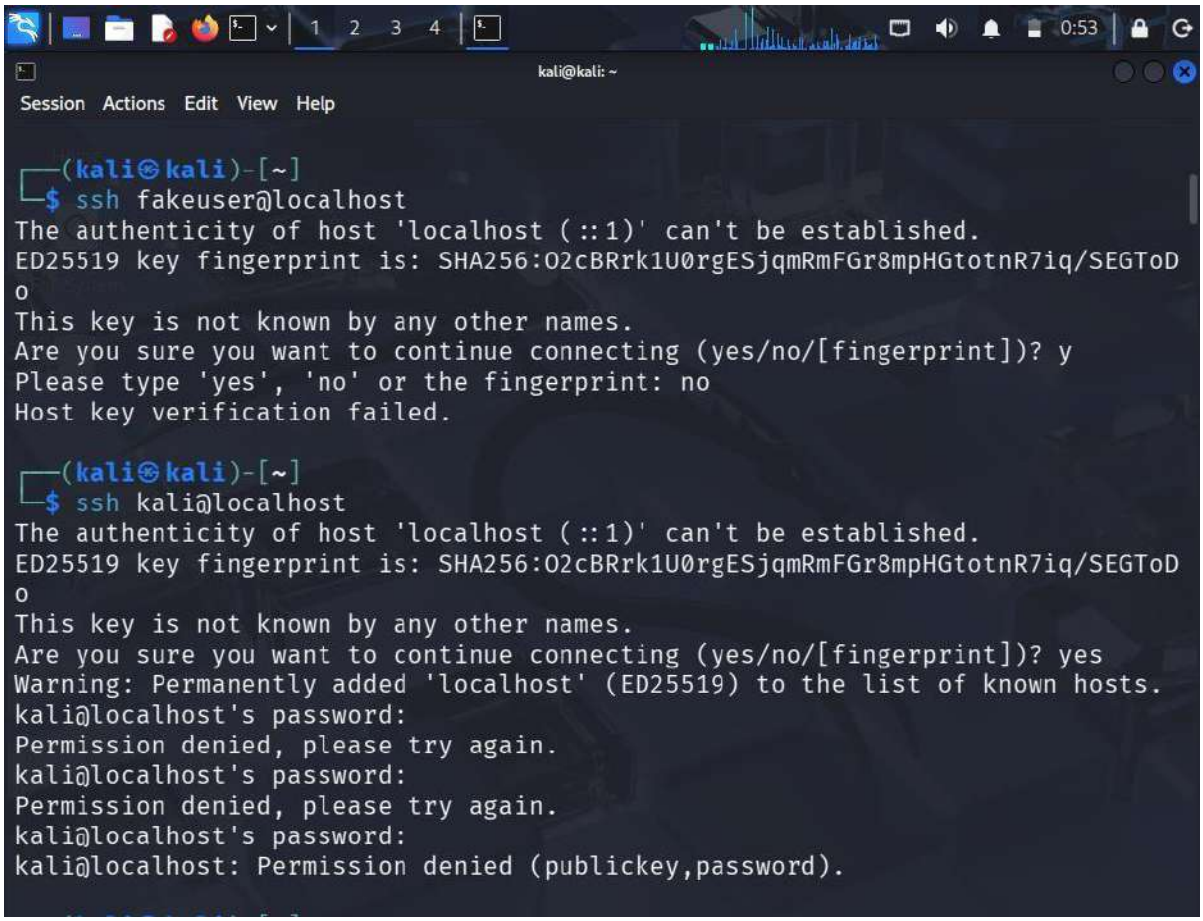
run:

ssh fakeuser@localhost

OR

ssh kali@localhost

Enter **wrong password multiple times (5–10 times)**



Now Analyze Logs

Step 5: Check Failed Logins

journalctl | grep "Failed password"

 You should now see:

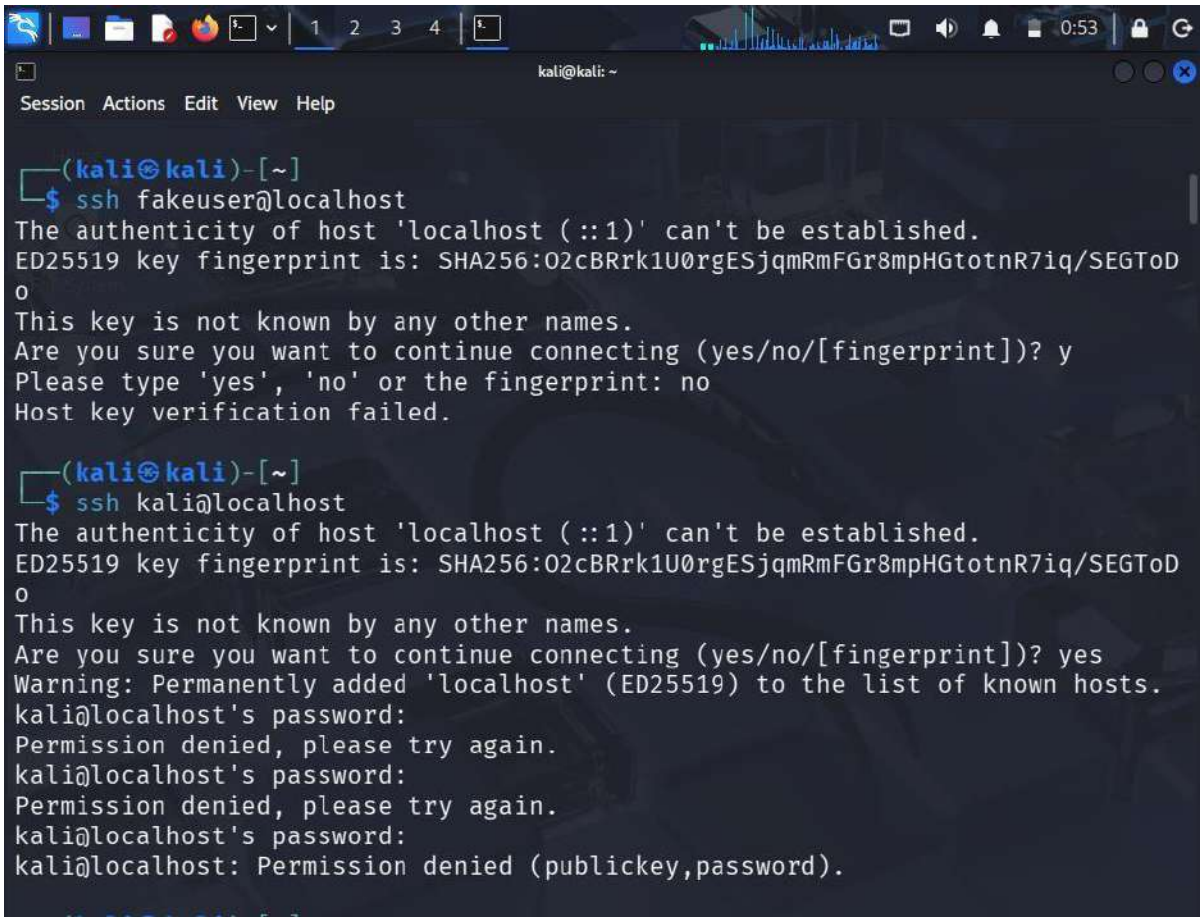
Failed password for kali from 127.0.0.1 port ...

Step 6: Extract Suspicious IP

journalctl | grep "Failed password" | awk '{print $11}'

Output:

127.0.0.1

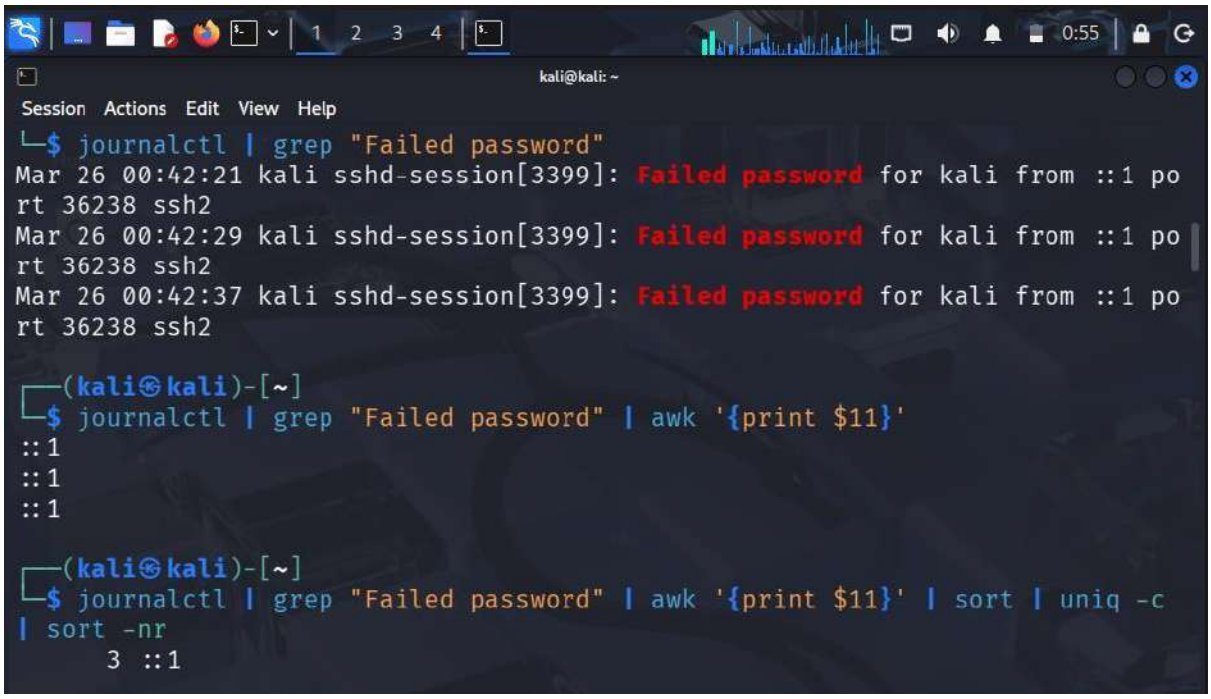


Step 7: Count Attempts Per IP

journalctl | grep "Failed password" | awk '{print $11}' | sort | uniq -c | sort -nr

Example:

10 127.0.0.1



INTERPRETATION (VERY IMPORTANT)

Multiple failed login attempts were detected from IP address 127.0.0.1, indicating a simulated brute force attack.

METHOD 2: Use lastb (Alternative)

sudo lastb

Shows failed login records

HOW TO IDENTIFY "SUSPICIOUS IP"

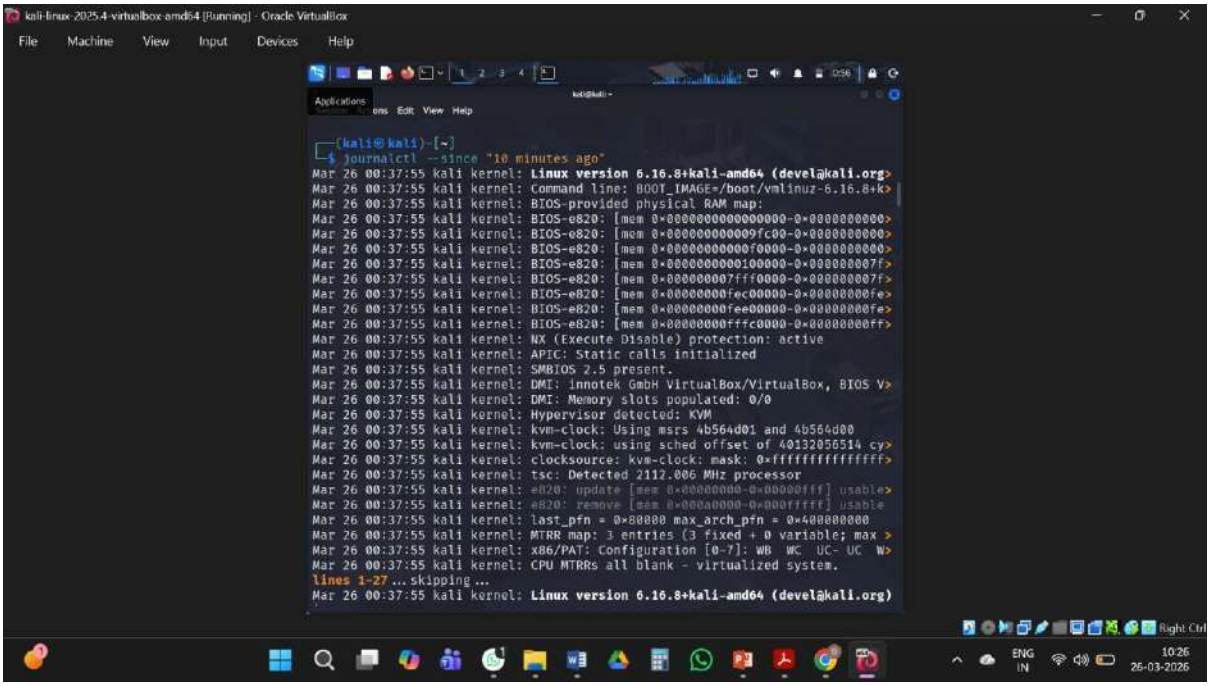
Simple rule:

Condition	Meaning
Same IP repeated many times	□ Suspicious
Unknown IP	□ Suspicious
High frequency attempts	□ Brute force

Run:

journalctl --since "10 minutes ago"

Shows recent attack activity



Experiment 19: NETWORK FORENSICS USING WIRESHARK

Aim

To capture and analyse network packets using Wireshark and understand network forensics techniques.

Objective

- To learn packet capturing using Wireshark
- To analyze different network protocols
- To apply filters for identifying specific traffic
- To understand how network forensics helps in detecting suspicious activities

Introduction

Network forensics is a branch of digital forensics that deals with monitoring, capturing, and analyzing network traffic to detect security breaches and malicious activities. Wireshark is a widely used open-source tool that helps in capturing real-time packets and analyzing them in detail.

Tools Required:

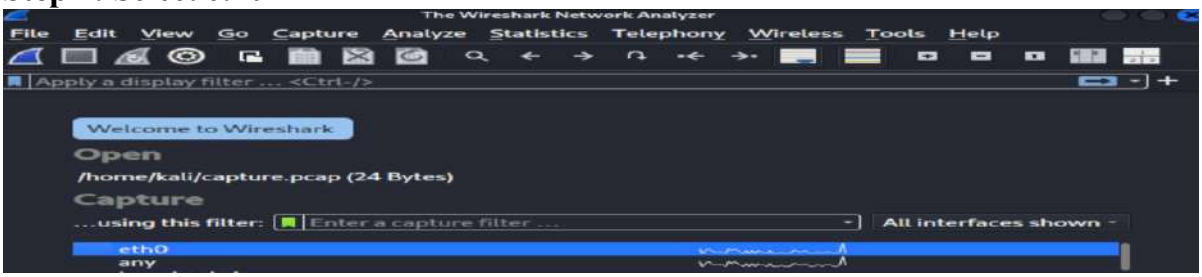
1. **Kali Linux**
2. **Wireshark**

Procedure:

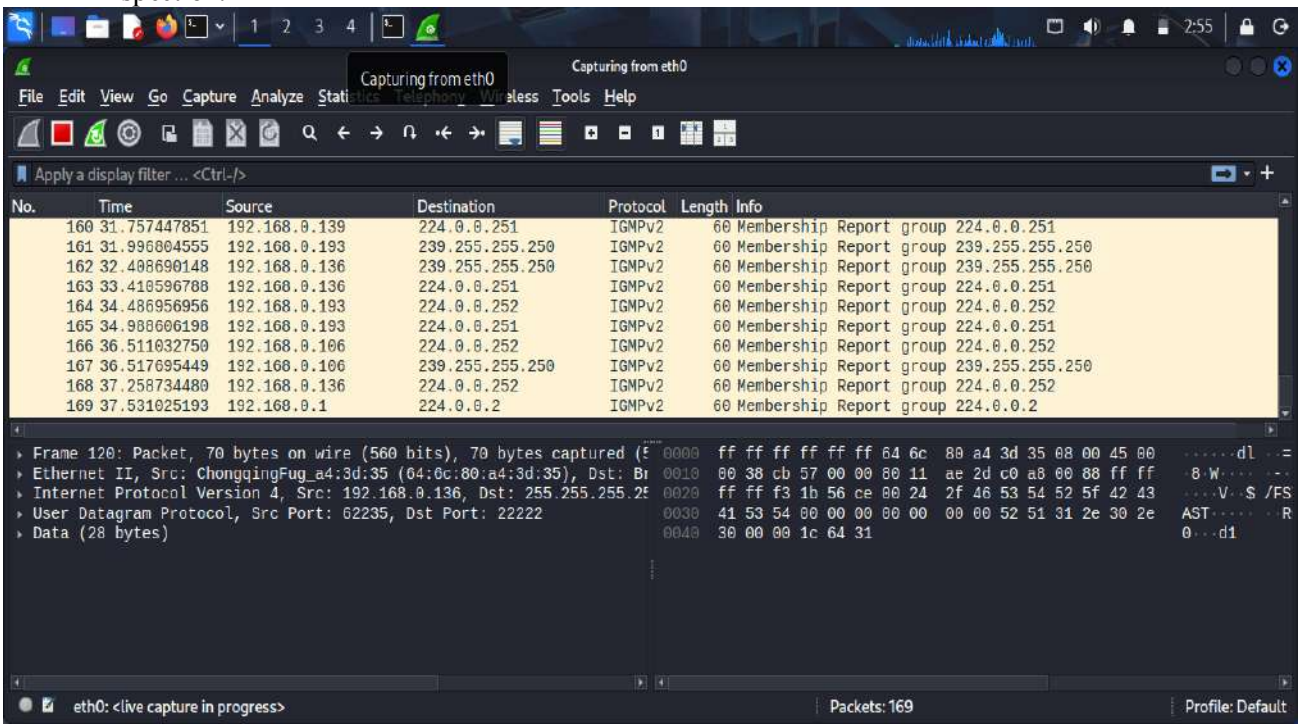
Step1:Open kali linux terminal

Type wireshark &

Step 2: Select eth0

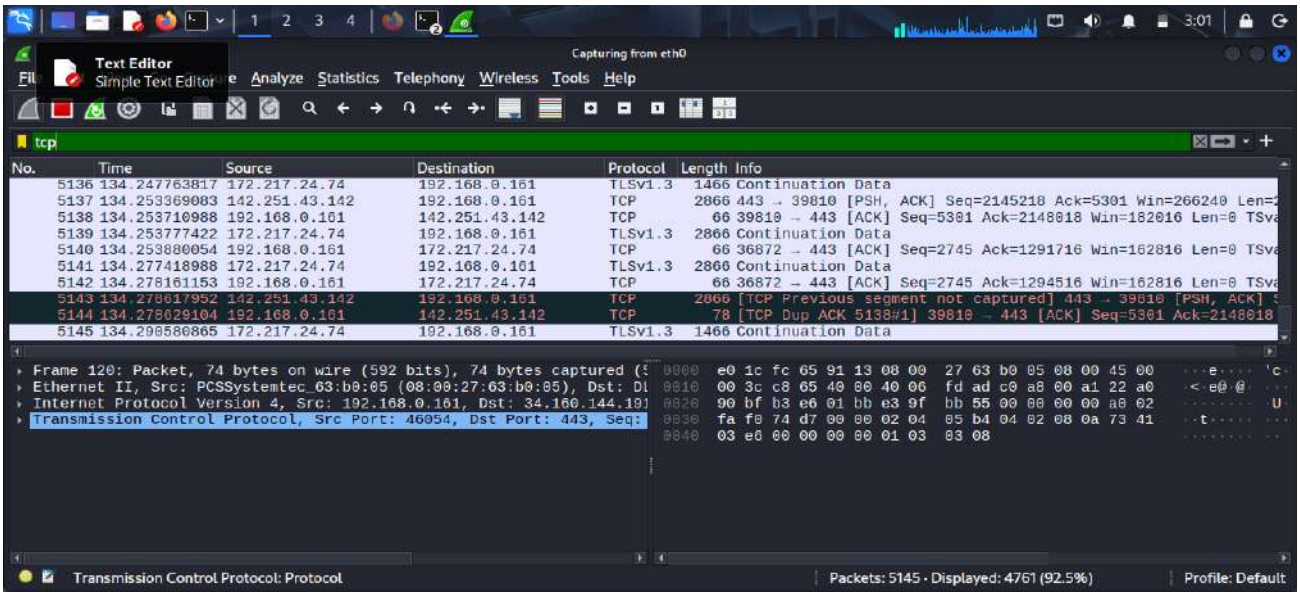


- Wireshark is launched to begin packet analysis. It provides a graphical interface to monitor all incoming and outgoing packets in a network. It supports multiple protocols and helps in detailed inspection.



Step 3: Protocol Analysis (TCP/UDP/HTTP): Wireshark supports deep inspection of protocols such as TCP, UDP, and HTTP. TCP ensures reliable communication, while UDP is faster but connectionless. HTTP packets help analyze web traffic and identify requests and responses.

- Apply filter as tcp…now you can able to see the only tcp packets. You can also try with UDP,HTTP

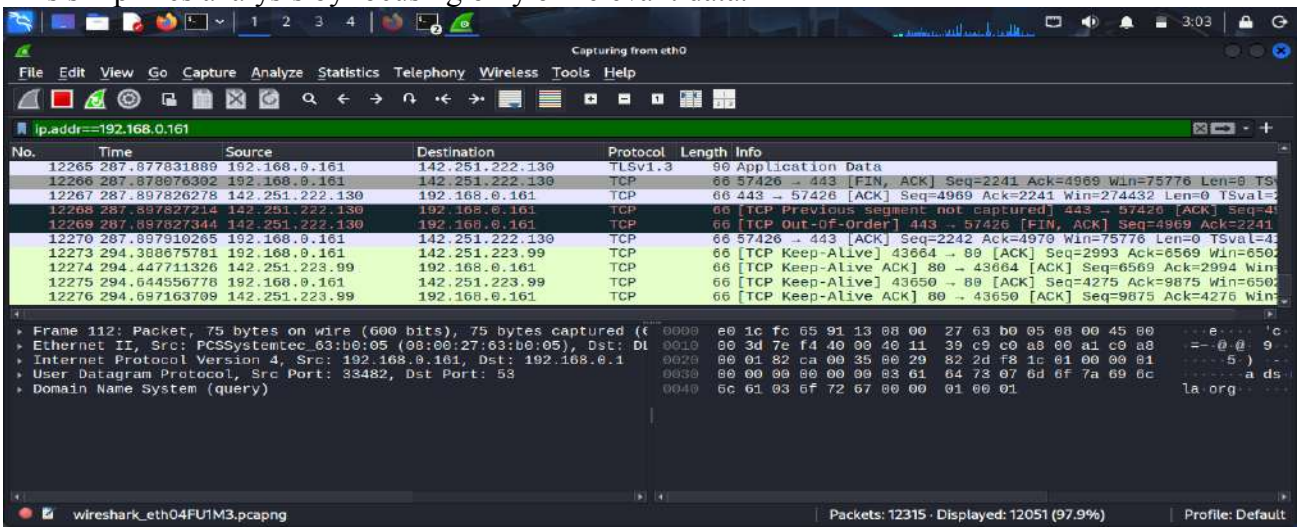


Step 4: Applying Filters

Filters are used to narrow down specific packets. For example:

- ip.addr == 192.168.0.161 → filters packets of a specific IP

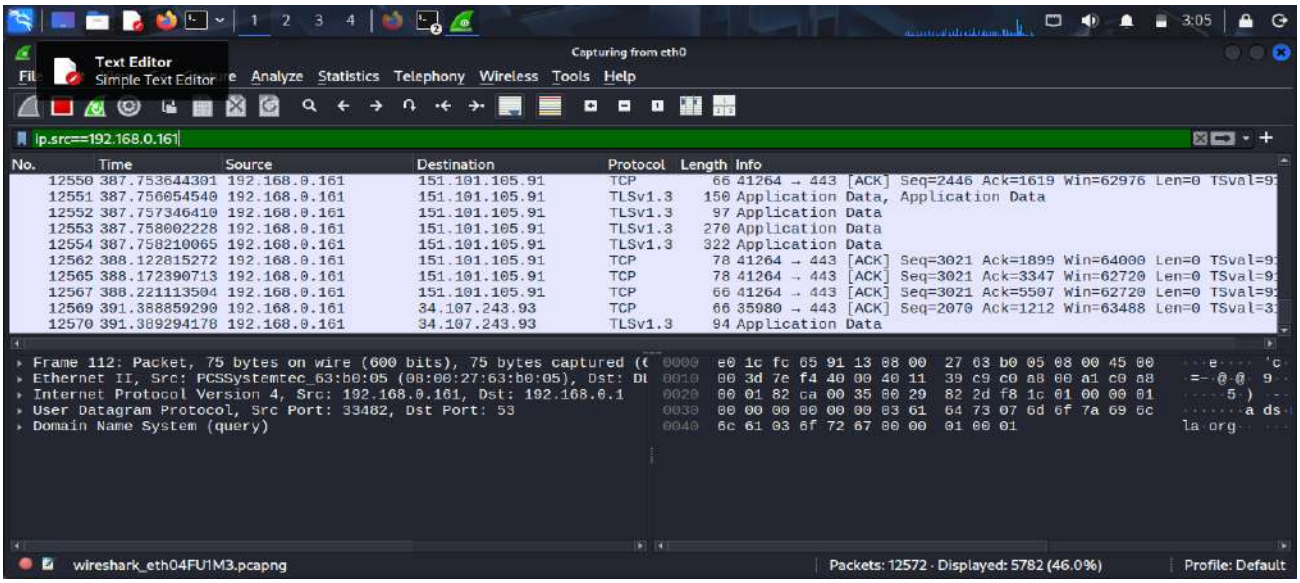
This simplifies analysis by focusing only on relevant data.



Step 5 :for particular Source address use

Ip.src==<IP_Address> like

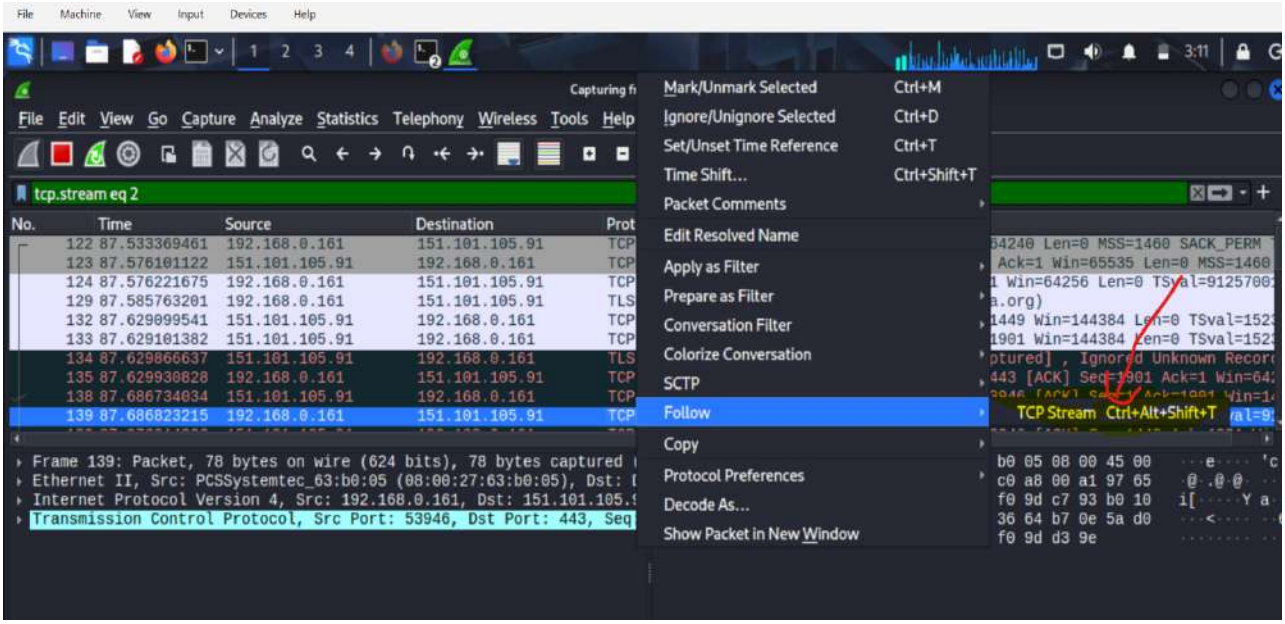
Ip.src== 192.168.0.161

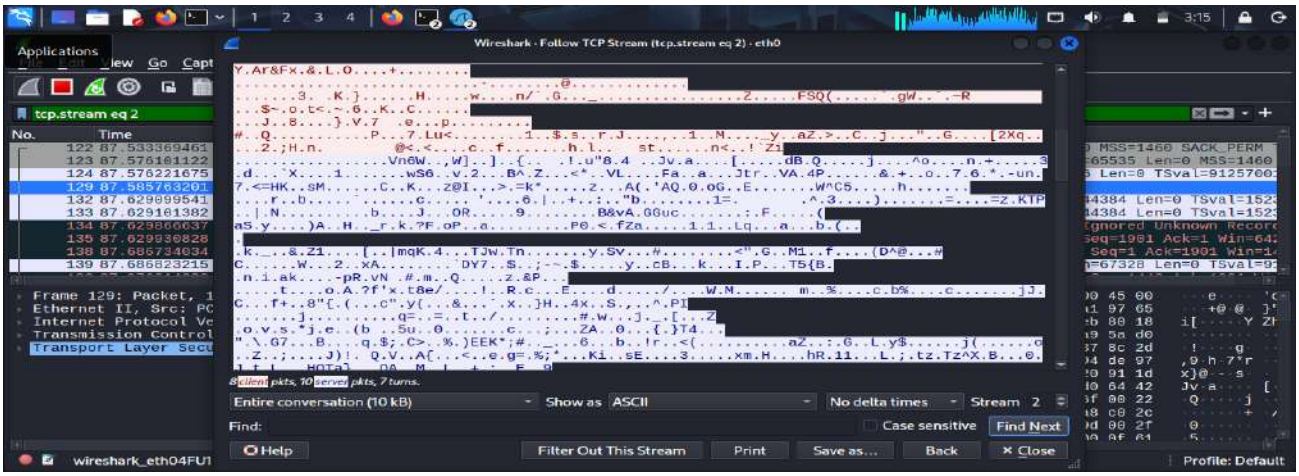


Step 6: Following TCP Stream

The "Follow TCP Stream" feature reconstructs communication between two endpoints. It helps in analyzing complete conversations, such as HTTP requests and responses, making it useful in forensic investigations.

- **Right click on any TCP packet->Follow->TCP Stream**





- **Observation on above tcp stream:**

Complete conversation between two devices over TCP

- **Red text** → Data sent from one device
- **Blue text** → Data sent from the other device

It reconstructs the **entire communication stream**

Mixed Content (Binary + Text)

- You see:
  - Random characters → **binary data**
  - Some readable text → **application data**

This means:

- It is **not plain HTTP**
- Likely:
  - Encrypted traffic (HTTPS)
  - Or application-level data

TCP Flags Visible

You can see:

[SYN], [ACK], [PSH], [FIN]

Meaning:

- **SYN** → Connection start
- **ACK** → Acknowledgement
- **PSH** → Data transfer
- **FIN** → Connection close

So this stream shows:

✔ Full TCP session lifecycle

Step 7: Identifying Suspicious Traffic

Open kali linux terminal

Type nmap –sS <target_IP>

Eg: nmap –sS 192.168.0.156 (use your windows ip)

Open wireshark

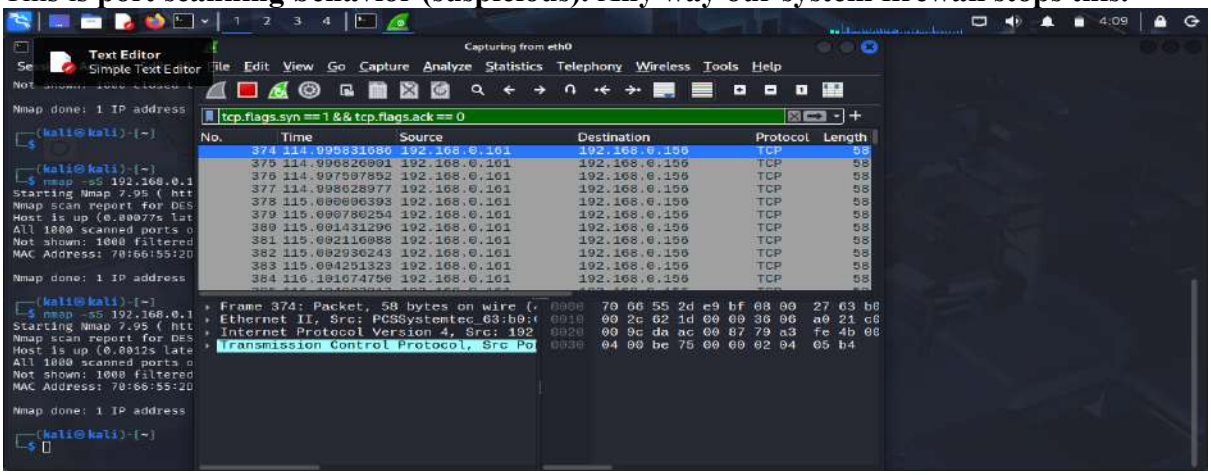
Use this filter:

tcp.flags.syn == 1 && tcp.flags.ack == 0

Now you will observe:

- Multiple SYN packets
- Different ports
- Incomplete connections
- Source address is Kali linux and Destination Address is Windows

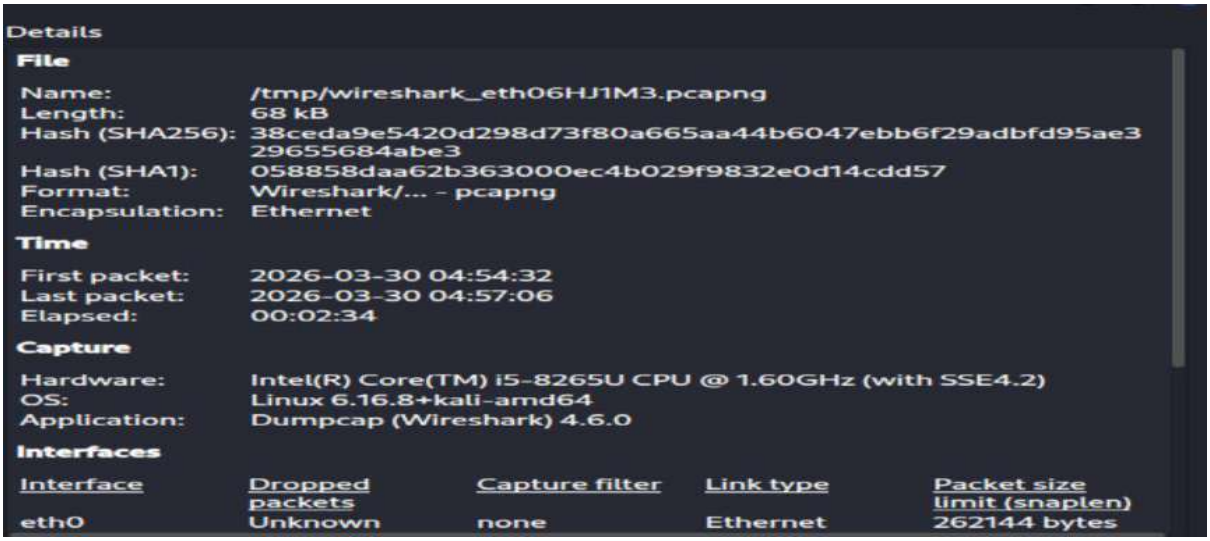
This is **port scanning behavior (suspicious). Any way our system firewall stops this.**



Step 8:

In wireshark

Go to Statistics->Capture File Properties



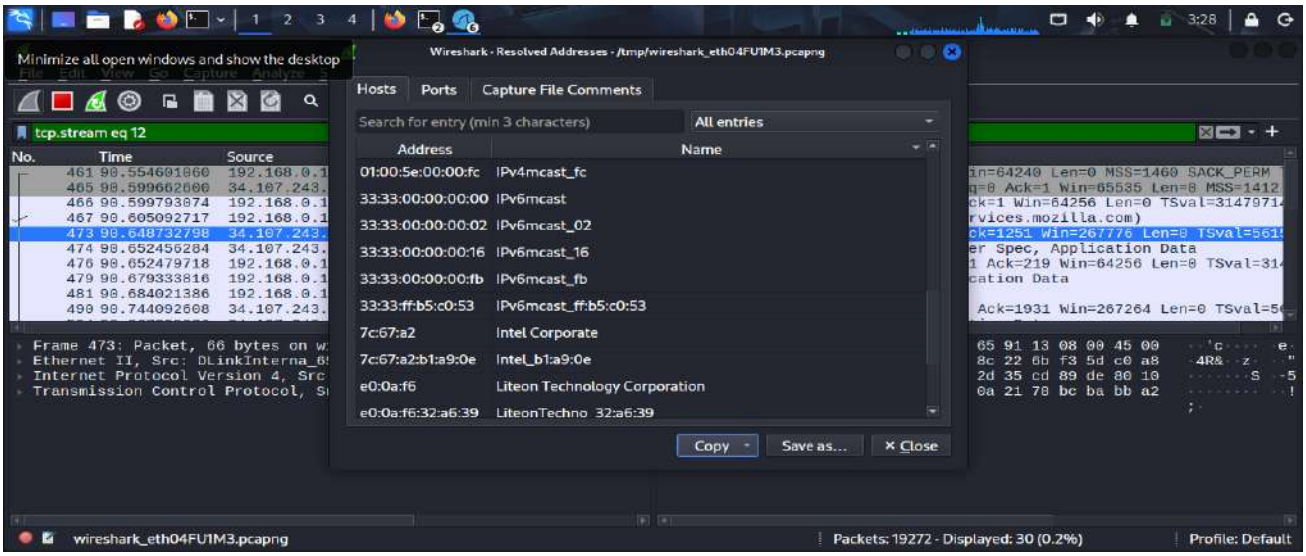
- It provides a quick overview of network traffic and helps detect suspicious activities without analyzing each packet individually.

Step 9:

In wireshark **Go To**

Statistics->Resolved Address

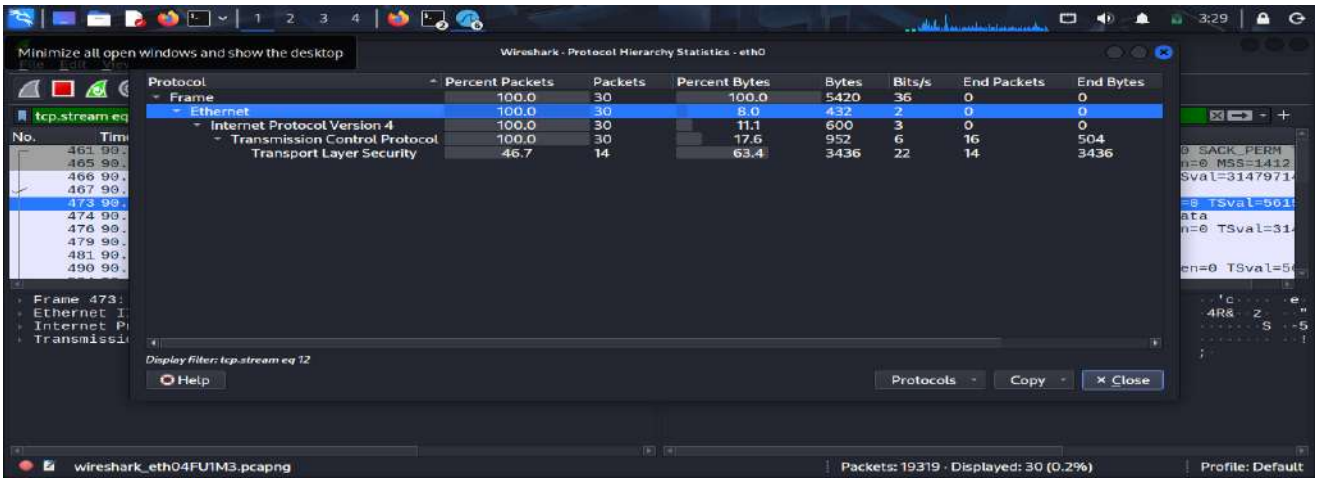
Resolve Address in Wireshark converts numerical network addresses (like IP or MAC) into **human-readable names** such as domain names or device names.



Step 10: Go to

Statistics-> Protocol Hierarchy Statistics

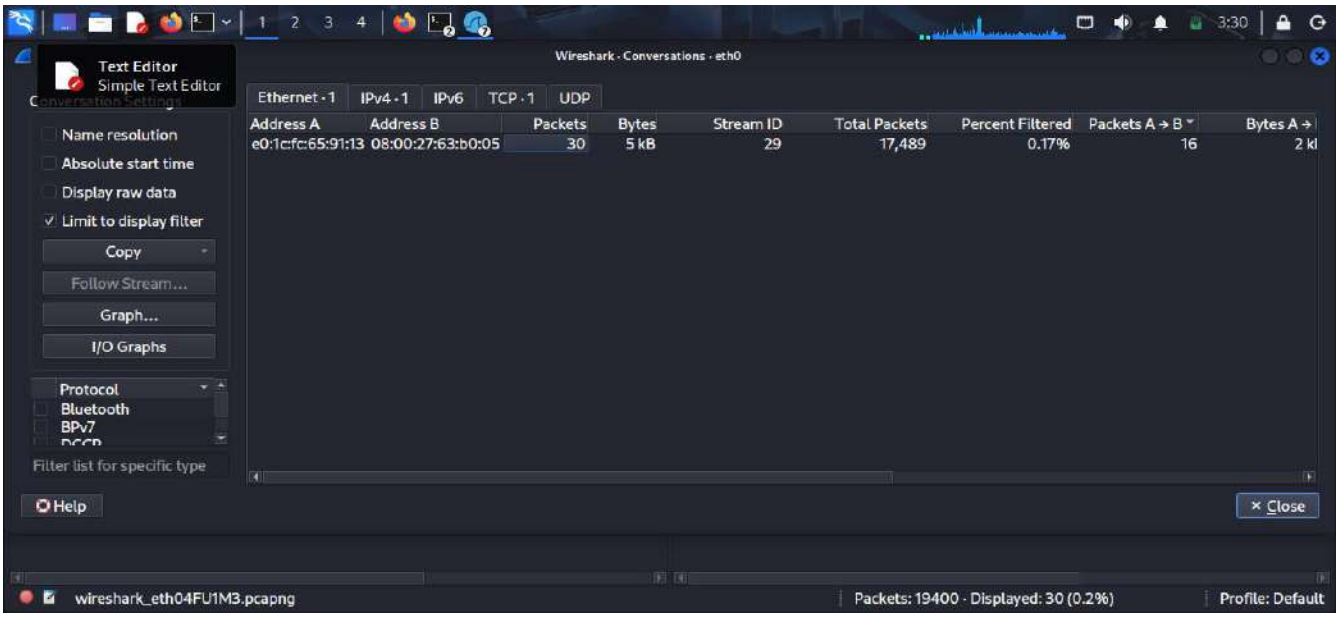
Protocol Hierarchy Statistics shows the breakdown of all network protocols in the captured traffic in a hierarchical (layer-wise) format along with their percentage and packet count.



Step 11: Go to

Statistics->Conversations

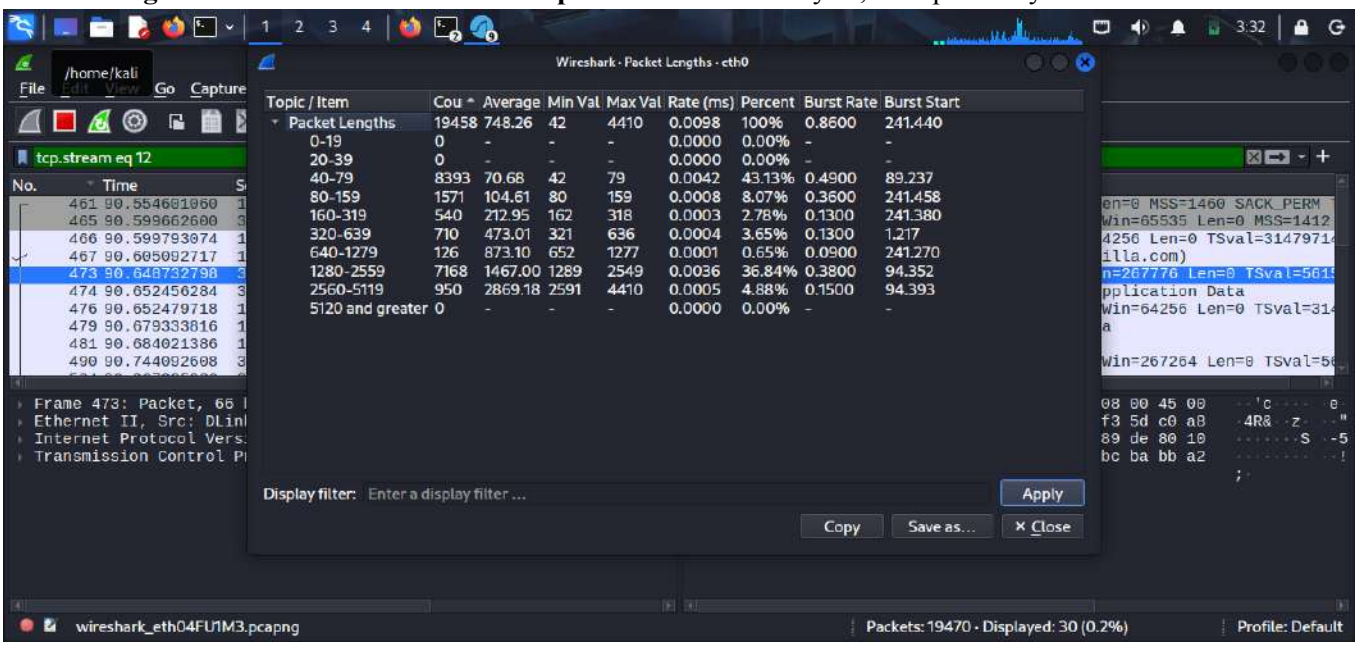
Conversations in Wireshark shows communication between two endpoints (devices), including source and destination addresses, number of packets, and data transferred.



Step 12:

Statistics-> Packet Length

Packet Length refers to the **size of a network packet** measured in bytes, as captured by Wireshark.



Step 13:

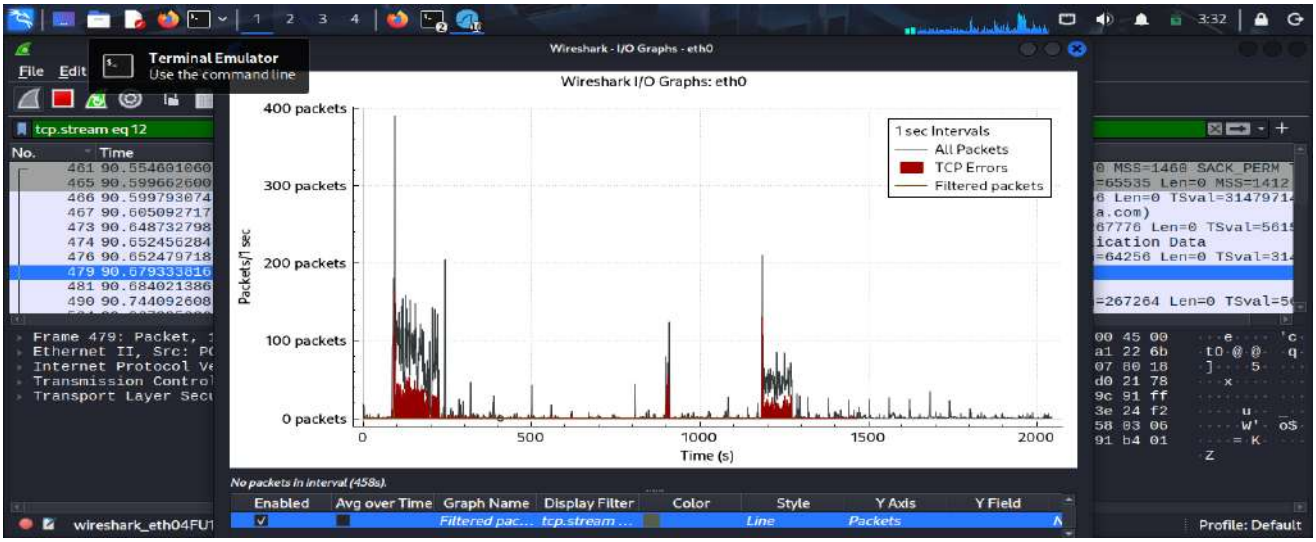
Statistics-> End Points

Endpoints in Wireshark represent individual network devices (hosts) involved in communication. It shows each device separately along with the number of packets sent/received and data usage.

Step 14:

Statistics-> I/O Graphs

I/O Graphs (Input/Output Graphs) in Wireshark are used to display network traffic over time in the form of a graph, showing the number of packets or bytes captured per second.



Result

Network packets were successfully captured and analyzed using Wireshark. Different protocols and communication patterns were observed and studied.

Conclusion

Wireshark is an effective tool for network forensics. It helps in capturing and analyzing network traffic, identifying suspicious activities, and understanding communication between systems. It plays a vital role in cybersecurity and incident investigation.

Viva Points

- What is packet sniffing?
- Difference between TCP and UDP
- What is a protocol analyzer?
- What is a filter in Wireshark?

EXPERIMENT 20:

Title: Privacy Audit of Popular Apps (Desktop **WhatsApp**) and Websites (**Facebook**) & Data Breach Case Study Analysis

Aim

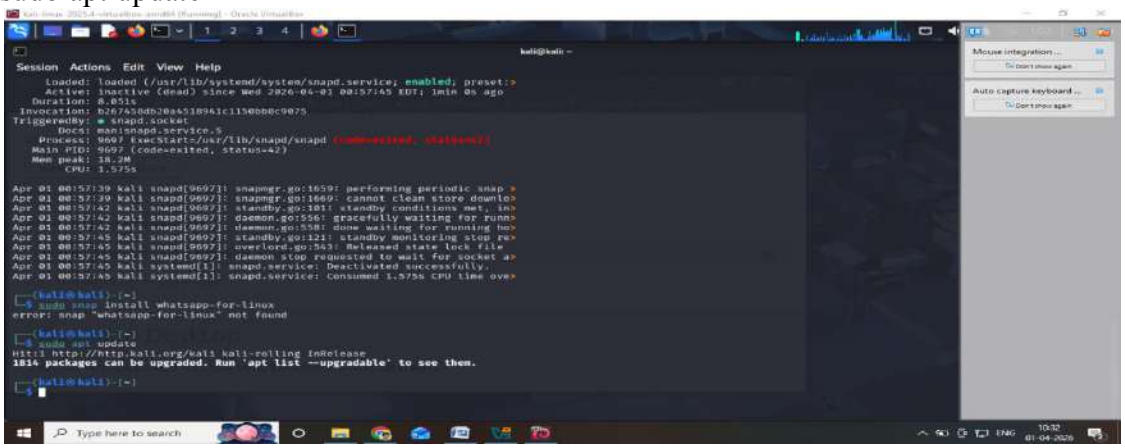
To analyze privacy, network behavior, and security risks of applications and websites using Kali Linux tools.

Tools Required

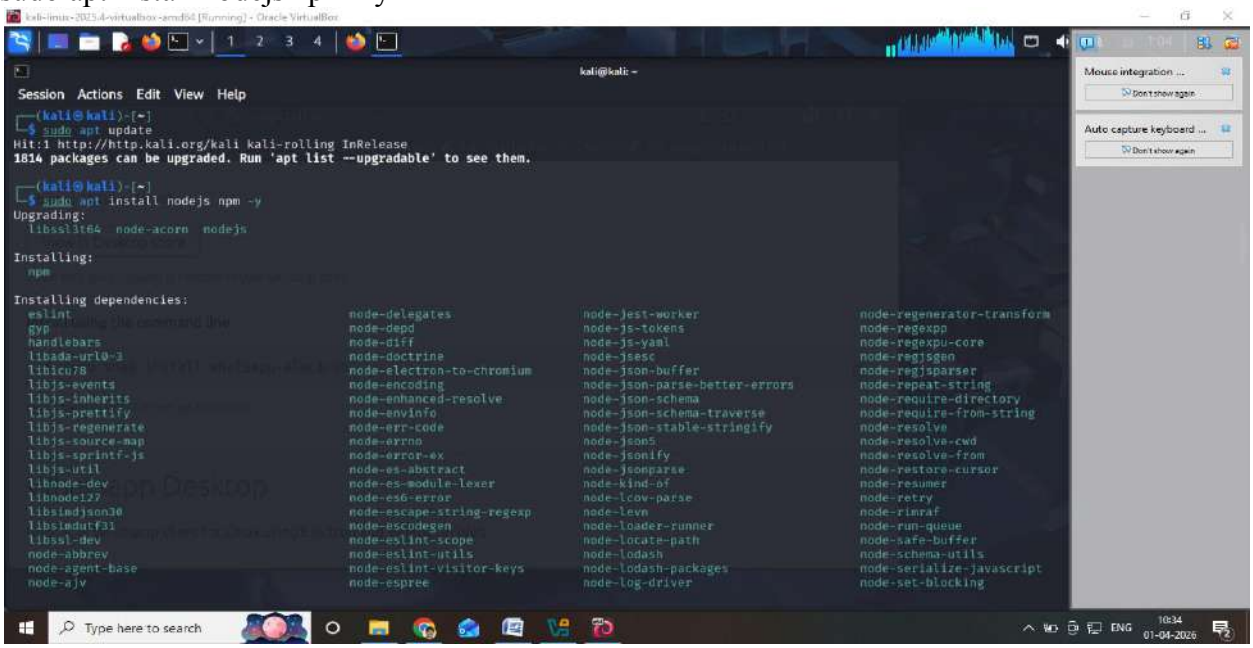
- **Wireshark**
- **Burp Suite**
- Kali Linux (VirtualBox)
- Desktop WhatsApp (Nativefier)
- Web Browser (Firefox)

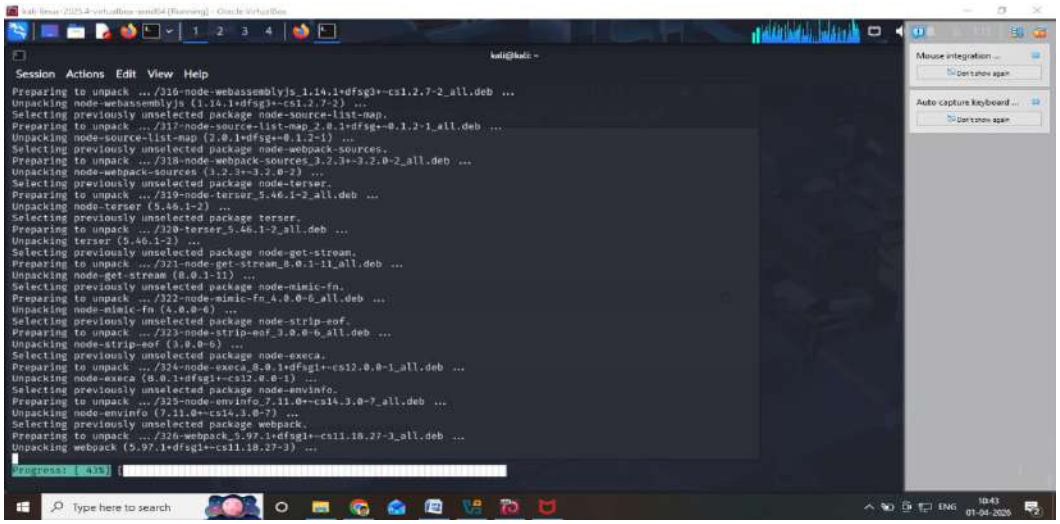
Step 1: Install Node.js & npm

sudo apt update



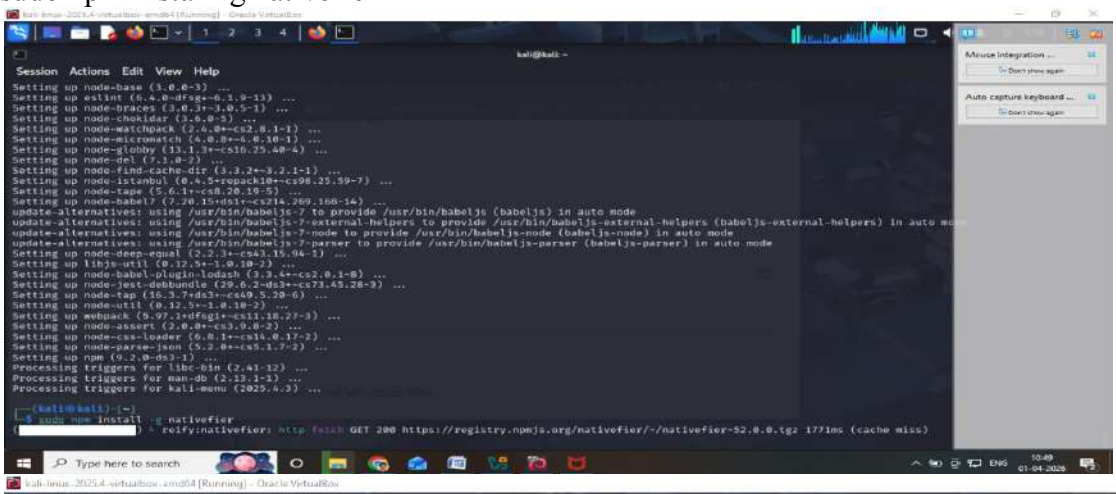
sudo apt install nodejs npm –y

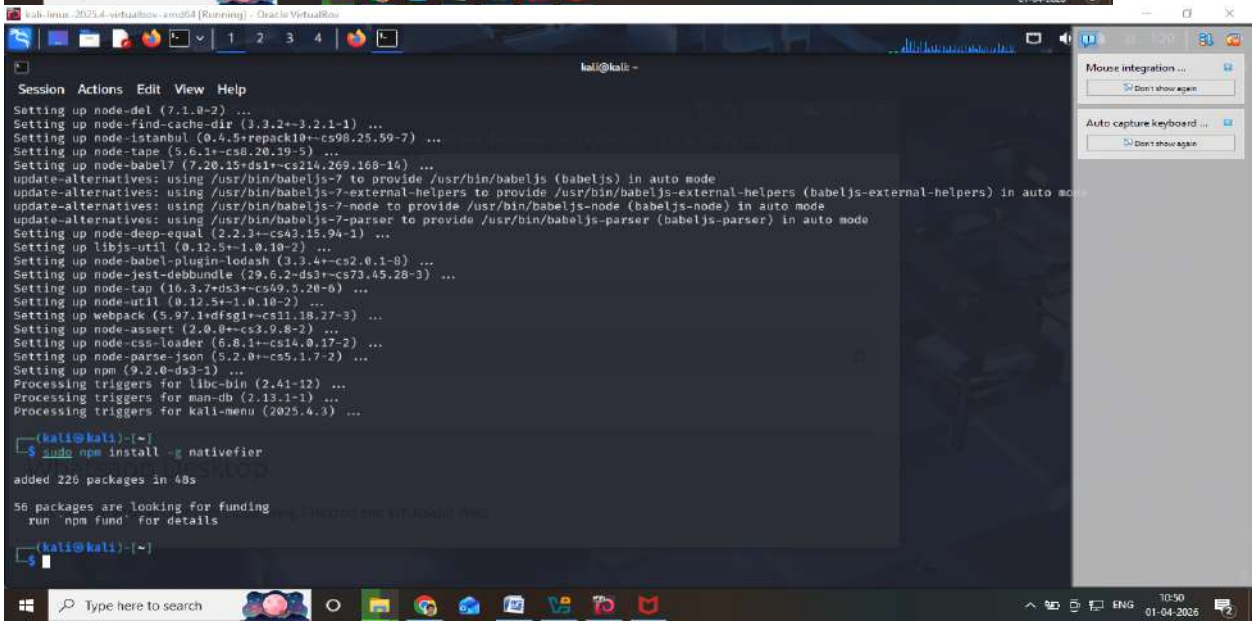




Step 2: Install Nativefier

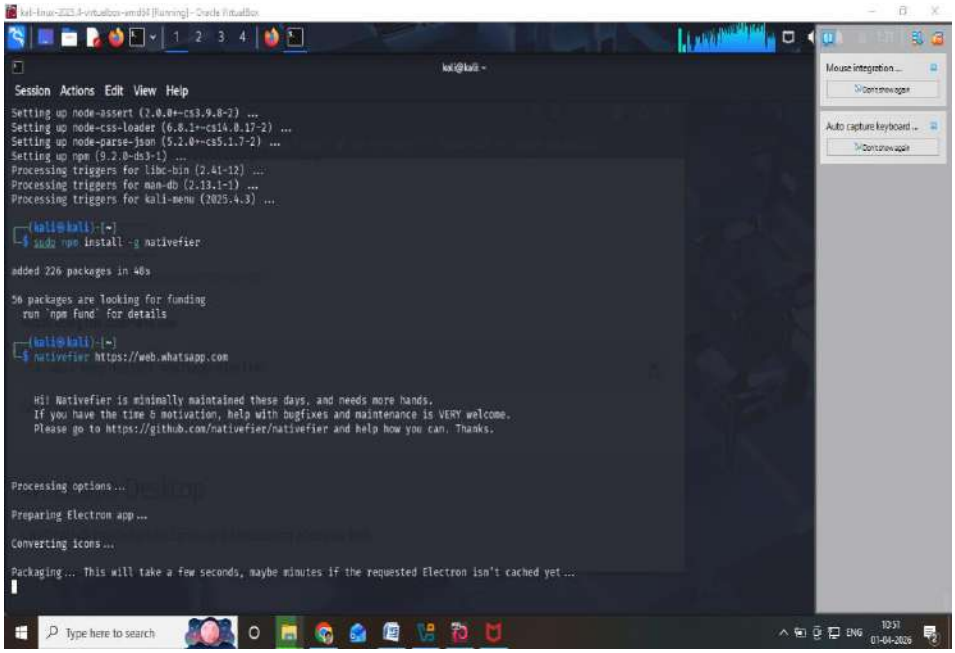
sudo npm install -g nativefier





Step 3: Create WhatsApp Desktop App

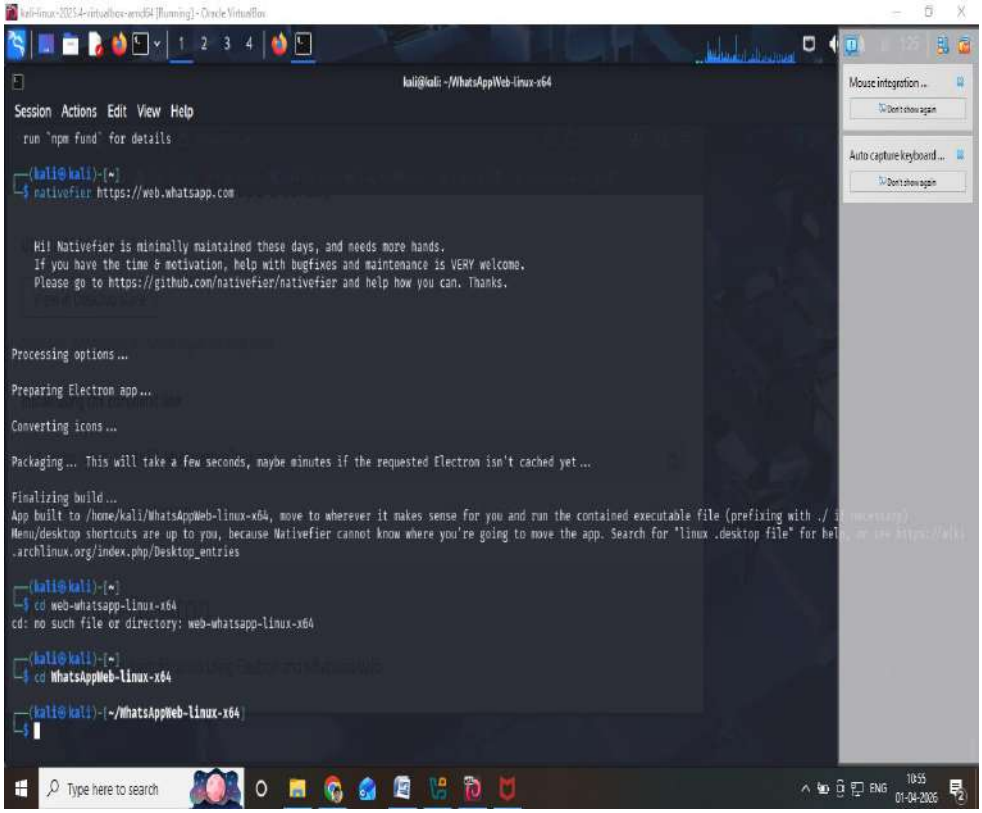
nativefier https://web.whatsapp.com



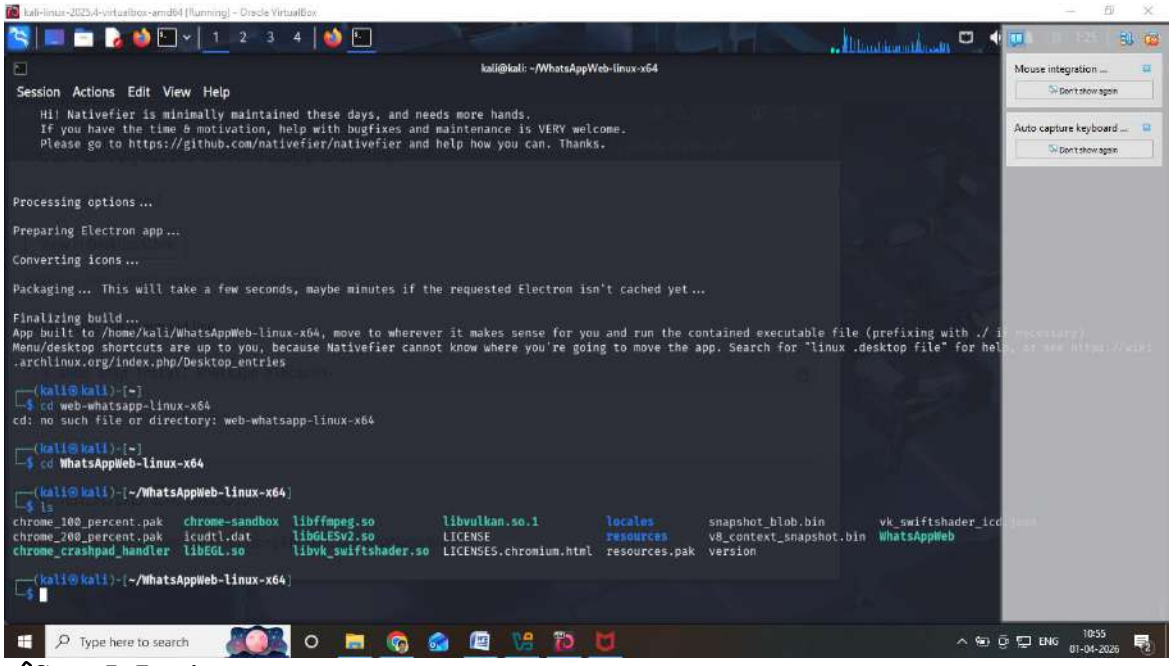
It will create a folder like:
web-whatsapp-linux-x64

Step 4: Run WhatsApp

cd WhatsAppWeb-linux-x64

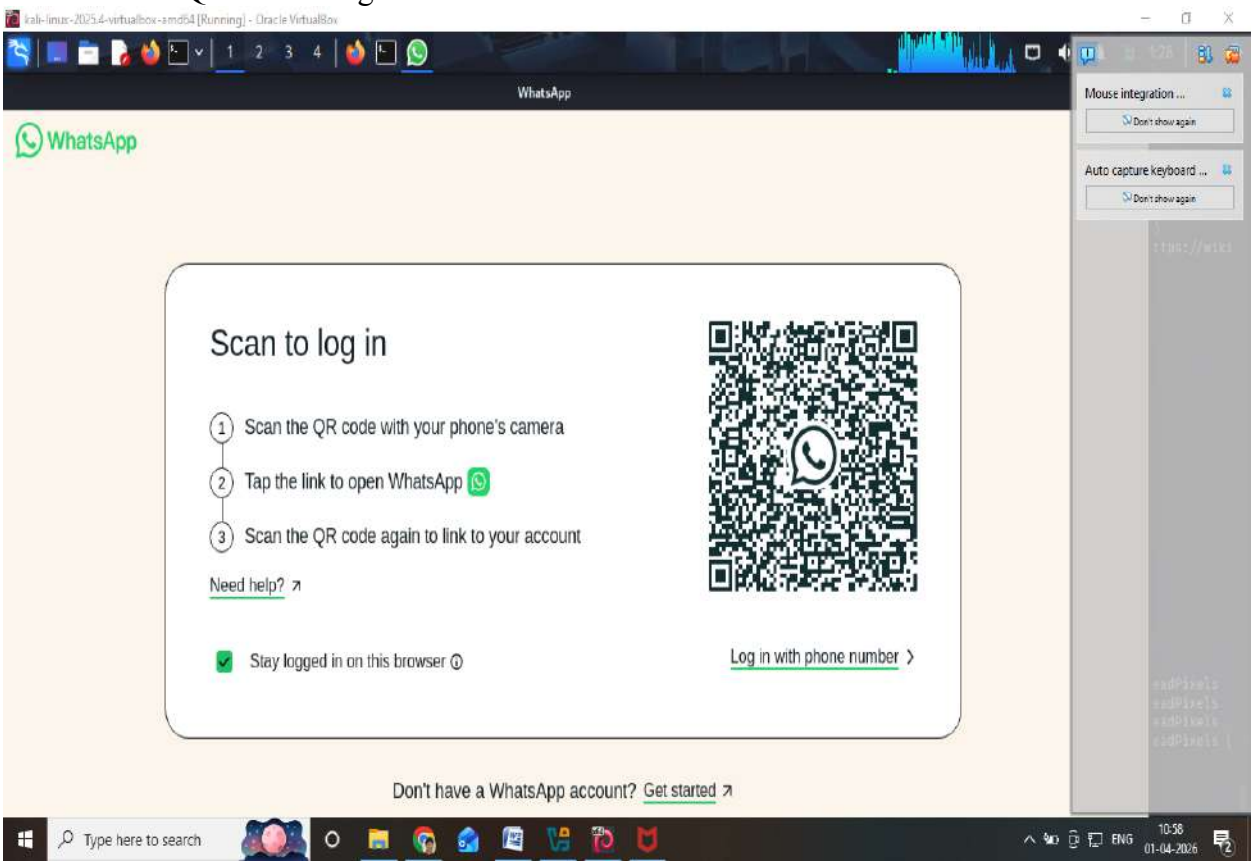


./WhatsAppWeb



✅Step 5: Login

- Scan QR code using mobile



- Done

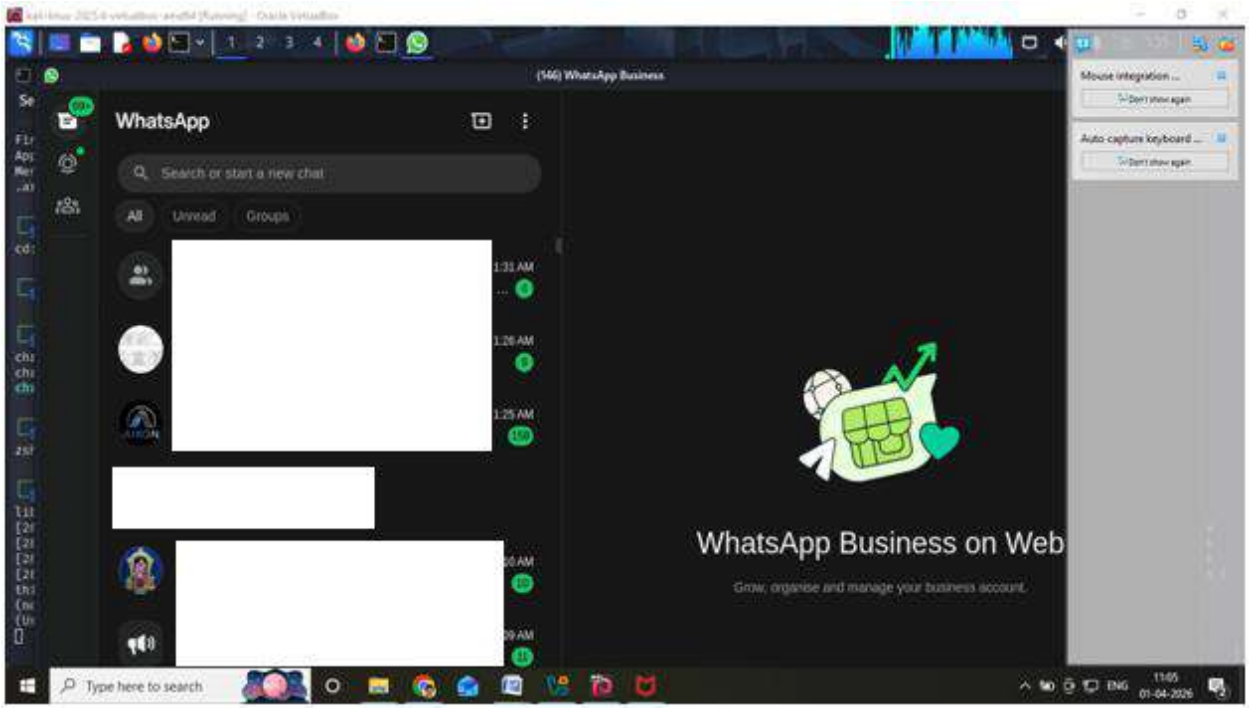
PART 1: Privacy Audit of Desktop WhatsApp

Step 1: Launch WhatsApp Desktop

cd WhatsAppWeb-linux-x64

./WhatsAppWeb

- QR Code appears
- Login using mobile → Linked Devices

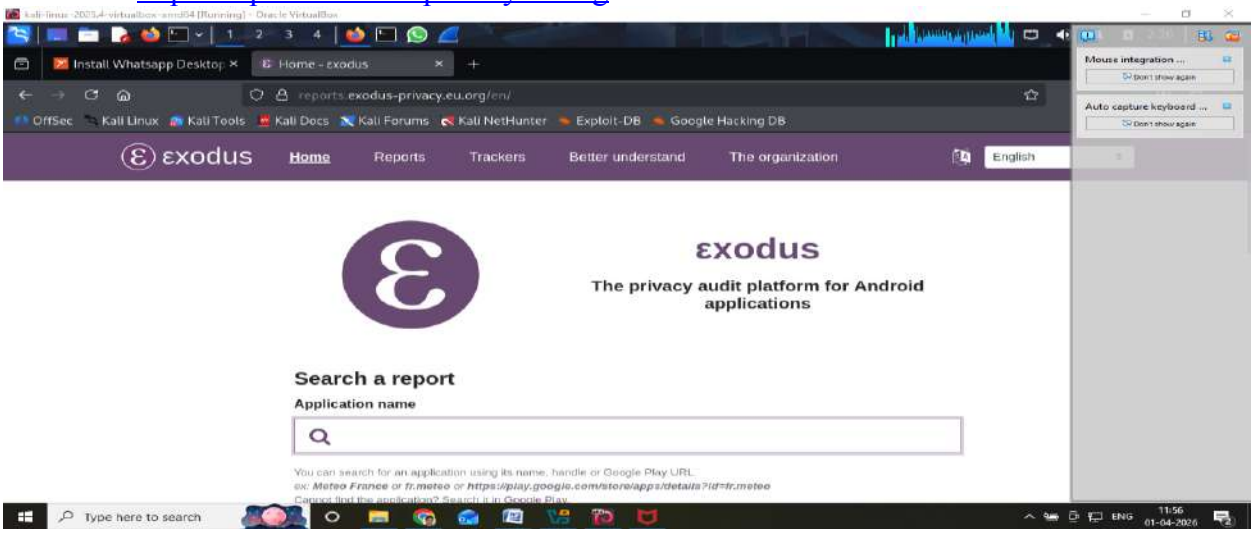


Step 1: Install WhatsApp APK (Optional in Emulator)

- Download APK from trusted source or use your mobile

Step 2: Analyze Trackers using Exodus Privacy

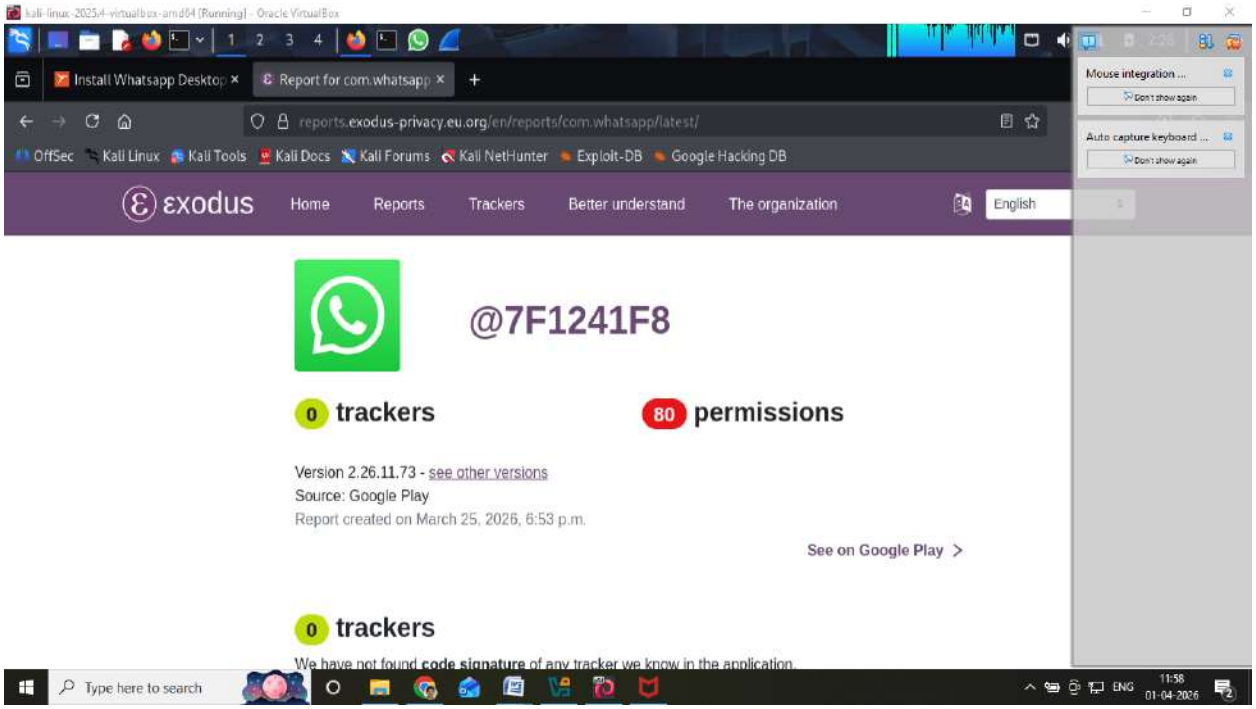
1. Open browser in Kali
2. Go to: https://reports.exodus-privacy.eu.org



3. Search for **WhatsApp**

Observation:

- Detects trackers like:
  - Google Analytics
  - Crashlytics
  - Facebook Analytics

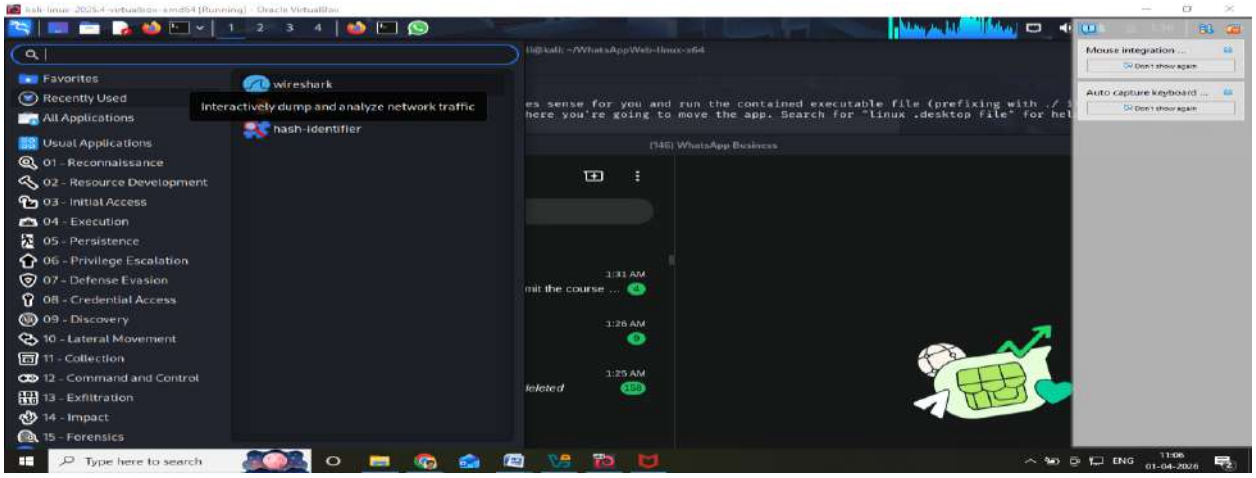


Record in lab:

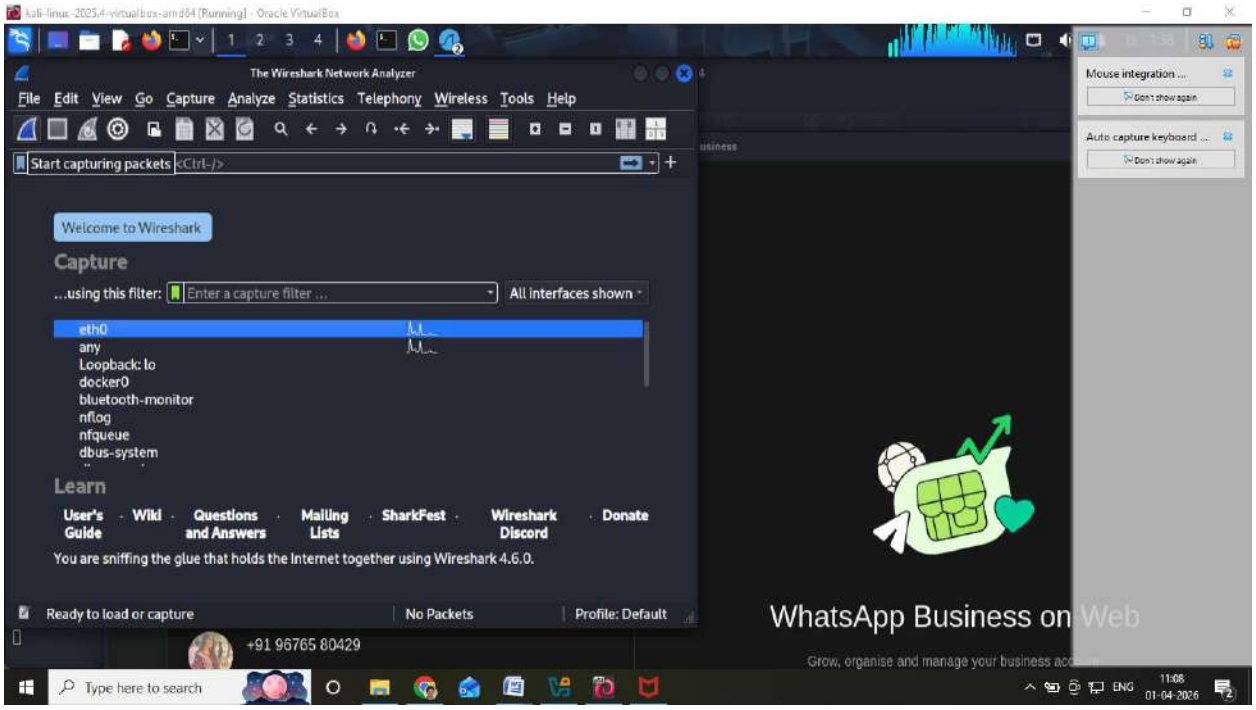
- Number of trackers
- Permissions used

Step 2: Start Wireshark

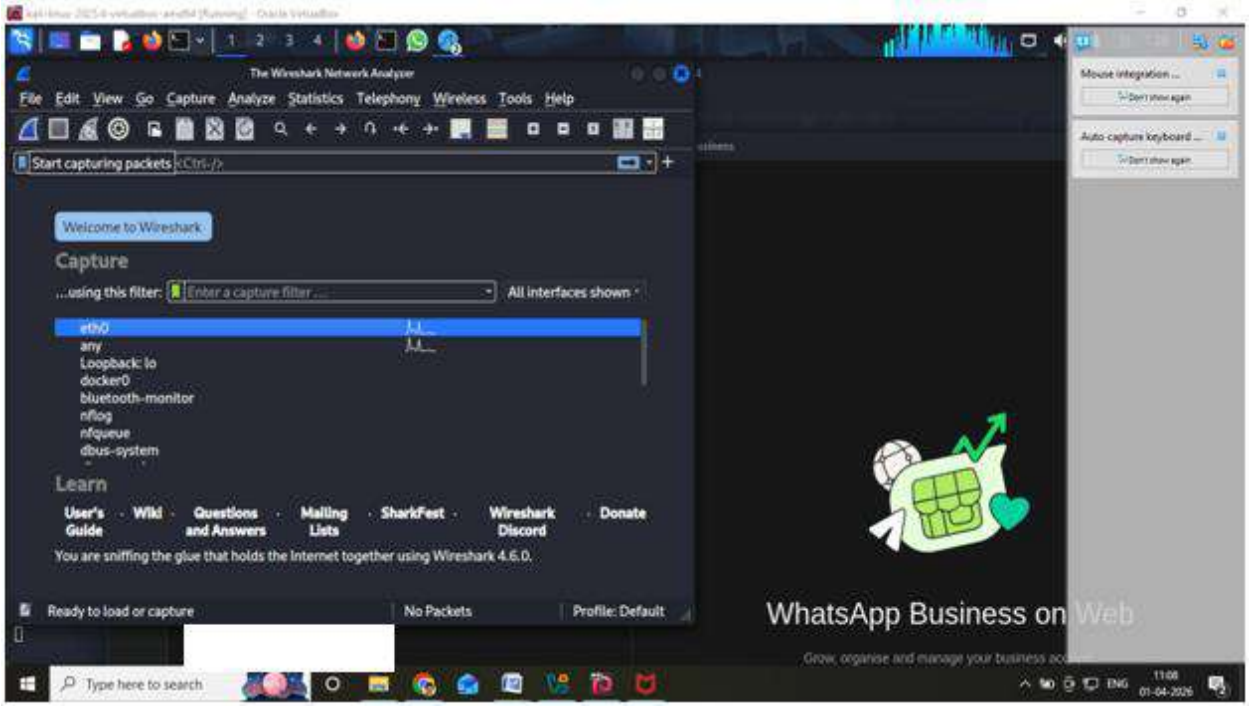
Wireshark



- Select interface (eth0 / wlan0)



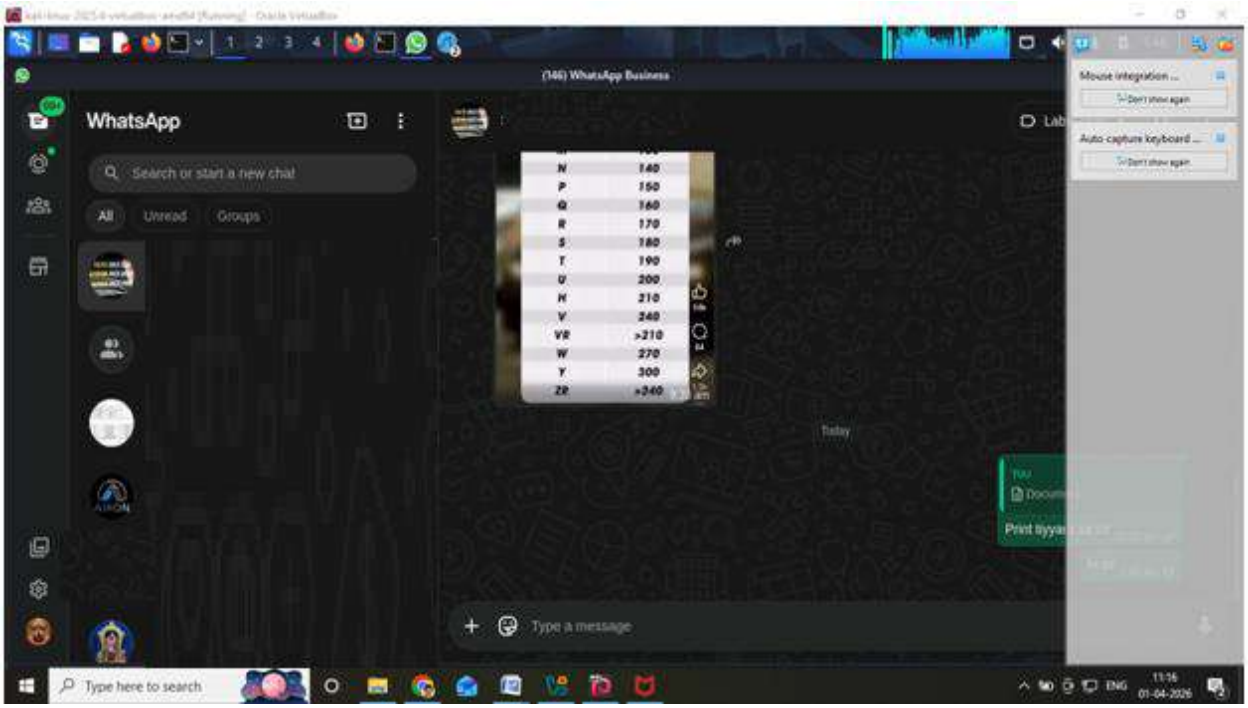
-
- Click **Start Capture**



Step 3: Generate Traffic

In WhatsApp Desktop:

- Send messages
- Send images
- Open chats

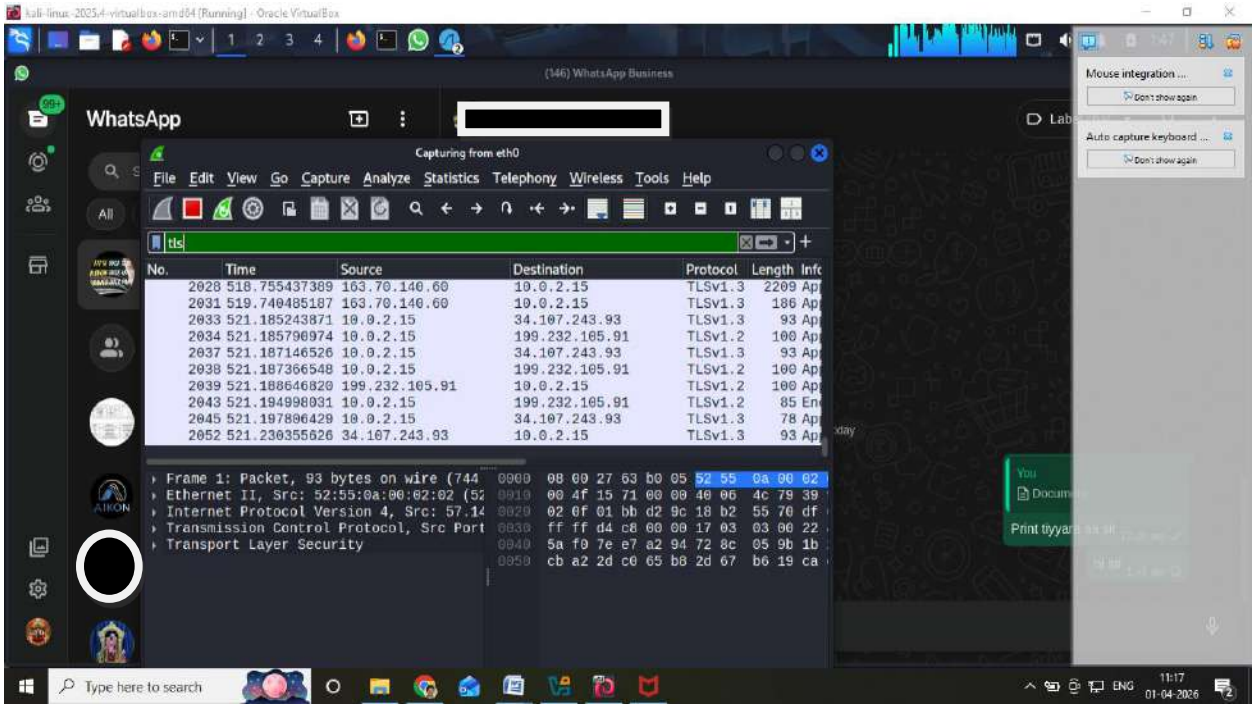


✅Step 4: Apply Filters

Use filters:

tls

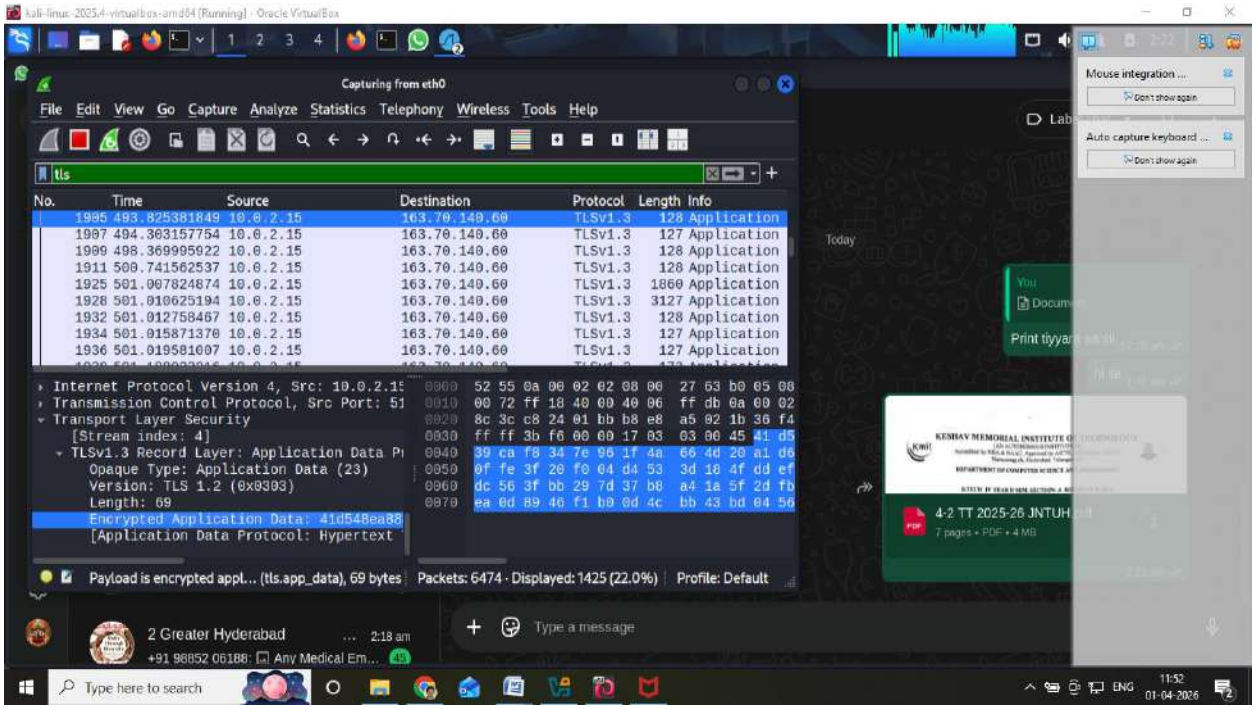
dns



Observations

- Encrypted traffic (TLS packets)
- Domains:
  - whatsapp.net
  - facebook.com

Message content NOT visible



Findings

End-to-End Encryption

Secure communication

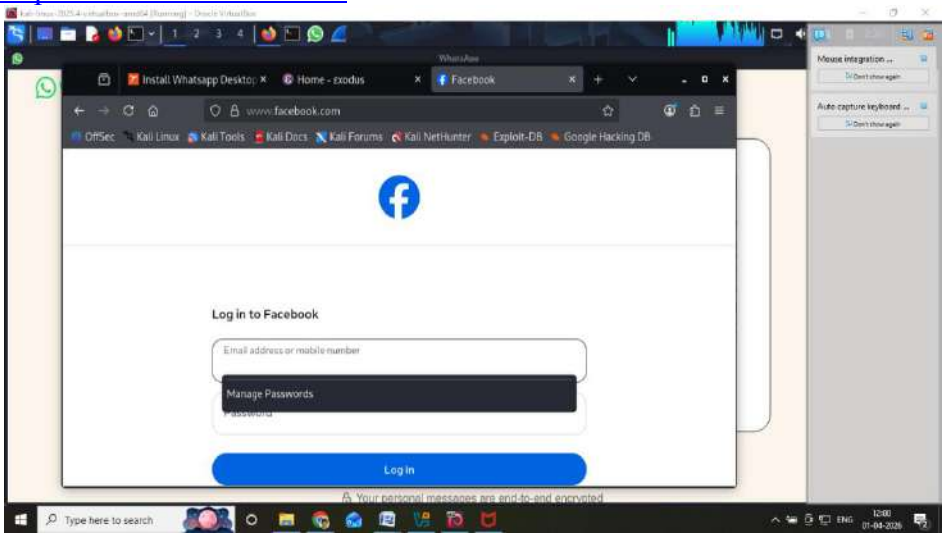
Metadata visible:

- IP address
- Time
- Server communication

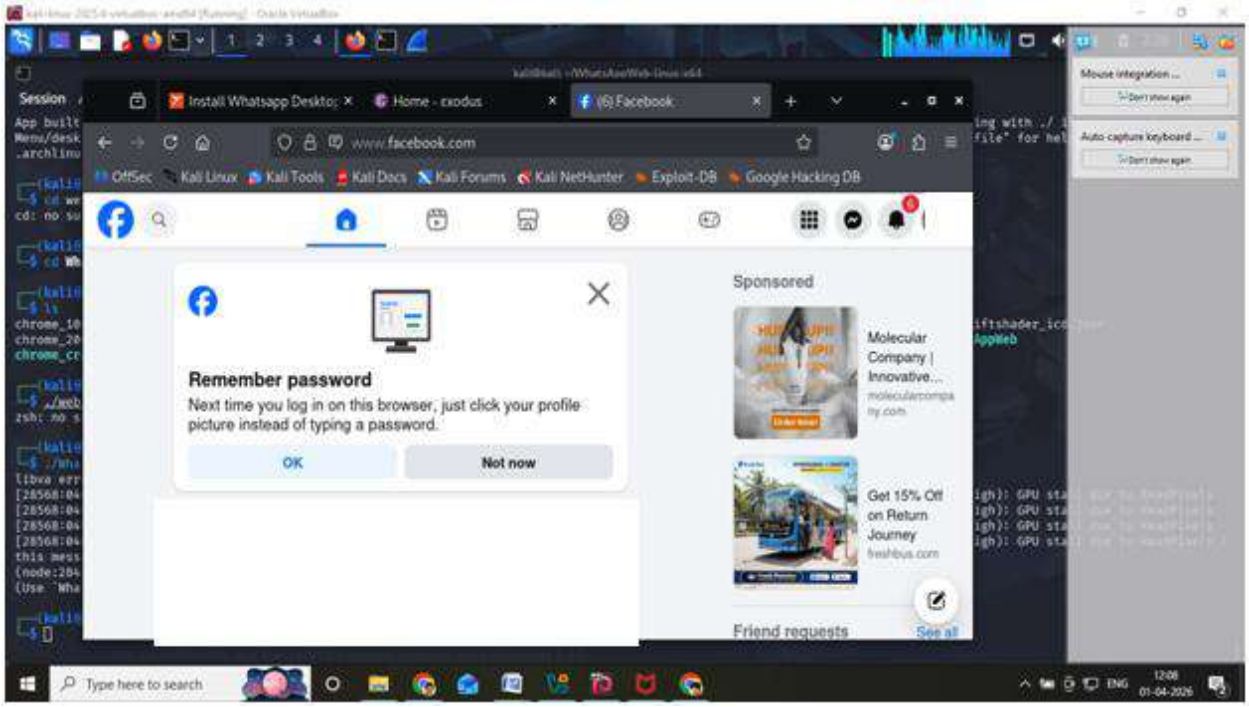
PART 2: Privacy Audit of Facebook Website

Step 1: Open Facebook

https://www.facebook.com

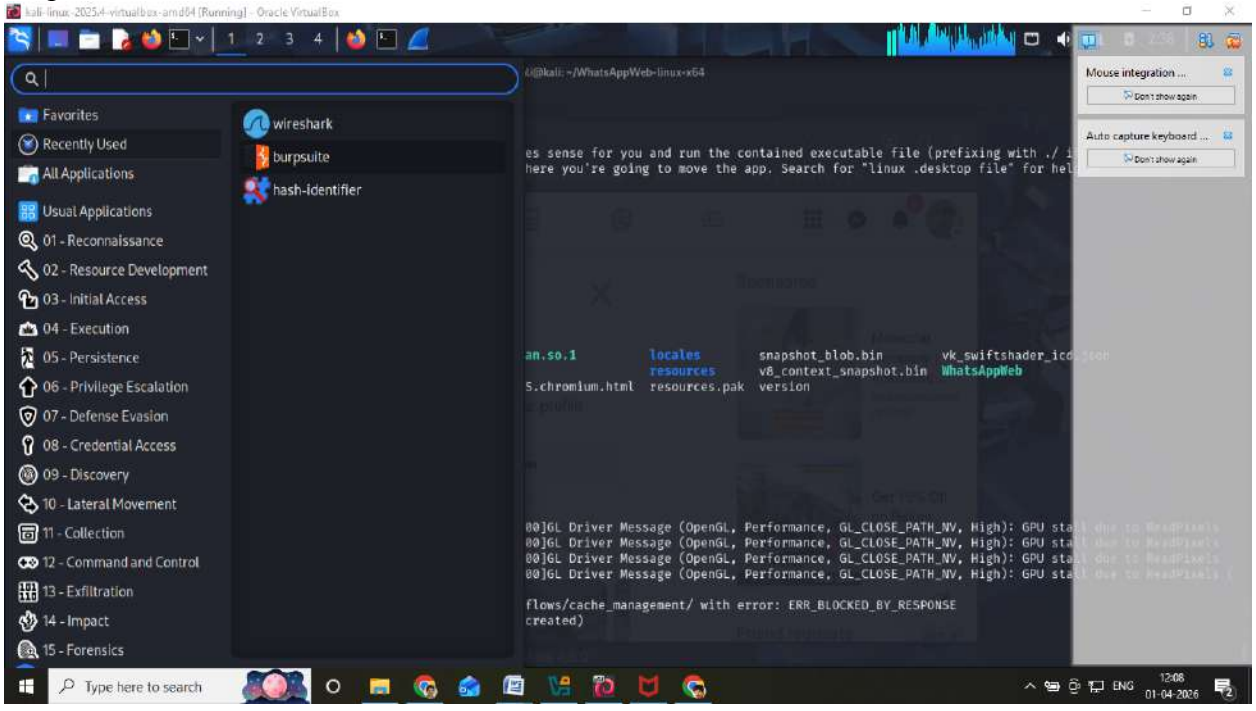


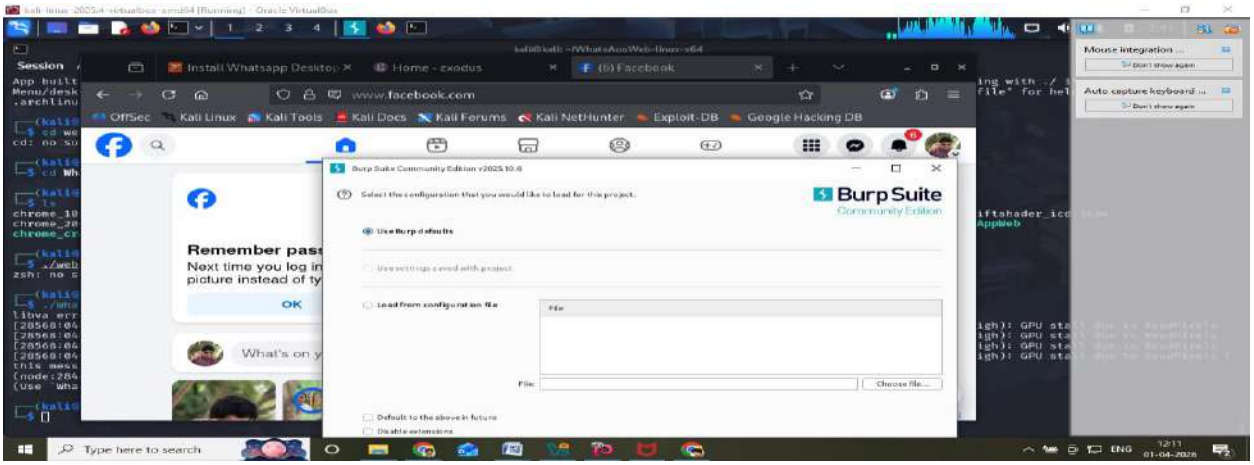
Login using test account



Step 2: Start Burp Suite

Burpsuite



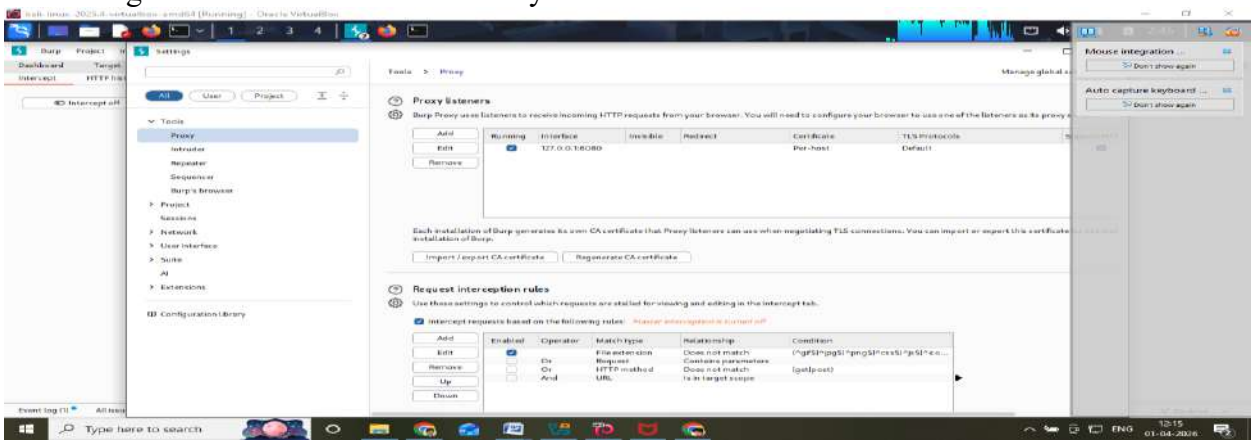


✓Step 3: Configure Proxy

- IP: 127.0.0.1
- Port: 8080

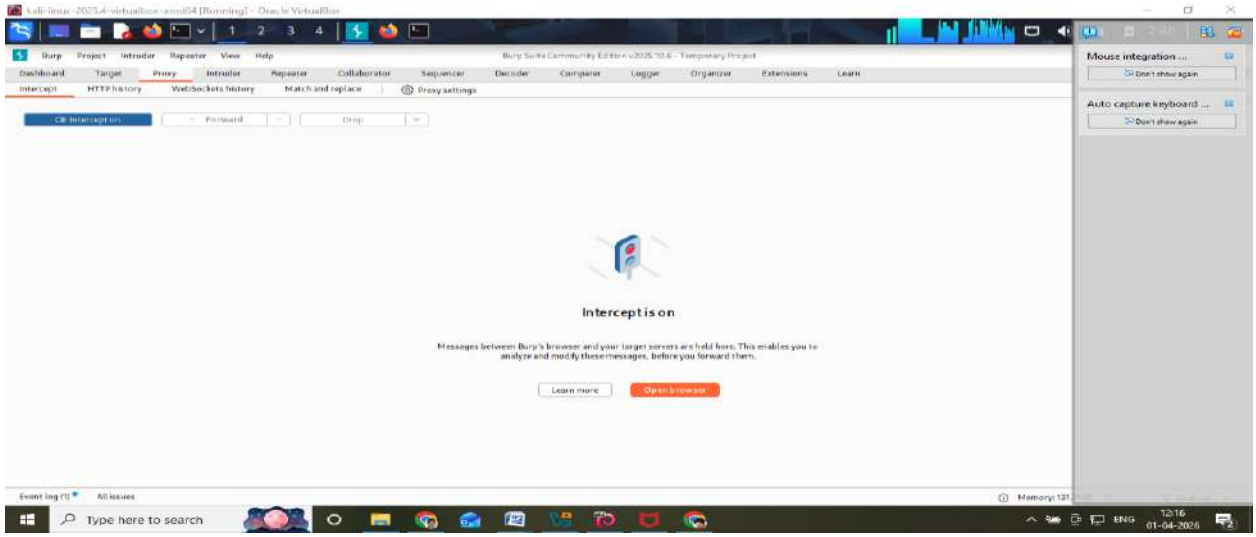
In browser:

- Settings → Network → Manual Proxy



✓Step 4: Intercept Traffic

- Turn **Intercept ON**
- Reload Facebook



Observe:

- Cookies (datr, fr)

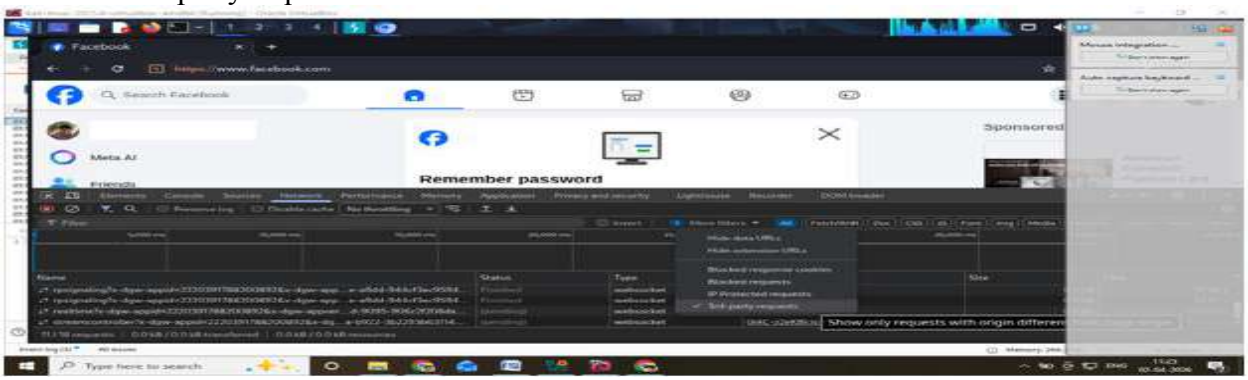
- Session IDs
- Headers

Step 5: Analyze Tracking

Press **F12 → Network tab**

Look for:

- Ads
- Analytics scripts
- Third-party requests



Findings

Heavy tracking

Uses cookies for personalization

Third-party integrations

PART 3: Data Breach Case Study

Case Study 1: Facebook Data Breach

- 533 million users affected
- Data leaked:
  - Phone numbers
  - Emails

Cause:

- API vulnerability

Case Study 2: WhatsApp Pegasus Attack

- Spyware attack via missed call
- Remote device access

Step 3: Check Email Breach

Go to:

 https://haveibeenpwned.com

Enter email → check breach status

Analysis Points

- Type of attack
- Vulnerability
- Data exposed
- Prevention

Observations

- WhatsApp uses strong encryption
- Facebook performs extensive tracking
- Data breaches occur due to vulnerabilities

Result

Privacy audit was successfully performed using Kali Linux tools. WhatsApp ensures secure communication through encryption, while Facebook shows extensive tracking behavior. Data breach analysis highlighted risks of poor security practices.

Experiment 21 :Conducting a Security Audit and Risk Assessment

1. Aim

To perform a security audit on a Windows system and identify potential vulnerabilities by conducting a risk assessment and suggesting mitigation strategies.

2. Tools Required

- Windows Operating System (Windows 10/11)
- Windows Security (Defender / Antivirus)
- Command Prompt
- Task Manager
- Web Browser (Google Chrome / Microsoft Edge)

3. Introduction

A security audit is the process of evaluating a system's security measures to identify vulnerabilities. Risk assessment involves analyzing potential threats, vulnerabilities, and their impact on the system. It helps in implementing appropriate security controls to protect data and resources from cyber threats.

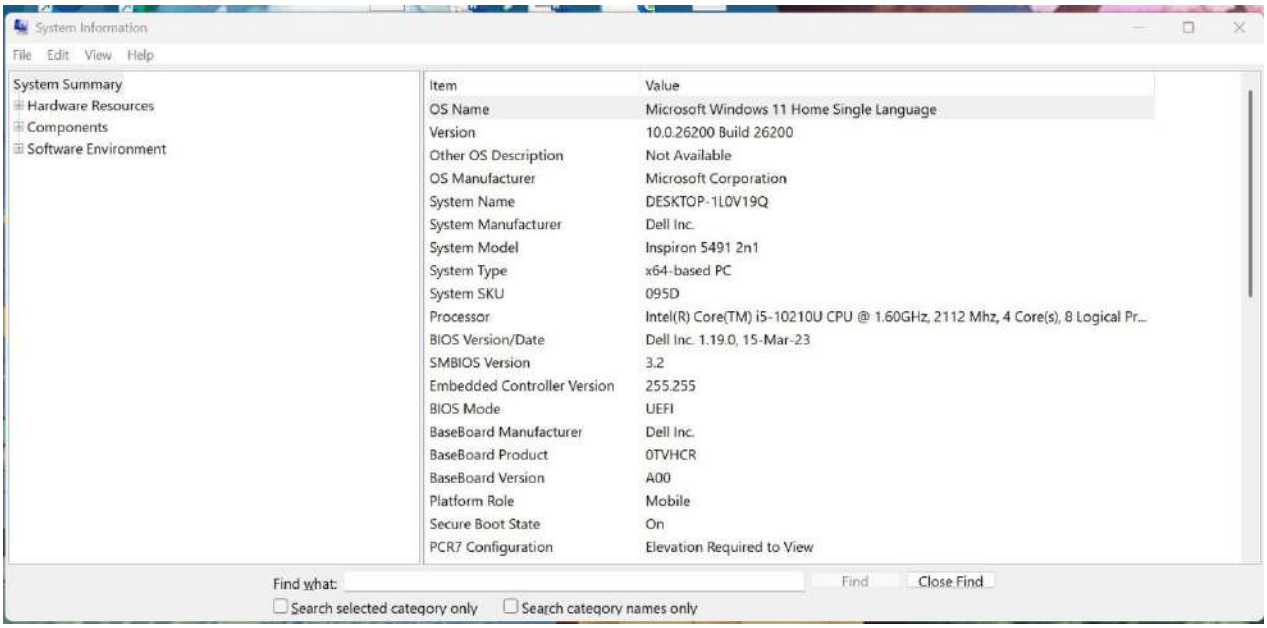
4. Procedure

STEP 1: System Information

- Press **Windows + R**
- Type: msinfo32

Note:

- OS Version
- System Type



Observation:

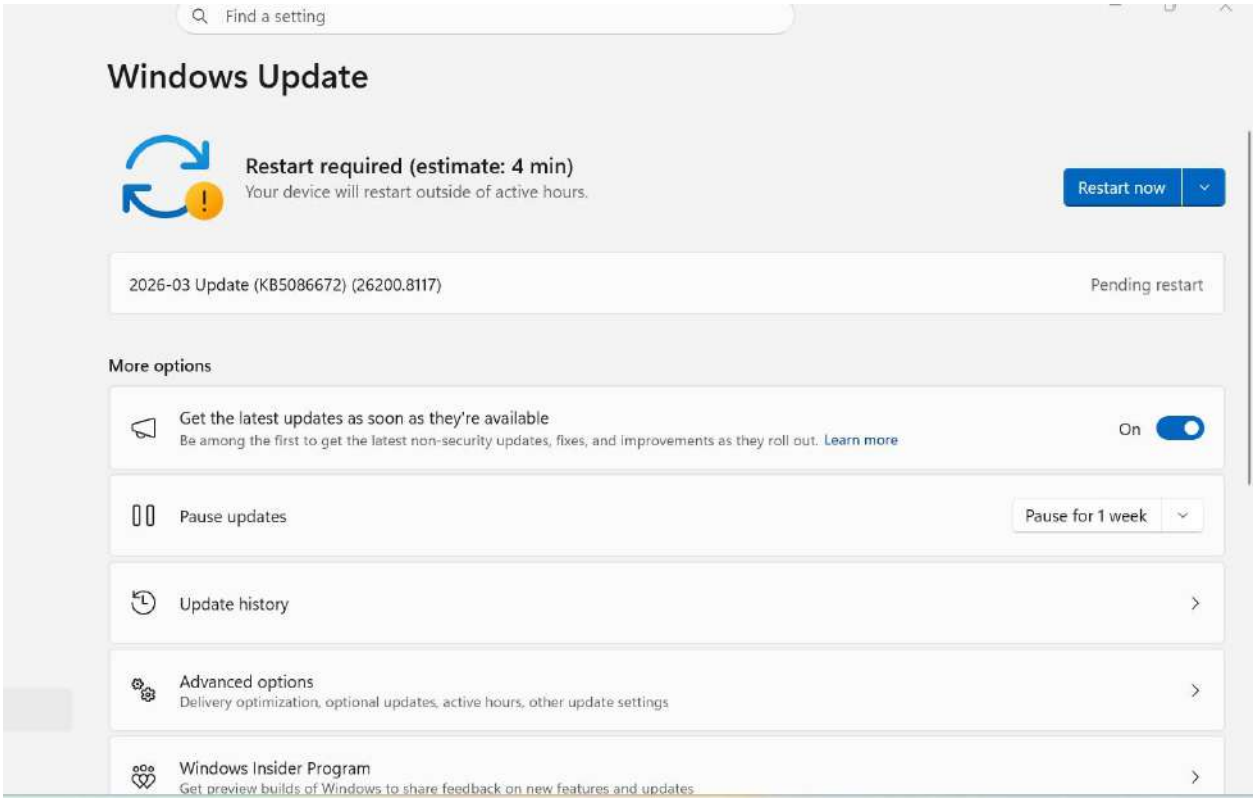
The system information was obtained using the *msinfo32* tool. The system is running on Microsoft Windows 11 Home Single Language, 64-bit operating system, with Intel i5 processor and UEFI-based architecture. Secure Boot is enabled, indicating a secure boot configuration. Secure Boot being enabled enhances system security by preventing unauthorized boot loaders.

STEP 2: Check

Windows

Updates Steps:

- Open **Settings**
- Go to **Windows Update**
- Click **Check for updates**



Observation:

The Windows Update section was checked, and it was found that updates are available and pending installation. The system requires a restart to complete the update process.
Pending updates may expose the system to known vulnerabilities. Regular updates are essential to maintain system security and protect against threats.

STEP 3: Check

Firewall Status

Steps:

- Search: **Windows Defender Firewall**
- Click **Turn Windows Defender Firewall on or off**

Observation:
The firewall settings were checked using Windows Defender Firewall. It was observed that firewall settings are managed by Kaspersky antivirus, indicating that a third-party firewall is active on the system.
The presence of a third-party firewall ensures network protection; however, misconfiguration may still expose the system to risks.

STEP 4: Check Antivirus

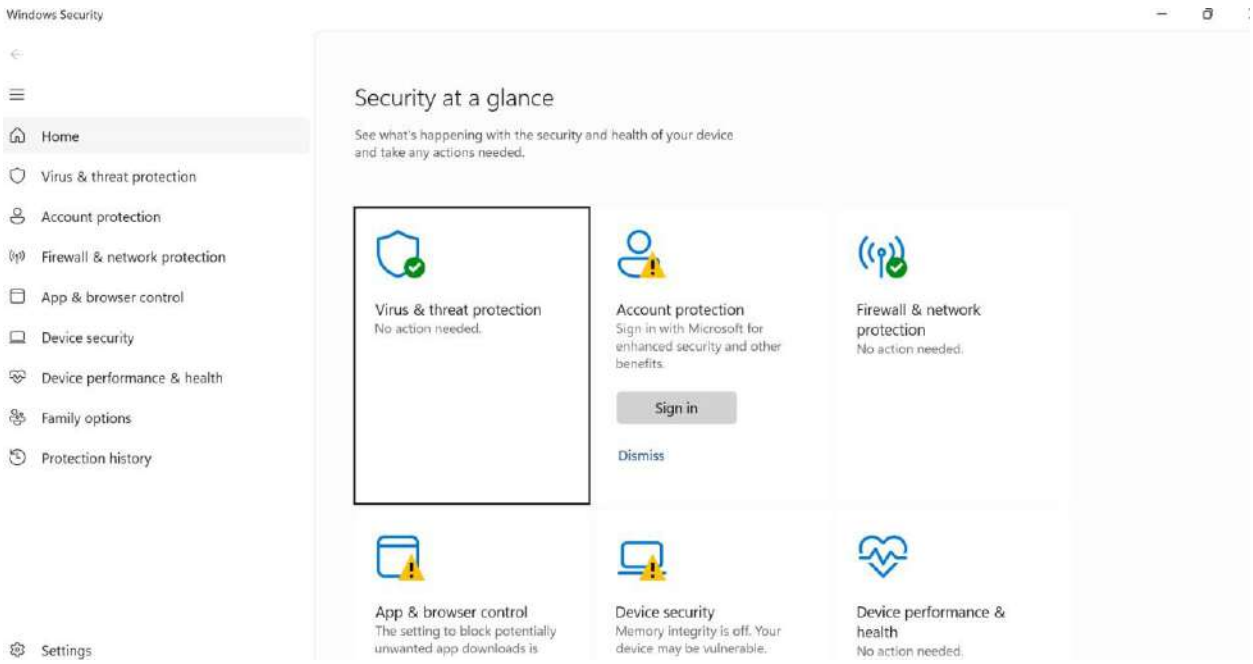
(Windows Security)

Steps:

- Open **Windows Security**
- Go to **Virus & Threat Protection**

Check:

- Real-time protection → ON
- Last scan → Recent



Observation:

The antivirus status was verified under Virus and Threat Protection. It was observed that Kaspersky antivirus is active, and no current threats were detected. Protection settings and updates are functioning properly. A third-party antivirus like Kaspersky provides advanced protection against malware, phishing, and other cyber threats.

STEP 5: Check Password

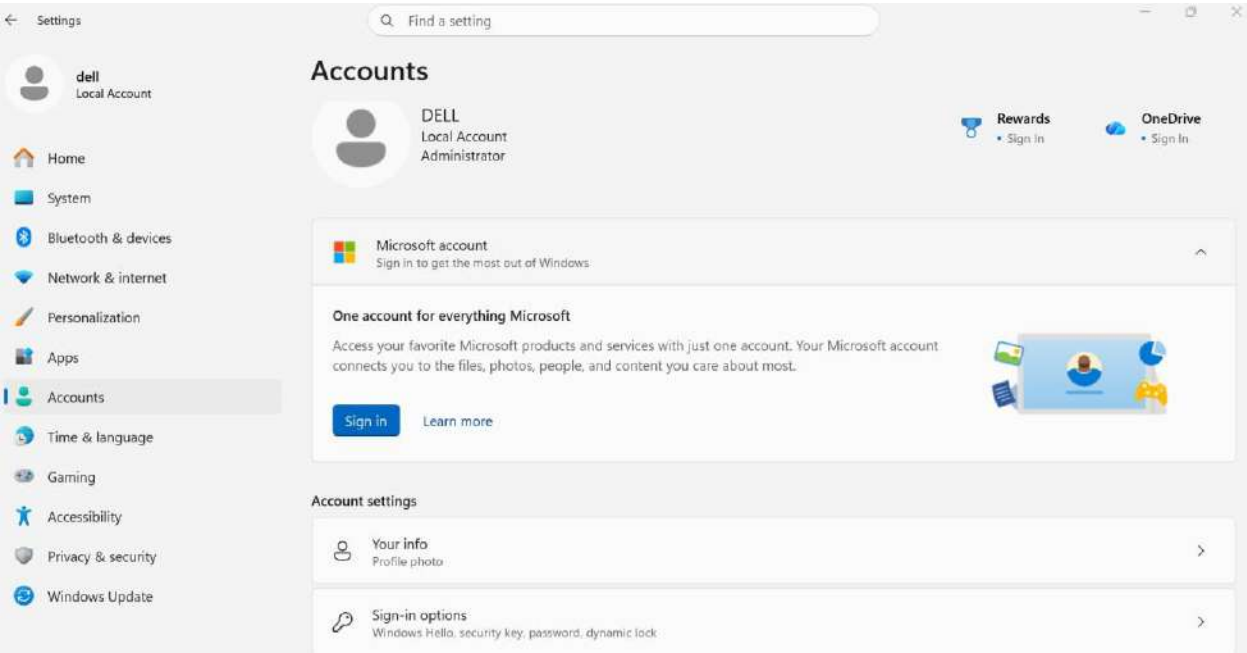
& Account Security

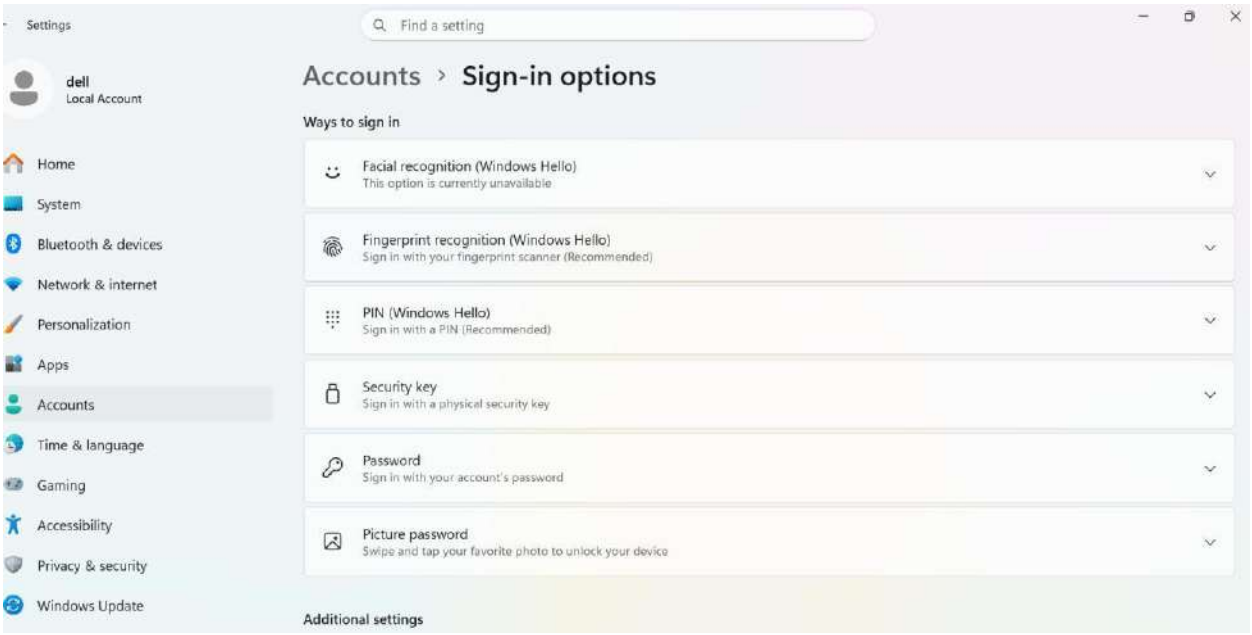
Steps:

- Settings → **Accounts** → **Sign-in options**

Check:

- Password / PIN enabled
- Windows Hello (optional)





Observation:

The account settings and sign-in options were checked. The system uses a local administrator account with multiple authentication methods such as password and PIN (Windows Hello), ensuring secure access.

Strong authentication mechanisms like PIN and biometric options enhance system security and prevent unauthorized access.

STEP 6: Check

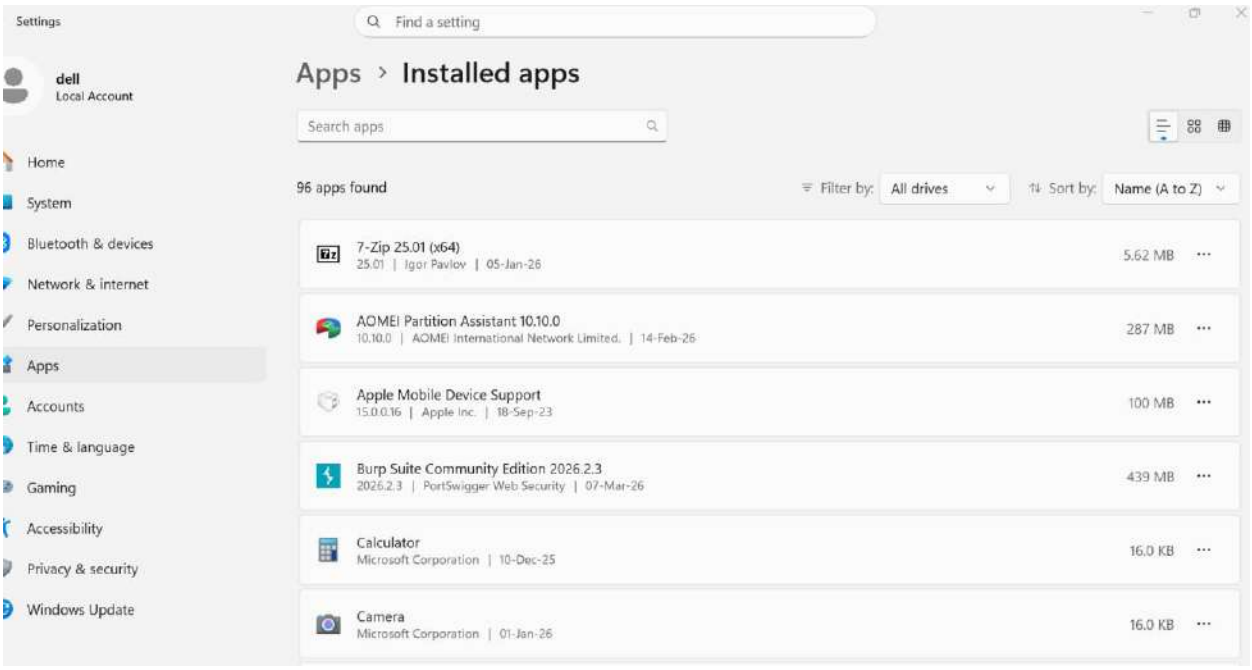
Installed

Applications Steps:

- Settings → **Apps** → **Installed apps**

Look for:

- Unknown software
- Cracked tools



Observation:

The list of installed applications was reviewed. No unauthorized or suspicious software was identified in the system.
Unverified or cracked software may introduce malware and compromise system security.

STEP 7: Check

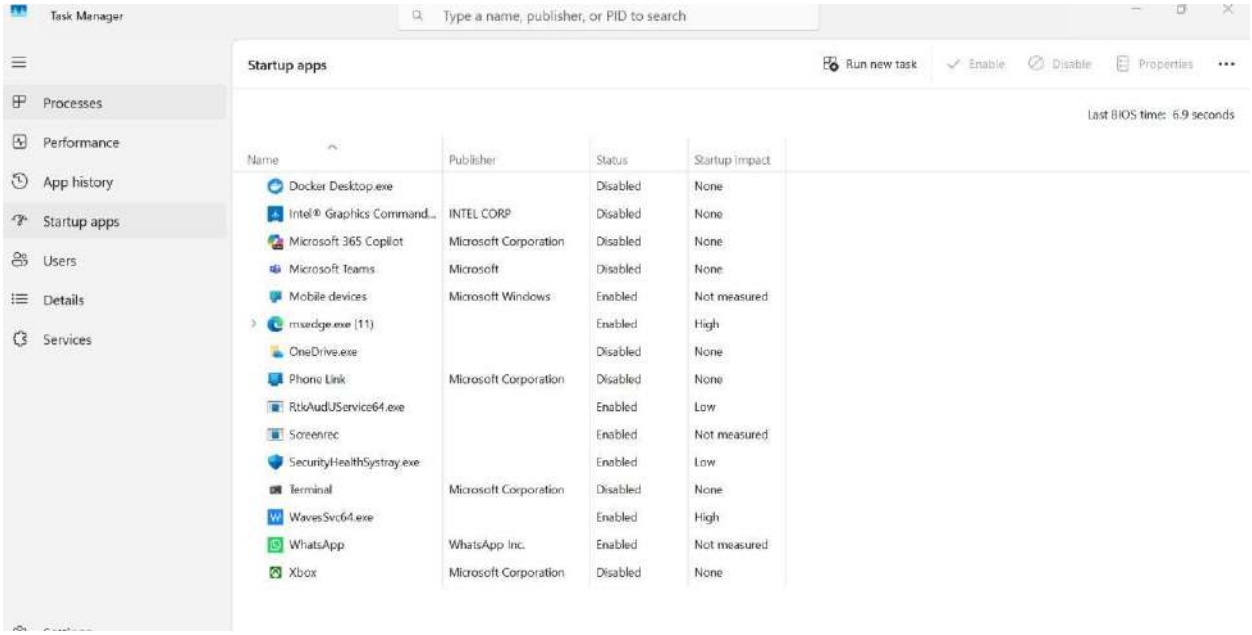
Startup

Programs Steps:

- Press **Ctrl + Shift + Esc**
- Go to **Startup tab**

Disable:

- Unknown apps



Observation:

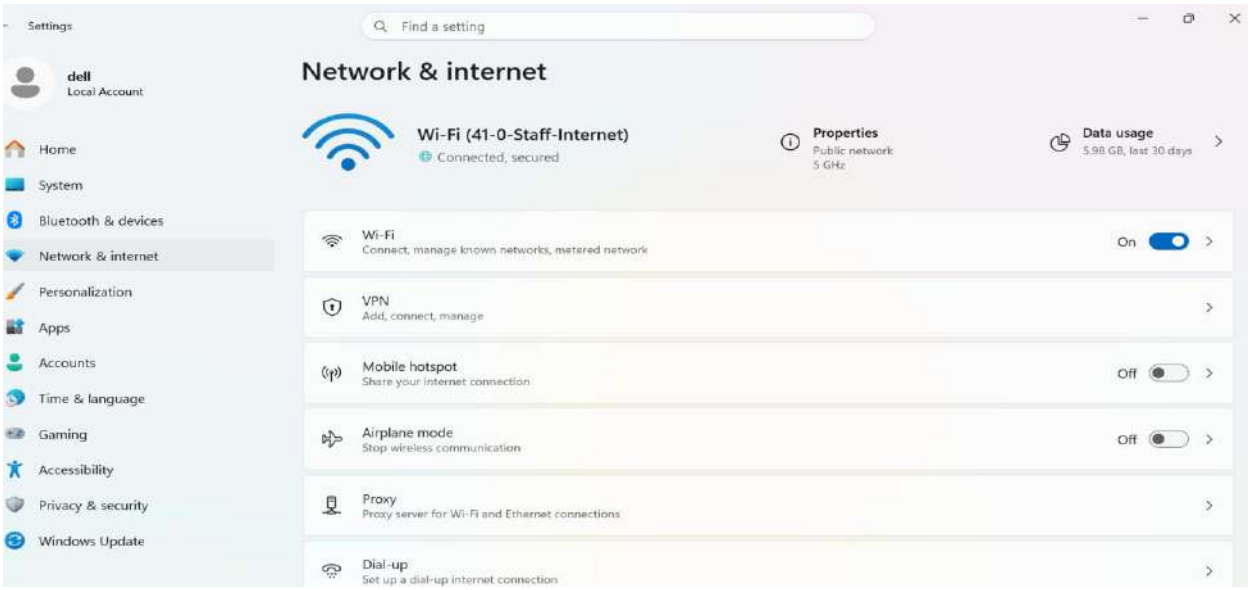
Startup applications were analyzed using Task Manager. Some applications were enabled at startup, which may impact system performance and security. Disabling unnecessary startup applications reduces attack surface and improves system performance.

STEP 8: Check

Network

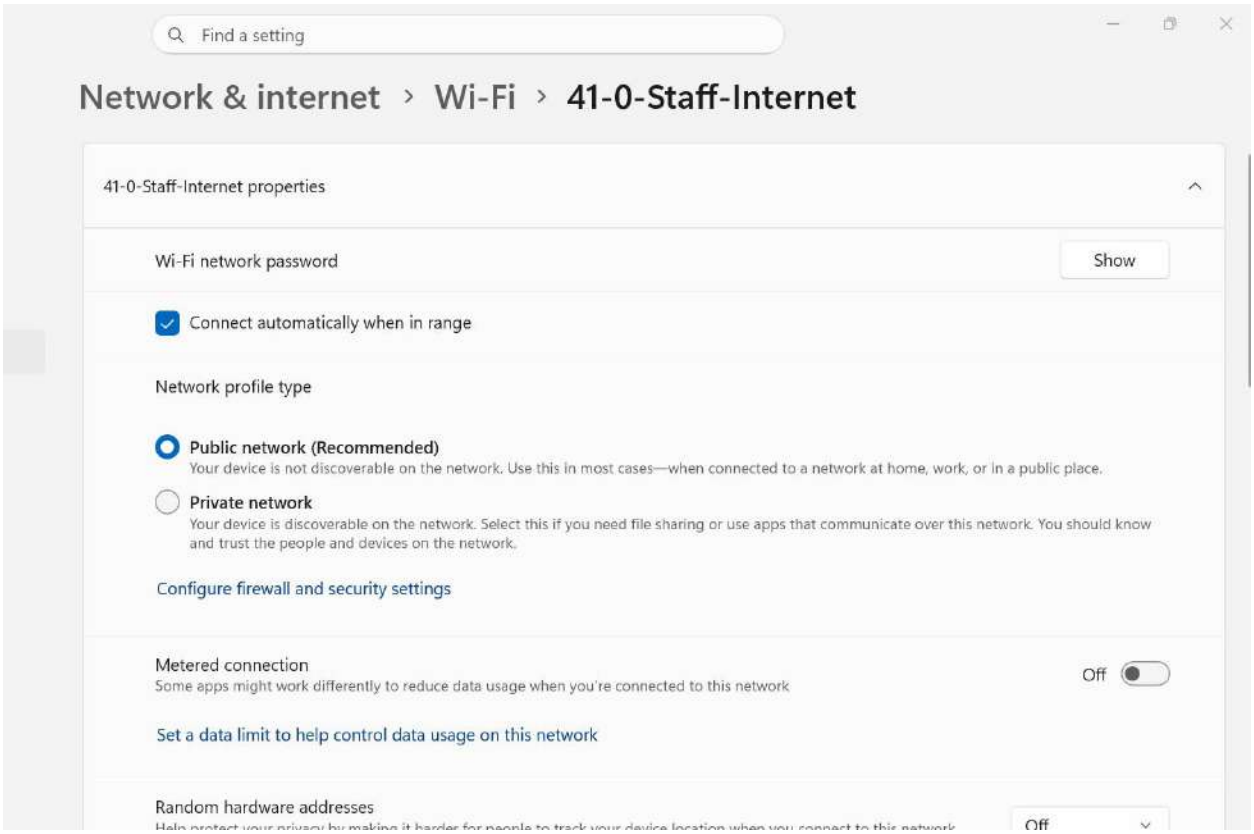
Security Steps:

- Settings → **Network & Internet**
- Check:
  - WiFi connection
  - Network type → Private/Public



Network Overview

- Wi-Fi connected → **41-0-Staff-Internet**
- Status → **Connected, secured**
- Shows general network info



Network Type

- Network profile → **Public network (selected)**

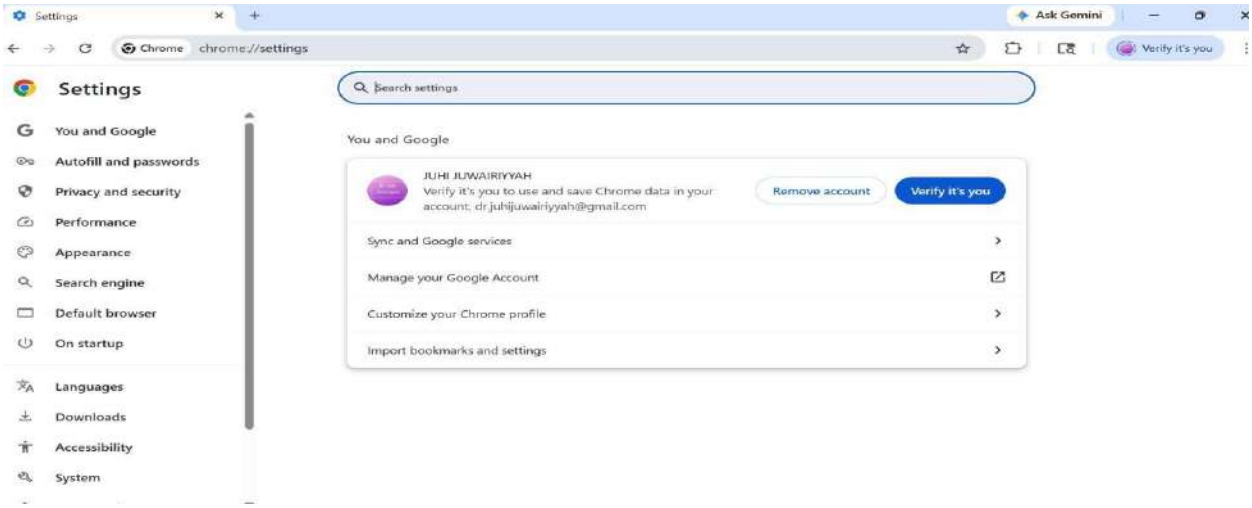
- Private option available

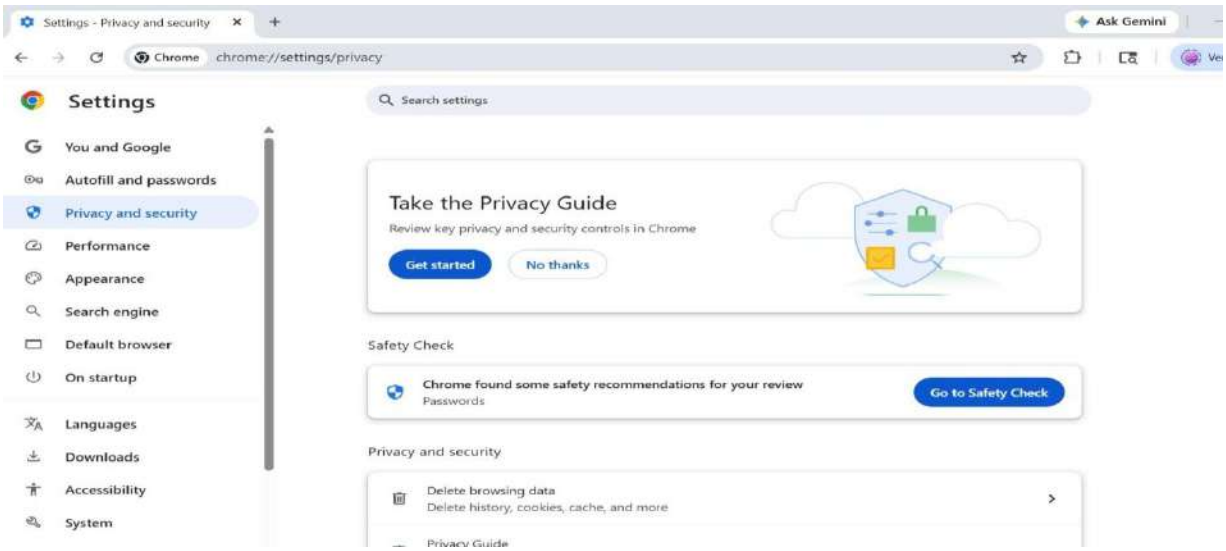
Observation:

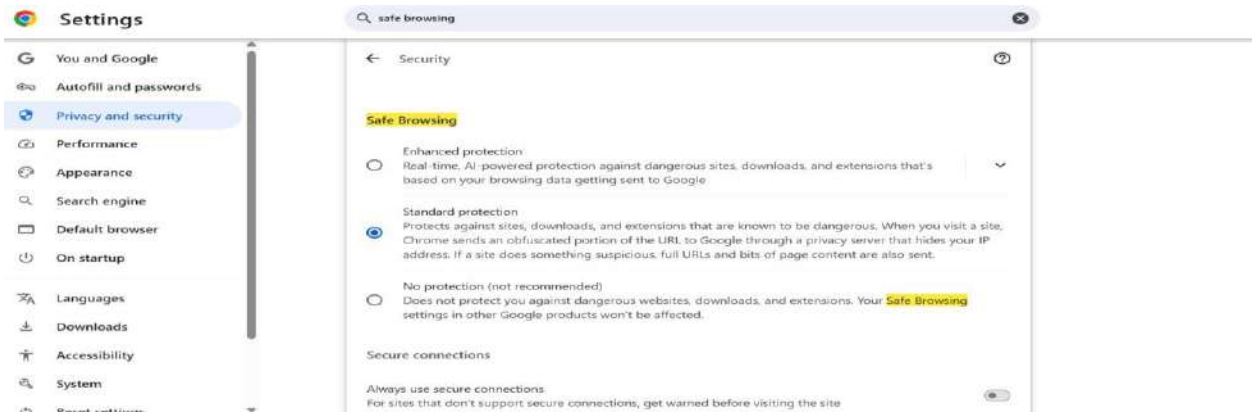
The network settings were analyzed, and the system is connected to a Wi-Fi network with a public network profile. This restricts device visibility and enhances security.
Using a public network profile improves security by limiting unauthorized access, especially in untrusted environments.

STEP 9: Check Browser Security Steps (Chrome/Edge):

- Go to **Settings → Privacy & Security**
- Check:
  - Safe browsing ON
  - Remove unknown extensions







Observation:

The browser security settings were examined under the Privacy and Security section. Safe Browsing was found to be enabled with standard protection, ensuring protection against malicious websites and unsafe downloads.

Safe Browsing helps prevent phishing attacks, malware infections, and unsafe browsing activities.

STEP 10:

Check Data Backup

Steps:

- Search: **Backup settings**
- Check:
  - OneDrive / External backup

: lab-user

Observation:

The backup settings were checked, and it was observed that OneDrive backup is not configured and system data is not being backed up.

Lack of proper backup mechanisms may lead to permanent data loss in case of system failure or cyber attacks such as ransomware.

STEP 11: Risk Assessment Table

Asset	Threat	Vulnerability	Risk Level	Solution
Personal Files	Data Loss	No Backup	High	Enable OneDrive
System	Malware	Unknown Apps	High	Remove apps
Network	Hacking	Public Network	Medium	Use Private
Account	Unauthorized Access	Weak Password	High	Strong password

STEP 12: Security Recommendations

✓ Technical Controls

- Enable firewall
- Enable antivirus
- Regular updates

Administrative Controls

- Strong passwords
- Avoid unknown downloads

User Awareness

- Avoid phishing links
- Safe browsing

STEP 13: Result

The system security audit identified potential vulnerabilities such as lack of backup and configuration risks. Suitable mitigation strategies were suggested to improve system security.

Experiment 22: Cloud Security Configuration-Lab – AWS Free Tier

Aim

To implement cloud security configuration using AWS Free Tier.

Note: Based on account type and permission levels, certain configurations may not be accessible to all users. The steps should be adapted based on available access.

Tools Required

AWS Free Tier Account Laptop/Desktop with

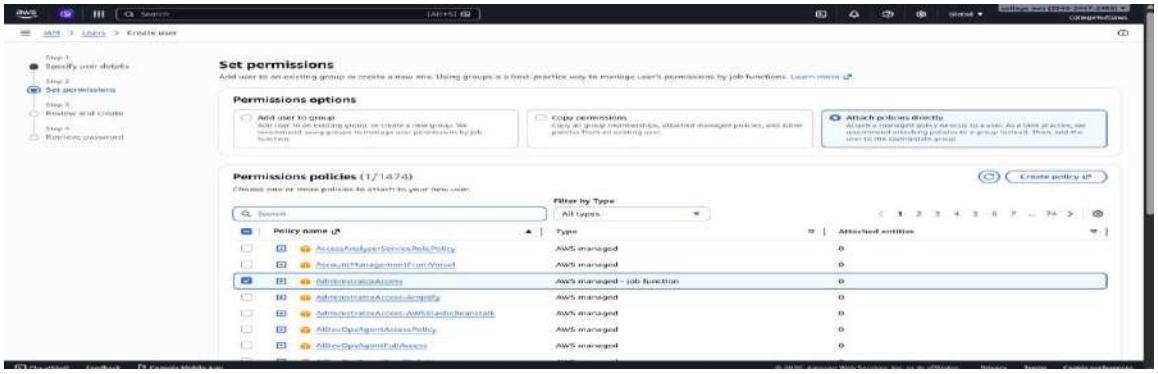
Internet

Web Browser (Chrome/Edge/Firefox)

Step 1 – Create IAM User

Login to AWS Console

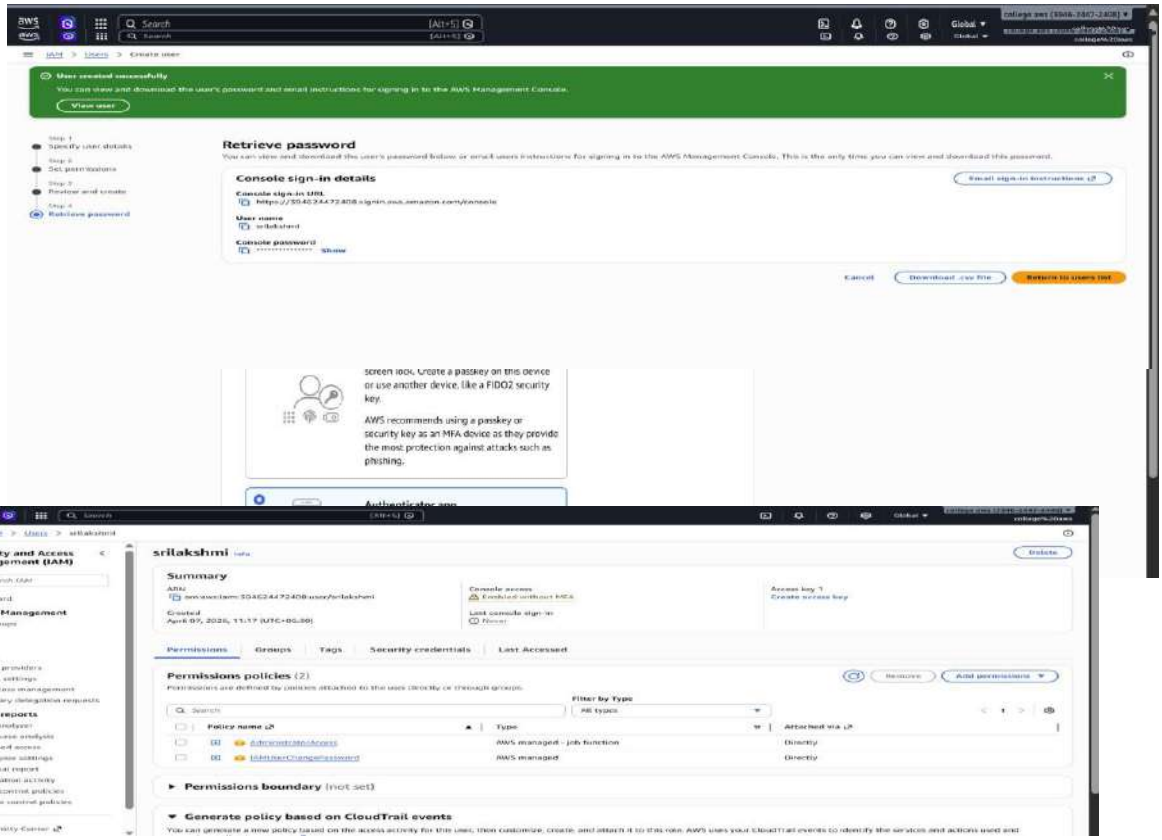
Attach Policy: Administrator Access



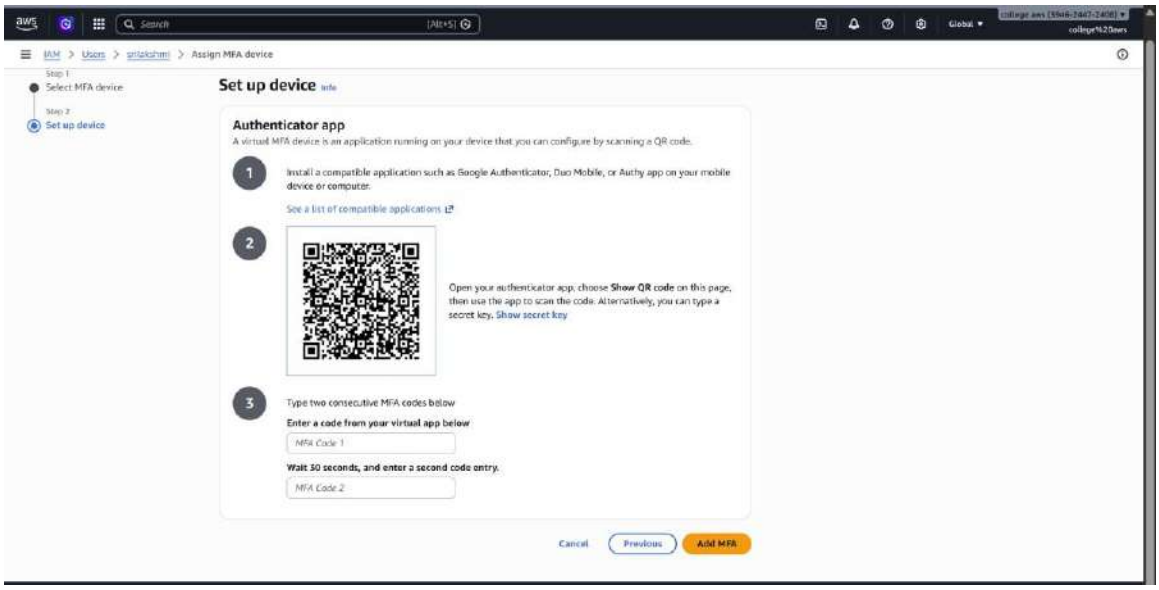
Enable Console Login Login

using IAM user

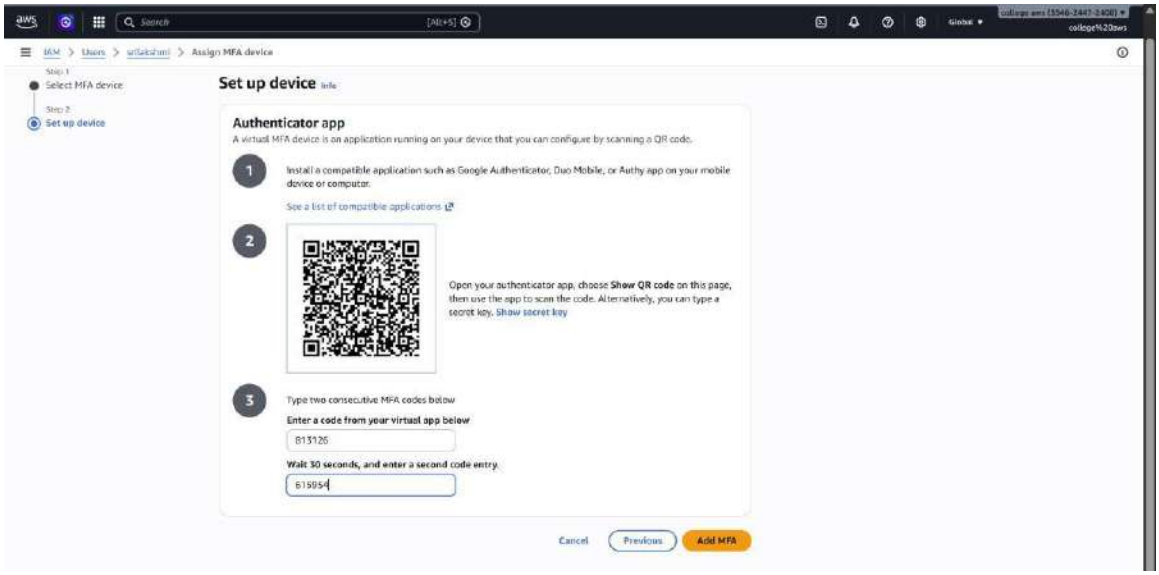
IAM User Created



Click Assign MFA

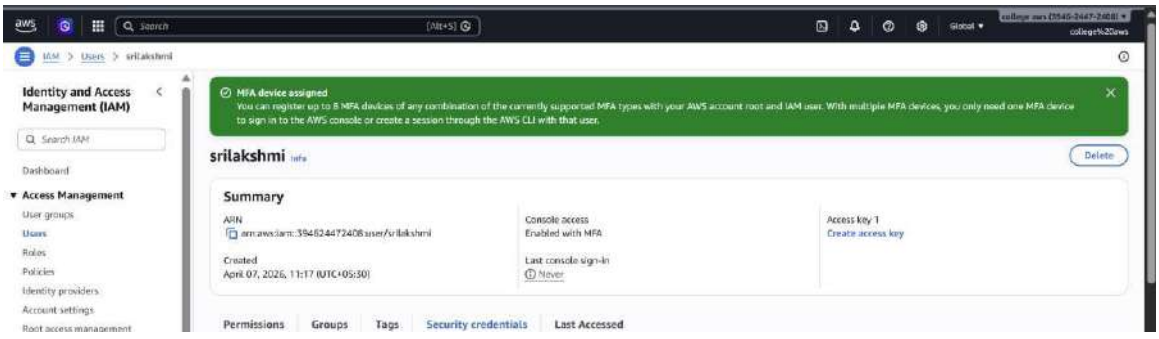


Scan QR code using Google Authent



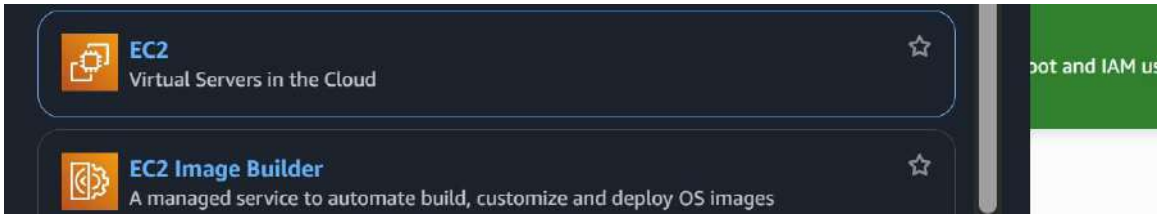
Enter two MFA codes MFA

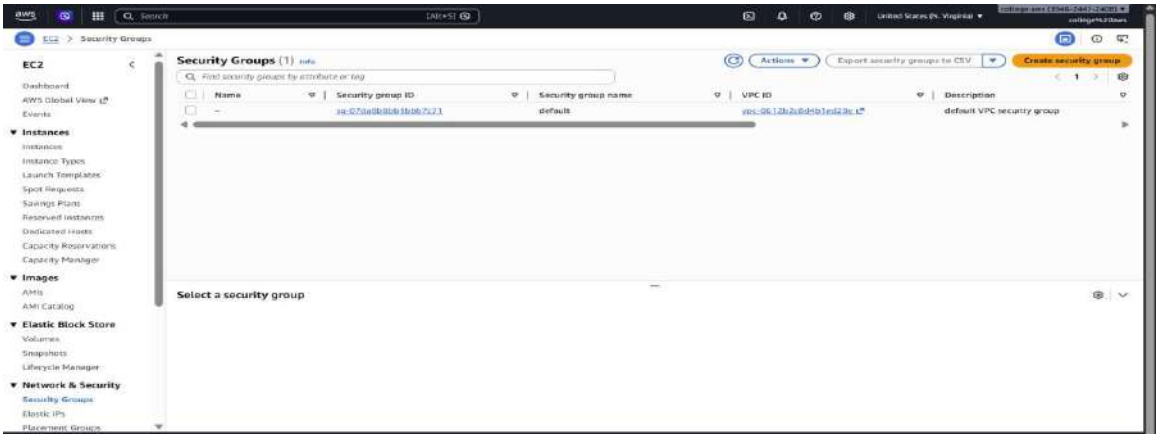
Enabled



Step 3 – Create Security Group

Go to EC2 → Security Groups → Create

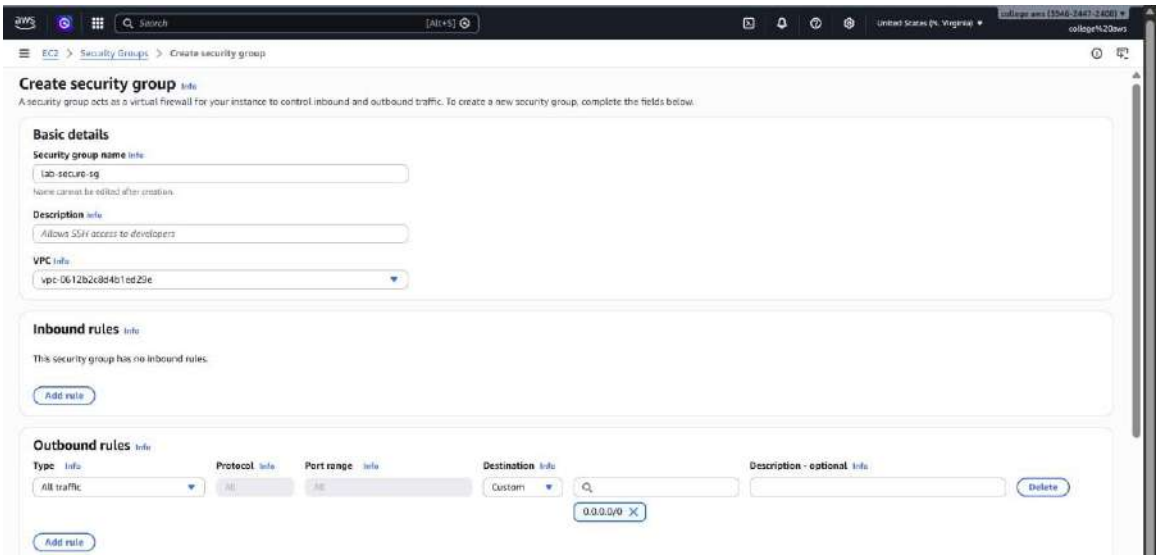




Security Group Name: lab-secure-sg Inbound

Rule: SSH, Port 22, My IP Outbound: Allow

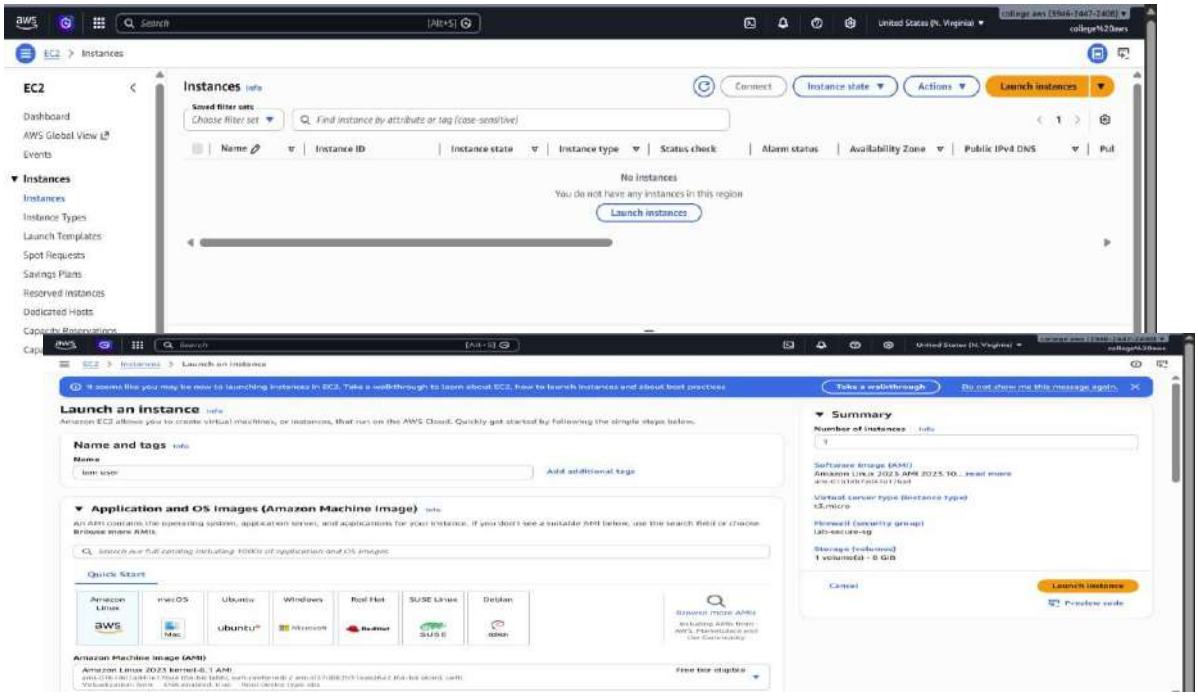
All



Security Group Created

Step 4 – Launch EC2

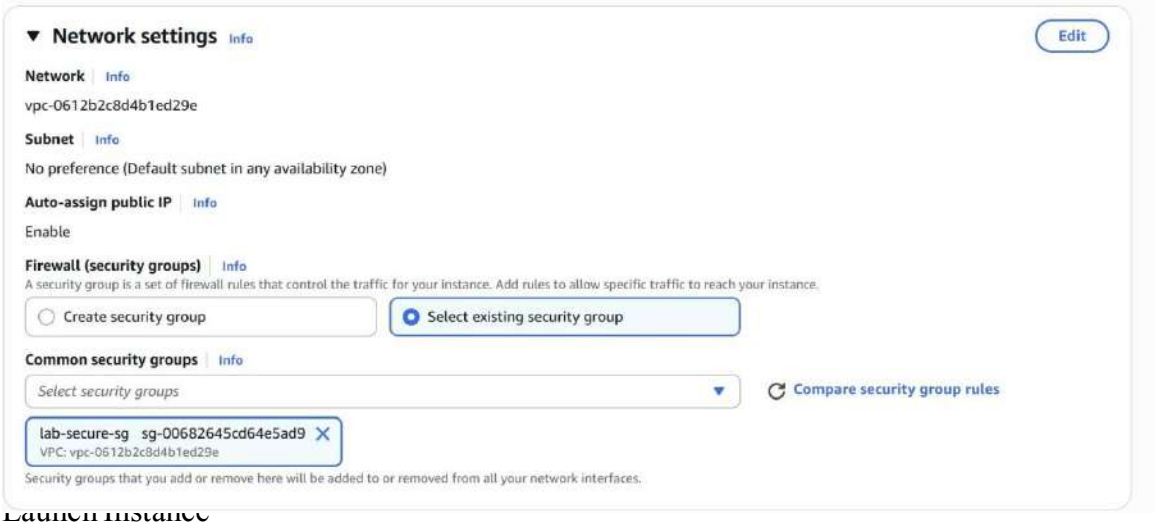
Go to EC2 → Launch Instance



Instance Type: t2.micro Create

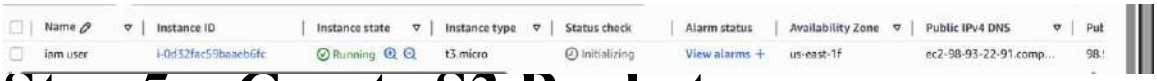
Key Pair

Select Security Group



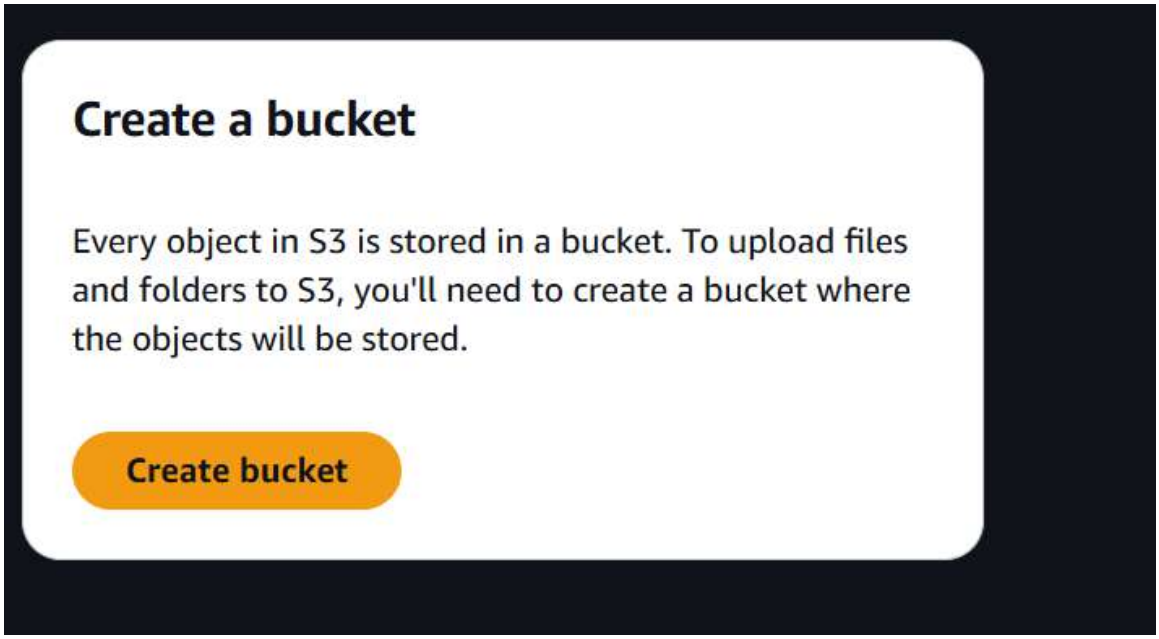
Launch Instance

EC2 Running

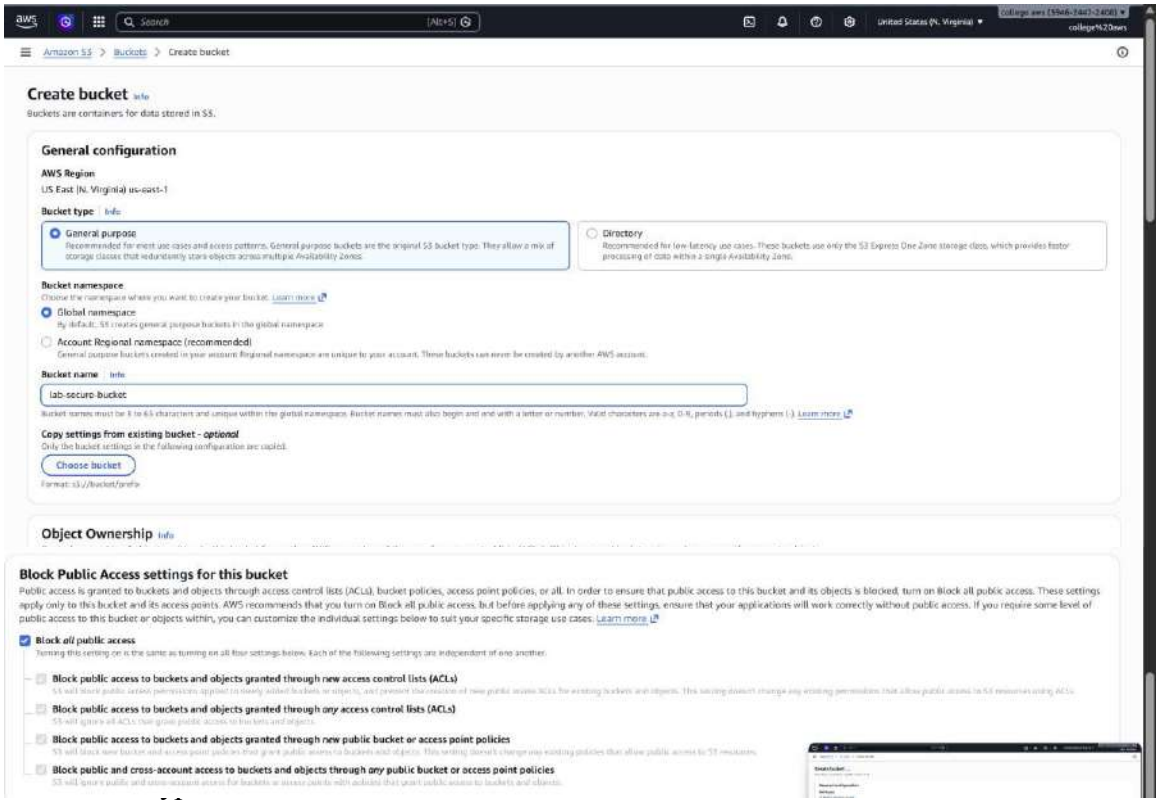


Step 5 – Create S3 Bucket

Go to S3 → Create Bucket

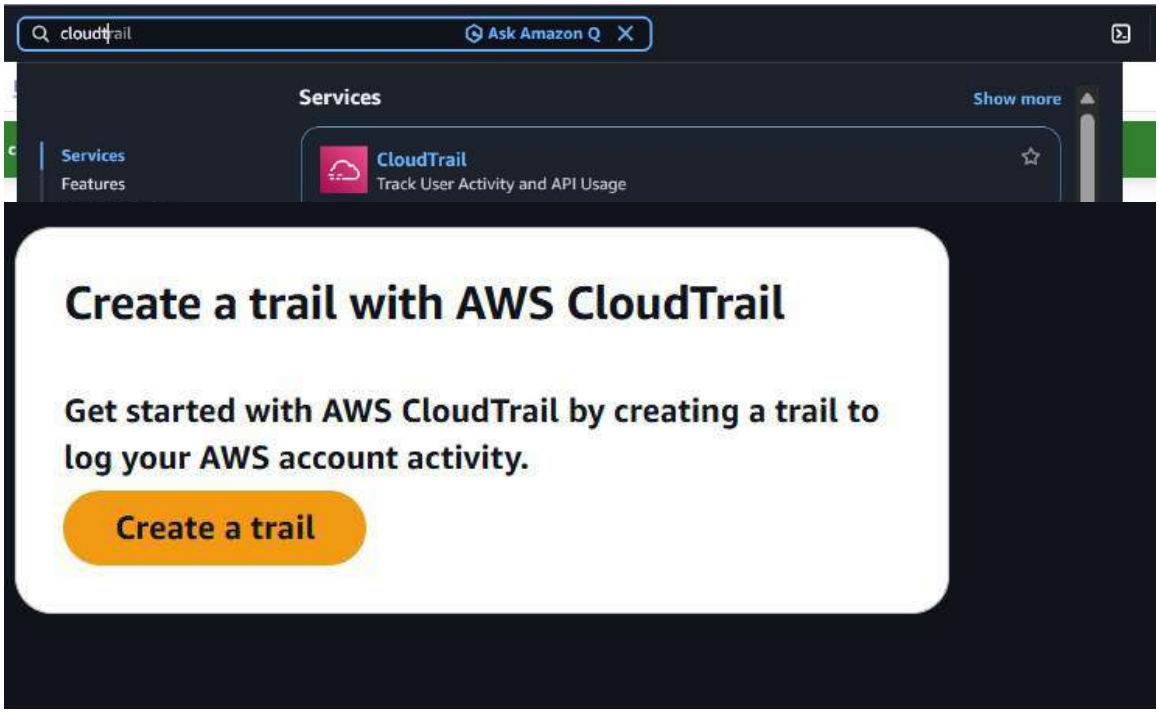


Bucket Name: lab-secure-bucket

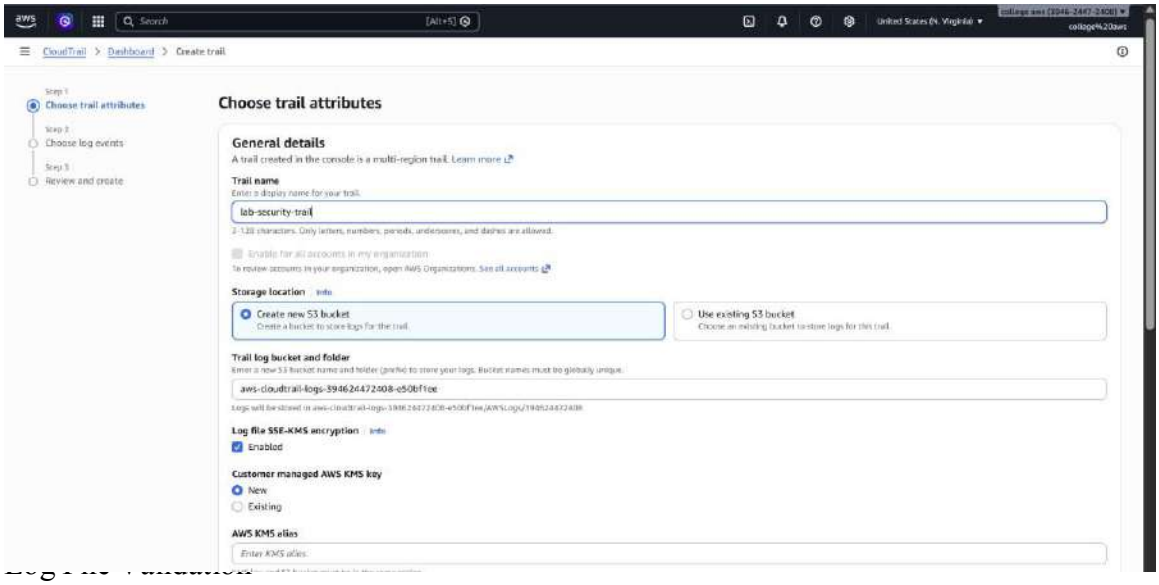


Step 6 – Enable CloudTrail

Go to CloudTrail → Create Trail

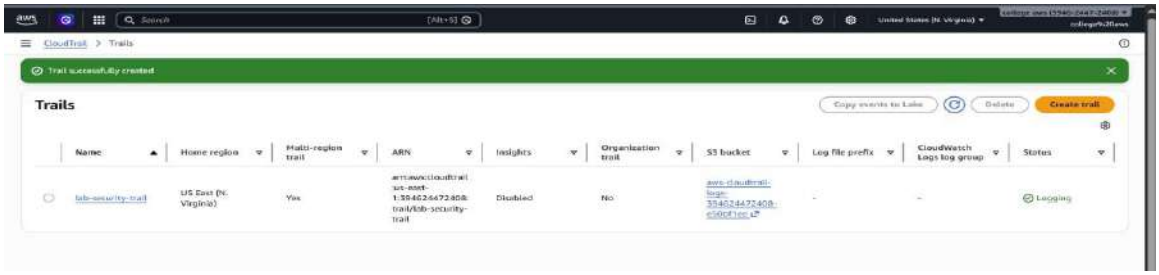


Trail Name: lab-security-trail



Log File Validation

Logging ON and CloudTrail Enabled



Result

IAM User created MFA enabled

Security Group configured EC2 instance launched

S3 bucket secured CloudTrail logging enabled

Secure cloud setup created successfully